

Part 01. AI 시대, 왜 프로그래머에게 기획서가 필요한가

Part 1에서는 바이트 코딩과 AI 코드 생성 도구가 게임 개발 방식을 어떻게 바꾸고 있는지 살펴봅니다.

이제 프로그래머는 자연어로 AI에게 구현을 요청하고, 빠르게 프로토타입을 만들 수 있게 되었습니다.

하지만 개발 속도가 빨라질수록 무엇을 만들지 명확히 설명하는 능력은 더 중요해집니다.

이 파트에서는 프로그래머가 왜 기획을 이해해야 하는지, 기획자가 없는 프로젝트에서 어떤 문제가 생기는지, AI에게 일을 시키기 위해 왜 구조화된 상세 기획서가 필요한지 설명합니다.

Chapter 01. 바이트 코딩 시대의 게임 개발 변화

게임 개발 방식이 빠르게 바뀌고 있습니다. 과거에는 아이디어가 있어도 엔진 사용법, 프로그래밍 언어, 그래픽 리소스, 빌드 과정까지 알아야 작은 게임 하나를 만들 수 있었습니다. 하지만 이제는 AI에게 자연어로 명령을 내리고, 필요한 코드를 생성하며, 오류를 다시 전달해 수정하는 방식으로 개발 속도를 크게 높일 수 있게 되었습니다. 흔히 말하는 바이트 코딩은 이러한 변화를 잘 보여줍니다.

바이트 코딩은 완벽한 설계 없이도 빠르게 결과물을 만들어볼 수 있다는 장점이 있습니다. “플레이어가 움직이게 해줘”, “적이 일정 시간마다 총알을 발사하게 해줘”, “게임 오버 화면을 만들어줘”처럼 원하는 기능을 문장으로 요청하면 AI가 코드와 구현 방향을 제안합니다. 프로그래머는 그 결과를 검토하고 수정하면서 빠르게 프로토타입을 완성할 수 있습니다.

그러나 개발 속도가 빨라졌다고 해서 기획이 덜 중요해진 것은 아닙니다. 오히려 반대입니다. AI는 모호한 아이디어를 정확한 게임으로 바꿔주지 않습니다. “재미있는 전투 시스템을 만들어줘”라는 요청만으로는 어떤 공격 방식인지, 데미지 계산은 어떻게 할지, 피격 판정은 어디서 발생하는지, 쿨타임은 있는지 알 수 없습니다. 결국 좋은 결과를 얻으려면 사람이 먼저 게임의 규칙과 조건을 정리해야 합니다.

이제 프로그래머는 단순히 코드를 작성하는 사람을 넘어, 게임의 구조를 설명할 수 있는 사람이 되어야 합니다. 기획자가 따로 없는 개인 프로젝트나 소규모 팀에서는 프로그래머가 직접 콘셉트, 플레이 루프, 시스템 규칙, 데이터 구조, UI 흐름까지 정리해야 하는 경우가 많습니다. 이때 필요한 문서가 바로 프로그래머가 쓰는 게임 상세 기획서입니다.

이 책에서 말하는 상세 기획서는 거창한 문서가 아닙니다. 개발을 시작하기 전에 무엇을 만들지 명확히 정리하고, AI와 동료 개발자가 이해할 수 있는 형태로 게임을 설명하는 문서입니다. 바이트 코딩 시대에는 코드를 빠르게 만드는 능력만큼이나, 만들 것을 정확히 설명하는 능력이 중요합니다. 이 책은 그 출발점에서 프로그래머가 어떻게 게임을 기획하고 문서화할 수 있는지 다룹니다.

Chapter 02. 코드를 잘 쓰는 것만으로는 부족한 이유

프로그래머에게 코드를 잘 쓰는 능력은 매우 중요합니다. 읽기 쉬운 코드, 유지보수하기 좋은 구조, 버그가 적은 구현은 좋은 개발자의 기본 역량입니다. 하지만 게임 개발에서는 코드만 잘 쓴다고 해서 좋은 결과물이 완성되지는 않습니다. 무엇을 만들지 명확하지 않다면 아무리 좋은 코드도 방향을 잃게 됩니다.

게임은 단순한 프로그램이 아닙니다. 입력, 규칙, 보상, 피드백, 난이도, UI, 사운드, 연출이 함께 작동해야 플레이 경험이 만들어집니다. 예를 들어 플레이어 이동 코드를 잘 작성했다고 해도 이동 속도는 어느 정도가 적절한지, 대시가 필요한지, 점프 중 방향 전환이 가능한지, 피격 중에는 이동할 수 있는지 정해져 있지 않다면 구현은 계속 흔들릴 수밖에 없습니다.

코드를 잘 쓰는 능력은 “어떻게 만들 것인가”에 가깝습니다. 반면 기획은 “무엇을 만들 것인가”와 “왜 그렇게 만들어야 하는가”를 정리하는 일입니다. 이 두 가지가 함께 있어야 개발이 안정적으로 진행됩니다. 기획이 부족하면 프로그래머는 매번 즉흥적으로 판단하게 되고, 그 결과 기능이 추가될수록 코드 구조도 함께 흔들립니다.

특히 AI를 활용하는 개발에서는 이 문제가 더 분명해집니다. AI는 사용자가 명확하게 설명한 만큼 결과를 만들어냅니다. “인벤토리 만들어줘”라고 요청하면 어떤 게임에 필요한 인벤토리인지 알 수 없습니다. 장비 슬롯이 있는지, 아이템 중첩이 가능한지, 사용 조건은 무엇인지, UI는 어떤 흐름으로 열리고 닫히는지 설명해야 합니다. 결국 좋은 프롬프트는 좋은 기획서에서 나옵니다.

프로그래머가 기획서를 작성해야 하는 이유는 기획자의 역할을 빼앗기 위해서가 아닙니다. 개발자가 구현할 대상을 더 정확히 이해하고, AI와 협업할 때 더 좋은 결과를 얻기 위해서입니다. 코드 작성 능력은 여전히 중요합니다. 그러나 AI 시대의 프로그래머에게는 코드를 잘 쓰는 능력에 더해, 만들고자 하는 게임을 명확히 설명하는 능력도 필요합니다.

Chapter 03. 프로그래머가 기획을 이해해야 하는 순간

프로그래머는 보통 “기획서가 나오면 그것을 코드로 구현하는 사람”으로 이해되곤 합니다. 하지만 실제 개발 현장에서는 기획서가 항상 완성된 상태로 전달되지 않습니다. 어떤 기능은 설명이 부족하고, 어떤 규칙은 예외 조건이 빠져 있으며, 어떤 시스템은 구현 중에야 문제가 드러납니다. 이때 프로그래머가 기획을 전혀 이해하지 못하면 단순히 “기획이 부족하다”고 멈춰 설 수밖에 없습니다.

프로그래머가 기획을 이해해야 하는 첫 번째 순간은 요구사항이 모호할 때입니다. 예를 들어 “플레이어가 공격하면 적이 피해를 입는다”라는 문장은 구현하기에 부족합니다. 공격 범위는 어디까지인지, 데미지는 언제 적용되는지, 적이 이미 죽은 상태라면 어떻게 처리하는지, 피격 애니메이션 중 다시 맞을 수 있는지 정해야 합니다. 이런 세부 조건을 질문하고 정리하는 능력이 기획 이해력입니다.

두 번째 순간은 기능 간 충돌이 발생할 때입니다. 대시 중에는 공격이 가능한지, 피격 중에는 아이템을 사용할 수 있는지, 게임 오버 이후에도 보상이 지급되는지 같은 문제는 코드만 보고 해결하기 어렵습니다. 기능은 각각 따로 존재하지 않고 서로 영향을 주기 때문에, 전체 게임 규칙을 이해해야 올바른 판단을 할 수 있습니다.

세 번째 순간은 AI에게 구현을 요청할 때입니다. AI에게 “전투 시스템을 만들어줘”라고 말하면 결과는 불안정합니다. 하지만 공격 조건, 데미지 계산, 피격 처리, 쿨타임, 상태 전환 규칙을 정리해서 요청하면 더 정확한 코드를 얻을 수 있습니다. 결국 AI를 잘 활용하는 프로그래머는 기획 내용을 구현 단위로 설명할 수 있어야 합니다.

프로그래머가 기획을 이해한다는 것은 기획자의 감각을 그대로 따라 한다는 뜻이 아닙니다. 게임의 규칙을 구조적으로 읽고, 빠진 조건을 발견하고, 구현 가능한 형태로 정리하는 능력을 갖춘다는 뜻입니다. 이 능력이 있을 때 프로그래머는 단순 구현자를 넘어, 게임의 완성도를 함께 책임지는 개발자가 될 수 있습니다.

Chapter 04. 기획자가 없는 프로젝트에서 생기는 문제

개인 개발이나 소규모 팀 프로젝트에서는 전담 기획자가 없는 경우가 많습니다. 이때 프로그래머는 자연스럽게 아이디어 정리, 기능 정의, 시스템 규칙 작성까지 함께 맡게 됩니다. 처음에는 “일단 만들어 보자”는 방식으로 진행할 수 있지만, 기능이 늘어날수록 기획의 빈자리는 점점 크게 드러납니다.

가장 먼저 생기는 문제는 개발 방향이 자주 바뀐다는 것입니다. 게임의 핵심 재미, 목표, 플레이 루프가 명확하지 않으면 오늘 만든 기능이 내일 필요 없어질 수 있습니다. 예를 들어 처음에는 간단한 슈팅 게임을 만들려고 했지만, 중간에 성장 시스템, 장비 시스템, 퀘스트 시스템을 추가하다 보면 프로젝트 범위가 빠르게 커집니다. 결국 완성도보다 기능 목록만 늘어나는 상황이 생깁니다.

두 번째 문제는 기능 간 규칙이 충돌한다는 점입니다. 대시 중 공격이 가능한지, 스킬 사용 중 피격되면 취소되는지, 게임 오버 직전에 획득한 보상은 저장되는지 같은 조건은 미리 정리하지 않으면 구현 중마다 즉흥적으로 결정하게 됩니다. 이런 결정이 쌓이면 코드 구조도 함께 복잡해지고, 나중에는 작은 수정 하나가 여러 시스템에 영향을 주게 됩니다.

세 번째 문제는 AI를 활용해도 결과가 불안정해진다는 것입니다. 기획자가 없다는 것은 AI에게 전달할 기준 문서도 부족하다는 뜻입니다. “상점 시스템을 만들어줘”라고 요청하면 AI는 일반적인 코드를 만들 수는 있지만, 이 게임에 맞는 구매 조건, 재화 종류, 아이템 제한, UI 흐름까지 정확히 알 수는 없습니다. 결국 사람이 다시 많은 부분을 수정해야 합니다.

기획자가 없는 프로젝트에서 중요한 것은 모든 문서를 완벽하게 작성하는 것이 아닙니다. 최소한 게임의 핵심 규칙, 기능 범위, 예외 조건, 데이터 구조는 정리해야 합니다. 프로그래머가 쓰는 상세 기획서는 이런 상황에서 프로젝트의 기준점이 됩니다. 기획자가 없기 때문에 문서가 필요 없는 것이 아니라, 오히려 기획자가 없기 때문에 더 짧고 명확한 기획서가 필요합니다.

Chapter 05. AI에게 일을 시키려면 먼저 구조화해야 한다

AI에게 좋은 결과를 얻으려면 먼저 사람이 만들고 싶은 내용을 구조화해야 합니다. 구조화한다는 것은 머릿속 아이디어를 기능, 규칙, 조건, 데이터, 예외 상황으로 나누어 정리하는 일입니다. 예를 들어 “플레이어가 적을 공격한다”는 문장은 너무 넓습니다. 공격 입력은 어떤 버튼인지, 공격 판정은 언제 발생하는지, 데미지는 어떤 공식으로 계산하는지, 적이 사망했을 때 보상은 언제 지급되는지, 공격 중 이동이 가능한지까지 나누어야 구현 가능한 요청이 됩니다.

AI에게 일을 시킬 때도 같은 원리가 적용됩니다. 좋은 프롬프트는 단순히 긴 문장이 아닙니다. 목적, 조건, 출력 형식, 제한 사항이 분명한 요청입니다. “상점 시스템을 만들어줘”보다 “골드를 사용해 아이템을 구매하는 상점 시스템을 만들어줘. 아이템은 ID, 이름, 가격, 설명을 가진다. 구매 전 골드가 부족하면 실패 메시지를 출력하고, 구매 성공 시 인벤토리에 아이템을 추가한다”라고 요청해야 더 안정적인 결과를 얻을 수 있습니다.

상세 기획서는 AI에게 전달할 구조화된 입력 자료가 됩니다. 기획서에 게임의 핵심 루프, 시스템 규칙, UI 흐름, 데이터 구조, 예외 조건이 정리되어 있다면 AI는 그 문서를 바탕으로 코드, 클래스 구조, 테스트 코드, UI 구현 방향을 제안할 수 있습니다. 반대로 기획서가 없다면 프로그래머는 매번 즉흥적으로 설명해야 하고, AI의 답변도 요청할 때마다 달라질 수 있습니다.

바이브 코딩 시대에 중요한 능력은 AI에게 빠르게 질문하는 능력만이 아닙니다. 질문하기 전에 문제를 작게 나누고, 구현 가능한 단위로 정리하는 능력이 더 중요합니다. AI는 구조화된 생각을 빠르게 코드로 바꾸는 도구입니다. 따라서 프로그래머가 먼저 게임을 구조화할 수 있어야 AI도 제대로 일을 할 수 있습니다.

Chapter 06. 상세 기획서는 개발을 느리게 만드는 문서가 아니다

상세 기획서라고 하면 많은 프로그래머가 먼저 부담을 느낍니다. 문서를 작성하는 시간만큼 개발이 늦어지고, 실제 코드를 작성하는 시간이 줄어든다고 생각하기 쉽습니다. 특히 개인 프로젝트나 소규모 팀에서는 “일단 만들어보면 되지 않을까?”라는 생각으로 문서 작성을 생략하는 경우가 많습니다.

하지만 상세 기획서는 개발을 느리게 만드는 문서가 아닙니다. 오히려 개발 중에 발생할 불필요한 되돌림을 줄여주는 기준점입니다. 무엇을 만들지 정하지 않은 상태에서 코드를 작성하면, 기능을 만들고 나서 다시 고치거나 삭제하는 일이 자주 발생합니다. 처음에는 빠르게 시작한 것처럼 보이지만, 시간이 지날수록 수정 비용이 커집니다.

예를 들어 상점 시스템을 만든다고 가정해보겠습니다. 처음에는 아이템을 구매하는 기능만 만들었지만, 나중에 재화 부족 처리, 구매 제한, 보유 수량 제한, 구매 확인 팝업, 아이템 중복 처리, 저장 처리까지 필요하다는 사실을 뒤늦게 알게 될 수 있습니다. 이 조건들을 처음부터 짧게라도 정리했다면 구조를 더 안정적으로 잡을 수 있습니다.

상세 기획서는 모든 것을 완벽하게 확정하는 문서가 아닙니다. 현재 알고 있는 내용을 기준으로 기능의 목적, 규칙, 입력, 출력, 예외 조건을 정리하는 문서입니다. 문서를 작성한다고 해서 변경이 금지되는 것은 아닙니다. 오히려 문서가 있으면 무엇이 바뀌었는지 추적할 수 있고, 변경이 코드에 어떤 영향을 주는지도 더 쉽게 판단할 수 있습니다.

AI를 활용할 때도 상세 기획서는 개발 속도를 높이는 도구가 됩니다. 기획서가 있으면 매번 AI에게 처음부터 설명할 필요가 없습니다. 정리된 규칙과 조건을 기반으로 구현 요청, 리팩토링 요청, 테스트 코드 요청을 할 수 있습니다. 결국 상세 기획서는 개발을 멈추게 하는 문서가 아니라, 개발이 엉뚱한 방향으로 가지 않도록 도와주는 가이드입니다.

Chapter 07. 프로그래머가 쓰는 기획서와 기획자가 쓰는 기획서의 차이

게임 기획서는 하나의 정답으로만 작성되는 문서가 아닙니다. 누가 읽고, 어떤 목적으로 사용하는지에 따라 내용과 형식이 달라집니다. 기획자가 쓰는 기획서는 게임의 재미, 콘셉트, 플레이 경험, 유저 흐름을 설득력 있게 설명하는 데 초점이 맞춰지는 경우가 많습니다. 반면 프로그래머가 쓰는 기획서는 실제 구현에 필요한 규칙, 조건, 데이터, 예외 상황을 명확히 정리하는 데 초점이 있습니다.

예를 들어 기획자가 “플레이어는 빠르고 시원한 액션을 경험한다”라고 표현할 수 있습니다. 이 문장은 게임의 방향을 설명하는 데는 유용하지만, 프로그래머가 바로 구현하기에는 부족합니다. 프로그래머 관점에서는 이동 속도, 공격 속도, 대시 거리, 쿨타임, 피격 경직, 카메라 흔들림, 이펙트 타이밍처럼 구체적인 값과 조건으로 바뀌어야 합니다.

기획자가 쓰는 문서가 “무엇이 재미있는가”를 설명한다면, 프로그래머가 쓰는 문서는 “그 재미를 어떤 시스템으로 구현할 것인가”를 설명합니다. 전투가 재미있어야 한다는 말은 공격 판정, 데미지 계산, 스킬 쿨타임, 적 행동 패턴, 보상 구조로 나누어져야 합니다. 성장의 재미는 경험치 공식, 레벨업 조건, 능력치 증가량, 아이템 드랍 테이블로 정리되어야 합니다.

또 하나의 차이는 예외 조건을 바라보는 방식입니다. 프로그래머는 정상 흐름뿐 아니라 비정상 흐름도 함께 생각해야 합니다. 골드가 부족할 때 구매 버튼은 어떻게 동작하는지, 인벤토리가 가득 찼을 때 아이템 획득은 어떻게 처리하는지, 보스가 사망하는 순간 플레이어도 죽으면 승패 판정은 어떻게 되는지 정리해야 합니다. 이런 조건들이 빠지면 구현 중에 즉흥적인 판단이 늘어납니다.

프로그래머가 쓰는 기획서는 기획자의 문서를 대체하기 위한 문서가 아닙니다. 게임의 의도를 구현 가능한 형태로 번역하는 문서에 가깝습니다. AI 시대에는 이 번역 능력이 더 중요해집니다. AI에게 코드를 요청하려면 감성적인 설명보다 명확한 규칙과 조건이 필요합니다. 따라서 프로그래머가 쓰는 기획서는 사람과 AI가 함께 이해할 수 있는 개발 명세서의 역할을 합니다.

Chapter 08. 이 책에서 작성할 상세 기획서의 최종 형태

이 책의 목표는 단순히 기획서 작성법을 설명하는 데 있지 않습니다. 최종적으로는 프로그래머가 직접 사용할 수 있는 게임 상세 기획서 한 벌을 완성하는 것이 목표입니다. 이 문서는 아이디어를 정리하는 메모가 아니라, 실제 구현과 AI 요청에 사용할 수 있는 개발 기준 문서입니다.

이 책에서 작성할 상세 기획서는 크게 게임 개요, 핵심 플레이 루프, 시스템 기획, UI 기획, 레벨 기획, 데이터 기획, 예외 조건, AI 개발 요청 프롬프트로 구성됩니다. 게임 개요에서는 어떤 게임을 만들 것인지 정리하고, 핵심 플레이 루프에서는 플레이어가 반복해서 경험할 행동 흐름을 설명합니다. 이후 각 시스템을 기능 단위로 나누어 구현 가능한 규칙으로 정리합니다.

예를 들어 전투 시스템 문서에는 공격 입력, 공격 판정, 데미지 계산, 피격 처리, 쿨타임, 사망 조건이 포함됩니다. 아이템 시스템 문서에는 아이템 종류, 사용 조건, 보유 제한, 획득 방식, 데이터 테이블 구조가 포함됩니다. UI 문서에는 화면 목록, 버튼 동작, 팝업 조건, 화면 전환 흐름이 들어갑니다. 이렇게 작성된 문서는 곧 구현 명세서가 됩니다.

또한 이 책의 상세 기획서는 AI에게 전달하기 쉬운 형태를 목표로 합니다. 문장은 짧고 명확해야 하며, 하나의 기능은 목적, 조건, 처리 흐름, 예외 상황으로 나누어 작성합니다. “상점 시스템을 만든다”가 아니라 “골드가 충분하면 아이템을 구매하고, 골드가 부족하면 실패 메시지를 출력한다”처럼 AI가 코드로 옮길 수 있는 문장으로 정리합니다.

최종적으로 이 책을 따라가면 하나의 작은 게임을 만들기 위한 상세 기획서가 완성됩니다. 이 기획서는 개발을 시작하기 전 기준점이 되고, 구현 중에는 판단 기준이 되며, AI에게 요청할 때는 프롬프트의 원본 자료가 됩니다. 프로그래머가 쓰는 상세 기획서는 문서 작업을 위한 문서가 아니라, 게임을 더 명확하게 만들기 위한 개발 도구입니다.

Part 02. 게임 아이디어를 개발 가능한 문장으로 바꾸기

Part 2에서는 막연한 게임 아이디어를 구현 가능한 기획으로 바꾸는 방법을 다룹니다.

“재미있는 액션 게임”, “성장 요소가 있는 슈팅 게임” 같은 표현은 출발점으로는 좋지만, 실제 개발에는 부족합니다.

이 파트에서는 게임 콘셉트를 한 문장으로 정리하고, 타겟 유저와 장르를 정의하며, 참고 게임을 프로그래머 관점에서 분석하는 방법을 설명합니다.

또한 핵심 재미를 기능 단위로 분해하고, MVP 기준으로 게임 범위를 줄이는 과정을 통해 아이디어를 현실적인 개발 범위로 정리합니다.

Chapter 09. 막연한 아이디어가 구현을 어렵게 만든다

게임 개발은 대부분 막연한 아이디어에서 시작됩니다. “재미있는 액션 게임을 만들고 싶다”, “성장 요소가 있는 슈팅 게임을 만들고 싶다”, “간단한 RPG를 만들어보고 싶다” 같은 생각은 출발점으로는 충분합니다. 하지만 이 상태로 바로 구현을 시작하면 곧 문제가 생깁니다. 아이디어는 방향을 알려줄 수는 있지만, 코드가 되기에는 아직 너무 넓고 모호하기 때문입니다.

프로그래머는 구현할 때 구체적인 조건이 필요합니다. 플레이어가 어떤 버튼을 누르면 어떤 행동을 하는지, 공격이 언제 성공하는지, 적은 어떤 기준으로 움직이는지, 보상은 언제 지급되는지 정해져 있어야 합니다. “빠른 전투”라는 표현은 느낌을 전달할 수는 있지만, 코드로 만들려면 이동 속도, 공격 속도, 쿨타임, 피격 경직, 카메라 연출 같은 구체적인 요소로 나누어야 합니다.

막연한 아이디어의 가장 큰 문제는 개발 중 판단 기준이 없어진다는 점입니다. 새로운 기능을 추가할 때 이 게임에 필요한 기능인지, 범위를 넘어서는 기능인지 결정하기 어렵습니다. 처음에는 간단한 게임을 만들려고 했지만, 어느 순간 장비, 스킬, 퀘스트, 상점, 업적까지 추가하고 싶어집니다. 기준이 없으면 프로젝트는 점점 커지고 완성은 멀어집니다.

AI를 활용할 때도 마찬가지입니다. “재미있는 게임 시스템을 만들어줘”라고 요청하면 AI는 일반적인 답을 줄 수밖에 없습니다. 반면 “플레이어가 3초마다 자동 공격하고, 적을 처치하면 경험치를 얻으며, 레벨업 시 공격력이 증가하는 방치형 전투 시스템을 만들어줘”라고 요청하면 구현 방향이 훨씬 명확해집니다. AI는 막연한 상상을 대신 정리해주는 도구가 아니라, 정리된 생각을 빠르게 확장해주는 도구에 가깝습니다.

따라서 아이디어를 구현하려면 먼저 작게 쪼개야 합니다. 장르, 목표, 핵심 플레이 루프, 주요 시스템, 조작 방식, 승패 조건처럼 개발에 필요한 단위로 나누어야 합니다. 이 과정을 거치면 막연한 아이디어는 구현 가능한 기획으로 바뀝니다. 프로그래머가 쓰는 상세 기획서는 바로 이 전환을 돕는 문서입니다.

Chapter 10. 게임 콘셉트를 한 문장으로 정리하기

게임 아이디어를 구현 가능한 기획으로 바꾸려면 먼저 콘셉트를 한 문장으로 정리해야 합니다. 콘셉트 문장은 게임의 방향을 잡아주는 기준입니다. “어떤 플레이어가, 어떤 상황에서, 무엇을 하며, 어떤 재미를 느끼는 게임인가”를 짧게 설명할 수 있어야 합니다. 이 문장이 명확할수록 이후 시스템, 조작, UI, 레벨, 데이터 기획도 흔들리지 않습니다.

예를 들어 “재미있는 슈팅 게임”이라는 표현은 콘셉트가 아닙니다. 너무 넓고 모호하기 때문입니다. 반면 “플레이어가 제한된 탄약으로 몰려오는 적을 처치하며 최대한 오래 생존하는 탑다운 생존 슈팅 게임”이라고 쓰면 구현 방향이 훨씬 분명해집니다. 이 문장 안에는 장르, 플레이 목표, 주요 행동, 긴장감의 원인이 들어 있습니다.

프로그래머에게 좋은 콘셉트 문장은 구현 범위를 줄여주는 역할을 합니다. 콘셉트가 “짧은 시간 안에 반복 플레이할 수 있는 캐주얼 퍼즐 게임”이라면 복잡한 성장 시스템이나 긴 스토리 연출은 우선순위에서 밀릴 수 있습니다. 반대로 “캐릭터를 성장시키며 스테이지를 돌파하는 액션 RPG”라면 레벨업, 장비, 스킬, 적 패턴 같은 시스템이 중요해집니다.

콘셉트 문장을 작성할 때는 감상적인 표현보다 구체적인 표현을 우선해야 합니다. “시원한 액션”보다는 “대시와 근접 공격을 빠르게 연계하는 액션”이 좋고, “전략적인 전투”보다는 “카드 선택과 위치 이동으로 적의 공격을 피하는 턴제 전투”가 좋습니다. 그래야 AI에게 설명할 때도 더 정확한 결과를 얻을 수 있습니다.

한 문장 콘셉트는 최종 답이 아니라 개발의 출발점입니다. 개발 중에 기능이 늘어나거나 방향이 흔들릴 때 이 문장으로 돌아와 판단할 수 있습니다. 지금 추가하려는 기능이 콘셉트와 맞는지, 핵심 재미를 강화하는지 확인하는 기준이 됩니다. 프로그래머가 상세 기획서를 쓰기 시작할 때 가장 먼저 해야 할 일은 게임을 한 문장으로 설명하는 것입니다.

Chapter 11. 타겟 유저를 정의해야 기능 범위가 보인다

게임을 기획할 때 “누가 이 게임을 플레이할 것인가”를 먼저 생각해야 합니다. 타겟 유저가 명확하지 않으면 기능의 범위도 쉽게 흔들립니다. 초보자를 위한 게임인지, 숙련자를 위한 게임인지, 짧게 즐기는 모바일 게임인지, 오래 몰입하는 PC 게임인지에 따라 필요한 시스템과 우선순위가 달라지기 때문입니다.

예를 들어 초보자를 대상으로 하는 액션 게임이라면 조작은 단순해야 합니다. 공격, 회피, 이동 정도만으로도 충분할 수 있습니다. 반대로 숙련자를 대상으로 한다면 콤보, 패링, 스킬 연계, 장비 세팅 같은 깊이 있는 시스템이 필요할 수 있습니다. 같은 액션 게임이라도 타겟 유저에 따라 구현해야 할 기능이 달라집니다.

타겟 유저는 난이도 설계에도 영향을 줍니다. 캐주얼 유저를 대상으로 한다면 실패해도 쉽게 다시 도전할 수 있어야 하고, 설명 없이도 규칙을 이해할 수 있어야 합니다. 하드코어 유저를 대상으로 한다면 높은 난이도, 세밀한 수치 조정, 반복 플레이 보상이 중요해질 수 있습니다. 즉, 타겟 유저를 정하는 일은 단순한 마케팅 문구가 아니라 게임의 구조를 결정하는 일입니다.

프로그래머 관점에서는 타겟 유저가 기능 제한의 기준이 됩니다. “이 유저에게 정말 필요한 기능인가?”라는 질문으로 기능 추가를 판단할 수 있습니다. 초보자용 퍼즐 게임에 복잡한 장비 시스템이 필요하지 않을 수 있고, 짧은 플레이를 목표로 한 게임에 긴 성장 곡선이 어울리지 않을 수 있습니다. 타겟이 명확하면 불필요한 기능을 줄일 수 있습니다.

AI에게 기획이나 구현을 요청할 때도 타겟 유저 정보는 중요합니다. “초보자도 5분 안에 이해할 수 있는 모바일 퍼즐 게임”과 “전략 게임 경험자가 오래 고민하며 플레이하는 턴제 전술 게임”은 AI가 제안해야 할 시스템이 다릅니다. 그래서 상세 기획서에는 반드시 타겟 유저를 포함해야 합니다. 타겟 유저를 정의하면 게임의 기능 범위와 구현 우선순위가 훨씬 분명해집니다.

Chapter 12. 장르를 정하면 필요한 시스템이 보인다

게임 장르는 단순한 분류가 아닙니다. 장르를 정하면 어떤 시스템이 필요한지, 어떤 기능을 우선 개발해야 하는지, 어떤 요소는 과감히 제외해도 되는지 판단할 수 있습니다. 액션, 퍼즐, RPG, 슈팅, 시뮬레이션, 전략 게임은 각각 플레이 방식이 다르고, 그에 따라 필요한 코드 구조와 데이터 구조도 달라집니다.

예를 들어 액션 게임이라면 입력 반응, 캐릭터 상태, 공격 판정, 피격 처리, 애니메이션 전환이 중요합니다. RPG라면 능력치, 경험치, 레벨업, 아이템, 장비, 퀘스트, 보상 시스템이 중심이 됩니다. 퍼즐 게임이라면 규칙 판정, 스테이지 데이터, 이동 제한, 클리어 조건, 힌트 시스템이 중요할 수 있습니다. 장르가 정해지면 필요한 시스템 목록이 자연스럽게 드러납니다.

장르를 정하지 않고 개발을 시작하면 기능 선택이 흔들립니다. 간단한 슈팅 게임을 만들다가 성장 시스템을 넣고, 다시 퀘스트를 넣고, 상점과 장비까지 추가하다 보면 프로젝트의 정체성이 흐려집니다. 물론 장르를 섞을 수는 있지만, 처음부터 모든 장르의 기능을 가져오면 개발 범위가 빠르게 커집니다. 그래서 먼저 중심 장르를 정해야 합니다.

프로그래머 관점에서 장르는 아키텍처의 방향을 잡는 기준이 됩니다. 액션 게임은 상태 머신과 입력 처리 구조가 중요하고, 카드 게임은 카드 데이터와 효과 처리 구조가 중요하며, 방치형 게임은 자동 진행, 보상 계산, 성장 수치 관리가 중요합니다. 장르를 정하면 어떤 클래스를 먼저 설계해야 하는지, 어떤 데이터를 테이블로 분리해야 하는지도 더 쉽게 판단할 수 있습니다.

AI에게 구현을 요청할 때도 장르는 중요한 정보입니다. “전투 시스템을 만들어줘”보다 “탐다운 액션 RPG의 근접 전투 시스템을 만들어줘”라고 요청하는 편이 훨씬 정확합니다. 장르는 AI에게 게임의 기본 문법을 알려주는 역할을 합니다. 따라서 상세 기획서를 작성할 때는 먼저 장르를 정하고, 그 장르에 필요한 핵심 시스템을 목록으로 정리해야 합니다.

Chapter 13. 참고 게임을 분석하는 프로그래머식 방법

게임을 기획할 때 참고 게임을 분석하는 일은 매우 중요합니다. 하지만 프로그래머가 참고 게임을 볼 때는 단순히 “재미있다”, “그래픽이 좋다”, “타격감이 좋다” 정도로 끝내면 부족합니다. 프로그래머식 분석은 게임을 기능, 규칙, 데이터, 흐름으로 나누어 보는 방식입니다. 즉, 플레이 경험을 구현 가능한 요소로 분해해야 합니다.

예를 들어 참고 게임의 전투가 재미있다면 왜 재미있는지 구체적으로 살펴봐야 합니다. 공격 속도가 빠르는지, 피격 반응이 강한지, 적이 공격 전에 예고 동작을 하는지, 회피 타이밍이 중요한지, 카메라 흔들림과 이펙트가 타격감을 만드는지 확인해야 합니다. 이런 요소를 나누어야 “타격감 좋은 전투”라는 감상이 “공격 판정, 피격 경직, 이펙트 타이밍, 사운드 재생 조건” 같은 구현 항목으로 바뀝니다.

UI도 마찬가지입니다. 참고 게임의 인벤토리가 편리하다면 화면 구성이 어떻게 되어 있는지, 아이템 정렬은 가능한지, 장비와 소비 아이템은 어떻게 구분되는지, 아이템을 선택했을 때 어떤 정보가 표시되는지 분석해야 합니다. 프로그래머는 화면을 보면서 필요한 데이터 구조와 이벤트 흐름을 함께 떠올릴 수 있어야 합니다.

참고 게임을 분석할 때는 그대로 베끼는 것이 목적이 아닙니다. 핵심은 구조를 이해하는 것입니다. 어떤 기능이 왜 필요한지, 어떤 규칙이 플레이 경험을 만드는지, 어떤 조건이 게임의 흐름을 안정적으로 유지하는지 살펴보는 것입니다. 그런 다음 내 게임에 필요한 요소만 골라 작게 적용해야 합니다.

AI에게도 참고 게임 분석 결과는 좋은 입력 자료가 됩니다. “뱀서라이크 게임처럼 만들어줘”라고만 요청하면 결과가 모호합니다. 대신 “자동 공격, 몰려오는 적, 경험치 보석, 레벨업 선택지, 제한 시간 생존 구조를 가진 게임”이라고 설명하면 AI가 훨씬 정확하게 이해할 수 있습니다. 참고 게임을 프로그래머식으로 분석한다는 것은 유명 게임의 이름을 빌리는 것이 아니라, 그 게임을 구성하는 시스템을 내 언어로 다시 정리하는 일입니다.

Chapter 14. 핵심 재미를 기능 단위로 분해하기

게임의 핵심 재미는 처음에는 감각적인 문장으로 표현되는 경우가 많습니다. “타격감이 좋은 게임”, “계속 성장하는 재미가 있는 게임”, “전략적으로 고민하는 게임”, “짧고 중독성 있는 게임” 같은 표현이 그렇습니다. 이런 문장은 게임의 방향을 설명하는 데는 도움이 되지만, 그대로는 구현하기 어렵습니다. 프로그래머는 핵심 재미를 기능 단위로 분해해야 합니다.

예를 들어 “타격감이 좋은 전투”를 만든다고 해보겠습니다. 이 표현을 코드로 옮기려면 여러 기능으로 나누어야 합니다. 공격 입력, 공격 판정, 피격 애니메이션, 넉백, 카메라 흔들림, 히트 이펙트, 사운드, 일시 정지 효과, 데미지 숫자 표시처럼 구체적인 요소가 필요합니다. 결국 타격감은 하나의 기능이 아니라 여러 작은 기능이 같은 타이밍에 맞물려 만들어지는 경험입니다.

“성장하는 재미”도 마찬가지입니다. 단순히 레벨업 기능만 있다고 성장의 재미가 생기지는 않습니다. 경험치 획득, 레벨업 조건, 능력치 증가, 스킬 해금, 장비 교체, 보상 선택, 난이도 상승이 함께 설계되어야 합니다. 프로그래머 관점에서는 이 요소들을 각각 데이터, 계산식, UI, 저장 구조, 밸런싱 수치로 나누어 정리해야 합니다.

핵심 재미를 기능 단위로 분해하면 개발 우선순위도 분명해집니다. 모든 기능을 한 번에 만들 필요는 없습니다. 먼저 핵심 재미를 확인할 수 있는 최소 기능부터 구현하면 됩니다. 타격감이 핵심이라면 인벤토리보다 공격 판정과 피격 피드백을 먼저 만들어야 합니다. 성장의 재미가 핵심이라면 화려한 연출보다 경험치와 보상 루프를 먼저 검증해야 합니다.

AI에게 구현을 요청할 때도 핵심 재미를 기능 단위로 나누어 전달해야 합니다. “재미있는 전투를 만들어줘”보다 “근접 공격 판정, 적 피격 처리, 넉백, 히트 이펙트, 카메라 흔들림을 포함한 전투 피드백 시스템을 만들어줘”가 훨씬 좋은 요청입니다. 핵심 재미를 분해하는 능력은 아이디어를 실제 개발 항목으로 바꾸는 첫 번째 기술입니다.

Chapter 15. 만들고 싶은 것과 만들 수 있는 것을 구분하기

게임을 기획하다 보면 만들고 싶은 기능이 계속 늘어납니다. 멋진 전투, 다양한 캐릭터, 성장 시스템, 장비 시스템, 퀘스트, 상점, 멀티플레이, 랭킹, 업적까지 넣고 싶어집니다. 아이디어가 많다는 것은 좋은 일입니다. 하지만 모든 아이디어를 한 프로젝트에 넣으려고 하면 개발은 금방 무거워집니다. 기획에서 중요한 것은 만들고 싶은 것을 모두 적는 것이 아니라, 지금 만들 수 있는 것을 구분하는 일입니다.

프로그래머는 기능의 매력뿐 아니라 구현 비용도 함께 생각해야 합니다. 예를 들어 “보스 몬스터가 다양한 패턴을 사용한다”는 아이디어는 좋아 보이지만, 실제로는 패턴 상태 관리, 공격 판정, 애니메이션, 이펙트, 난이도 조정, 테스트가 필요합니다. “아이템 조합 시스템”도 데이터 구조, UI, 저장, 예외 처리까지 고려해야 합니다. 기능 하나는 단순한 문장처럼 보이지만, 구현 단계에서는 여러 작업으로 나뉩니다.

만들 수 있는 것을 판단하려면 시간, 실력, 리소스, 플랫폼, 테스트 범위를 함께 봐야 합니다. 혼자 개발하는 프로젝트라면 대규모 RPG보다 작은 액션 게임이나 퍼즐 게임이 더 현실적일 수 있습니다. 그래픽 리소스가 부족하다면 화려한 연출보다 규칙이 명확한 게임이 적합할 수 있습니다. 모바일 출시를 목표로 한다면 조작과 UI도 그에 맞게 줄여야 합니다.

상세 기획서는 이런 판단을 돕는 도구입니다. 기능을 문서로 정리하면 각 기능이 얼마나 많은 하위 작업을 요구하는지 보입니다. 단순히 “상점 추가”라고 적는 대신 구매 조건, 재화 확인, 인벤토리 추가, UI 갱신, 저장 처리까지 적어보면 실제 작업량을 더 정확히 알 수 있습니다. 그 과정에서 지금 필요한 기능과 나중에 추가해도 되는 기능을 나눌 수 있습니다.

AI를 활용하면 개발 속도는 빨라질 수 있지만, 프로젝트 범위가 자동으로 줄어들지는 않습니다. 오히려 AI가 빠르게 코드를 만들어주기 때문에 더 많은 기능을 넣고 싶어질 수 있습니다. 그래서 프로그래머는 “만들고 싶은 것”과 “지금 완성할 수 있는 것”을 구분해야 합니다. 완성 가능한 범위를 정하는 능력은 좋은 기획의 핵심이며, 작은 게임을 끝까지 완성하게 만드는 현실적인 기준입니다.

Chapter 16. MVP 기준으로 게임 범위 줄이기

게임을 끝까지 완성하려면 처음부터 큰 게임을 만들려고 하면 안 됩니다. 특히 개인 개발이나 소규모 프로젝트에서는 범위를 줄이는 능력이 중요합니다. 이때 사용할 수 있는 기준이 MVP입니다. MVP는 Minimum Viable Product의 약자로, 게임의 핵심 재미를 확인할 수 있는 최소한의 완성 버전을 의미합니다.

MVP는 기능이 적은 게임이 아니라, 핵심을 검증할 수 있는 게임입니다. 예를 들어 탑다운 슈팅 게임을 만든다면 처음부터 상점, 성장, 업적, 다양한 무기, 보스전까지 만들 필요는 없습니다. 먼저 플레이어 이동, 조준, 발사, 적 생성, 피격, 점수, 게임 오버만 있어도 기본 재미를 확인할 수 있습니다. 이 최소 구조가 재미없다면 기능을 더 추가해도 문제가 해결되지 않을 수 있습니다.

프로그래머 관점에서 MVP는 개발 우선순위를 정하는 기준입니다. “이 기능이 없으면 게임의 핵심 루프를 테스트할 수 없는가?”라고 질문해볼 수 있습니다. 없다면 아직 필수 기능이고, 없어도 핵심 플레이가 가능하다면 후순위 기능입니다. 예를 들어 전투 게임에서 공격 판정은 필수지만, 스킨 시스템은 나중에 추가해도 됩니다. 퍼즐 게임에서 클리어 조건은 필수지만, 꾸미기 요소는 MVP 이후로 미룰 수 있습니다.

상세 기획서를 작성할 때도 모든 기능을 같은 비중으로 적으면 안 됩니다. 기능을 필수, 중요, 선택으로 나누어야 합니다. 필수 기능은 MVP에 반드시 포함되는 기능이고, 중요 기능은 재미를 강화하지만 없어도 테스트 가능한 기능입니다. 선택 기능은 완성 이후 확장할 수 있는 기능입니다. 이렇게 구분하면 개발 중에 범위가 계속 커지는 것을 막을 수 있습니다.

AI를 활용할 때도 MVP 기준은 필요합니다. AI가 빠르게 코드를 만들어준다고 해서 모든 기능을 한 번에 요청하면 구조가 복잡해지고 검증하기 어려워집니다. 먼저 MVP 범위의 기능만 요청하고, 실행 가능한 작은 버전을 만든 뒤 개선하는 방식이 좋습니다. 좋은 상세 기획서는 큰 꿈을 작게 포기하는 문서가 아니라, 완성 가능한 첫 번째 버전을 정의하는 문서입니다.

Part 03. 게임 상세 기획서의 기본 구조

Part 3에서는 게임 상세 기획서에 어떤 항목이 들어가야 하는지 전체 구조를 살펴봅니다.

게임 개요, 핵심 플레이 루프, 주요 시스템 목록, 화면 목록, 데이터 목록, 기능 우선순위, 예외 조건은 프로그래머가 구현을 시작하기 전에 반드시 정리해야 할 기본 요소입니다.

이 파트는 상세 기획서를 처음 작성하는 독자가 문서의 뼈대를 잡을 수 있도록 돕습니다.

단순한 아이디어 메모가 아니라, 실제 개발과 AI 요청에 사용할 수 있는 기준 문서로 기획서를 구성하는 방법을 설명합니다.

Chapter 17. 상세 기획서에는 무엇이 들어가야 하는가

상세 기획서는 게임을 만들기 전에 모든 아이디어를 길게 적어두는 문서가 아닙니다. 프로그래머가 실제 구현에 참고할 수 있도록 게임의 목적, 규칙, 흐름, 데이터, 예외 조건을 정리한 문서입니다. 좋은 상세 기획서는 읽는 사람이 “무엇을 만들어야 하는지”와 “어떤 조건에서 어떻게 동작해야 하는지”를 이해할 수 있어야 합니다.

가장 먼저 들어가야 할 것은 게임 개요입니다. 어떤 장르의 게임인지, 플레이어가 무엇을 하는지, 핵심 재미가 무엇인지 한눈에 파악할 수 있어야 합니다. 이 부분이 명확해야 이후 시스템 기획도 흔들리지 않습니다. 게임 개요가 모호하면 전투, 성장, UI, 레벨 설계가 각각 다른 방향으로 흘러갈 수 있습니다.

두 번째로 필요한 것은 핵심 플레이 루프입니다. 플레이어가 반복해서 수행하는 행동 흐름을 정리해야 합니다. 예를 들어 “전투한다 → 보상을 얻는다 → 성장한다 → 더 어려운 스테이지에 도전한다”처럼 게임의 반복 구조를 작성합니다. 핵심 플레이 루프는 어떤 기능이 필수인지 판단하는 기준이 됩니다.

세 번째는 시스템별 상세 규칙입니다. 조작, 전투, 아이템, 성장, 상점, 퀘스트, UI 같은 주요 시스템을 나누고, 각 시스템의 입력, 처리, 결과, 예외 조건을 작성해야 합니다. “아이템을 구매한다”라고만 쓰는 것이 아니라, 재화가 충분할 때와 부족할 때, 인벤토리가 가득 찼을 때, 이미 보유한 아이템일 때의 처리를 함께 정리해야 합니다.

네 번째는 데이터와 수치입니다. 프로그래머에게 데이터는 구현의 기준이 됩니다. 캐릭터 능력치, 아이템 속성, 스킬 쿨타임, 보상량, 스테이지 난이도 같은 값은 표로 정리하는 것이 좋습니다. 이렇게 정리하면 하드코딩할 값과 데이터 테이블로 분리할 값을 판단하기 쉬워집니다.

마지막으로 AI 개발 요청에 사용할 프롬프트 구조도 포함하면 좋습니다. 상세 기획서가 잘 정리되어 있으면 AI에게 기능 구현, 클래스 설계, UI 구현, 테스트 코드 작성을 요청하기 쉬워집니다. 결국 상세 기획서에는 게임을 설명하는 문장뿐 아니라, 실제 개발로 이어질 수 있는 규칙과 기준이 들어가야 합니다.

Chapter 18. 개요 문서와 상세 기획서의 차이

게임을 문서로 정리할 때 가장 먼저 구분해야 할 것은 개요 문서와 상세 기획서입니다. 두 문서는 모두 필요하지만 목적이 다릅니다. 개요 문서는 게임의 큰 방향을 설명하는 문서이고, 상세 기획서는 실제 구현에 필요한 규칙과 조건을 정리하는 문서입니다. 이 차이를 구분하지 못하면 문서는 길어지지만 개발에는 도움이 되지 않을 수 있습니다.

개요 문서는 게임을 처음 이해하기 위한 문서입니다. 어떤 장르인지, 플레이어가 무엇을 하는지, 핵심 재미가 무엇인지, 어떤 분위기와 목표를 가진 게임인지 설명합니다. 예를 들어 “플레이어가 몰려오는 적을 피하고 자동 공격으로 생존하는 탐다운 액션 게임”이라는 문장은 개요 문서에 어울립니다. 이 문장은 게임의 방향을 잡아주지만, 바로 코드로 옮기기에는 아직 부족합니다.

상세 기획서는 개요 문서를 구현 가능한 수준으로 풀어낸 문서입니다. 자동 공격은 몇 초마다 실행되는지, 적은 어디서 생성되는지, 플레이어가 피격되면 체력이 얼마나 감소하는지, 경험치는 언제 지급되는지, 게임 오버 조건은 무엇인지 작성해야 합니다. 상세 기획서는 추상적인 설명을 입력, 처리, 결과, 예외 조건으로 나누는 작업에 가깝습니다.

프로그래머 관점에서 개요 문서는 방향을 확인하는 지도이고, 상세 기획서는 구현을 진행하기 위한 작업 명세서입니다. 개요 문서만 있으면 “무슨 게임인지”는 알 수 있지만 “어떻게 만들어야 하는지”는 알기 어렵습니다. 반대로 상세 기획서만 있고 개요가 없으면 기능은 많지만 게임의 목적과 핵심 재미가 흐려질 수 있습니다.

AI에게 요청할 때도 두 문서의 역할은 다릅니다. 개요 문서는 AI에게 게임의 전체 맥락을 설명하는 데 유용하고, 상세 기획서는 실제 코드 생성이나 시스템 설계 요청에 유용합니다. 좋은 개발 흐름은 먼저 개요 문서로 방향을 잡고, 그다음 상세 기획서로 기능을 나누는 것입니다. 이 두 문서를 구분할 수 있어야 게임 아이디어가 안정적인 구현 계획으로 이어집니다.

Chapter 19. 게임 개요 작성하기

게임 개요는 상세 기획서의 첫 페이지에 해당합니다. 이 문서는 게임을 처음 읽는 사람이 “무슨 게임을 만들려는가”를 빠르게 이해하도록 돕습니다. 프로그래머에게 게임 개요는 단순한 소개문이 아니라 이후 시스템, UI, 데이터, 레벨 기획의 기준점이 됩니다. 개요가 명확해야 상세 기획도 같은 방향으로 정리됩니다.

게임 개요에는 먼저 장르와 플랫폼을 적어야 합니다. 예를 들어 “PC 기반 탐다운 액션 슈팅 게임”처럼 작성하면 게임의 기본 형태가 분명해집니다. 플랫폼이 PC인지 모바일인지에 따라 입력 방식, 화면 구성, 조작 복잡도가 달라지기 때문에 처음부터 함께 정리하는 것이 좋습니다.

다음으로 플레이어의 목표와 핵심 행동을 작성합니다. 플레이어가 적을 처치하는 게임인지, 퍼즐을 해결하는 게임인지, 자원을 모아 성장하는 게임인지 분명해야 합니다. “플레이어는 제한 시간 동안 몰려오는 적을 피하고 처치하며 최대한 오래 생존한다”처럼 쓰면 게임의 목표와 반복 행동이 한눈에 보입니다.

핵심 재미도 짧게 정리해야 합니다. 이 게임이 조작의 재미를 주는지, 성장의 재미를 주는지, 전략적 선택의 재미를 주는지, 반복 도전의 재미를 주는지 적어야 합니다. 핵심 재미가 정리되어 있으면 기능 우선순위를 정할 때 도움이 됩니다. 핵심 재미와 관련 없는 기능은 뒤로 미룰 수 있기 때문입니다.

마지막으로 게임의 범위를 간단히 제한해야 합니다. 처음부터 모든 시스템을 넣겠다고 쓰는 대신, 이번 버전에서 만들 기능과 제외할 기능을 구분하는 것이 좋습니다. 예를 들어 “MVP 버전에서는 플레이어 이동, 적 생성, 자동 공격, 경험치 획득, 레벨업 선택지만 구현한다”라고 적으면 개발 범위가 명확해집니다. 좋은 게임 개요는 멋진 소개문이 아니라, 개발 방향을 흔들리지 않게 잡아주는 짧은 기준 문서입니다.

Chapter 20. 핵심 플레이 루프 작성하기

핵심 플레이 루프는 플레이어가 게임 안에서 반복해서 경험하는 행동의 흐름입니다. 게임은 한 번의 행동으로 끝나지 않습니다. 플레이어는 이동하고, 선택하고, 전투하고, 보상을 얻고, 다시 도전합니다. 이 반복 구조가 명확해야 어떤 기능이 필요한지, 어떤 시스템을 먼저 구현해야 하는지 판단할 수 있습니다.

예를 들어 액션 RPG라면 핵심 플레이 루프는 “스테이지 입장 → 적과 전투 → 보상 획득 → 캐릭터 성장 → 더 어려운 스테이지 도전”처럼 정리할 수 있습니다. 퍼즐 게임이라면 “문제 확인 → 조작 시도 → 결과 확인 → 클리어 또는 재도전 → 다음 스테이지 이동”이 될 수 있습니다. 장르마다 루프는 다르지만, 중요한 것은 플레이어가 반복하는 핵심 행동을 짧고 분명하게 쓰는 것입니다.

프로그래머 관점에서 핵심 플레이 루프는 시스템 목록을 뽑아내는 기준이 됩니다. “전투한다”가 루프에 포함되어 있다면 입력, 공격 판정, 적 AI, 체력, 데미지, 피격 처리 시스템이 필요합니다. “보상을 얻는다”가 포함되어 있다면 아이템, 재화, 경험치, 드랍 테이블, 보상 UI가 필요할 수 있습니다. 루프를 작성하면 필요한 기능과 불필요한 기능이 자연스럽게 구분됩니다.

핵심 플레이 루프를 작성할 때는 너무 많은 단계를 넣지 않는 것이 좋습니다. 처음에는 4~6단계 정도로 단순하게 정리합니다. 루프가 길고 복잡하면 게임의 핵심이 흐려질 수 있습니다. 특히 MVP 단계에서는 “이 루프만 구현해도 게임의 기본 재미를 확인할 수 있는가?”를 기준으로 삼아야 합니다.

AI에게 구현을 요청할 때도 핵심 플레이 루프는 중요한 입력 자료가 됩니다. “이 게임은 적을 처치하고 경험치를 얻어 레벨업한 뒤 다시 전투하는 루프를 가진다”라고 설명하면 AI는 어떤 시스템이 필요한지 더 정확하게 추론할 수 있습니다. 좋은 상세 기획서는 개별 기능을 나열하는 데서 끝나지 않습니다. 그 기능들이 어떤 반복 경험을 만들기 위해 연결되는지 보여주어야 합니다.

Chapter 21. 주요 시스템 목록 작성하기

게임 개요와 핵심 플레이 루프를 작성했다면 다음 단계는 주요 시스템 목록을 정리하는 일입니다. 시스템 목록은 이 게임을 만들기 위해 어떤 기능 묶음이 필요한지 보여주는 개발 지도와 같습니다. 프로그래머는 이 목록을 기준으로 클래스, 데이터, UI, 작업 일정을 나눌 수 있습니다.

시스템은 단순한 기능 하나가 아니라 서로 관련된 기능들의 묶음입니다. 예를 들어 전투 시스템에는 공격 입력, 공격 판정, 데미지 계산, 피격 처리, 사망 처리, 전투 피드백이 포함될 수 있습니다. 성장 시스템에는 경험치, 레벨업, 능력치 증가, 스킬 해금이 포함될 수 있습니다. 이렇게 묶어야 기능 간 관계를 이해하기 쉽습니다.

주요 시스템 목록을 작성할 때는 핵심 플레이 루프에서 출발하는 것이 좋습니다. 루프에 “적과 전투한다”가 있다면 전투 시스템과 적 AI 시스템이 필요합니다. “보상을 얻는다”가 있다면 아이템, 재화, 보상 시스템이 필요합니다. “성장한다”가 있다면 경험치, 레벨, 능력치 시스템이 필요합니다. 루프의 각 단계를 시스템으로 바꾸면 필요한 기능을 빠뜨릴 가능성이 줄어듭니다.

처음부터 너무 많은 시스템을 넣지 않는 것도 중요합니다. MVP 단계에서는 핵심 재미를 확인하는 데 필요한 시스템만 우선 작성합니다. 예를 들어 생존 슈팅 게임이라면 플레이어 조작, 적 생성, 공격, 체력, 점수, 게임 오버 정도가 먼저 필요할 수 있습니다. 상점, 업적, 스킨, 랭킹은 이후 확장 시스템으로 분리할 수 있습니다.

AI에게 구현을 요청할 때도 시스템 목록은 유용합니다. “게임 전체를 만들어줘”라고 요청하기보다 “플레이어 조작 시스템”, “적 생성 시스템”, “보상 시스템”처럼 나누어 요청하면 결과를 검토하기 쉽습니다. 좋은 상세 기획서는 아이디어를 하나의 큰 덩어리로 두지 않고, 구현 가능한 시스템 단위로 나누어 개발 순서를 정리합니다.

Chapter 22. 화면 목록 작성하기

게임 상세 기획서에는 어떤 화면이 필요한지도 정리해야 합니다. 화면 목록은 UI 개발의 출발점입니다. 게임에 필요한 화면을 미리 나열해두면 버튼, 팝업, 데이터 표시, 화면 전환 흐름을 더 쉽게 설계할 수 있습니다. 반대로 화면 목록 없이 개발을 시작하면 기능이 추가될 때마다 UI가 즉흥적으로 늘어나고 구조가 복잡해질 수 있습니다.

화면 목록을 작성할 때는 플레이어가 게임을 시작해서 종료하기까지 어떤 화면을 거치는지 생각하면 됩니다. 일반적으로 타이틀 화면, 메인 메뉴, 로비, 스테이지 선택, 인게임 HUD, 결과 화면, 설정 화면 등이 필요할 수 있습니다. RPG나 성장형 게임이라면 캐릭터 정보, 인벤토리, 장비, 스킬, 상점, 퀘스트 화면도 추가될 수 있습니다.

프로그래머 관점에서 화면 목록은 단순한 UI 이름 목록이 아닙니다. 각 화면이 어떤 데이터를 보여주는지, 어떤 버튼을 가지는지, 어떤 시스템과 연결되는지 함께 정리해야 합니다. 예를 들어 인벤토리 화면은 아이템 목록 데이터를 사용하고, 아이템 선택 버튼, 사용 버튼, 버리기 버튼, 상세 정보 패널을 가질 수 있습니다. 이 정보를 적어두면 UI 구현 범위가 명확해집니다.

화면 목록에는 화면 간 이동 흐름도 함께 포함하는 것이 좋습니다. 타이틀 화면에서 게임 시작 버튼을 누르면 어디로 이동하는지, 인게임 중 일시정지 버튼을 누르면 어떤 팝업이 뜨는지, 결과 화면에서 다시하기 버튼을 누르면 어떤 상태로 돌아가는지 정리해야 합니다. 화면 전환 조건이 정리되어 있으면 게임 상태 관리도 더 안정적으로 설계할 수 있습니다.

AI에게 UI 구현을 요청할 때도 화면 목록은 중요한 입력 자료가 됩니다. “게임 UI를 만들어줘”라고 요청하기보다 “타이틀, 인게임 HUD, 결과 화면을 만들고, 각 화면의 버튼과 전환 흐름은 다음과 같다”라고 설명해야 결과가 좋아집니다. 좋은 화면 목록은 UI를 꾸미기 위한 자료가 아니라, 플레이어가 게임을 어떻게 경험하는지 정리하는 개발 문서입니다.

Chapter 23. 데이터 목록 작성하기

게임 상세 기획서에서 데이터 목록은 매우 중요합니다. 프로그래머에게 데이터는 시스템을 움직이는 기준값입니다. 캐릭터의 체력, 공격력, 이동 속도, 아이템 가격, 스킬 쿨타임, 스테이지 보상처럼 게임 안에서 변하거나 조정될 수 있는 값은 미리 목록으로 정리해두는 것이 좋습니다.

데이터 목록을 작성하면 어떤 값을 코드에 직접 넣어도 되는지, 어떤 값을 외부 데이터로 분리해야 하는지 판단하기 쉬워집니다. 예를 들어 테스트용 임시 값은 코드에 넣을 수 있지만, 밸런싱이 자주 바뀌는 값은 테이블이나 ScriptableObject로 분리하는 편이 좋습니다. 특히 아이템, 스킬, 적, 스테이지처럼 종류가 늘어나는 요소는 처음부터 데이터 구조를 생각해야 합니다.

데이터 목록은 시스템별로 나누어 작성하면 좋습니다. 캐릭터 데이터에는 ID, 이름, 체력, 공격력, 이동 속도, 방어력 같은 항목이 들어갈 수 있습니다. 아이템 데이터에는 ID, 이름, 종류, 가격, 설명, 효과 값이 필요할 수 있습니다. 스킬 데이터에는 쿨타임, 데미지, 범위, 지속 시간, 필요 레벨 같은 항목을 정리할 수 있습니다.

프로그래머 관점에서 데이터 목록은 클래스와 테이블 설계의 출발점이 됩니다. 어떤 필드가 필요한지 알 수 있고, 데이터 간 관계도 파악할 수 있습니다. 예를 들어 아이템이 스킬 효과를 부여한다면 아이템 데이터와 스킬 데이터가 연결되어야 합니다. 스테이지마다 등장하는 적이 다르다면 스테이지 데이터와 적 데이터도 연결되어야 합니다.

AI에게 구현을 요청할 때도 데이터 목록은 좋은 기준이 됩니다. “아이템 시스템을 만들어줘”라고 요청하는 것보다 “아이템 데이터는 ID, 이름, 타입, 가격, 효과 값을 가진다”라고 설명하는 편이 훨씬 정확합니다. 좋은 상세 기획서는 기능만 적는 것이 아니라, 그 기능이 어떤 데이터로 동작하는지까지 함께 정리해야 합니다.

Chapter 24. 기능 우선순위 작성하기

게임 상세 기획서를 작성할 때 모든 기능을 같은 중요도로 다루면 안 됩니다. 기능에는 반드시 먼저 만들어야 하는 기능이 있고, 나중에 추가해도 되는 기능이 있습니다. 우선순위를 정하지 않으면 개발 중에 눈에 띄는 기능부터 만들게 되고, 정작 게임의 핵심 재미를 확인하는 데 필요한 기능은 뒤로 밀릴 수 있습니다.

기능 우선순위는 핵심 플레이 루프를 기준으로 정하는 것이 좋습니다. 플레이어가 이동하고, 적을 공격하고, 보상을 얻고, 다시 도전하는 구조라면 이동, 공격, 적, 피격, 보상, 게임 오버는 우선순위가 높습니다. 반면 스킨, 업적, 랭킹, 옵션 세부 설정 같은 기능은 핵심 루프가 동작한 뒤에 추가해도 됩니다.

우선순위를 작성할 때는 기능을 필수, 중요, 선택으로 나누면 편합니다. 필수 기능은 없으면 게임이 성립하지 않는 기능입니다. 중요 기능은 핵심 재미를 강화하지만 MVP 단계에서는 단순화할 수 있는 기능입니다. 선택 기능은 완성도를 높이는 확장 요소입니다. 이렇게 나누면 개발 범위가 커지는 것을 막고, 일정이 부족할 때 무엇을 줄여야 하는지도 판단할 수 있습니다.

프로그래머 관점에서는 구현 난이도도 함께 고려해야 합니다. 재미에 꼭 필요한 기능이라도 구현 비용이 너무 크다면 먼저 단순한 버전으로 만들 수 있습니다. 예를 들어 복잡한 적 AI 대신 플레이어를 향해 이동하는 기본 AI부터 만들고, 다양한 패턴은 이후 단계로 미룰 수 있습니다. 중요한 것은 처음부터 완벽한 기능을 만드는 것이 아니라, 검증 가능한 형태로 구현하는 것입니다.

AI에게 작업을 요청할 때도 우선순위는 필요합니다. 모든 기능을 한 번에 요청하면 결과를 검토하기 어렵고 코드 구조도 복잡해질 수 있습니다. 먼저 필수 기능부터 요청하고, 실행 가능한 버전을 만든 뒤 중요 기능과 선택 기능을 단계적으로 추가하는 편이 좋습니다. 좋은 상세 기획서는 기능 목록만 적는 문서가 아니라, 어떤 순서로 만들어야 하는지 알려주는 개발 계획서입니다.

Chapter 25. 예외 조건과 제한 사항 작성하기

상세 기획서에서 자주 빠지는 부분이 예외 조건과 제한 사항입니다. 많은 기획서는 정상적으로 동작하는 상황만 설명합니다. “아이템을 구매한다”, “스킬을 사용한다”, “보상을 획득한다”처럼 기본 흐름만 적는 것입니다. 하지만 실제 게임 개발에서는 정상 흐름보다 예외 상황에서 더 많은 문제가 발생합니다.

예외 조건은 “원래 흐름대로 진행되지 않을 때 어떻게 처리할 것인가”를 정리하는 항목입니다. 예를 들어 상점에서 아이템을 구매할 때 골드가 부족하면 어떻게 할지, 인벤토리가 가득 차 있으면 구매를 막을지, 이미 보유한 아이템을 다시 구매할 수 있는지 정해야 합니다. 이런 조건이 없으면 구현 중에 프로그래머가 즉흥적으로 판단하게 됩니다.

제한 사항은 기능이 동작할 수 있는 범위를 정하는 기준입니다. 스킬은 쿨타임 중에는 사용할 수 없는지, 사망 상태에서는 입력을 받을 수 없는지, 전투 중에는 장비를 변경할 수 없는지 같은 규칙이 여기에 해당합니다. 제한 사항이 명확해야 시스템 간 충돌을 줄일 수 있습니다. 특히 게임은 여러 기능이 동시에 작동하기 때문에 제한 조건을 미리 정리하는 것이 중요합니다.

프로그래머 관점에서 예외 조건은 버그를 줄이는 설계 자료입니다. 보스가 사망하는 순간 플레이어도 죽으면 승리인지 패배인지, 네트워크가 끊긴 상태에서 보상을 받았다면 저장할 것인지, 일시정지 중에도 시간이 흘러가는지 같은 문제는 나중에 발견되면 수정 비용이 커집니다. 상세 기획서에 이런 조건을 적어두면 구현 기준이 분명해집니다.

AI에게 구현을 요청할 때도 예외 조건은 반드시 포함해야 합니다. “아이템 구매 기능을 만들어줘”보다 “골드가 부족하면 구매하지 않고 경고 메시지를 출력하며, 인벤토리가 가득 찬 경우에도 구매를 막는다”라고 요청해야 더 안정적인 코드를 얻을 수 있습니다. 좋은 상세 기획서는 정상 동작만 설명하지 않습니다. 잘못된 입력, 부족한 자원, 중복 처리, 실패 상황까지 함께 정의하는 문서입니다.

Part 04. 플레이어 조작과 캐릭터 기획하기

Part 4에서는 플레이어 조작과 캐릭터 동작을 상세 기획서로 정리하는 방법을 다룹니다.

게임에서 플레이어가 어떤 입력을 하고, 캐릭터가 어떤 상태로 전환되며, 어떤 조건에서 행동이 가능한지 명확히 정리해야 안정적인 구현이 가능합니다.

이 파트에서는 이동, 점프, 대시 같은 기본 조작부터 캐릭터 상태 머신, 애니메이션 전환 조건, 능력치 정의까지 프로그래머가 구현에 참고할 수 있는 형태로 작성하는 방법을 설명합니다.

Chapter 26. 조작 기획은 입력 명세에서 시작한다

게임에서 조작은 플레이어가 시스템과 처음 만나는 지점입니다. 아무리 전투 시스템이나 성장 시스템이 잘 만들어져 있어도, 플레이어가 원하는 대로 캐릭터를 움직일 수 없다면 게임의 첫인상은 나빠집니다. 그래서 조작 기획은 “캐릭터가 움직인다”는 설명에서 끝나면 안 됩니다. 어떤 입력을 받았을 때 어떤 행동이 실행되는지 명확히 정리해야 합니다.

입력 명세는 조작 기획의 출발점입니다. 예를 들어 PC 게임이라면 이동은 WASD, 점프는 Space, 공격은 마우스 왼쪽 버튼, 회피는 Shift처럼 작성할 수 있습니다. 모바일 게임이라면 가상 조이스틱, 터치 버튼, 드래그, 스와이프 같은 입력 방식을 정리해야 합니다. 플랫폼에 따라 입력 방식이 달라지므로, 조작 기획은 반드시 대상 플랫폼과 함께 작성해야 합니다.

프로그래머 관점에서 입력 명세에는 버튼 이름만 적는 것으로 부족합니다. 입력 시점도 함께 정의해야 합니다. 버튼을 누르는 순간 실행되는지, 누르고 있는 동안 유지되는지, 버튼을 뗄 때 실행되는지 구분해야 합니다. 예를 들어 점프는 버튼을 누르는 순간 실행될 수 있고, 차지 공격은 버튼을 누르고 있는 시간에 따라 위력이 달라질 수 있습니다. 이런 차이가 코드 구조를 결정합니다.

또한 입력이 항상 허용되는 것은 아닙니다. 캐릭터가 사망한 상태에서는 이동할 수 없고, 피격 중에는 공격할 수 없으며, 대시 중에는 다시 대시할 수 없을 수 있습니다. 따라서 입력 명세에는 가능한 조건과 불가능한 조건을 함께 적어야 합니다. “공격 버튼을 누르면 공격한다”가 아니라 “지상 또는 공중에서 공격 가능하지만, 피격 상태와 사망 상태에서는 공격할 수 없다”처럼 작성해야 구현 기준이 분명해집니다.

AI에게 조작 기능을 요청할 때도 입력 명세는 중요합니다. “플레이어 조작을 만들어줘”라고 요청하면 결과가 모호할 수 있습니다. 반면 “WASD로 이동하고, Space로 점프하며, 마우스 왼쪽 버튼으로 공격한다. 피격 상태와 사망 상태에서는 입력을 무시한다”라고 설명하면 훨씬 정확한 코드를 얻을 수 있습니다. 좋은 조작 기획은 입력, 행동, 조건, 제한 사항을 함께 정리하는 문서에서 시작됩니다.

Chapter 27. 이동, 점프, 대시 같은 기본 조작 정의하기

플레이어 조작을 기획할 때 가장 먼저 정리해야 할 것은 기본 행동입니다. 대부분의 게임에서 기본 행동은 이동, 점프, 대시, 공격, 상호작용처럼 플레이어가 반복해서 사용하는 조작입니다. 이 기본 조작이 명확하지 않으면 이후 전투, 레벨, UI, 카메라 기획도 흔들릴 수 있습니다. 플레이어가 어떤 방식으로 움직이고, 어떤 조건에서 행동할 수 있는지를 먼저 정의해야 합니다.

이동 기획에서는 입력 방식과 이동 방향, 이동 속도를 정리해야 합니다. 예를 들어 2D 플랫폼머라면 좌우 이동이 중심이고, 탑다운 게임이라면 상하좌우 또는 8방향 이동이 필요할 수 있습니다. 3D 게임이라면 카메라 방향 기준으로 이동할지, 월드 좌표 기준으로 이동할지도 정해야 합니다. 단순히 “캐릭터가 이동한다”가 아니라 “WASD 입력을 받아 카메라 기준 방향으로 이동하며, 기본 이동 속도는 5로 설정한다”처럼 작성해야 구현하기 쉽습니다.

점프는 조건이 중요한 조작입니다. 지상에서만 가능한지, 공중 점프가 가능한지, 점프 높이는 고정인지, 버튼을 오래 누르면 더 높이 뛰는지 정해야 합니다. 또한 벽 점프, 낙하 속도, 착지 판정, 점프 중 이동 가능 여부도 게임 장르에 따라 달라질 수 있습니다. 점프 하나만 보더라도 입력, 물리 처리, 애니메이션, 충돌 판정이 함께 연결되므로 기획서에서 조건을 분명히 적어야 합니다.

대시는 이동보다 더 많은 제한 조건이 필요합니다. 대시 방향은 입력 방향을 따르는지, 바라보는 방향으로만 가능한지, 쿨타임이 있는지, 대시 중 무적인지, 대시 중 공격이 가능한지 정해야 합니다. 대시는 전투와 회피에 직접 영향을 주기 때문에 잘못 설계하면 난이도와 밸런스가 크게 흔들릴 수 있습니다. 따라서 지속 시간, 이동 거리, 재사용 대기 시간, 사용 불가 상태를 함께 작성하는 것이 좋습니다.

AI에게 기본 조작 구현을 요청할 때도 이런 정의가 필요합니다. “플레이어 이동과 점프, 대시를 만들어 줘”라고만 요청하면 원하는 결과와 다른 코드가 나올 수 있습니다. 대신 “WASD로 이동하고, Space로 지상 점프하며, Shift로 입력 방향 대시를 한다. 대시는 0.2초 동안 유지되고 1초의 쿨타임을 가진다”처럼 작성하면 구현 방향이 분명해집니다. 기본 조작 기획은 플레이 감각을 코드로 옮기기 위한 첫 번째 명세입니다.

Chapter 28. 입력 조건과 불가능 조건 작성하기

게임 조작 기획에서 중요한 것은 어떤 버튼을 누르면 어떤 행동이 실행되는지만 적는 것이 아닙니다. 그 행동이 언제 가능한지, 언제 불가능한지도 함께 작성해야 합니다. 플레이어가 같은 버튼을 눌러도 캐릭터의 상태, 위치, 자원, 쿨타임, 게임 진행 상황에 따라 결과가 달라질 수 있기 때문입니다. 입력 조건과 불가능 조건을 정리해야 조작 시스템이 안정적으로 동작합니다.

입력 조건은 행동이 실행되기 위해 필요한 조건입니다. 예를 들어 점프는 캐릭터가 지면에 닿아 있을 때만 가능할 수 있습니다. 대시는 대기 쿨타임이 끝났을 때만 가능하고, 스킬은 마나가 충분할 때만 사용할 수 있습니다. 공격은 무기를 장착한 상태에서만 가능할 수도 있습니다. 이런 조건을 작성하면 프로그래머는 입력을 받았을 때 바로 행동을 실행하는 것이 아니라, 먼저 조건을 검사하는 구조로 구현할 수 있습니다.

불가능 조건은 입력을 받아도 행동을 실행하지 않아야 하는 상황입니다. 캐릭터가 사망 상태라면 이동, 공격, 점프 입력을 모두 무시해야 합니다. 피격 상태에서는 공격할 수 없고, 일시정지 상태에서는 게임 플레이 입력을 막아야 합니다. UI 팝업이 열린 상태에서는 캐릭터 조작보다 UI 입력이 우선될 수 있습니다. 이런 제한을 명확히 하지 않으면 기능 간 충돌이 발생합니다.

입력 조건과 불가능 조건은 상태 시스템과 밀접하게 연결됩니다. Idle, Move, Jump, Attack, Hit, Dead 같은 상태가 있다면 각 상태에서 허용되는 입력을 표로 정리하는 것이 좋습니다. 예를 들어 Idle 상태에서는 이동, 점프, 공격이 가능하지만, Hit 상태에서는 이동과 공격을 막고, Dead 상태에서는 모든 플레이 입력을 막는 식입니다. 이렇게 작성하면 상태 전환과 입력 처리 기준이 분명해집니다.

AI에게 조작 코드를 요청할 때도 이 정보는 반드시 필요합니다. “점프와 대시를 만들어줘”라고 요청하기보다 “점프는 지상 상태에서만 가능하고, 대시는 쿨타임이 끝났을 때만 가능하며, 피격 상태와 사망 상태에서는 모든 입력을 무시한다”라고 설명해야 합니다. 좋은 조작 기획은 가능한 입력뿐 아니라 불가능한 입력까지 정의합니다. 게임의 조작감은 입력을 허용하는 순간뿐 아니라, 입력을 막아야 하는 순간을 정확히 처리할 때 완성됩니다.

Chapter 29. 캐릭터 상태를 상태 머신으로 정리하기

캐릭터 조작이 복잡해질수록 단순히 입력을 받는 방식만으로는 안정적인 구현이 어렵습니다. 이동 중 공격할 수 있는지, 점프 중 피격되면 어떻게 되는지, 대기 중에는 무적인지, 사망 후에도 입력을 받을 수 있는지 같은 조건이 계속 생기기 때문입니다. 이런 문제를 정리할 때 유용한 방식이 상태 머신입니다.

상태 머신은 캐릭터가 현재 어떤 상태에 있는지 정의하고, 조건에 따라 다른 상태로 전환되는 구조입니다. 예를 들어 캐릭터는 Idle, Move, Jump, Attack, Hit, Dead 같은 상태를 가질 수 있습니다. Idle 상태에서는 이동과 공격이 가능하고, Jump 상태에서는 공중 이동은 가능하지만 다시 점프할 수 없을 수 있습니다. Dead 상태에서는 모든 조작을 막아야 합니다.

프로그래머 관점에서 상태 머신은 입력 조건과 불가능 조건을 정리하는 기준이 됩니다. “공격 버튼을 누르면 공격한다”가 아니라 “Idle 또는 Move 상태에서 공격 버튼을 누르면 Attack 상태로 전환한다”처럼 작성할 수 있습니다. 피격도 마찬가지입니다. 체력이 0보다 크면 Hit 상태로 전환하고, 체력이 0 이하라면 Dead 상태로 전환한다고 정의할 수 있습니다.

상태 머신을 기획서에 작성할 때는 각 상태의 의미, 진입 조건, 유지 중 가능한 행동, 종료 조건을 함께 적는 것이 좋습니다. 예를 들어 Attack 상태는 공격 입력으로 진입하고, 공격 애니메이션이 끝나면 Idle 또는 Move 상태로 돌아갑니다. 이 상태에서는 이동을 제한할지, 방향 전환을 허용할지, 피격 시 공격을 취소할지 같은 조건도 함께 정리해야 합니다.

AI에게 캐릭터 제어 코드를 요청할 때도 상태 머신은 매우 좋은 입력 자료가 됩니다. “캐릭터 조작을 만들어줘”보다 “캐릭터는 Idle, Move, Jump, Attack, Hit, Dead 상태를 가지며, 각 상태에서 허용되는 입력과 전환 조건은 다음과 같다”라고 요청하면 훨씬 안정적인 구조를 얻을 수 있습니다. 캐릭터 상태를 상태 머신으로 정리하면 조작, 애니메이션, 전투, 피격 처리를 하나의 기준으로 연결할 수 있습니다.

Chapter 30. Idle, Move, Attack, Hit, Dead 상태 정의하기

캐릭터 상태 머신을 작성할 때 처음부터 너무 많은 상태를 만들 필요는 없습니다. 대부분의 게임에서는 Idle, Move, Attack, Hit, Dead 같은 기본 상태만으로도 캐릭터의 핵심 흐름을 정리할 수 있습니다. 중요한 것은 상태 이름을 나열하는 것이 아니라, 각 상태가 어떤 의미를 가지며 어떤 조건에서 시작되고 끝나는지 명확히 정의하는 것입니다.

Idle은 캐릭터가 아무 행동도 하지 않고 대기하는 상태입니다. 이동 입력이 없고, 공격이나 피격 같은 특별한 행동이 없을 때 유지됩니다. 이 상태에서는 이동, 공격, 점프, 대시 같은 기본 입력을 받을 수 있습니다. Idle은 다른 상태로 전환되기 전의 기준 상태이므로, 캐릭터 제어의 출발점으로 생각하면 됩니다.

Move는 캐릭터가 이동 입력에 따라 움직이는 상태입니다. 이동 입력이 들어오면 Move 상태로 전환되고, 입력이 사라지면 다시 Idle 상태로 돌아갑니다. Move 상태에서는 공격이나 대시를 허용할지, 이동 중 방향 전환이 가능한지, 이동 속도가 어떤 값으로 적용되는지를 정리해야 합니다. 단순히 움직인다고 쓰기보다 이동 방식과 전환 조건을 함께 작성해야 합니다.

Attack은 공격 입력으로 진입하는 상태입니다. 이 상태에서는 공격 판정이 언제 발생하는지, 공격 중 이동이 가능한지, 공격이 끝나면 어떤 상태로 돌아가는지 정의해야 합니다. 예를 들어 공격 애니메이션이 끝나면 이동 입력이 있으면 Move로, 입력이 없으면 Idle로 돌아갈 수 있습니다. 피격을 받으면 공격이 취소되는지도 함께 정리해야 합니다.

Hit와 Dead는 캐릭터가 피해를 입었을 때 필요한 상태입니다. Hit는 피격 반응을 처리하는 상태이고, 일정 시간이 지나면 Idle 또는 Move로 돌아갈 수 있습니다. Dead는 체력이 0 이하가 되었을 때 진입하는 최종 상태입니다. Dead 상태에서는 이동, 공격, 점프 같은 모든 플레이 입력을 막고 사망 연출이나 게임 오버 흐름으로 연결합니다. 이렇게 기본 상태를 명확히 정의하면 조작, 전투, 애니메이션, AI 요청까지 하나의 기준으로 정리할 수 있습니다.

Chapter 31. 애니메이션 전환 조건 작성하기

캐릭터 상태를 정의했다면 다음으로 애니메이션 전환 조건을 정리해야 합니다. 게임에서 애니메이션은 단순히 보기 좋은 움직임이 아닙니다. 플레이어가 현재 어떤 상태인지, 어떤 행동을 하고 있는지, 다음에 무엇을 할 수 있는지 알려주는 중요한 피드백입니다. 그래서 애니메이션 기획은 “걷기 애니메이션이 있다”, “공격 애니메이션이 있다” 정도로 끝나면 안 됩니다. 어떤 조건에서 어떤 애니메이션으로 전환되는지 명확히 작성해야 합니다.

애니메이션 전환 조건은 캐릭터 상태와 연결해서 작성하는 것이 좋습니다. Idle 상태에서는 대기 애니메이션을 재생하고, 이동 입력이 들어오면 Move 상태로 전환되면서 이동 애니메이션을 재생합니다. 공격 입력이 들어오면 Attack 상태로 전환되고 공격 애니메이션이 재생됩니다. 피격되면 Hit 애니메이션을 재생하고, 체력이 0 이하가 되면 Dead 애니메이션으로 전환됩니다. 이렇게 상태와 애니메이션을 연결하면 구현 기준이 분명해집니다.

전환 조건에는 입력뿐 아니라 수치와 이벤트도 포함됩니다. 이동 속도가 0보다 크면 Move 애니메이션을 재생하고, 속도가 0이면 Idle로 돌아갈 수 있습니다. 공격 애니메이션이 특정 프레임에 도달했을 때 공격 판정을 발생시킬 수도 있습니다. 피격 애니메이션이 끝나면 다시 Idle 또는 Move로 돌아가고, 사망 애니메이션이 끝나면 게임 오버 화면으로 넘어갈 수 있습니다. 애니메이션은 상태 전환과 게임 이벤트를 연결하는 기준이 됩니다.

또한 애니메이션 전환에는 우선순위가 필요합니다. 이동 중 공격 입력이 들어오면 이동 애니메이션보다 공격 애니메이션이 우선될 수 있습니다. 공격 중 피격을 허용한다면 Hit 애니메이션이 Attack 애니메이션을 끊고 재생되어야 합니다. 반대로 특정 스킬은 피격으로 취소되지 않도록 만들 수도 있습니다. 이런 우선순위를 정하지 않으면 애니메이션과 실제 상태가 어긋나는 문제가 생깁니다.

AI에게 캐릭터 애니메이션 제어를 요청할 때도 전환 조건은 반드시 필요합니다. “애니메이션을 연결해줘”라고 요청하기보다 “Idle, Move, Attack, Hit, Dead 상태에 따라 애니메이션을 전환하고, Attack 중 Hit가 발생하면 공격을 취소하고 Hit 애니메이션을 재생한다”처럼 작성해야 합니다. 좋은 애니메이션 기획은 어떤 애니메이션이 있는지가 아니라, 언제 시작되고 언제 끝나며 어떤 상태와 연결되는지를 정의하는 문서입니다.

Chapter 32. 플레이어 능력치 정의하기

플레이어 능력치는 캐릭터가 게임 안에서 어떤 성능을 가지는지 나타내는 기준값입니다. 체력, 공격력, 방어력, 이동 속도, 점프력, 공격 속도, 치명타 확률처럼 게임 규칙에 영향을 주는 값들이 여기에 해당합니다. 능력치를 명확히 정의하지 않으면 전투, 성장, 아이템, 밸런싱 시스템이 모두 흔들릴 수 있습니다.

능력치를 작성할 때는 먼저 어떤 값이 게임에 필요한지 구분해야 합니다. 모든 게임에 복잡한 능력치가 필요한 것은 아닙니다. 간단한 액션 게임이라면 체력, 이동 속도, 공격력 정도로 충분할 수 있습니다. 반대로 RPG나 성장형 게임이라면 방어력, 치명타, 공격 속도, 회피율, 속성 저항, 경험치 배율 같은 값이 필요할 수 있습니다. 장르와 핵심 재미에 맞는 능력치만 선택해야 합니다.

프로그래머 관점에서 능력치는 단순한 숫자가 아니라 데이터 구조입니다. 각 능력치는 이름, 의미, 기본값, 최소값, 최대값, 증가 방식, 적용 위치를 함께 정의해야 합니다. 예를 들어 MoveSpeed는 캐릭터 이동 계산에 사용되고, AttackPower는 데미지 계산식에 사용되며, MaxHp는 체력 UI와 사망 판정에 연결됩니다. 어떤 시스템이 어떤 능력치를 참조하는지 명확히 적어두면 구현이 쉬워집니다.

능력치에는 계산 규칙도 필요합니다. 장비나 버프가 적용될 때 기본값에 더할지, 곱할지, 최종값에 제한을 둘지 정해야 합니다. 예를 들어 공격력은 기본 공격력에 장비 공격력을 더한 뒤 버프 배율을 곱할 수 있습니다. 이동 속도는 너무 빨라지면 게임이 깨질 수 있으므로 최대값을 제한할 수도 있습니다. 이런 규칙은 나중에 밸런싱 문제를 줄이는 데 도움이 됩니다.

AI에게 플레이어 능력치 시스템을 요청할 때도 구체적인 정의가 필요합니다. “능력치 시스템을 만들어 줘”보다 “플레이어는 MaxHp, CurrentHp, AttackPower, Defense, MoveSpeed를 가지며, 데미지는 $\text{AttackPower} - \text{Defense}$ 로 계산하되 최소 1 이상 적용한다”라고 요청해야 더 정확한 결과를 얻을 수 있습니다. 좋은 능력치 기획은 숫자 목록이 아니라, 게임의 규칙과 데이터 구조를 연결하는 문서입니다.

Chapter 33. 캐릭터 기획서를 코드 구조로 연결하기

캐릭터 기획서는 문서로만 끝나면 의미가 약합니다. 프로그래머가 쓰는 기획서는 결국 코드 구조로 이어져야 합니다. 플레이어 조작, 상태, 능력치, 애니메이션, 피격 처리 같은 내용을 정리했다면, 이제 이 정보를 어떤 클래스와 컴포넌트로 나눌지 생각해야 합니다. 좋은 캐릭터 기획서는 구현 전에 코드의 큰 구조를 미리 보여줍니다.

예를 들어 플레이어 조작 기획서에는 이동, 점프, 대시, 공격 입력이 정리되어 있을 수 있습니다. 이 내용을 코드로 옮길 때는 입력을 담당하는 `PlayerInputController`, 이동을 처리하는 `PlayerMovement`, 공격을 처리하는 `PlayerCombat`처럼 역할을 나누어볼 수 있습니다. 모든 기능을 하나의 클래스에 넣으면 처음에는 편하지만, 기능이 늘어날수록 수정하기 어려운 구조가 됩니다.

캐릭터 상태 기획서는 상태 머신 코드로 연결할 수 있습니다. `Idle`, `Move`, `Attack`, `Hit`, `Dead` 상태가 문서에 정리되어 있다면, 이를 `PlayerState` enum이나 상태 클래스 구조로 바꿀 수 있습니다. 각 상태에는 진입 조건, 유지 중 동작, 종료 조건이 필요합니다. 기획서에 이런 조건이 명확히 적혀 있으면 AI에게 상태 머신 코드를 요청하거나 직접 구현할 때 기준이 분명해집니다.

능력치 기획서는 데이터 클래스로 연결됩니다. `MaxHp`, `CurrentHp`, `AttackPower`, `Defense`, `MoveSpeed` 같은 항목은 `PlayerStats` 또는 `CharacterStats` 같은 클래스로 묶을 수 있습니다. 밸런싱이 자주 바뀌는 값이라면 `ScriptableObject`나 데이터 테이블로 분리하는 것이 좋습니다. 이렇게 하면 코드 수정 없이 수치를 조정할 수 있고, 여러 캐릭터에 같은 구조를 재사용할 수 있습니다.

캐릭터 기획서를 코드 구조로 연결한다는 것은 문장을 클래스 이름으로 단순히 바꾸는 일이 아닙니다. 각 기능의 책임을 나누고, 데이터와 로직을 분리하며, 상태와 입력의 관계를 명확히 하는 일입니다. AI에게 구현을 요청할 때도 “플레이어 캐릭터를 만들어줘”보다 “입력, 이동, 전투, 상태 머신, 능력치를 각각 분리한 구조로 작성해줘”라고 요청해야 합니다. 좋은 기획서는 구현을 시작하기 전에 코드의 방향을 정리해주는 설계 문서가 됩니다.

Part 05. 전투 시스템 상세 기획하기

Part 5에서는 게임 개발에서 가장 복잡해지기 쉬운 전투 시스템을 기획하는 방법을 다룹니다.

전투는 공격 입력, 판정, 데미지 계산, 피격 처리, 쿨타임, 적 AI, 보스 패턴처럼 여러 요소가 맞물려 작동하는 시스템입니다.

이 파트에서는 “타격감 있는 전투” 같은 추상적인 표현을 구체적인 규칙과 조건으로 나누어 작성하는 방법을 설명합니다.

또한 작성한 전투 기획서를 AI에게 전달할 수 있는 구현 요청 프롬프트로 바꾸는 과정도 함께 다룹니다.

Chapter 34. 전투 시스템은 규칙의 집합이다

전투 시스템은 단순히 공격 버튼을 누르고 적의 체력을 줄이는 기능이 아닙니다. 전투는 입력, 판정, 데미지, 피격, 회피, 쿨타임, 상태 전환, 보상 같은 여러 규칙이 연결된 시스템입니다. 플레이어가 전투를 재미있다고 느끼려면 이 규칙들이 일관되게 동작해야 합니다. 그래서 전투 기획은 “공격한다”라는 문장으로 끝나면 안 됩니다.

전투를 기획할 때는 먼저 공격이 성립하는 조건을 정리해야 합니다. 플레이어가 어떤 입력을 했을 때 공격하는지, 공격 중 이동이 가능한지, 공격 판정은 애니메이션의 어느 시점에 발생하는지, 공격 범위는 어디까지인지 작성해야 합니다. 같은 공격이라도 근접 공격인지, 원거리 공격인지, 범위 공격인지에 따라 필요한 구현 방식이 달라집니다.

그다음은 데미지와 피격 규칙입니다. 데미지는 고정값인지, 공격력과 방어력으로 계산되는지, 치명타나 속성 보정이 있는지 정해야 합니다. 피격 시에는 체력 감소뿐 아니라 피격 애니메이션, 넉백, 무적 시간, 입력 제한, 사망 판정까지 함께 고려해야 합니다. 전투는 숫자 계산만으로 완성되지 않고, 플레이어에게 피드백을 주는 여러 요소가 함께 작동해야 합니다.

전투 시스템에서는 예외 조건도 중요합니다. 적이 이미 죽은 상태에서 다시 공격을 받으면 어떻게 할지, 플레이어가 공격하는 순간 피격되면 공격이 취소되는지, 스킬 쿨타임 중에 다시 입력하면 어떤 반응을 줄지 정해야 합니다. 이런 조건이 빠지면 구현 중마다 즉흥적으로 판단하게 되고, 기능이 늘어날수록 코드가 복잡해집니다.

AI에게 전투 시스템 구현을 요청할 때도 규칙을 나누어 전달해야 합니다. “전투 시스템을 만들어줘”보다 “공격 입력, 공격 판정, 데미지 계산, 피격 처리, 무적 시간, 사망 판정을 각각 분리해서 구현해줘”라고 요청하는 편이 좋습니다. 전투 시스템은 하나의 기능이 아니라 규칙들의 집합입니다. 좋은 전투 기획서는 이 규칙들을 명확히 정리해 구현과 테스트의 기준을 만들어줍니다.

Chapter 35. 공격 입력과 공격 판정 정의하기

전투 시스템을 기획할 때 가장 먼저 정리해야 할 것은 공격 입력과 공격 판정입니다. 공격 입력은 플레이어가 어떤 조작으로 공격을 시작하는지를 의미하고, 공격 판정은 그 공격이 실제로 적에게 맞았는지를 판단하는 규칙입니다. “공격 버튼을 누르면 적을 공격한다”는 설명만으로는 부족합니다. 어떤 버튼을 누르는지, 언제 공격이 시작되는지, 어느 범위에 있는 적이 맞는지까지 정의해야 구현할 수 있습니다.

공격 입력을 작성할 때는 입력 방식과 입력 조건을 함께 적어야 합니다. 예를 들어 PC 게임이라면 마우스 왼쪽 버튼으로 기본 공격을 하고, 키보드 숫자 키로 스킬을 사용할 수 있습니다. 모바일 게임이라면 공격 버튼 터치나 자동 공격 방식이 될 수 있습니다. 또한 공격은 Idle 또는 Move 상태에서만 가능한지, 점프 중에도 가능한지, 피격 중에는 입력을 무시할지 같은 조건도 함께 작성해야 합니다.

공격 판정은 전투의 핵심 규칙입니다. 근접 공격이라면 캐릭터 앞의 일정 범위 안에 있는 적을 검사할 수 있고, 원거리 공격이라면 발사체가 적과 충돌했는지를 판단할 수 있습니다. 범위 공격이라면 중심 위치와 반경을 기준으로 여러 적을 동시에 타격할 수 있습니다. 이처럼 공격 방식에 따라 판정 방식이 달라지므로, 기획서에는 공격 타입과 판정 범위를 명확히 적어야 합니다.

공격 판정의 발생 시점도 중요합니다. 공격 버튼을 누른 즉시 데미지가 들어가는지, 공격 애니메이션의 특정 프레임에서 판정이 발생하는지 정해야 합니다. 타격감이 중요한 액션 게임이라면 애니메이션, 이펙트, 사운드, 데미지 적용 시점이 어긋나지 않아야 합니다. 따라서 공격 기획서에는 “공격 애니메이션 시작 후 0.2초 시점에 판정 발생”처럼 가능한 구체적으로 작성하는 것이 좋습니다.

AI에게 공격 기능 구현을 요청할 때도 입력과 판정을 나누어 설명해야 합니다. “공격 시스템을 만들어줘”보다 “마우스 왼쪽 버튼 입력 시 기본 공격을 시작하고, 공격 애니메이션 0.2초 후 캐릭터 전방 2m 범위의 적에게 데미지를 적용한다”라고 요청하는 편이 훨씬 정확합니다. 좋은 전투 기획은 공격을 감각적인 표현으로 설명하는 데서 끝나지 않고, 입력 조건과 판정 규칙을 구현 가능한 형태로 정의합니다.

Chapter 36. 데미지 계산식 작성하기

전투 시스템에서 데미지 계산식은 매우 중요한 기준입니다. 공격이 적에게 맞았을 때 체력이 얼마나 줄어드는지, 방어력이 어떤 방식으로 적용되는지, 치명타나 속성 보정이 있는지에 따라 전투의 난이도와 밸런스가 달라집니다. 그래서 상세 기획서에는 “적에게 데미지를 준다”라고만 쓰면 안 됩니다. 어떤 값으로, 어떤 순서로, 어떤 제한을 적용해 데미지를 계산하는지 명확히 작성해야 합니다.

가장 단순한 방식은 고정 데미지입니다. 예를 들어 기본 공격은 항상 10의 데미지를 주고, 강공격은 25의 데미지를 주는 식입니다. 이런 방식은 구현이 쉽고 초반 프로토타입에 적합합니다. 하지만 성장 시스템이나 장비 시스템이 있는 게임이라면 공격력, 방어력, 스킬 계수 같은 값을 함께 사용하는 계산식이 필요합니다.

예를 들어 기본 데미지 계산식은 “최종 데미지 = 공격력 - 방어력”처럼 작성할 수 있습니다. 다만 이 방식은 방어력이 공격력보다 높을 때 데미지가 0 이하가 될 수 있으므로 최소 데미지 조건이 필요합니다. “최종 데미지는 최소 1 이상 적용한다”라는 조건을 함께 작성해야 합니다. 이렇게 계산식과 예외 조건을 함께 적어야 구현 기준이 분명해집니다.

데미지 계산식에는 보정값도 포함될 수 있습니다. 스킬 데미지는 공격력에 스킬 계수를 곱해 계산할 수 있고, 치명타가 발생하면 최종 데미지에 배율을 적용할 수 있습니다. 속성 상성이 있는 게임이라면 불, 물, 전기 같은 속성 보정도 들어갈 수 있습니다. 중요한 것은 모든 보정값을 감으로 처리하지 않고, 적용 순서를 기획서에 명확히 쓰는 것입니다.

AI에게 전투 시스템 구현을 요청할 때도 데미지 계산식은 반드시 포함해야 합니다. “데미지를 계산해 줘”보다 “기본 데미지는 공격력에서 방어력을 뺀 값으로 계산하고, 최소 데미지는 1로 보장한다. 치명타 발생 시 최종 데미지에 1.5배를 적용한다”라고 요청해야 합니다. 좋은 데미지 기획은 숫자를 정하는 데서 끝나지 않습니다. 계산식, 최소값, 보정 순서, 예외 조건까지 함께 정의해야 안정적인 전투 시스템을 만들 수 있습니다.

Chapter 37. 체력, 방어력, 치명타 같은 수치 정의하기

전투 시스템을 안정적으로 만들려면 전투에 사용되는 수치를 명확히 정의해야 합니다. 체력, 공격력, 방어력, 치명타 확률, 치명타 배율, 공격 속도, 이동 속도 같은 값은 전투의 흐름과 난이도를 결정합니다. 이런 수치가 정리되어 있지 않으면 전투는 감으로 조정하게 되고, 기능이 늘어날수록 밸런스를 맞추기 어려워집니다.

체력은 캐릭터가 얼마나 버틸 수 있는지를 나타내는 기본 수치입니다. 기획서에는 최대 체력, 현재 체력, 체력이 0 이하가 되었을 때의 처리, 회복 가능 여부를 함께 적어야 합니다. 예를 들어 “플레이어의 최대 체력은 100이며, 현재 체력이 0 이하가 되면 Dead 상태로 전환하고 게임 오버 UI를 표시한다”처럼 작성하면 구현 기준이 분명해집니다.

방어력은 받은 데미지를 줄이는 수치입니다. 단순히 “방어력이 있다”라고 쓰기보다 방어력이 데미지 계산에 어떻게 적용되는지 정해야 합니다. 공격력에서 방어력을 빼는 방식인지, 최종 데미지에 퍼센트 감소를 적용하는 방식인지, 최소 데미지를 보장하는지 함께 작성해야 합니다. 방어력 계산 방식에 따라 전투 밸런스가 크게 달라지기 때문에 초기에 기준을 잡는 것이 좋습니다.

치명타는 전투에 변동성과 긴장감을 주는 수치입니다. 치명타 확률은 얼마나 자주 발생하는지를 나타내고, 치명타 배율은 발생했을 때 데미지가 얼마나 증가하는지를 나타냅니다. 예를 들어 “치명타 확률은 10%, 치명타 배율은 1.5배이며, 치명타는 최종 데미지 계산 후 적용한다”처럼 작성할 수 있습니다. 이때 치명타가 모든 공격에 적용되는지, 특정 스킬에는 적용되지 않는지도 함께 정리해야 합니다.

AI에게 전투 수치 시스템을 요청할 때는 수치 이름과 의미, 기본값, 계산 방식을 함께 제공해야 합니다. “체력과 방어력, 치명타를 만들어줘”라고 요청하는 것보다 “MaxHp, CurrentHp, Defense, CriticalChance, CriticalMultiplier를 정의하고, 방어력 적용 후 치명타 배율을 계산하는 구조로 작성해줘”라고 요청하는 편이 좋습니다. 좋은 수치 기획은 숫자 목록이 아니라, 전투 규칙과 밸런싱 기준을 연결하는 문서입니다.

Chapter 38. 피격, 넉백, 무적 시간 작성하기

전투 시스템에서 공격만큼 중요한 것이 피격 처리입니다. 적에게 맞았을 때 체력만 줄어들면 전투는 밋밋하게 느껴질 수 있습니다. 플레이어가 공격을 받았다는 사실을 명확히 인식하려면 피격 애니메이션, 넉백, 무적 시간, 사운드, 이펙트 같은 피드백이 함께 동작해야 합니다. 따라서 상세 기획서에는 “데미지를 받는다”가 아니라 “어떻게 맞고, 어떻게 밀려나며, 언제 다시 피해를 받을 수 있는지”를 작성해야 합니다.

피격은 캐릭터가 공격 판정에 맞았을 때 실행되는 처리입니다. 이때 체력이 감소하고, Hit 상태로 전환되며, 피격 애니메이션이 재생될 수 있습니다. 기획서에는 피격 시 입력을 막을지, 이동을 유지할지, 공격 중이었다면 공격을 취소할지 정리해야 합니다. 예를 들어 “플레이어가 공격 중 피격되면 Attack 상태를 취소하고 Hit 상태로 전환한다”처럼 작성하면 상태 전환 기준이 분명해집니다.

넉백은 피격 대상이 충격 방향으로 밀려나는 처리입니다. 넉백을 작성할 때는 방향, 거리, 지속 시간, 적용 대상을 정의해야 합니다. 적에게 맞은 방향의 반대쪽으로 밀려나는지, 공격자의 바라보는 방향으로 밀려나는지 정해야 하며, 보스나 무거운 적에게는 넉백을 적용하지 않을 수도 있습니다. 넉백은 전투 감각을 강화하지만, 너무 강하면 조작감을 해칠 수 있으므로 제한 조건도 함께 필요합니다.

무적 시간은 연속 피격을 막기 위한 규칙입니다. 플레이어가 한 번 맞은 직후 계속 데미지를 받으면 체력이 순식간에 줄어들 수 있습니다. 그래서 피격 후 1초 동안 추가 피해를 받지 않도록 설정할 수 있습니다. 이때 무적 시간 동안 피격 판정만 무시할지, 캐릭터가 깜빡이는 연출을 할지, 특정 강력한 공격은 무적을 무시할 수 있는지도 함께 정리해야 합니다.

AI에게 피격 시스템을 요청할 때는 체력 감소만 요청하지 말고 피격 흐름 전체를 설명해야 합니다. “피격되면 체력이 줄어들고, 공격자의 반대 방향으로 0.3초 동안 넉백되며, 이후 1초 동안 무적 상태가 된다”처럼 작성하면 구현 방향이 명확해집니다. 좋은 피격 기획은 전투의 공정성과 타격감을 함께 만드는 문서입니다.

Chapter 39. 스킬과 쿨타임 기획하기

스킬은 전투 시스템에 선택과 변화를 만들어주는 중요한 요소입니다. 기본 공격만 있는 전투는 단순하게 느껴질 수 있지만, 스킬이 추가되면 플레이어는 언제 어떤 행동을 할지 판단하게 됩니다. 다만 스킬을 기획할 때 “강력한 공격 스킬”, “회복 스킬”, “범위 공격 스킬”처럼 이름과 느낌만 적어서는 부족합니다. 스킬은 입력 조건, 효과, 대상, 범위, 자원 소모, 쿨타임까지 함께 정의해야 구현할 수 있습니다.

스킬 기획에서 먼저 정해야 할 것은 스킬의 역할입니다. 적에게 큰 데미지를 주는 공격 스킬인지, 여러 적을 타격하는 범위 스킬인지, 체력을 회복하는 지원 스킬인지, 일정 시간 능력치를 올리는 버프 스킬인지 구분해야 합니다. 역할이 명확해야 데미지, 범위, 지속 시간, 사용 조건을 정하기 쉽습니다. 모든 스킬이 단순히 데미지만 높다면 전투 선택지는 늘어나지 않습니다.

쿨타임은 스킬 사용 빈도를 조절하는 규칙입니다. 강력한 스킬일수록 자주 사용할 수 없도록 쿨타임을 길게 설정할 수 있습니다. 반대로 기본 전투 흐름에 자주 쓰이는 스킬은 짧은 쿨타임을 가질 수 있습니다. 기획서에는 스킬을 사용한 직후 쿨타임이 시작되는지, 스킬 시전이 끝난 뒤 시작되는지, 쿨타임 중 입력하면 어떤 반응을 줄지 작성해야 합니다.

스킬에는 사용 불가능 조건도 필요합니다. 마나가 부족할 때, 쿨타임 중일 때, 피격 상태일 때, 사망 상태일 때는 스킬을 사용할 수 없을 수 있습니다. 또한 일부 스킬은 지상에서만 사용할 수 있고, 일부 스킬은 공중에서도 사용할 수 있습니다. 이런 조건을 정리하지 않으면 스킬 시스템은 다른 조작, 상태, 전투 시스템과 충돌하기 쉽습니다.

AI에게 스킬 시스템 구현을 요청할 때는 스킬의 이름보다 규칙을 먼저 전달해야 합니다. “파이어볼 스킬을 만들어줘”보다 “파이어볼은 마우스 오른쪽 버튼으로 사용하며, 전방으로 투사체를 발사하고, 적에게 30 데미지를 주며, 사용 후 3초 쿨타임을 가진다. 쿨타임 중에는 다시 사용할 수 없다”라고 작성하는 편이 좋습니다. 좋은 스킬 기획은 화려한 아이디어가 아니라, 사용 조건과 제한 규칙이 명확한 전투 명세입니다.

Chapter 40. 적 AI 행동 패턴 정의하기

전투 시스템에서 적 AI는 플레이어에게 도전 과제를 제공하는 핵심 요소입니다. 적이 아무 생각 없이 플레이어에게 다가오기만 한다면 전투는 단조로워질 수 있습니다. 반대로 적 AI가 너무 복잡하면 구현과 테스트가 어려워지고, 플레이어가 규칙을 이해하기 힘들어집니다. 따라서 적 AI 기획은 “똑똑한 적”을 만드는 것이 아니라, 플레이어가 이해하고 대응할 수 있는 행동 패턴을 정의하는 일입니다.

적 AI 행동 패턴을 작성할 때는 먼저 상태를 나누는 것이 좋습니다. 예를 들어 대기, 추적, 공격, 후퇴, 피격, 사망 상태로 구분할 수 있습니다. 대기 상태에서는 플레이어가 일정 거리 안에 들어올 때까지 멈춰 있고, 추적 상태에서는 플레이어를 향해 이동하며, 공격 범위에 들어오면 공격 상태로 전환합니다. 이렇게 상태와 전환 조건을 함께 적으면 구현 기준이 분명해집니다.

행동 패턴에는 감지 조건과 공격 조건이 필요합니다. 적이 플레이어를 언제 발견하는지, 어느 거리부터 추적하는지, 공격 범위는 얼마나 되는지, 공격 후 쿨타임은 몇 초인지 정해야 합니다. 예를 들어 “플레이어가 8m 안에 들어오면 추적하고, 2m 안에 들어오면 근접 공격을 한다. 공격 후 1.5초 동안 다시 공격할 수 없다”처럼 작성할 수 있습니다.

적 AI는 종류별로 다르게 정의해야 합니다. 근접 적은 플레이어에게 접근해 공격하고, 원거리 적은 일정 거리를 유지하며 투사체를 발사할 수 있습니다. 돌진형 적은 짧은 예고 동작 후 빠르게 돌진하고, 방어형 적은 일정 시간 동안 피해를 줄일 수 있습니다. 적마다 역할이 다르다면 전투가 더 다양해지지만, 각 행동은 명확한 조건과 제한을 가져야 합니다.

AI에게 적 AI 구현을 요청할 때는 “적을 똑똑하게 만들어줘”라고 하지 말고 행동 규칙을 구체적으로 전달해야 합니다. “적은 플레이어가 8m 안에 있으면 추적하고, 2m 안에 있으면 공격하며, 공격 후 1.5초 쿨타임을 가진다. 체력이 0 이하가 되면 Dead 상태로 전환한다”처럼 작성하면 구현 결과가 안정적입니다. 좋은 적 AI 기획은 복잡한 지능을 만드는 것이 아니라, 예측 가능하면서도 재미있는 행동 규칙을 정리하는 문서입니다.

Chapter 41. 보스 패턴을 단계별로 작성하기

보스 전투는 일반 적과 달리 하나의 긴 전투 흐름을 설계해야 합니다. 일반 적은 대기, 추적, 공격, 사망 정도의 단순한 행동 패턴으로도 충분할 수 있지만, 보스는 플레이어가 전투 중 변화를 느낄 수 있어야 합니다. 그래서 보스 패턴 기획은 “강한 적을 만든다”가 아니라, 체력 구간과 시간 흐름에 따라 어떤 공격을 어떤 순서로 사용할지 단계별로 작성하는 일입니다.

보스 패턴을 작성할 때는 먼저 페이지를 나누는 것이 좋습니다. 예를 들어 보스 체력이 100~70%일 때는 1페이지, 70~40%일 때는 2페이지, 40% 이하일 때는 3페이지로 구분할 수 있습니다. 각 페이지마다 사용하는 공격, 이동 방식, 공격 속도, 소환 패턴, 난이도 변화를 정리합니다. 이렇게 나누면 전투가 단조롭지 않고, 플레이어는 보스가 점점 위험해진다는 흐름을 느낄 수 있습니다.

각 패턴은 실행 조건과 예고 동작을 함께 작성해야 합니다. 예를 들어 “보스가 플레이어 방향으로 돌진한다”라고만 쓰면 부족합니다. 돌진 전에 1초 동안 몸을 낮추는 예고 동작을 하고, 이후 플레이어의 현재 위치를 향해 빠르게 이동하며, 충돌 시 데미지를 주고, 돌진 후 2초 동안 빈틈을 보인다고 작성해야 합니다. 보스 패턴은 강력해야 하지만, 플레이어가 보고 피할 수 있는 단서도 필요합니다.

패턴 사이의 순서와 쿨타임도 중요합니다. 보스가 같은 강력한 공격을 계속 반복하면 전투가 불공정하게 느껴질 수 있습니다. 따라서 기본 공격, 범위 공격, 돌진 공격, 소환 패턴처럼 여러 공격을 조합하고, 각 패턴에 재사용 대기 시간을 설정해야 합니다. 특정 패턴은 페이지 전환 시 한 번만 실행되도록 만들 수도 있습니다. 이런 규칙이 있어야 보스 AI가 예측 가능하면서도 지루하지 않게 동작합니다.

AI에게 보스 패턴 구현을 요청할 때는 보스의 체력 구간, 패턴 목록, 실행 조건, 예고 시간, 데미지, 쿨타임을 함께 전달해야 합니다. “보스 AI를 만들어줘”보다 “체력 70% 이하에서 2페이지로 전환하고, 돌진 패턴과 원형 탄막 패턴을 추가한다. 돌진은 1초 예고 후 실행되며, 사용 후 5초 쿨타임을 가진다”라고 요청하는 편이 좋습니다. 좋은 보스 기획은 강한 적을 만드는 것이 아니라, 플레이어가 읽고 대응할 수 있는 단계적 전투 흐름을 설계하는 문서입니다.

Chapter 42. 전투 시스템을 AI 프롬프트로 바꾸기

전투 시스템 기획서를 작성했다면 다음 단계는 이를 AI에게 전달할 수 있는 프롬프트로 바꾸는 것입니다. AI는 “전투 시스템을 만들어줘” 같은 넓은 요청보다, 입력 조건과 처리 흐름이 명확한 요청에서 더 좋은 결과를 냅니다. 따라서 전투 기획서는 단순한 문서로 끝나는 것이 아니라, 구현 요청 프롬프트의 원본 자료가 되어야 합니다.

전투 시스템 프롬프트에는 먼저 구현 목적을 적어야 합니다. 예를 들어 “Unity C#으로 탑다운 액션 게임의 기본 전투 시스템을 구현해줘”처럼 어떤 환경에서 어떤 시스템을 만들지 설명합니다. 그다음 공격 입력, 공격 판정, 데미지 계산, 피격 처리, 넉백, 무적 시간, 사망 판정처럼 필요한 기능을 항목별로 나누어 전달해야 합니다. 이렇게 나누면 AI가 기능을 더 구조적으로 설계할 수 있습니다.

좋은 프롬프트는 전투 규칙을 구체적으로 포함합니다. “마우스 왼쪽 버튼을 누르면 기본 공격을 실행한다. 공격 애니메이션 시작 후 0.2초 뒤 캐릭터 전방 2m 범위에 있는 적을 판정한다. 데미지는 공격력에서 방어력을 뺀 값으로 계산하고, 최소 1 이상 적용한다. 피격된 적은 0.3초 동안 넉백되고, 체력이 0 이하가 되면 Dead 상태로 전환한다”처럼 작성하면 구현 기준이 분명해집니다.

또한 AI에게 코드 구조도 함께 요청하는 것이 좋습니다. 예를 들어 PlayerCombat, DamageCalculator, Health, HitReceiver, EnemyStateMachine처럼 역할을 분리해달라고 요청할 수 있습니다. “하나의 클래스에 모두 작성하지 말고, 입력 처리, 데미지 계산, 체력 관리, 피격 처리를 분리해줘”라고 조건을 붙이면 유지보수하기 좋은 구조를 얻을 가능성이 높아집니다.

AI가 만든 결과는 그대로 믿기보다 기획서와 비교하며 검토해야 합니다. 공격 판정 시점이 맞는지, 무적 시간이 적용되는지, 사망 상태에서 추가 입력이 막히는지, 데미지 계산식이 기획서와 같은지 확인해야 합니다. 전투 시스템을 AI 프롬프트로 바꾸는 목적은 AI에게 모든 판단을 맡기는 것이 아닙니다. 프로그래머가 정리한 규칙을 바탕으로 AI가 더 빠르게 구현하도록 돕는 것입니다.

Part 06. 성장, 아이템, 보상 시스템 기획하기

Part 6에서는 플레이어의 반복 플레이를 이끄는 성장, 아이템, 보상 시스템을 정리합니다.

경험치, 레벨업, 능력치 상승, 장비, 소비 아이템, 드랍 확률, 재화, 상점 시스템은 게임의 장기적인 재미와 밀접하게 연결됩니다.

이 파트에서는 각 시스템을 데이터 구조와 규칙 중심으로 작성하는 방법을 설명합니다.

특히 프로그래머 관점에서 어떤 값을 테이블로 분리해야 하는지, 어떤 조건을 예외 처리해야 하는지, 보상과 성장이 어떻게 연결되는지 정리하는 데 초점을 둡니다.

Chapter 43. 성장 시스템이 필요한 이유

성장 시스템은 플레이어가 게임을 계속 플레이하게 만드는 중요한 장치입니다. 전투에서 적을 처치하고, 경험치를 얻고, 레벨이 오르고, 능력치가 강해지는 흐름은 플레이어에게 “내가 점점 강해지고 있다”는 느낌을 줍니다. 이 감각은 반복 플레이를 유지하는 힘이 됩니다. 그래서 성장 시스템은 단순히 숫자를 올리는 기능이 아니라, 플레이어의 행동에 보상을 연결하는 구조입니다.

성장 시스템이 없는 게임도 만들 수 있습니다. 퍼즐 게임이나 순수 액션 게임처럼 플레이어의 실력 향상 자체가 재미가 되는 장르도 있습니다. 하지만 RPG, 방치형 게임, 로그라이트, 액션 RPG, 수집형 게임처럼 반복 플레이가 중요한 장르에서는 성장 시스템이 게임의 중심 구조가 됩니다. 플레이어는 전투를 통해 자원을 얻고, 그 자원으로 캐릭터를 강화하며, 더 어려운 콘텐츠에 도전합니다.

프로그래머 관점에서 성장 시스템은 여러 시스템을 연결하는 허브 역할을 합니다. 전투 시스템은 경험치와 보상을 지급하고, 아이템 시스템은 능력치를 바꾸며, UI는 성장 결과를 보여주고, 저장 시스템은 성장 상태를 보존합니다. 따라서 성장 시스템을 기획할 때는 레벨업만 생각하면 안 됩니다. 경험치 획득 조건, 레벨업 공식, 능력치 증가 방식, 보상 지급, 저장 처리까지 함께 정리해야 합니다.

성장 시스템은 밸런싱과도 밀접합니다. 너무 빠르게 성장하면 게임이 쉬워지고, 너무 느리게 성장하면 플레이어가 지루함을 느낄 수 있습니다. 레벨이 오를 때 체력과 공격력이 얼마나 증가하는지, 새로운 스킬은 언제 해금되는지, 다음 레벨까지 필요한 경험치는 어떻게 늘어나는지 기준을 정해야 합니다. 이런 수치가 정리되어 있어야 게임의 난이도 곡선을 조절할 수 있습니다.

AI에게 성장 시스템 구현을 요청할 때도 구체적인 규칙이 필요합니다. “레벨업 시스템을 만들어줘”보다 “적을 처치하면 경험치를 얻고, 경험치가 필요량 이상이 되면 레벨이 오르며, 레벨업 시 최대 체력과 공격력이 증가한다. 남은 경험치는 다음 레벨에 반영한다”라고 작성해야 더 정확한 결과를 얻을 수 있습니다. 좋은 성장 기획은 플레이어의 반복 행동과 보상을 연결하고, 게임을 계속 진행할 이유를 만들어주는 문서입니다.

Chapter 44. 경험치와 레벨업 규칙 작성하기

성장 시스템을 만들 때 가장 먼저 정리해야 할 것은 경험치와 레벨업 규칙입니다. 플레이어가 어떤 행동을 했을 때 경험치를 얻고, 어느 정도 경험치를 모으면 레벨이 오르며, 레벨업 후 어떤 변화가 생기는지를 명확히 작성해야 합니다. “적을 처치하면 레벨이 오른다”는 설명만으로는 부족합니다. 경험치 획득 조건, 필요 경험치, 초과 경험치 처리, 레벨업 보상까지 함께 정의해야 합니다.

경험치 획득 조건은 게임의 반복 행동과 연결됩니다. 전투 게임이라면 적 처치 시 경험치를 지급할 수 있고, 퀘스트형 게임이라면 목표 완료 시 경험치를 줄 수 있습니다. 방치형 게임이라면 일정 시간마다 자동으로 경험치를 얻을 수도 있습니다. 중요한 것은 어떤 행동이 성장으로 이어지는지 분명히 정하는 것입니다. 이 규칙이 명확해야 플레이어는 무엇을 해야 강해지는지 이해할 수 있습니다.

레벨업 규칙에는 다음 레벨까지 필요한 경험치를 어떻게 계산할지 포함해야 합니다. 가장 단순하게는 레벨마다 고정된 필요 경험치를 사용할 수 있습니다. 조금 더 성장감을 주고 싶다면 레벨이 오를수록 필요 경험치를 증가시킬 수 있습니다. 예를 들어 “다음 레벨 필요 경험치 = 현재 레벨 $\times$ 100”처럼 간단한 공식을 사용할 수 있습니다. 또는 레벨별 필요 경험치를 테이블로 작성해 직접 조정할 수도 있습니다.

초과 경험치 처리도 반드시 정해야 합니다. 플레이어가 레벨업에 필요한 경험치보다 더 많은 경험치를 얻었을 때 남은 경험치를 다음 레벨에 반영할지, 버릴지 결정해야 합니다. 일반적으로는 남은 경험치를 유지하는 편이 자연스럽습니다. 예를 들어 필요 경험치가 100이고 현재 경험치가 90인 상태에서 30을 얻었다면, 레벨업 후 20의 경험치를 다음 레벨에 남길 수 있습니다.

AI에게 경험치와 레벨업 시스템을 요청할 때는 규칙을 구체적으로 전달해야 합니다. “레벨업 시스템을 만들어줘”보다 “적 처치 시 경험치를 얻고, 현재 경험치가 필요 경험치 이상이면 레벨업한다. 레벨업 후 남은 경험치는 유지하며, 최대 체력과 공격력을 증가시킨다”라고 작성하는 편이 좋습니다. 좋은 레벨업 기획은 단순히 숫자를 올리는 기능이 아니라, 플레이어의 행동과 성장 보상을 연결하는 규칙 문서입니다.

Chapter 45. 능력치 상승 구조 정의하기

성장 시스템에서 중요한 것은 레벨이 오르는 사실 자체가 아니라, 레벨업 후 무엇이 어떻게 달라지는 지입니다. 플레이어가 레벨업을 했는데 체감되는 변화가 없다면 성장의 재미는 약해집니다. 반대로 능력치가 너무 빠르게 상승하면 게임 난이도가 무너질 수 있습니다. 따라서 상세 기획서에는 레벨업 시 어떤 능력치가 얼마나 증가하는지, 어떤 방식으로 계산되는지 명확히 작성해야 합니다.

능력치 상승 구조는 먼저 성장 대상 능력치를 정하는 것에서 시작합니다. 일반적으로 최대 체력, 공격력, 방어력, 이동 속도, 공격 속도, 치명타 확률 같은 값이 성장 대상이 될 수 있습니다. 하지만 모든 능력치를 성장시킬 필요는 없습니다. 액션 게임이라면 공격력과 체력만 증가해도 충분할 수 있고, RPG라면 방어력, 마나, 치명타, 스킬 피해량까지 포함할 수 있습니다. 장르와 핵심 재미에 맞는 능력치만 선택해야 합니다.

능력치 상승 방식도 정해야 합니다. 가장 단순한 방식은 레벨업할 때마다 고정값을 더하는 것입니다. 예를 들어 레벨업 시 최대 체력은 10, 공격력은 2씩 증가할 수 있습니다. 조금 더 세밀하게 만들고 싶다면 레벨별 성장 테이블을 사용할 수 있습니다. 성장 테이블을 사용하면 특정 구간에서 성장 폭을 크게 하거나 작게 조절할 수 있어 밸런싱에 유리합니다.

프로그래머 관점에서는 기본 능력치와 최종 능력치를 구분하는 것이 중요합니다. 기본 능력치는 캐릭터가 레벨에 따라 가지는 값이고, 최종 능력치는 장비, 버프, 패시브 효과가 모두 적용된 결과값입니다. 예를 들어 기본 공격력이 20이고 장비 공격력 5, 버프 20%가 적용된다면 최종 공격력은 별도의 계산 규칙으로 산출되어야 합니다. 이 구조를 미리 정리하지 않으면 장비와 버프 시스템이 추가될 때 코드가 복잡해집니다.

AI에게 능력치 상승 시스템을 요청할 때도 상승 규칙을 구체적으로 전달해야 합니다. “레벨업하면 강해지게 해줘”보다 “레벨업 시 MaxHp는 10, AttackPower는 2씩 증가한다. 최종 능력치는 기본 능력치에 장비 보너스를 더한 뒤 버프 배율을 적용한다”라고 작성하는 편이 좋습니다. 좋은 성장 기획은 레벨업 연출만 다루지 않습니다. 플레이어가 강해지는 규칙을 수치, 계산식, 데이터 구조로 정리하는 문서입니다.

Chapter 46. 아이템 종류와 속성 정의하기

아이템 시스템을 기획할 때 가장 먼저 해야 할 일은 아이템의 종류와 속성을 정의하는 것입니다. 아이템은 단순히 플레이어가 획득하는 물건이 아닙니다. 전투, 성장, 보상, 상점, 인벤토리, 장비 시스템과 연결되는 중요한 데이터입니다. 따라서 “아이템을 추가한다”라고만 작성하면 부족합니다. 어떤 종류의 아이템이 있고, 각 아이템이 어떤 값을 가지며, 어떤 조건에서 사용되는지 정리해야 합니다.

아이템 종류는 게임의 구조에 맞게 나누어야 합니다. 일반적으로 장비 아이템, 소비 아이템, 재료 아이템, 퀘스트 아이템, 재화 아이템처럼 구분할 수 있습니다. 장비 아이템은 캐릭터 능력치를 변경하고, 소비 아이템은 사용 즉시 효과를 발생시키며, 재료 아이템은 제작이나 강화에 사용될 수 있습니다. 아이템 종류가 명확하면 인벤토리 UI, 정렬 방식, 사용 조건, 저장 구조를 설계하기 쉬워집니다.

아이템 속성은 데이터 필드로 생각하면 좋습니다. 모든 아이템은 ID, 이름, 설명, 아이콘, 종류, 등급, 가격, 최대 보유 수량 같은 기본 속성을 가질 수 있습니다. 장비 아이템이라면 공격력 증가, 방어력 증가, 장착 부위가 필요할 수 있고, 소비 아이템이라면 회복량, 사용 가능 상태, 쿨타임이 필요할 수 있습니다. 재료 아이템이라면 제작에 필요한 수량이나 사용처를 함께 정의할 수 있습니다.

프로그래머 관점에서는 공통 속성과 종류별 속성을 구분하는 것이 중요합니다. 모든 아이템이 같은 필드를 가질 필요는 없습니다. 예를 들어 포션에는 공격력이 필요 없고, 검에는 회복량이 필요 없습니다. 따라서 공통 데이터에는 모든 아이템이 공유하는 정보를 넣고, 장비 데이터, 소비 데이터, 재료 데이터처럼 종류별 데이터를 분리하는 구조를 고려할 수 있습니다.

AI에게 아이템 시스템 구현을 요청할 때도 아이템 종류와 속성을 구체적으로 전달해야 합니다. “아이템 시스템을 만들어줘”보다 “아이템은 ID, 이름, 설명, 타입, 가격, 최대 보유 수량을 가지며, 장비 아이템은 공격력 보너스와 장착 부위를 추가로 가진다”라고 작성하는 편이 좋습니다. 좋은 아이템 기획은 아이템 이름 목록이 아니라, 게임 시스템과 연결될 데이터 구조를 정의하는 문서입니다.

Chapter 47. 장비, 소비, 재료 아이템 구분하기

아이템 시스템을 안정적으로 설계하려면 아이템을 종류별로 구분해야 합니다. 모든 아이템을 하나의 방식으로 처리하면 처음에는 단순해 보이지만, 기능이 늘어날수록 예외가 많아집니다. 장비 아이템, 소비 아이템, 재료 아이템은 사용 목적과 동작 방식이 다르기 때문에 상세 기획서에서도 분리해서 정의하는 것이 좋습니다.

장비 아이템은 캐릭터에게 장착되어 능력치를 변경하는 아이템입니다. 무기, 방어구, 장신구처럼 특정 슬롯에 장착되며, 공격력, 방어력, 체력, 치명타 확률 같은 수치를 증가시킬 수 있습니다. 기획서에는 장착 부위, 장착 가능 조건, 능력치 보너스, 장착 해제 조건을 함께 작성해야 합니다. 예를 들어 “검은 무기 슬롯에만 장착 가능하며, 장착 시 공격력 +10을 적용한다”처럼 구체적으로 정의합니다.

소비 아이템은 사용하면 효과가 발생하고 수량이 줄어드는 아이템입니다. 체력 포션, 마나 포션, 버프 물약, 폭탄 같은 아이템이 여기에 해당합니다. 소비 아이템을 기획할 때는 사용 조건, 효과, 수량 감소 방식, 쿨타임, 사용 불가능 상태를 정리해야 합니다. 예를 들어 체력이 가득 찬 상태에서 회복 포션을 사용할 수 있는지, 전투 중에도 사용할 수 있는지 같은 조건이 필요합니다.

재료 아이템은 직접 사용하는 것보다 제작, 강화, 교환 같은 다른 시스템에 사용되는 아이템입니다. 몬스터가 드랍하는 가죽, 광석, 보석, 조각 같은 아이템이 여기에 해당합니다. 재료 아이템은 즉시 효과를 만들지 않지만, 성장과 보상 구조를 연결하는 역할을 합니다. 기획서에는 재료의 획득처, 사용처, 필요 수량, 최대 보유 수량을 작성하는 것이 좋습니다.

AI에게 아이템 시스템 구현을 요청할 때도 아이템 종류를 명확히 나누어 전달해야 합니다. “아이템을 만들어줘”보다 “아이템은 장비, 소비, 재료 타입으로 구분한다. 장비는 슬롯에 장착되어 능력치를 변경하고, 소비 아이템은 사용 시 효과를 적용한 뒤 수량이 감소하며, 재료 아이템은 제작 시스템에서 사용된다”라고 요청하는 편이 좋습니다. 좋은 아이템 기획은 아이템을 이름별로 나열하는 것이 아니라, 사용 방식과 시스템 연결 방식에 따라 분류하는 문서입니다.

Chapter 48. 보상 테이블 작성하기

보상 시스템을 안정적으로 만들려면 보상을 감으로 정하지 말고 테이블로 정리해야 합니다. 플레이어가 적을 처치했을 때 무엇을 얻는지, 스테이지를 클리어했을 때 어떤 아이템이 지급되는지, 퀘스트를 완료했을 때 경험치와 재화를 얼마나 받는지 명확히 작성해야 합니다. 보상 테이블은 플레이어의 반복 행동과 성장 흐름을 연결하는 기준 문서입니다.

보상 테이블에는 보상 지급 조건과 보상 내용을 함께 작성해야 합니다. 예를 들어 “슬라임 처치 시 경험치 5, 골드 3, 낮은 확률로 젤리 조각 1개 지급”처럼 정리할 수 있습니다. 스테이지 보상이라면 클리어 보상, 최초 클리어 보상, 반복 클리어 보상을 구분하는 것이 좋습니다. 같은 스테이지라도 처음 클리어했을 때와 다시 플레이했을 때의 보상이 다를 수 있기 때문입니다.

프로그래머 관점에서 보상 테이블은 데이터 구조로 이어집니다. 보상 ID, 보상 타입, 아이템 ID, 수량, 확률, 지급 조건 같은 항목을 필드로 정의할 수 있습니다. 보상 타입은 경험치, 재화, 아이템, 장비, 스킬 포인트처럼 나눌 수 있습니다. 이렇게 작성하면 보상 시스템을 코드에 직접 하드코딩하지 않고 데이터 기반으로 처리할 수 있습니다.

보상 확률도 명확히 작성해야 합니다. 아이템이 항상 지급되는지, 일정 확률로 지급되는지, 여러 보상 중 하나만 선택되는지 구분해야 합니다. 예를 들어 드랍 보상은 “각 아이템을 독립 확률로 검사한다”와 “보상 목록 중 하나만 가중치로 선택한다”가 서로 다른 구조입니다. 이 차이를 적어두지 않으면 구현 결과가 기획 의도와 달라질 수 있습니다.

AI에게 보상 시스템 구현을 요청할 때도 테이블 구조를 함께 전달해야 합니다. “보상 시스템을 만들어 줘”보다 “보상 데이터는 RewardId, RewardType, ItemId, Amount, DropRate를 가지며, 적 처치 시 해당 보상 테이블을 조회해 지급한다”라고 요청하는 편이 좋습니다. 좋은 보상 기획은 플레이어에게 무엇을 줄지 정하는 문서이면서, 동시에 그 보상을 어떤 조건과 데이터로 지급할지 정의하는 개발 명세입니다.

Chapter 49. 드랍 확률과 획득 조건 작성하기

아이템 보상에서 중요한 것은 무엇을 지급할지뿐 아니라, 어떤 조건에서 얼마나 자주 지급할지를 정하는 것입니다. 적을 처치했을 때 아이템이 항상 나오는지, 일정 확률로 나오는지, 특정 스테이지에서만 나오는지에 따라 플레이어의 성장 속도와 반복 플레이 동기가 달라집니다. 그래서 상세 기획서에는 드랍 아이템 이름만 적는 것이 아니라 드랍 확률과 획득 조건을 함께 작성해야 합니다.

드랍 확률은 아이템이 지급될 가능성을 나타냅니다. 예를 들어 슬라임을 처치했을 때 골드는 100% 지급하고, 젤리 조각은 30%, 희귀 젤리는 5% 확률로 지급할 수 있습니다. 이때 각 아이템을 독립적으로 확률 검사할지, 보상 목록 중 하나만 선택할지 정해야 합니다. 독립 확률 방식은 여러 아이템이 동시에 나올 수 있고, 가중치 선택 방식은 목록 중 하나만 지급됩니다. 두 방식은 결과가 다르기 때문에 기획서에 명확히 구분해야 합니다.

획득 조건도 함께 정의해야 합니다. 특정 아이템은 특정 적에게서만 드랍될 수 있고, 특정 난이도 이상에서만 등장할 수도 있습니다. 보스 장비는 최초 클리어 시에만 지급하고, 일반 재료는 반복 클리어에서도 얻을 수 있게 만들 수 있습니다. 또한 플레이어 레벨, 퀘스트 진행 상태, 스테이지 지역, 이벤트 기간에 따라 드랍 조건을 다르게 설정할 수도 있습니다.

프로그래머 관점에서 드랍 확률과 획득 조건은 데이터 테이블로 정리하는 것이 좋습니다. 드랍 테이블에는 DropTableId, EnemyId, ItemId, DropRate, MinAmount, MaxAmount, Condition 같은 항목을 둘 수 있습니다. 이렇게 작성하면 적마다 보상을 코드에 직접 작성하지 않고 데이터로 관리할 수 있습니다. 밸런싱이 필요할 때도 코드 수정 없이 확률과 수량을 조정할 수 있습니다.

AI에게 드랍 시스템 구현을 요청할 때도 확률과 조건을 구체적으로 전달해야 합니다. “아이템 드랍 시스템을 만들어줘”보다 “적 처치 시 드랍 테이블을 조회하고, 각 보상 항목의 DropRate에 따라 독립 확률 검사를 수행한다. 조건을 만족한 아이템만 지급하며, 수량은 MinAmount와 MaxAmount 사이에서 랜덤으로 결정한다”라고 요청하는 편이 좋습니다. 좋은 드랍 기획은 보상의 기대감을 만들면서도, 획득 조건과 확률을 명확히 정의하는 문서입니다.

Chapter 50. 재화 시스템 기획하기

재화 시스템은 플레이어가 게임 안에서 보상을 모으고 사용하는 흐름을 관리하는 시스템입니다. 골드, 코인, 보석, 티켓, 에너지, 강화석처럼 게임마다 다양한 재화가 존재할 수 있습니다. 재화는 단순한 숫자가 아니라 상점, 강화, 제작, 뽑기, 입장 제한, 보상 시스템과 연결됩니다. 그래서 상세 기획서에는 어떤 재화가 있고, 어디서 얻으며, 어디에 사용하는지 명확히 작성해야 합니다.

먼저 재화의 종류를 정의해야 합니다. 기본 재화는 일반적으로 게임 플레이를 통해 자주 얻고 자주 사용하는 재화입니다. 예를 들어 골드는 적 처치, 스테이지 클리어, 퀘스트 보상으로 얻고, 상점 구매나 장비 강화에 사용할 수 있습니다. 희귀 재화는 획득량이 제한적이며, 특별한 아이템 구매나 고급 성장 요소에 사용될 수 있습니다. 재화마다 역할이 달라야 경제 구조가 단순히 숫자 누적에 그치지 않습니다.

재화 기획에서 중요한 것은 획득처와 사용처의 균형입니다. 플레이어가 재화를 얻을 수만 있고 쓸 곳이 부족하면 재화는 의미가 약해집니다. 반대로 사용할 곳은 많은데 획득량이 너무 적으면 답답함이 생깁니다. 따라서 기획서에는 재화별 획득 방법, 평균 획득량, 주요 사용처, 소비량을 함께 작성하는 것이 좋습니다. 예를 들어 “스테이지 1회 클리어 시 골드 100을 지급하고, 기본 장비 강화에는 골드 50을 사용한다”처럼 적을 수 있습니다.

프로그래머 관점에서는 재화를 데이터와 저장 구조로 봐야 합니다. 재화 ID, 이름, 최대 보유량, 표시 아이콘, 획득 가능 여부, 사용 가능 시스템 같은 속성을 정의할 수 있습니다. 또한 재화가 증가하거나 감소할 때 UI 갱신, 저장 처리, 부족 메시지, 중복 지급 방지 같은 로직도 필요합니다. 특히 재화는 플레이어 진행 상태와 직접 연결되므로 저장과 불러오기 기준을 반드시 정리해야 합니다.

AI에게 재화 시스템 구현을 요청할 때도 규칙을 구체적으로 전달해야 합니다. “골드 시스템을 만들어 줘”보다 “플레이어는 Gold를 보유하며, 적 처치 시 Gold가 증가하고, 상점 구매 시 가격만큼 감소한다. Gold가 부족하면 구매를 실패시키고 안내 메시지를 출력한다”라고 작성하는 편이 좋습니다. 좋은 재화 기획은 획득과 소비의 흐름을 정리하고, 게임의 성장과 보상 구조를 안정적으로 연결하는 문서입니다.

Chapter 51. 상점 시스템 상세 기획하기

상점 시스템은 플레이어가 획득한 재화를 사용하는 대표적인 구조입니다. 전투나 스테이지 클리어를 통해 얻은 골드, 코인, 보석 같은 재화를 아이템, 장비, 스킬, 강화 재료로 교환하게 만들면 보상과 성장 흐름이 자연스럽게 연결됩니다. 그래서 상점 시스템은 단순히 “아이템을 구매하는 기능”이 아니라, 재화 소비와 성장 선택을 관리하는 시스템으로 기획해야 합니다.

상점 시스템을 작성할 때는 먼저 판매 항목을 정의해야 합니다. 상점에서 어떤 아이템을 판매하는지, 가격은 얼마인지, 구매 가능 조건은 무엇인지 정리합니다. 예를 들어 체력 포션은 항상 구매 가능하지만, 고급 장비는 특정 스테이지를 클리어한 뒤에만 구매할 수 있습니다. 또한 한 번만 구매 가능한 아이템인지, 반복 구매 가능한 아이템인지도 함께 작성해야 합니다.

구매 흐름도 명확히 정의해야 합니다. 플레이어가 아이템을 선택하고, 구매 버튼을 누르고, 보유 재화가 충분한지 확인한 뒤, 재화를 차감하고 아이템을 인벤토리에 추가하는 순서입니다. 이때 재화가 부족하면 구매를 실패시키고 안내 메시지를 보여주어야 합니다. 인벤토리가 가득 찬 경우에도 구매를 막을지, 우편함 같은 별도 보관함으로 보낼지 정해야 합니다.

상점에는 예외 조건이 많습니다. 이미 보유한 장비를 다시 구매할 수 있는지, 구매 제한 수량이 있는지, 구매 후 환불이 가능한지, 판매 목록이 시간에 따라 갱신되는지 정리해야 합니다. 할인, 일일 구매 제한, 랜덤 상점, 이벤트 상점 같은 기능을 넣는다면 조건은 더 복잡해집니다. 따라서 처음에는 MVP 기준으로 기본 구매, 재화 부족 처리, 아이템 지급, UI 갱신 정도부터 정의하는 것이 좋습니다.

AI에게 상점 시스템 구현을 요청할 때도 구매 규칙과 예외 조건을 함께 전달해야 합니다. “상점 시스템을 만들어줘”보다 “상점은 아이템 ID, 가격, 구매 제한을 가진 판매 목록을 표시한다. 구매 시 Gold가 충분하면 Gold를 차감하고 인벤토리에 아이템을 추가하며, Gold가 부족하거나 인벤토리가 가득 차면 구매를 실패시키고 메시지를 출력한다”라고 작성하는 편이 좋습니다. 좋은 상점 기획은 판매 목록을 나열하는 문서가 아니라, 재화 소비와 아이템 지급 흐름을 명확히 정의하는 개발 명세입니다.

Chapter 52. 성장 시스템을 데이터 구조로 바꾸기

성장 시스템은 경험치, 레벨, 능력치 상승, 아이템 효과, 장비 보너스처럼 여러 데이터가 함께 움직이는 구조입니다. 기획서에 “레벨이 오르면 강해진다”라고만 적으면 구현 기준이 부족합니다. 프로그래머 관점에서는 어떤 데이터를 저장하고, 어떤 값을 계산하며, 어떤 시스템이 그 데이터를 참조하는지까지 정리해야 합니다. 성장 시스템은 결국 데이터 구조로 바뀌어야 안정적으로 구현할 수 있습니다.

먼저 플레이어의 성장 상태 데이터를 정의해야 합니다. 현재 레벨, 현재 경험치, 다음 레벨에 필요한 경험치, 누적 경험치, 현재 체력, 최대 체력, 기본 공격력, 방어력 같은 값이 여기에 해당합니다. 이 데이터는 저장과 불러오기에 직접 연결되므로 명확히 분리해야 합니다. 예를 들어 PlayerGrowthData에는 레벨과 경험치를 저장하고, PlayerStatsData에는 능력치 정보를 저장하는 식으로 나눌 수 있습니다.

다음으로 레벨별 성장 테이블이 필요합니다. 레벨마다 필요한 경험치, 최대 체력 증가량, 공격력 증가량, 해금되는 스킬이나 콘텐츠를 표로 정리할 수 있습니다. 이렇게 작성하면 코드에 레벨업 수치를 직접 넣지 않고 데이터 기반으로 처리할 수 있습니다. 밸런싱이 필요할 때도 코드 수정 없이 테이블 값만 조정하면 됩니다.

능력치 계산 구조도 함께 정의해야 합니다. 기본 능력치, 레벨 상승 보너스, 장비 보너스, 버프 효과를 어떤 순서로 계산할지 정리합니다. 예를 들어 최종 공격력은 기본 공격력에 레벨 보너스와 장비 보너스를 더한 뒤, 버프 배율을 곱해 계산할 수 있습니다. 이 계산 순서가 명확하지 않으면 장비나 버프 시스템이 추가될 때 결과가 달라질 수 있습니다.

AI에게 성장 시스템 구현을 요청할 때는 데이터 구조를 함께 제시해야 합니다. “성장 시스템을 만들어줘”보다 “플레이어 성장 데이터는 Level, CurrentExp, RequiredExp, BaseStats를 가지고, 레벨 테이블을 참조해 필요 경험치와 능력치 증가량을 계산한다”라고 요청하는 편이 좋습니다. 좋은 성장 기획은 성장 규칙을 설명하는 데서 끝나지 않고, 그 규칙을 저장 가능한 데이터와 계산 가능한 구조로 바꾸는 문서입니다.

Part 07. UI와 화면 흐름 상세 기획하기

Part 7에서는 게임 UI와 화면 전환 흐름을 상세 기획서로 작성하는 방법을 다룹니다.

UI는 단순히 화면을 꾸미는 요소가 아니라 플레이어가 게임 시스템과 상호작용하는 통로입니다.

이 파트에서는 타이틀, 로비, 인게임 HUD, 인벤토리, 상점, 설정 화면처럼 게임에 필요한 화면을 목록화하고, 각 화면의 버튼 동작, 표시 데이터, 팝업 조건, 전환 흐름을 정리합니다.

이를 통해 UI 구현 범위를 명확히 하고, AI에게 화면 구현을 요청할 때 필요한 정보를 체계적으로 준비할 수 있습니다.

Chapter 53. UI 기획은 화면 목록에서 시작한다

UI 기획은 버튼을 어디에 배치할지 정하는 일에서 시작하지 않습니다. 먼저 게임에 어떤 화면이 필요한지 정리해야 합니다. 타이틀 화면, 로비 화면, 인게임 HUD, 인벤토리, 상점, 결과 화면, 설정 화면처럼 플레이어가 거쳐 가는 화면을 목록으로 만들면 UI의 전체 구조가 보입니다. 화면 목록은 UI 기획의 출발점이자, UI 구현 범위를 정하는 기준입니다.

화면 목록을 작성할 때는 플레이어의 흐름을 기준으로 생각하는 것이 좋습니다. 플레이어는 게임을 실행한 뒤 타이틀 화면을 보고, 시작 버튼을 눌러 로비나 스테이지 선택 화면으로 이동합니다. 게임이 시작되면 인게임 HUD를 보고, 일시정지나 설정 화면을 열 수 있습니다. 게임이 끝나면 결과 화면에서 보상과 점수를 확인하고 다시 도전할 수 있습니다. 이 흐름을 따라가면 필요한 화면을 빠뜨릴 가능성이 줄어듭니다.

프로그래머 관점에서 화면 목록은 단순한 이름 목록이 아닙니다. 각 화면이 어떤 데이터를 표시하는지, 어떤 버튼을 가지는지, 어떤 시스템과 연결되는지 함께 정리해야 합니다. 예를 들어 인게임 HUD는 체력, 스킬 쿨타임, 재화, 미니맵 같은 데이터를 표시할 수 있고, 상점 화면은 보유 재화, 판매 아이템, 구매 버튼, 부족 메시지를 포함할 수 있습니다. 이런 정보가 있어야 UI 코드와 데이터 연결 구조를 설계할 수 있습니다.

화면 목록에는 화면 전환 조건도 포함해야 합니다. 타이틀에서 시작 버튼을 누르면 어디로 이동하는지, 인게임에서 사망하면 결과 화면으로 이동하는지, 상점에서 닫기 버튼을 누르면 로비로 돌아가는지 작성해야 합니다. 화면 전환 조건이 명확하지 않으면 UI 상태 관리가 복잡해지고, 팝업과 화면이 겹치거나 잘못 닫히는 문제가 생길 수 있습니다.

AI에게 UI 구현을 요청할 때도 화면 목록은 중요한 입력 자료입니다. “게임 UI를 만들어줘”라고 요청하기보다 “타이틀 화면, 로비 화면, 인게임 HUD, 결과 화면을 만들고, 각 화면의 버튼과 전환 조건은 다음과 같다”라고 설명해야 더 안정적인 결과를 얻을 수 있습니다. 좋은 UI 기획은 예쁜 화면을 상상하는 데서 끝나지 않습니다. 플레이어가 어떤 화면을 거쳐 게임을 경험하는지 구조화하는 일에서 시작합니다.

Chapter 54. 타이틀 화면 기획하기

타이틀 화면은 플레이어가 게임을 실행했을 때 가장 먼저 만나는 화면입니다. 그래서 타이틀 화면은 단순히 게임 제목을 보여주는 화면이 아니라, 게임의 첫인상을 만드는 진입점입니다. 플레이어는 이 화면에서 게임을 시작하고, 설정을 변경하고, 저장된 데이터를 불러오거나 종료할 수 있습니다. 따라서 타이틀 화면 기획은 화면 구성, 버튼 기능, 전환 흐름을 함께 정리해야 합니다.

타이틀 화면에는 보통 게임 로고나 제목, 시작 버튼, 이어하기 버튼, 설정 버튼, 종료 버튼이 포함됩니다. 게임에 따라 크레딧, 언어 선택, 계정 로그인, 서버 선택 같은 항목이 추가될 수 있습니다. 하지만 처음부터 많은 기능을 넣을 필요는 없습니다. MVP 단계에서는 게임 시작, 설정, 종료 정도만 있어도 충분합니다. 중요한 것은 플레이어가 다음 행동을 쉽게 이해할 수 있도록 화면의 역할을 단순하게 만드는 것입니다.

각 버튼의 동작도 명확히 작성해야 합니다. 시작 버튼을 누르면 새 게임을 시작하는지, 스테이지 선택 화면으로 이동하는지, 튜토리얼로 진입하는지 정해야 합니다. 이어하기 버튼은 저장 데이터가 있을 때만 활성화할지, 저장 데이터가 없으면 비활성화할지 결정해야 합니다. 설정 버튼은 설정 팝업을 여는지 별도 설정 화면으로 이동하는지도 정리해야 합니다.

프로그래머 관점에서는 타이틀 화면이 게임 상태와 어떻게 연결되는지 중요합니다. 타이틀 화면에서 게임 시작을 누르면 씬을 전환할지, 현재 씬에서 UI만 바꿀지 결정해야 합니다. 저장 데이터 확인, 버튼 활성화 상태, 로딩 처리, 입력 차단, 배경 음악 재생 같은 요소도 함께 고려해야 합니다. 단순한 버튼 UI처럼 보여도 실제로는 게임 흐름의 시작점이기 때문입니다.

AI에게 타이틀 화면 구현을 요청할 때는 화면 요소와 버튼 동작을 함께 전달해야 합니다. “타이틀 화면을 만들어줘”보다 “게임 제목, 시작 버튼, 이어하기 버튼, 설정 버튼, 종료 버튼을 포함하고, 이어하기 버튼은 저장 데이터가 있을 때만 활성화되며, 시작 버튼을 누르면 게임 씬으로 이동한다”라고 요청하는 편이 좋습니다. 좋은 타이틀 화면 기획은 첫 화면을 꾸미는 문서가 아니라, 플레이어가 게임에 진입하는 흐름을 정의하는 문서입니다.

Chapter 55. 로비 화면 기획하기

로비 화면은 플레이어가 게임을 본격적으로 시작하기 전에 머무는 중심 화면입니다. 타이틀 화면이 게임의 첫 진입점이라면, 로비 화면은 플레이어가 다음 행동을 선택하는 허브입니다. 스테이지 선택, 캐릭터 정보 확인, 장비 변경, 상점 이용, 퀘스트 확인, 설정 변경 같은 기능이 로비에서 연결될 수 있습니다. 그래서 로비 화면은 단순한 메뉴 화면이 아니라 게임의 주요 시스템을 연결하는 출발점으로 기획해야 합니다.

로비 화면을 작성할 때는 먼저 플레이어가 로비에서 무엇을 할 수 있는지 정리해야 합니다. 예를 들어 “스테이지 입장”, “캐릭터 성장 확인”, “인벤토리 열기”, “상점 열기”, “일일 보상 받기” 같은 기능을 목록으로 작성할 수 있습니다. 이 기능 목록이 곧 로비 화면의 버튼과 UI 구성 요소가 됩니다. MVP 단계에서는 스테이지 입장, 캐릭터 정보, 상점 정도만 넣고 나머지는 이후 확장으로 미룰 수 있습니다.

프로그래머 관점에서 로비 화면은 여러 시스템의 데이터를 보여주는 화면입니다. 플레이어 레벨, 보유 재화, 장착 장비, 진행 중인 퀘스트, 남은 에너지, 알림 상태 같은 정보를 표시할 수 있습니다. 따라서 로비 화면 기획서에는 어떤 데이터를 어디에 표시할지, 데이터가 변경되면 UI를 어떻게 갱신할지 함께 적어야 합니다. 예를 들어 상점에서 아이템을 구매해 골드가 줄어들면 로비의 보유 골드 표시도 갱신되어야 합니다.

로비 화면은 화면 전환 흐름도 중요합니다. 스테이지 버튼을 누르면 스테이지 선택 화면으로 이동할지, 바로 마지막 스테이지에 입장할지 정해야 합니다. 상점 버튼을 누르면 상점 팝업을 띄울지 별도 상점 화면으로 전환할지도 결정해야 합니다. 뒤로 가기 버튼을 누르면 타이틀 화면으로 돌아갈지, 게임 종료 확인 팝업을 띄울지도 기획서에 작성해야 합니다.

AI에게 로비 화면 구현을 요청할 때는 화면 요소, 표시 데이터, 버튼 동작을 함께 전달해야 합니다. “로비 UI를 만들어줘”보다 “로비 화면에는 플레이어 레벨, 보유 골드, 스테이지 입장 버튼, 인벤토리 버튼, 상점 버튼을 표시한다. 각 버튼을 누르면 해당 화면 또는 팝업을 연다”라고 작성하는 편이 좋습니다. 좋은 로비 화면 기획은 메뉴를 나열하는 문서가 아니라, 게임의 주요 시스템을 플레이어가 접근할 수 있도록 연결하는 흐름 문서입니다.

Chapter 56. 인게임 HUD 기획하기

인게임 HUD는 플레이어가 게임을 플레이하는 동안 계속 확인하는 화면 정보입니다. 체력, 스킬 쿨타임, 탄약, 점수, 미니맵, 목표, 보유 재화처럼 현재 플레이에 필요한 정보를 실시간으로 보여줍니다. HUD는 단순히 정보를 많이 표시하는 화면이 아닙니다. 플레이어가 지금 어떤 상태인지, 무엇을 해야 하는지, 어떤 위험이 있는지 빠르게 이해하도록 돕는 화면입니다.

HUD를 기획할 때는 먼저 플레이 중 반드시 필요한 정보를 구분해야 합니다. 액션 게임이라면 체력, 스킬 쿨타임, 적 위치, 현재 목표가 중요할 수 있습니다. 슈팅 게임이라면 탄약, 조준점, 남은 적 수, 점수가 필요할 수 있습니다. RPG라면 체력, 마나, 버프 상태, 퀘스트 목표, 미니맵이 중요할 수 있습니다. 장르에 따라 필요한 정보가 다르므로 모든 요소를 한 화면에 넣기보다 핵심 플레이에 필요한 정보만 우선 배치해야 합니다.

프로그래머 관점에서 HUD는 여러 시스템의 데이터를 실시간으로 받아 표시하는 UI입니다. 체력 UI는 Health 시스템과 연결되고, 스킬 쿨타임 UI는 Skill 시스템과 연결되며, 점수 UI는 Score 시스템과 연결됩니다. 따라서 HUD 기획서에는 어떤 UI 요소가 어떤 데이터와 연결되는지 명확히 작성해야 합니다. 예를 들어 “플레이어 체력이 변경되면 체력 바와 체력 숫자를 함께 갱신한다”처럼 데이터 갱신 조건을 적어두는 것이 좋습니다.

HUD는 표시 방식도 중요합니다. 체력은 바 형태로 보여줄지 숫자로 보여줄지, 스킬 쿨타임은 아이콘 위에 어두운 오버레이로 표시할지, 남은 시간을 숫자로 표시할지 정해야 합니다. 목표 정보는 항상 보여줄지, 일정 시간만 표시할지도 결정해야 합니다. 정보가 너무 많으면 플레이 화면을 가리고, 너무 적으면 플레이어가 상황을 파악하기 어렵습니다.

AI에게 HUD 구현을 요청할 때는 UI 요소와 연결 데이터를 함께 전달해야 합니다. “인게임 HUD를 만들어줘”보다 “HUD에는 체력 바, 스킬 아이콘 3개, 쿨타임 표시, 점수 텍스트, 일시정지 버튼을 포함한다. 체력이 변경되면 체력 바를 갱신하고, 스킬 사용 후 쿨타임 동안 아이콘에 남은 시간을 표시한다”라고 작성하는 편이 좋습니다. 좋은 HUD 기획은 화면을 꾸미는 문서가 아니라, 플레이 중 필요한 정보를 정확한 타이밍에 보여주는 정보 설계 문서입니다.

Chapter 57. 인벤토리 화면 기획하기

인벤토리 화면은 플레이어가 보유한 아이템을 확인하고 관리하는 화면입니다. 아이템을 획득하는 게임이라면 인벤토리는 단순한 목록 화면이 아니라, 성장과 선택을 연결하는 중요한 UI입니다. 플레이어는 이 화면에서 장비를 착용하고, 소비 아이템을 사용하고, 재료 아이템을 확인하며, 필요 없는 아이템을 판매하거나 정리할 수 있습니다. 따라서 인벤토리 화면은 아이템 데이터와 플레이어 행동을 함께 고려해 기획해야 합니다.

먼저 인벤토리에서 어떤 아이템을 보여줄지 정해야 합니다. 장비, 소비, 재료, 퀘스트 아이템을 모두 한 화면에 표시할지, 탭으로 나누어 보여줄지 결정합니다. 아이템 수가 적은 게임이라면 하나의 목록으로 충분할 수 있지만, 아이템 종류가 많다면 탭 분리가 필요합니다. 예를 들어 “전체, 장비, 소비, 재료” 탭을 만들면 플레이어가 원하는 아이템을 빠르게 찾을 수 있습니다.

아이템 목록에 표시할 정보도 정리해야 합니다. 아이콘, 이름, 등급, 수량, 장착 여부, 잠금 상태, 판매 가능 여부 같은 정보가 필요할 수 있습니다. 아이템을 선택했을 때는 상세 설명, 능력치 변화, 사용 효과, 판매 가격, 장착 버튼, 사용 버튼 등을 보여줄 수 있습니다. 특히 장비 아이템은 현재 장착 중인 아이템과 비교할 수 있어야 플레이어가 선택하기 쉽습니다.

프로그래머 관점에서 인벤토리 화면은 아이템 데이터와 UI 갱신 규칙이 중요합니다. 아이템을 획득하면 목록에 추가되고, 소비 아이템을 사용하면 수량이 감소하며, 장비를 착용하면 장착 상태가 변경됩니다. 이때 UI가 즉시 갱신되어야 합니다. 또한 아이템 수량이 0이 되었을 때 목록에서 제거할지, 비활성 상태로 남길지도 기획서에 작성해야 합니다.

AI에게 인벤토리 화면 구현을 요청할 때는 화면 구성과 동작 규칙을 함께 전달해야 합니다. “인벤토리 UI를 만들어줘”보다 “인벤토리 화면은 장비, 소비, 재료 탭으로 구분하며, 아이템 목록에는 아이콘, 이름, 수량을 표시한다. 아이템을 선택하면 상세 정보와 사용 또는 장착 버튼을 보여주고, 사용 후 수량이 0이 되면 목록에서 제거한다”라고 요청하는 편이 좋습니다. 좋은 인벤토리 기획은 아이템을 보여주는 화면이 아니라, 플레이어가 보유 자원을 이해하고 사용할 수 있게 만드는 관리 화면 설계입니다.

Chapter 58. 상점 화면 기획하기

상점 화면은 플레이어가 재화를 사용해 아이템을 구매하는 화면입니다. 로비나 인벤토리와 연결되는 경우가 많으며, 게임의 보상 구조와 성장 흐름을 소비 구조로 이어주는 역할을 합니다. 플레이어가 모은 골드나 코인을 어디에 사용할 수 있는지 보여주고, 필요한 아이템을 선택해 구매할 수 있게 해야 합니다. 따라서 상점 화면은 판매 목록, 가격 표시, 보유 재화, 구매 조건을 함께 기획해야 합니다.

먼저 상점에서 어떤 상품을 보여줄지 정해야 합니다. 체력 포션, 장비, 강화 재료, 스킬 아이템, 특수 아이템처럼 판매 항목을 분류할 수 있습니다. 상품이 많다면 “소비”, “장비”, “재료”, “특수”처럼 탭을 나누는 것이 좋습니다. 상품이 적은 MVP 단계라면 하나의 목록으로 시작해도 충분합니다. 중요한 것은 플레이어가 무엇을 살 수 있는지 빠르게 이해하도록 구성하는 것입니다.

상품 정보에는 아이콘, 이름, 설명, 가격, 보유 수량, 구매 가능 여부가 포함될 수 있습니다. 구매 제한이 있는 상품이라면 남은 구매 횟수도 표시해야 합니다. 예를 들어 “체력 포션 100골드, 보유 수량 3개, 구매 가능”처럼 보여줄 수 있습니다. 재화가 부족한 상품은 구매 버튼을 비활성화하거나, 클릭 시 “골드가 부족합니다”라는 메시지를 출력하도록 기획할 수 있습니다.

프로그래머 관점에서 상점 화면은 재화 시스템, 아이템 데이터, 인벤토리 시스템과 연결됩니다. 구매 버튼을 누르면 보유 재화를 확인하고, 가격만큼 차감한 뒤, 아이템을 인벤토리에 추가해야 합니다. 구매 성공 후에는 보유 재화 표시와 상품 상태, 인벤토리 데이터가 함께 갱신되어야 합니다. 인벤토리가 가득 찬 경우 구매를 막을지, 별도 보관함으로 보낼지도 미리 정해야 합니다.

AI에게 상점 화면 구현을 요청할 때는 UI 구성과 구매 흐름을 함께 전달해야 합니다. “상점 UI를 만들어줘”보다 “상점 화면에는 보유 골드, 상품 목록, 상품 아이콘, 가격, 구매 버튼을 표시한다. 구매 시 골드가 충분하면 골드를 차감하고 아이템을 인벤토리에 추가하며, 부족하면 안내 메시지를 출력한다”라고 요청하는 편이 좋습니다. 좋은 상점 화면 기획은 상품을 보여주는 화면을 넘어서, 재화 소비와 아이템 획득 흐름을 명확히 연결하는 UI 설계입니다.

Chapter 59. 설정 화면 기획하기

설정 화면은 플레이어가 게임 환경을 자신의 취향에 맞게 조정하는 화면입니다. 사운드 크기, 그래픽 품질, 조작 방식, 언어, 진동, 알림 같은 항목을 변경할 수 있습니다. 설정 화면은 게임의 핵심 재미를 직접 만들지는 않지만, 플레이 경험의 편의성과 접근성을 높이는 중요한 UI입니다. 따라서 설정 화면은 단순한 옵션 목록이 아니라, 플레이어가 불편함 없이 게임을 조정할 수 있게 만드는 화면으로 기획해야 합니다.

먼저 어떤 설정 항목이 필요한지 정해야 합니다. 기본적으로 배경음, 효과음, 전체 음량 같은 사운드 설정을 포함할 수 있습니다. 모바일 게임이라면 진동, 터치 조작 감도, 절전 모드가 필요할 수 있고, PC 게임이라면 해상도, 전체 화면 여부, 그래픽 품질, 키 설정이 필요할 수 있습니다. MVP 단계에서는 사운드 설정과 종료 또는 저장 관련 옵션 정도만 넣고, 복잡한 그래픽 설정은 이후 확장으로 미룰 수 있습니다.

설정 항목마다 입력 방식도 정리해야 합니다. 음량은 슬라이더로 조절할 수 있고, 전체 화면 여부는 토글로 처리할 수 있습니다. 언어 선택은 드롭다운이나 버튼 목록으로 구성할 수 있습니다. 조작 방식은 “키보드”, “게임패드”, “터치”처럼 선택형 옵션이 될 수 있습니다. 기획서에는 각 설정 항목의 기본값, 변경 가능 범위, 저장 여부를 함께 작성하는 것이 좋습니다.

프로그래머 관점에서 설정 화면은 저장 데이터와 직접 연결됩니다. 플레이어가 음량을 변경하면 즉시 사운드 시스템에 반영하고, 게임을 다시 실행해도 이전 설정이 유지되어야 합니다. 설정값을 언제 저장할지도 정해야 합니다. 옵션을 바꾸는 즉시 저장할지, 확인 버튼을 눌렀을 때만 저장할지에 따라 구현 방식이 달라집니다. 취소 버튼이 있다면 변경 전 값으로 되돌리는 규칙도 필요합니다.

AI에게 설정 화면 구현을 요청할 때는 설정 항목과 저장 규칙을 함께 전달해야 합니다. “설정 UI를 만들어줘”보다 “설정 화면에는 BGM 볼륨 슬라이더, SFX 볼륨 슬라이더, 전체 화면 토글, 언어 선택 버튼을 포함한다. 값이 변경되면 즉시 적용하고, 확인 버튼을 누르면 저장한다”라고 요청하는 편이 좋습니다. 좋은 설정 화면 기획은 옵션을 나열하는 문서가 아니라, 플레이어의 환경 설정을 안전하게 적용하고 저장하는 흐름을 정의하는 문서입니다.

Chapter 60. 팝업, 토스트, 확인창 규칙 작성하기

게임 UI에서는 모든 정보를 하나의 화면에 표시할 수 없습니다. 때로는 플레이어에게 짧은 안내를 보여주고, 때로는 선택을 요구하며, 때로는 중요한 행동을 다시 확인해야 합니다. 이때 사용하는 UI가 팝업, 토스트, 확인창입니다. 세 가지는 비슷해 보이지만 역할이 다르기 때문에 상세 기획서에서 사용 규칙을 명확히 나누어야 합니다.

토스트는 짧은 안내 메시지를 보여줄 때 사용합니다. 예를 들어 “골드가 부족합니다”, “아이템을 획득했습니다”, “저장되었습니다”처럼 플레이어가 확인 버튼을 누르지 않아도 되는 정보에 적합합니다. 토스트는 일정 시간이 지나면 자동으로 사라져야 하며, 플레이 흐름을 막지 않는 것이 좋습니다. 따라서 토스트 기획에는 표시 위치, 표시 시간, 중복 메시지 처리 방식을 작성해야 합니다.

팝업은 플레이어에게 추가 정보를 보여주거나 별도 기능을 제공할 때 사용합니다. 상점 구매 결과, 아이템 상세 정보, 퀘스트 완료 보상, 설정 화면 같은 UI가 팝업 형태로 열릴 수 있습니다. 팝업은 배경 화면 위에 표시되므로, 뒤쪽 화면을 조작할 수 있는지 막을 것인지 정해야 합니다. 또한 팝업이 여러 개 겹칠 수 있는지, 하나만 열 수 있는지도 규칙으로 정리해야 합니다.

확인창은 플레이어의 중요한 선택을 다시 확인할 때 사용합니다. 아이템 판매, 장비 강화, 게임 종료, 데이터 초기화처럼 되돌리기 어렵거나 재화를 소비하는 행동에는 확인창이 필요합니다. 확인창에는 확인 버튼과 취소 버튼을 명확히 제공해야 하며, 어떤 버튼이 기본 선택인지도 정할 수 있습니다. 중요한 행동일수록 메시지를 구체적으로 작성해야 합니다. “진행할까요?”보다 “골드 500을 사용해 장비를 강화하시겠습니까?”가 더 명확합니다.

AI에게 UI 규칙 구현을 요청할 때는 세 UI의 역할을 구분해 전달해야 합니다. “팝업 시스템을 만들어줘”보다 “토스트는 자동으로 사라지는 짧은 안내 메시지, 팝업은 별도 정보를 표시하는 창, 확인창은 확인과 취소 선택이 필요한 창으로 구분한다. 확인창이 열려 있을 때는 뒤쪽 UI 입력을 막는다”라고 요청하는 편이 좋습니다. 좋은 UI 기획은 화면을 만드는 것뿐 아니라, 어떤 상황에서 어떤 방식으로 플레이어에게 정보를 전달할지 정하는 규칙 문서입니다.

Chapter 61. 화면 전환 흐름도 작성하기

화면 전환 흐름도는 플레이어가 어떤 화면에서 어떤 화면으로 이동하는지 정리한 문서입니다. 게임 UI는 화면을 하나씩 따로 기획하는 것만으로는 부족합니다. 타이틀 화면에서 로비로 이동하고, 로비에서 상점이나 인벤토리로 이동하며, 인게임에서 결과 화면으로 이어지는 전체 흐름이 필요합니다. 이 흐름이 명확해야 UI 구현 중 화면 이동이 꼬이지 않습니다.

먼저 주요 화면을 노드처럼 나열합니다. 타이틀, 로비, 스테이지 선택, 인게임 HUD, 일시정지, 결과 화면, 인벤토리, 상점, 설정 화면처럼 플레이어가 실제로 이동할 수 있는 화면을 정리합니다. 그다음 각 화면 사이의 이동 조건을 화살표로 연결합니다. 예를 들어 “타이틀 → 시작 버튼 → 로비”, “로비 → 스테이지 입장 → 인게임”, “인게임 → 플레이어 사망 → 결과 화면”처럼 작성할 수 있습니다.

화면 전환 흐름도에는 버튼 입력뿐 아니라 시스템에 의한 자동 전환도 포함해야 합니다. 플레이어가 사망했을 때 결과 화면으로 이동하거나, 스테이지 클리어 후 보상 팝업이 열리는 경우가 여기에 해당합니다. 또한 로딩 화면을 거쳐 이동하는지, 팝업만 열리는지, 씬 자체가 바뀌는지도 구분해야 합니다. 화면 전환과 팝업 호출은 구현 방식이 다르기 때문에 기획서에서 분리해두는 것이 좋습니다.

프로그래머 관점에서는 화면 전환 흐름도가 UI 상태 관리 기준이 됩니다. 현재 열린 화면을 닫고 새 화면을 열지, 이전 화면을 유지한 채 팝업을 추가로 띄울지 정해야 합니다. 뒤로 가기 버튼을 눌렀을 때 어디로 돌아갈지도 중요합니다. 예를 들어 상점 화면에서 뒤로 가기를 누르면 로비로 돌아가지만, 상점 구매 확인창에서는 확인창만 닫혀야 합니다. 이런 규칙이 없으면 UI 스택 관리가 복잡해집니다.

AI에게 화면 전환 구현을 요청할 때는 흐름도를 문장으로 풀어 전달해야 합니다. “UI 화면 전환을 만들어줘”보다 “타이틀 화면에서 시작 버튼을 누르면 로비 화면으로 이동하고, 로비에서 상점 버튼을 누르면 상점 팝업을 연다. 인게임에서 플레이어가 사망하면 결과 화면으로 전환한다”라고 작성하는 편이 좋습니다. 좋은 화면 전환 기획은 화면 목록을 연결해 플레이어의 이동 경로와 UI 상태 변화를 명확히 정의하는 문서입니다.

Chapter 62. UI 기획서를 구현 요청 프롬프트로 바꾸기

UI 기획서는 화면 구성과 흐름을 정리하는 문서이지만, AI와 함께 개발할 때는 이 문서를 구현 요청 프롬프트로 바꾸는 과정이 필요합니다. AI는 “상점 UI를 만들어줘” 같은 짧은 요청만으로는 기획 의도를 정확히 알기 어렵습니다. 어떤 화면 요소가 필요하고, 어떤 데이터를 표시하며, 버튼을 누르면 어떤 동작이 발생하는지 구체적으로 전달해야 합니다. 좋은 UI 프롬프트는 화면 설명이 아니라 구현 조건을 정리한 요청문입니다.

먼저 화면의 목적을 한 문장으로 설명합니다. 예를 들어 “상점 화면은 플레이어가 보유 골드를 사용해 아이템을 구매하는 화면이다”처럼 작성합니다. 그다음 화면에 포함될 UI 요소를 나열합니다. 보유 골드 텍스트, 상품 목록, 상품 아이콘, 가격 텍스트, 구매 버튼, 닫기 버튼처럼 실제 구현해야 할 요소를 구체적으로 적어야 합니다. 이 단계가 빠지면 AI는 임의로 화면 구성을 만들 가능성이 높습니다.

다음으로 각 UI 요소가 어떤 데이터와 연결되는지 작성해야 합니다. 보유 골드 텍스트는 `PlayerCurrencyData`의 `Gold` 값을 표시하고, 상품 목록은 `ShopItemData` 리스트를 기반으로 생성하며, 가격 텍스트는 각 상품의 `Price` 값을 표시한다는 식입니다. 프로그래머 관점에서는 이 연결 정보가 중요합니다. UI는 단순한 오브젝트 배치가 아니라 데이터와 상호작용하는 구조이기 때문입니다.

버튼 동작과 예외 조건도 프롬프트에 포함해야 합니다. 구매 버튼을 눌렀을 때 골드가 충분하면 재화를 차감하고 아이템을 인벤토리에 추가하며, 골드가 부족하면 토스트 메시지를 출력하도록 작성할 수 있습니다. 닫기 버튼을 누르면 이전 화면으로 돌아가거나 팝업을 닫는 규칙도 필요합니다. 입력 차단, 중복 클릭 방지, 화면 갱신 조건 같은 세부 동작도 구현 품질에 영향을 줍니다.

AI에게 UI 구현을 요청할 때는 “화면 목적 → UI 요소 → 연결 데이터 → 버튼 동작 → 예외 조건” 순서로 작성하는 것이 좋습니다. 예를 들어 “상점 화면을 구현한다. 보유 골드, 상품 목록, 구매 버튼을 표시한다. 상품 데이터는 `ShopItemData`를 사용한다. 구매 시 골드가 충분하면 아이템을 지급하고, 부족하면 ‘골드가 부족합니다’ 토스트를 표시한다”처럼 요청할 수 있습니다. 좋은 UI 기획서는 AI에게 그대로 붙여 넣는 문서가 아니라, 구현자가 이해할 수 있는 명확한 작업 지시로 변환되어야 합니다.

Part 08. 레벨과 스테이지 기획하기

Part 8에서는 레벨과 스테이지를 어떻게 문서화할지 다룹니다.

스테이지는 단순한 배경이나 맵이 아니라 플레이어가 목표를 수행하고, 장애물을 만나고, 보상을 얻는 경험의 구조입니다.

이 파트에서는 스테이지 목표, 진행 흐름, 오브젝트 배치, 적 배치, 난이도 곡선, 튜토리얼 구성, 체크 포인트와 실패 조건을 정리하는 방법을 설명합니다.

특히 레벨 기획을 표와 조건 중심으로 작성하여 프로그래머가 실제 배치와 구현 작업에 활용할 수 있도록 만드는 데 초점을 둡니다.

Chapter 63. 레벨 디자인 문서가 필요한 이유

레벨 디자인 문서는 플레이어가 어떤 공간에서 어떤 순서로 게임을 경험하는지 정리한 문서입니다. 여기서 말하는 레벨은 단순히 배경 맵만 의미하지 않습니다. 스테이지 구조, 적 배치, 오브젝트 위치, 목표, 실패 조건, 보상, 난이도 흐름까지 포함합니다. 게임 시스템이 잘 만들어져 있어도 레벨 구성이 불명확하면 플레이어는 무엇을 해야 하는지 알기 어렵고, 재미의 흐름도 끊길 수 있습니다.

레벨 디자인 문서가 필요한 이유는 구현 전에 플레이 경험을 예측할 수 있기 때문입니다. 예를 들어 첫 번째 구간에서는 이동을 익히고, 두 번째 구간에서는 적과 전투하며, 세 번째 구간에서는 보상을 얻고, 마지막 구간에서는 보스와 싸우는 식으로 흐름을 정리할 수 있습니다. 이런 구조가 없으면 스테이지는 단순히 오브젝트를 배치한 공간이 되고, 플레이어의 학습과 긴장감이 자연스럽게 이어지지 않습니다.

프로그래머 관점에서 레벨 디자인 문서는 배치 데이터와 이벤트 조건을 정리하는 기준이 됩니다. 어떤 위치에 적을 배치할지, 문은 어떤 조건에서 열리는지, 체크포인트는 어디에 둘지, 스테이지 클리어는 어떤 조건으로 판단할지 작성해야 합니다. 이 정보가 있어야 맵 제작, 스폰 시스템, 트리거 이벤트, 보상 지급 로직을 안정적으로 구현할 수 있습니다.

레벨 디자인 문서는 난이도 조절에도 도움이 됩니다. 적이 너무 많이 배치되면 초반부터 어렵고, 보상이 너무 늦게 나오면 플레이 동기가 떨어질 수 있습니다. 반대로 위험 요소 없이 보상만 많으면 긴장감이 사라집니다. 따라서 각 구간의 목표, 적 수, 보상, 회복 지점, 실패 가능성을 함께 정리해야 합니다. 레벨 디자인은 공간을 꾸미는 일이 아니라 플레이어의 감정 흐름을 설계하는 일입니다.

AI에게 레벨 구현을 요청할 때도 레벨 디자인 문서가 필요합니다. “스테이지를 만들어줘”보다 “스테이지는 시작 구간, 전투 구간, 보상 구간, 보스 구간으로 구성한다. 전투 구간에는 적 5마리를 배치하고, 모든 적을 처치하면 다음 문이 열린다”라고 요청하는 편이 좋습니다. 좋은 레벨 디자인 문서는 맵의 모양을 설명하는 문서가 아니라, 플레이어가 어떤 순서로 배우고 도전하고 보상받는지 정의하는 개발 기준 문서입니다.

Chapter 64. 스테이지 목표 작성하기

스테이지 목표는 플레이어가 해당 스테이지에서 무엇을 해야 하는지 알려주는 기준입니다. 적을 모두 처치해야 하는지, 특정 지점까지 이동해야 하는지, 제한 시간 안에 탈출해야 하는지, 보스를 쓰러뜨려야 하는지에 따라 스테이지의 구조와 플레이 방식이 달라집니다. 목표가 명확하지 않으면 플레이어는 어디로 가야 하는지, 무엇을 성공으로 판단해야 하는지 알기 어렵습니다.

스테이지 목표는 가능한 한 구체적으로 작성해야 합니다. “스테이지를 클리어한다”는 표현은 너무 넓습니다. “모든 적을 처치한다”, “제한 시간 3분 안에 목적지에 도착한다”, “보스 몬스터의 체력을 0으로 만든다”, “열쇠를 획득한 뒤 출구로 이동한다”처럼 성공 조건을 명확히 적어야 합니다. 이 목표가 곧 클리어 조건과 UI 안내 문구, 이벤트 트리거의 기준이 됩니다.

목표는 하나만 있을 수도 있고 여러 단계로 나뉘는 수도 있습니다. 예를 들어 첫 번째 목표는 “열쇠 찾기”, 두 번째 목표는 “잠긴 문 열기”, 세 번째 목표는 “출구 도달”이 될 수 있습니다. 이런 단계형 목표를 사용하면 플레이어가 스테이지를 자연스럽게 따라가도록 유도할 수 있습니다. 반대로 단순한 액션 게임이라면 “모든 적 처치”처럼 하나의 목표만으로도 충분할 수 있습니다.

프로그래머 관점에서 스테이지 목표는 상태와 조건으로 구현됩니다. 목표가 시작 전인지, 진행 중인지, 완료되었는지, 실패했는지 관리해야 합니다. 예를 들어 KillAllEnemies 목표라면 남은 적 수가 0이 되었을 때 완료 처리하고, ReachGoal 목표라면 플레이어가 목표 지점 트리거에 들어왔을 때 완료 처리할 수 있습니다. 따라서 기획서에는 목표 타입, 완료 조건, 실패 조건, 완료 후 동작을 함께 작성하는 것이 좋습니다.

AI에게 스테이지 목표 구현을 요청할 때는 목표 조건을 구체적으로 전달해야 합니다. “스테이지 클리어 기능을 만들어줘”보다 “스테이지 목표는 모든 적 처치이며, EnemyManager의 남은 적 수가 0이 되면 클리어 팝업을 표시하고 보상을 지급한다”라고 작성하는 편이 좋습니다. 좋은 스테이지 목표 기획은 플레이어의 목적을 명확히 보여주고, 개발자가 성공과 실패를 판단할 수 있는 조건을 제공하는 문서입니다.

Chapter 65. 스테이지 구조와 진행 흐름 작성하기

스테이지 목표를 정했다면 다음에는 스테이지가 어떤 구조로 진행되는지 작성해야 합니다. 플레이어가 어디에서 시작하고, 어떤 구간을 지나며, 어떤 이벤트를 만나고, 어디에서 클리어하는지를 정리하는 과정입니다. 스테이지 구조와 진행 흐름이 명확해야 맵 배치, 적 스폰, 오브젝트 배치, 이벤트 트리거를 안정적으로 구현할 수 있습니다.

스테이지 구조는 구간 단위로 나누어 작성하는 것이 좋습니다. 예를 들어 시작 구간, 학습 구간, 전투 구간, 보상 구간, 보스 구간, 탈출 구간처럼 나눌 수 있습니다. 각 구간에는 플레이어가 해야 할 행동이 있어야 합니다. 시작 구간에서는 조작을 확인하고, 전투 구간에서는 적을 처치하며, 보상 구간에서는 아이템을 얻고, 보스 구간에서는 최종 도전을 하게 만들 수 있습니다.

진행 흐름은 플레이어가 어떤 순서로 경험하는지를 설명합니다. “시작 지점에서 이동한다 → 첫 번째 적을 만난다 → 모든 적을 처치하면 문이 열린다 → 보상 상자를 획득한다 → 보스 방에 진입한다 → 보스를 처치하면 클리어된다”처럼 작성할 수 있습니다. 이 흐름은 플레이어의 실제 플레이 순서를 기준으로 작성해야 합니다. 개발자는 이 흐름을 보고 트리거, 조건, 연출, 보상 지급 시점을 구현할 수 있습니다.

프로그래머 관점에서는 스테이지 흐름을 상태와 이벤트로 바꾸어 생각해야 합니다. 스테이지 시작, 전투 시작, 전투 완료, 문 열림, 보상 획득, 보스 등장, 클리어 같은 상태를 정의할 수 있습니다. 각 상태는 특정 조건이 만족되면 다음 상태로 넘어갑니다. 예를 들어 “남은 적 수가 0이면 문 열림 상태로 전환한다”처럼 조건을 명확히 작성해야 합니다.

AI에게 스테이지 진행 흐름 구현을 요청할 때는 구간과 조건을 함께 전달해야 합니다. “스테이지를 만들어줘”보다 “스테이지는 시작 구간, 전투 구간, 보상 구간, 보스 구간으로 구성한다. 전투 구간의 적을 모두 처치하면 문이 열리고, 보스 구간에서 보스를 처치하면 클리어 처리한다”라고 요청하는 편이 좋습니다. 좋은 스테이지 기획은 맵의 생김새만 설명하는 것이 아니라, 플레이어가 어떤 순서로 도전하고 보상받는지를 구조화하는 문서입니다.

Chapter 66. 오브젝트 배치 기준 정하기

스테이지를 만들 때 오브젝트를 아무 위치에나 배치하면 플레이 흐름이 쉽게 무너집니다. 적, 장애물, 보상 상자, 회복 아이템, 문, 스위치, 체크포인트 같은 오브젝트는 각각의 역할에 맞는 위치가 필요합니다. 오브젝트 배치 기준은 플레이어가 어디에서 무엇을 보고, 어떤 행동을 하며, 어떤 위험과 보상을 경험할지 정하는 규칙입니다.

먼저 오브젝트의 역할을 구분해야 합니다. 적은 플레이어에게 도전을 제공하고, 장애물은 이동 경로를 제한하거나 판단을 요구합니다. 보상 상자는 목표 달성 후 만족감을 주고, 회복 아이템은 다음 전투를 준비하게 만듭니다. 체크포인트는 실패했을 때 다시 시작할 위치를 정합니다. 같은 오브젝트라도 어느 위치에 배치하느냐에 따라 플레이 경험이 달라집니다.

오브젝트 배치는 플레이어의 시야와 이동 경로를 기준으로 정하는 것이 좋습니다. 중요한 목표물은 플레이어가 자연스럽게 볼 수 있는 위치에 배치하고, 위험 요소는 너무 갑작스럽게 등장하지 않도록 해야 합니다. 적을 배치할 때도 시작 지점 바로 앞에 강한 적을 두기보다, 플레이어가 조작에 익숙해진 뒤 만날 수 있도록 흐름을 조절해야 합니다. 배치는 난이도와 학습 순서를 함께 설계하는 과정입니다.

프로그래머 관점에서는 오브젝트 배치 기준이 좌표, 프리팹, 스폰 포인트, 트리거 영역으로 연결됩니다. 기획서에는 어떤 오브젝트를 어느 구간에 배치할지, 고정 배치인지 동적 스폰인지, 플레이어가 어떤 조건을 만족했을 때 활성화되는지 작성해야 합니다. 예를 들어 “전투 구간 진입 시 적 3마리를 지정된 스폰 포인트에 생성한다”처럼 구현 가능한 문장으로 정리할 수 있습니다.

AI에게 오브젝트 배치 구현을 요청할 때는 배치 의도와 조건을 함께 전달해야 합니다. “맵에 적과 아이템을 배치해줘”보다 “시작 구간에는 튜토리얼 안내 오브젝트를 배치하고, 첫 전투 구간에는 근접 적 2마리와 원거리 적 1마리를 스폰한다. 전투 완료 후 보상 상자를 활성화한다”라고 요청하는 편이 좋습니다. 좋은 오브젝트 배치 기획은 맵을 채우는 작업이 아니라, 플레이어의 동선과 경험을 설계하는 문서입니다.

Chapter 67. 적 배치와 난이도 곡선 작성하기

적 배치는 스테이지 난이도를 결정하는 핵심 요소입니다. 같은 적이라도 어디에, 몇 마리나, 어떤 순서로 등장하느냐에 따라 플레이 경험은 크게 달라집니다. 초반부터 강한 적을 많이 배치하면 플레이어는 조작을 익히기 전에 실패할 수 있고, 반대로 너무 쉬운 적만 반복되면 긴장감이 사라집니다. 따라서 적 배치는 단순한 배치 작업이 아니라 난이도 흐름을 설계하는 일입니다.

난이도 곡선은 스테이지가 진행될수록 도전이 어떻게 변하는지 보여주는 기준입니다. 처음에는 약한 적 1~2마리로 조작과 전투 방식을 익히게 하고, 중반에는 적 수를 늘리거나 다른 공격 패턴을 가진 적을 섞을 수 있습니다. 후반에는 강한 적, 좁은 공간, 제한 시간, 보스 패턴처럼 플레이어에게 더 높은 집중을 요구하는 요소를 배치할 수 있습니다. 중요한 것은 난이도가 갑자기 튀지 않게 점진적으로 상승하도록 만드는 것입니다.

적 배치를 작성할 때는 적의 종류, 수량, 위치, 등장 조건을 함께 정리해야 합니다. 예를 들어 “전투 구간 1에는 근접 적 2마리, 전투 구간 2에는 근접 적 3마리와 원거리 적 1마리, 보스 전에는 회복 아이템 1개를 배치한다”처럼 쓸 수 있습니다. 이 정보가 있어야 스폰 포인트, 적 프리팹, 트리거 이벤트를 구현할 수 있습니다.

프로그래머 관점에서는 적 배치를 데이터화하는 방식도 고려해야 합니다. `EnemyId`, `SpawnPointId`, `SpawnCount`, `SpawnCondition`, `WaveIndex` 같은 항목을 사용하면 적 배치를 테이블로 관리할 수 있습니다. 웨이브 방식 게임이라면 몇 번째 웨이브에 어떤 적이 몇 마리 등장하는지 작성해야 하고, 탐험형 스테이지라면 위치 기반 배치와 트리거 조건을 함께 정의해야 합니다.

AI에게 적 배치와 난이도 흐름 구현을 요청할 때는 적의 종류와 등장 조건을 구체적으로 전달해야 합니다. “스테이지에 적을 배치해줘”보다 “초반 구간에는 근접 적 2마리만 배치하고, 중반 구간에는 근접 적 3마리와 원거리 적 1마리를 배치한다. 플레이어가 전투 구간에 진입하면 해당 적들을 스폰하고, 모두 처치하면 다음 문을 연다”라고 요청하는 편이 좋습니다. 좋은 적 배치 기획은 적을 많이 놓는 문서가 아니라, 플레이어가 점진적으로 배우고 도전하도록 난이도 곡선을 설계하는 문서입니다.

Chapter 68. 튜토리얼 스테이지 기획하기

튜토리얼 스테이지는 플레이어가 게임의 기본 규칙을 자연스럽게 익히는 첫 번째 학습 구간입니다. 단순히 조작 방법을 설명하는 화면이 아니라, 플레이어가 직접 움직이고, 공격하고, 회피하고, 목표를 달성하면서 게임 방식을 이해하도록 만드는 스테이지입니다. 좋은 튜토리얼은 설명을 많이 보여주는 것이 아니라, 플레이어가 행동을 통해 배울 수 있게 설계합니다.

튜토리얼 스테이지를 기획할 때는 한 번에 너무 많은 기능을 가르치지 않는 것이 중요합니다. 먼저 이동을 익히고, 다음에는 점프나 대시를 사용하게 하며, 그다음에는 적을 공격하거나 아이템을 획득하게 만들 수 있습니다. 각 기능은 별도의 짧은 구간으로 나누어 배치하는 것이 좋습니다. 예를 들어 첫 구간은 이동, 두 번째 구간은 공격, 세 번째 구간은 회복 아이템 사용, 마지막 구간은 간단한 전투로 구성할 수 있습니다.

튜토리얼에서는 실패 가능성을 낮게 설계해야 합니다. 처음 만나는 적은 공격 속도가 느리거나 체력이 낮아야 하며, 플레이어가 실수해도 다시 시도할 수 있어야 합니다. 위험 요소를 넘더라도 플레이어가 규칙을 이해한 뒤 등장시키는 편이 좋습니다. 튜토리얼의 목적은 플레이어를 시험하는 것이 아니라, 앞으로의 게임을 플레이할 준비를 시키는 것입니다.

프로그래머 관점에서 튜토리얼 스테이지는 안내 메시지, 트리거, 목표 상태, 입력 제한 같은 요소와 연결됩니다. 플레이어가 특정 위치에 도달하면 이동 안내를 숨기고, 공격 구간에 진입하면 공격 버튼 설명을 보여줄 수 있습니다. 특정 행동을 완료해야 다음 문이 열리도록 만들 수도 있습니다. 따라서 기획서에는 “어떤 행동을 하면 어떤 안내가 사라지고, 다음 단계가 열리는지”를 조건 중심으로 작성해야 합니다.

AI에게 튜토리얼 스테이지 구현을 요청할 때는 학습 순서와 완료 조건을 구체적으로 전달해야 합니다. “튜토리얼을 만들어줘”보다 “튜토리얼은 이동, 공격, 회복 아이템 사용, 적 처치 순서로 진행된다. 각 구간에 진입하면 안내 메시지를 표시하고, 해당 행동을 완료하면 다음 구간으로 이동할 수 있게 문을 연다”라고 요청하는 편이 좋습니다. 좋은 튜토리얼 기획은 플레이어에게 설명을 쏟아내는 문서가 아니라, 게임의 기본 규칙을 단계적으로 경험하게 만드는 학습 설계 문서입니다.

Chapter 69. 반복 플레이 구간 설계하기

반복 플레이 구간은 플레이어가 같은 스테이지나 콘텐츠를 여러 번 플레이해도 의미를 느끼도록 만드는 구조입니다. 게임은 한 번 클리어하고 끝나는 경우도 있지만, 많은 게임은 재화, 경험치, 아이템, 장비, 재료를 얻기 위해 같은 구간을 반복하게 만듭니다. 이때 반복 플레이가 단순한 노가다처럼 느껴지지 않으려면 목표, 보상, 난이도, 변화를 함께 설계해야 합니다.

먼저 반복 플레이의 목적을 정해야 합니다. 플레이어가 왜 이 구간을 다시 플레이해야 하는지 설명할 수 있어야 합니다. 경험치를 얻기 위해 반복하는지, 특정 재료를 모으기 위해 반복하는지, 더 좋은 장비를 얻기 위해 반복하는지에 따라 보상 구조가 달라집니다. 목적이 명확하지 않은 반복은 플레이어에게 지루함만 줄 수 있습니다.

반복 플레이 구간에는 일정한 보상과 변동 보상이 함께 있는 것이 좋습니다. 예를 들어 스테이지를 클리어할 때마다 골드와 경험치는 항상 지급하고, 특정 재료나 희귀 장비는 확률적으로 지급할 수 있습니다. 항상 얻는 보상은 플레이어에게 안정감을 주고, 확률 보상은 다시 도전할 이유를 만듭니다. 다만 확률 보상이 너무 낮으면 피로감이 커질 수 있으므로 획득 기대값을 고려해야 합니다.

프로그래머 관점에서는 반복 플레이 구간을 데이터와 조건으로 관리해야 합니다. 반복 보상, 최초 클리어 보상, 일일 제한 보상, 난이도별 보상처럼 보상 종류를 구분할 수 있습니다. 또한 스테이지를 몇 번 클리어했는지, 오늘 몇 번 보상을 받았는지, 특정 보상을 이미 받았는지 저장해야 합니다. 이 정보가 없으면 중복 지급이나 보상 누락 문제가 발생할 수 있습니다.

AI에게 반복 플레이 구간 구현을 요청할 때는 반복 목적과 보상 규칙을 함께 전달해야 합니다. “반복 플레이가 가능하게 해줘”보다 “스테이지는 반복 클리어 가능하며, 최초 클리어 시 보석 10개를 지급하고 이후 클리어 시 골드와 경험치를 지급한다. 희귀 재료는 10% 확률로 지급하며, 일일 보상은 하루 3회까지만 받을 수 있다”라고 요청하는 편이 좋습니다. 좋은 반복 플레이 기획은 같은 구간을 다시 하게 만드는 것이 아니라, 반복할 이유와 보상 구조를 명확히 설계하는 문서입니다.

Chapter 70. 체크포인트와 실패 조건 정의하기

체크포인트와 실패 조건은 플레이어가 스테이지에서 실수했을 때 어떻게 처리할지 정하는 기준입니다. 플레이어가 사망했을 때 처음부터 다시 시작할지, 중간 지점에서 재시작할지, 제한 시간이 끝나면 실패로 처리할지에 따라 플레이 경험이 달라집니다. 이 규칙이 명확하지 않으면 실패 후 재시작 흐름, 보상 지급, 상태 복구가 복잡해질 수 있습니다.

체크포인트는 플레이어의 진행 상태를 임시로 저장하는 지점입니다. 긴 스테이지나 난이도가 높은 스테이지에서는 체크포인트가 플레이 피로도를 줄여줍니다. 예를 들어 전투 구간을 완료한 뒤 체크포인트를 저장하면, 이후 보스전에서 실패하더라도 처음부터 다시 하지 않아도 됩니다. 체크포인트는 위치 뿐 아니라 체력, 보유 아이템, 처치한 적, 열린 문 같은 상태를 어디까지 복구할지도 함께 정의해야 합니다.

실패 조건은 스테이지를 더 이상 진행할 수 없다고 판단하는 기준입니다. 플레이어 체력이 0이 되는 경우, 제한 시간이 끝나는 경우, 보호해야 할 NPC가 사망하는 경우, 목표 오브젝트가 파괴되는 경우가 실패 조건이 될 수 있습니다. 실패 조건은 스테이지 목표와 함께 작성해야 합니다. “보스를 처치하면 성공”이라는 조건만 있으면 부족하고, “플레이어가 사망하면 실패”처럼 반대 상황도 명확히 적어야 합니다.

프로그래머 관점에서 체크포인트와 실패 조건은 상태 저장과 복구 로직으로 이어집니다. 체크포인트에 도달하면 현재 위치와 진행 상태를 저장하고, 실패 시 저장된 체크포인트 상태로 되돌릴지 결과 화면으로 이동할지 결정해야 합니다. 또한 실패 후 적을 다시 생성할지, 이미 획득한 보상은 유지할지, 사용한 소비 아이템은 복구할지도 정해야 합니다. 이런 규칙이 없으면 플레이어가 의도치 않게 보상을 중복 획득하거나 손해를 볼 수 있습니다.

AI에게 체크포인트와 실패 조건 구현을 요청할 때는 저장 대상과 복구 흐름을 구체적으로 전달해야 합니다. “체크포인트를 만들어줘”보다 “플레이어가 체크포인트 트리거에 진입하면 위치와 현재 체력을 저장한다. 플레이어 체력이 0이 되면 마지막 체크포인트 위치에서 부활하고, 체크포인트가 없으면 결과 화면으로 이동한다”라고 요청하는 편이 좋습니다. 좋은 체크포인트 기획은 실패를 없애는 문서가 아니라, 실패 후에도 플레이어가 납득 가능한 흐름으로 다시 도전할 수 있게 만드는 규칙 문서입니다.

Chapter 71. 레벨 기획서를 표로 정리하기

레벨 기획서는 문장으로만 작성하면 전체 구조를 한눈에 보기 어렵습니다. 스테이지 목표, 적 배치, 보상, 실패 조건, 체크포인트, 난이도 정보를 각각 따로 적어두면 구현 과정에서 빠뜨리는 항목이 생길 수 있습니다. 그래서 레벨 기획서는 문장 설명과 함께 표로 정리하는 것이 좋습니다. 표는 레벨의 주요 정보를 압축해서 보여주는 개발 기준표 역할을 합니다.

레벨 기획 표에는 스테이지 ID, 스테이지 이름, 목표, 등장 적, 주요 오브젝트, 보상, 실패 조건, 체크포인트, 난이도 같은 항목을 넣을 수 있습니다. 예를 들어 “Stage_01, 숲 입구, 모든 적 처치, 슬라임 3 마리, 보상 상자 1개, 골드 100, 플레이어 사망 시 실패”처럼 작성할 수 있습니다. 이렇게 정리하면 각 스테이지의 차이를 비교하기 쉽고, 구현해야 할 요소도 명확해집니다.

표로 정리할 때는 너무 많은 정보를 한 칸에 넣지 않는 것이 중요합니다. 적 배치가 복잡하다면 별도의 적 배치 테이블로 분리하고, 보상이 복잡하다면 보상 테이블과 연결하는 편이 좋습니다. 레벨 기획 표는 전체 스테이지를 빠르게 파악하는 요약표이고, 세부 정보는 별도 테이블이나 문서로 나누어 관리할 수 있습니다. 모든 정보를 하나의 표에 억지로 넣으면 오히려 읽기 어려워집니다.

프로그래머 관점에서 레벨 기획 표는 데이터 구조로 변환하기 쉽습니다. StageId, ObjectiveType, EnemyGroupId, RewardTableId, Difficulty, NextStageId 같은 필드를 정의하면 스테이지 데이터를 코드에서 참조할 수 있습니다. 예를 들어 스테이지 선택 화면은 StageId와 이름을 표시하고, 게임 시작 시 해당 StageId에 연결된 적 배치와 보상 테이블을 불러올 수 있습니다. 표는 기획 문서이면서 동시에 데이터 설계의 출발점이 됩니다.

AI에게 레벨 시스템 구현을 요청할 때도 표 형태의 기획은 큰 도움이 됩니다. “스테이지 여러 개를 만들어줘”보다 “스테이지 데이터는 StageId, StageName, ObjectiveType, EnemyGroupId, RewardTableId, Difficulty를 가진다. StageId에 따라 적 배치와 보상 테이블을 불러와 스테이지를 시작한다”라고 요청하는 편이 좋습니다. 좋은 레벨 기획서는 설명을 길게 늘어놓는 문서가 아니라, 스테이지의 목표와 조건을 비교 가능한 표로 정리해 구현 기준으로 사용할 수 있는 문서입니다.

Part 09. 데이터 중심으로 상세 기획서 작성하기

Part 9에서는 게임 상세 기획서를 데이터 중심으로 작성하는 방법을 다룹니다.

프로그래머에게 데이터는 게임 시스템을 움직이는 기준입니다.

캐릭터 능력치, 아이템 속성, 스킬 쿨타임, 스테이지 보상, 퀘스트 조건, 밸런싱 수치처럼 변경 가능성이 높은 값은 문서와 테이블로 정리되어야 합니다.

이 파트에서는 하드코딩할 값과 데이터화할 값을 구분하고, 캐릭터, 아이템, 스킬, 스테이지, 퀘스트 데이터를 표로 구성하는 방법을 설명합니다.

Chapter 72. 프로그래머에게 데이터 기획이 중요한 이유

프로그래머에게 데이터 기획이 중요한 이유는 게임의 많은 규칙이 결국 데이터로 표현되기 때문입니다. 캐릭터 능력치, 아이템 가격, 스킬 쿨타임, 적 체력, 스테이지 보상, 퀘스트 조건은 모두 코드 안의 로직이면서 동시에 조정 가능한 데이터입니다. 이 값을 아무 기준 없이 코드에 직접 넣으면 처음에는 빠르게 만들 수 있지만, 기능이 늘어날수록 수정하기 어려운 구조가 됩니다.

데이터 기획은 무엇을 코드로 처리하고, 무엇을 데이터로 관리할지 구분하는 작업입니다. 예를 들어 “공격 버튼을 누르면 공격한다”는 로직에 가깝지만, “검의 공격력은 10이고, 활의 공격력은 7이다”는 데이터에 가깝습니다. 이런 값을 코드에 직접 작성하면 아이템을 추가할 때마다 코드를 수정해야 합니다. 반대로 데이터 테이블로 관리하면 새로운 아이템을 추가하거나 수치를 조정하기 쉬워집니다.

게임 개발에서 밸런싱은 반복적으로 이루어집니다. 처음 정한 체력, 공격력, 보상량, 드랍 확률이 한번에 맞는 경우는 거의 없습니다. 플레이 테스트를 하면서 수치를 바꾸고, 난이도를 조절하고, 보상을 수정해야 합니다. 이때 데이터 구조가 잘 정리되어 있으면 코드 변경 없이 테이블 값만 조정할 수 있습니다. 데이터 기획은 개발 속도뿐 아니라 테스트와 개선 속도에도 직접 영향을 줍니다.

프로그래머 관점에서 데이터 기획은 저장 구조, 참조 구조, 확장성을 함께 고려해야 합니다. 캐릭터 데이터는 스탯 시스템에서 사용되고, 아이템 데이터는 인벤토리와 상점에서 사용되며, 보상 데이터는 스테이지와 퀘스트에서 사용됩니다. 하나의 데이터가 여러 시스템에서 참조될 수 있으므로 ID, 타입, 수치, 조건을 일관되게 설계해야 합니다. 데이터 설계가 흐트러지면 시스템 간 연결도 함께 복잡해집니다.

AI에게 구현을 요청할 때도 데이터 기획은 중요합니다. “아이템 시스템을 만들어줘”보다 “아이템 데이터는 ItemId, Name, ItemType, Price, MaxStack, EffectValue를 가진다. 인벤토리와 상점은 이 데이터를 참조한다”라고 요청하는 편이 훨씬 명확합니다. 좋은 데이터 기획은 단순히 표를 만드는 작업이 아니라, 게임 규칙을 수정 가능하고 확장 가능한 구조로 정리하는 개발 설계입니다.

Chapter 73. 하드코딩할 값과 데이터화할 값 구분하기

게임을 만들 때 모든 값을 데이터로 빼야 하는 것은 아닙니다. 반대로 모든 값을 코드 안에 직접 넣는 것도 좋지 않습니다. 중요한 것은 어떤 값은 코드에 고정해도 되고, 어떤 값은 기획 변경과 밸런싱을 위해 데이터로 관리해야 하는지 구분하는 것입니다. 이 기준이 없으면 코드가 지저분해지거나, 작은 수치 변경에도 매번 코드를 수정해야 하는 문제가 생깁니다.

하드코딩해도 되는 값은 자주 바뀌지 않고, 게임 규칙 자체에 가까운 값입니다. 예를 들어 배열의 최소 인덱스, 시스템 내부 상태값, 특정 로직의 기본 분기 조건처럼 기획자가 자주 조정하지 않는 값은 코드에 있어도 괜찮습니다. 또한 임시 프로토타입 단계에서는 빠른 검증을 위해 일부 값을 하드코딩할 수도 있습니다. 다만 프로토타입이 끝난 뒤에는 이 값이 계속 고정되어도 되는지 다시 검토해야 합니다.

데이터화해야 하는 값은 밸런싱과 콘텐츠 추가에 자주 영향을 받는 값입니다. 캐릭터 체력, 공격력, 아이템 가격, 스킬 쿨타임, 적 스폰 수, 드랍 확률, 스테이지 보상 같은 값이 여기에 해당합니다. 이런 값은 플레이 테스트 과정에서 계속 바뀔 가능성이 높습니다. 코드에 직접 넣어두면 변경할 때마다 빌드와 테스트가 번거워지고, 실수로 다른 로직까지 건드릴 위험이 생깁니다.

프로그래머 관점에서는 “변경 주체”와 “변경 빈도”를 기준으로 판단하면 좋습니다. 개발자만 이해해야 하는 내부 규칙은 코드에 둘 수 있지만, 기획자나 밸런싱 담당자가 조정해야 하는 값은 데이터로 빼는 편이 좋습니다. 또한 한 번 정하면 거의 바뀌지 않는 값은 하드코딩해도 되지만, 여러 번 테스트하며 조정할 값은 데이터 테이블이나 ScriptableObject, JSON, CSV 같은 형태로 관리하는 것이 좋습니다.

AI에게 데이터 구조를 요청할 때도 이 구분을 명확히 전달해야 합니다. “게임 수치를 관리하게 해줘”보다 “캐릭터 체력, 공격력, 이동 속도, 스킬 쿨타임은 데이터로 관리하고, 내부 상태 전환 로직은 코드에서 처리한다”라고 요청하는 편이 좋습니다. 좋은 데이터 기획은 모든 값을 밖으로 빼는 것이 아니라, 자주 바뀌는 값과 고정되어야 할 규칙을 구분해 유지보수하기 쉬운 구조를 만드는 일입니다.

Chapter 74. 캐릭터 데이터 테이블 작성하기

캐릭터 데이터 테이블은 플레이어 캐릭터, 적 캐릭터, NPC처럼 게임에 등장하는 캐릭터의 기본 정보를 정리하는 표입니다. 캐릭터의 이름, 체력, 공격력, 이동 속도, 방어력 같은 값을 코드에 직접 넣으면 캐릭터가 늘어날수록 관리가 어려워집니다. 따라서 캐릭터별 수치와 속성을 데이터 테이블로 정리해 두면 밸런싱과 확장이 쉬워집니다.

먼저 캐릭터를 구분할 수 있는 ID가 필요합니다. 예를 들어 `Player_001`, `Enemy_Slime_001`, `NPC_Merchant_001`처럼 고유한 `CharacterId`를 작성할 수 있습니다. ID는 코드와 데이터가 캐릭터를 찾는 기준이 되므로 중복되지 않아야 합니다. 이름은 화면에 표시되는 값이고, ID는 내부 참조용 값이라는 점을 구분해야 합니다.

캐릭터 데이터 테이블에는 기본 능력치를 포함해야 합니다. 최대 체력, 공격력, 방어력, 이동 속도, 공격 속도, 사거리, 경험치 보상, 드랍 테이블 ID 같은 항목이 들어갈 수 있습니다. 플레이어 캐릭터와 적 캐릭터가 같은 필드를 공유할 수도 있지만, 모든 캐릭터가 모든 값을 필요로 하는 것은 아닙니다. 필요하다면 플레이어 데이터, 적 데이터, NPC 데이터를 별도 테이블로 분리할 수 있습니다.

프로그래머 관점에서는 캐릭터 데이터가 여러 시스템에서 참조된다는 점을 고려해야 합니다. 전투 시스템은 체력과 공격력을 사용하고, AI 시스템은 이동 속도와 탐지 범위를 사용하며, 보상 시스템은 경험치와 드랍 테이블을 참조합니다. 따라서 테이블의 필드 이름은 명확하고 일관되게 작성해야 합니다. `Hp`, `Health`, `MaxHealth`처럼 비슷한 이름을 섞어 쓰면 데이터 해석이 어려워집니다.

AI에게 캐릭터 데이터 구조를 요청할 때는 필요한 필드를 구체적으로 전달해야 합니다. “캐릭터 데이터를 만들어줘”보다 “캐릭터 데이터 테이블은 `CharacterId`, `DisplayName`, `CharacterType`, `MaxHp`, `AttackPower`, `Defense`, `MoveSpeed`, `ExpReward`, `DropTableId`를 가진다. 전투 시작 시 `CharacterId`를 기준으로 데이터를 불러와 능력치를 초기화한다”라고 요청하는 편이 좋습니다. 좋은 캐릭터 데이터 테이블은 캐릭터 목록을 정리하는 표가 아니라, 전투와 성장, 보상 시스템이 함께 참조할 수 있는 기준 데이터입니다.

Chapter 75. 아이템 데이터 테이블 작성하기

아이템 데이터 테이블은 게임에 등장하는 모든 아이템의 기본 정보를 정리하는 표입니다. 아이템 이름, 종류, 가격, 최대 보유 수량, 사용 효과 같은 값을 코드에 직접 넣으면 아이템이 늘어날수록 관리가 어려워집니다. 특히 상점, 인벤토리, 보상, 제작, 강화 시스템이 아이템 데이터를 함께 참조하기 때문에 아이템 정보는 데이터 테이블로 분리해두는 것이 좋습니다.

먼저 아이템을 구분할 수 있는 고유 ID가 필요합니다. 예를 들어 `Item_Potion_001`, `Item_Sword_001`, `Item_Material_Stone_001`처럼 작성할 수 있습니다. 아이템 이름은 플레이어에게 표시되는 값이고, 아이템 ID는 코드와 데이터가 참조하는 내부 값입니다. 같은 이름의 아이템이 나중에 추가될 수도 있으므로 내부 참조에는 반드시 중복되지 않는 ID를 사용하는 것이 좋습니다.

아이템 데이터 테이블에는 공통 속성과 종류별 속성을 구분해 작성해야 합니다. 공통 속성에는 `ItemId`, `DisplayName`, `ItemType`, `Description`, `IconId`, `Price`, `MaxStack` 같은 항목을 넣을 수 있습니다. 장비 아이템이라면 `EquipSlot`, `AttackBonus`, `DefenseBonus` 같은 값이 필요하고, 소비 아이템이라면 `EffectType`, `EffectValue`, `Cooldown` 같은 값이 필요할 수 있습니다. 재료 아이템은 사용처나 제작 재료 여부를 표시하는 필드가 필요합니다.

프로그래머 관점에서는 하나의 테이블에 모든 속성을 넣을지, 아이템 종류별로 테이블을 분리할지 결정해야 합니다. 아이템 수가 적고 구조가 단순하다면 하나의 테이블로도 충분합니다. 하지만 장비, 소비, 재료의 속성이 많이 다르다면 공통 아이템 테이블과 장비 상세 테이블, 소비 아이템 상세 테이블을 분리하는 편이 좋습니다. 중요한 것은 불필요한 빈 필드가 너무 많아지지 않도록 구조를 정리하는 것입니다.

AI에게 아이템 데이터 구조를 요청할 때는 필드와 사용처를 함께 전달해야 합니다. “아이템 데이터를 만들어줘”보다 “아이템 데이터 테이블은 `ItemId`, `DisplayName`, `ItemType`, `Description`, `IconId`, `Price`, `MaxStack`을 가진다. 인벤토리와 상점은 `ItemId`를 기준으로 이 데이터를 참조한다”라고 작성하는 편이 좋습니다. 좋은 아이템 데이터 테이블은 아이템 목록을 저장하는 표가 아니라, 여러 게임 시스템이 안정적으로 공유할 수 있는 기준 데이터입니다.

Chapter 76. 스킬 데이터 테이블 작성하기

스킬 데이터 테이블은 게임에 등장하는 스킬의 기본 정보와 동작 기준을 정리하는 표입니다. 스킬 이름, 쿨타임, 소비 자원, 데미지, 범위, 지속 시간 같은 값은 전투 밸런싱에 직접 영향을 줍니다. 이 값을 코드에 직접 넣으면 스킬을 추가하거나 수정할 때마다 코드 변경이 필요합니다. 따라서 스킬 정보는 데이터 테이블로 분리해 관리하는 것이 좋습니다.

먼저 스킬을 구분할 수 있는 고유 ID를 작성해야 합니다. 예를 들어 Skill_Fireball_001, Skill_Dash_001, Skill_Heal_001처럼 스킬의 역할이 드러나도록 만들 수 있습니다. 스킬 이름은 플레이어에게 표시되는 값이고, 스킬 ID는 코드에서 해당 스킬 데이터를 찾는 기준입니다. 스킬이 많아질수록 ID 규칙이 중요해지므로 이름 규칙을 미리 정해두는 것이 좋습니다.

스킬 데이터 테이블에는 공통 필드를 먼저 정의합니다. SkillId, DisplayName, Description, IconId, SkillType, Cooldown, CostType, CostAmount, Range, Duration 같은 항목이 들어갈 수 있습니다. 공격 스킬이라면 Damage, HitCount, AttackRange가 필요하고, 회복 스킬이라면 HealAmount, TargetType이 필요할 수 있습니다. 이동 스킬이라면 MoveDistance, MoveSpeed 같은 값이 필요합니다.

프로그래머 관점에서는 스킬의 효과와 연출을 분리해서 생각해야 합니다. 데미지를 주는 규칙은 전투 로직이고, 이펙트나 사운드는 표현 요소입니다. 따라서 스킬 데이터에는 EffectId, AnimationId, SoundId 같은 참조 값을 둘 수 있습니다. 이렇게 구성하면 같은 스킬 로직에 다른 이펙트를 적용하거나, 같은 이펙트를 여러 스킬에서 재사용하기 쉬워집니다.

AI에게 스킬 데이터 구조를 요청할 때는 필드와 사용 방식을 함께 전달해야 합니다. “스킬 시스템을 만들어줘”보다 “스킬 데이터 테이블은 SkillId, SkillType, Cooldown, CostAmount, Damage, Range, Duration, EffectId를 가진다. 캐릭터가 스킬을 사용할 때 SkillId로 데이터를 조회하고, 쿨타임과 자원 조건을 확인한 뒤 효과를 실행한다”라고 요청하는 편이 좋습니다. 좋은 스킬 데이터 테이블은 스킬 목록을 저장하는 표가 아니라, 전투 시스템과 UI, 이펙트 시스템이 함께 참조할 수 있는 기준 데이터입니다.

Chapter 77. 스테이지 데이터 테이블 작성하기

스테이지 데이터 테이블은 게임에 등장하는 각 스테이지의 기본 정보와 진행 조건을 정리하는 표입니다. 스테이지 이름, 목표, 난이도, 적 배치, 보상, 다음 스테이지 연결 같은 정보를 코드에 직접 넣으면 스테이지가 늘어날수록 관리가 어려워집니다. 따라서 스테이지 정보는 데이터 테이블로 분리해두는 것이 좋습니다. 이렇게 하면 스테이지 선택 화면, 전투 시작, 보상 지급, 진행도 저장을 하나의 기준 데이터로 연결할 수 있습니다.

먼저 스테이지를 구분할 수 있는 고유 ID가 필요합니다. 예를 들어 Stage_001, Stage_Forest_001, Stage_Boss_001처럼 작성할 수 있습니다. 스테이지 이름은 플레이어에게 표시되는 값이고, 스테이지 ID는 코드와 데이터가 참조하는 내부 값입니다. 스테이지 ID가 명확해야 저장 데이터에서 어떤 스테이지를 클리어했는지 관리하고, 다음 스테이지 해금 조건을 판단할 수 있습니다.

스테이지 데이터 테이블에는 StageId, DisplayName, StageType, Difficulty, ObjectiveType, EnemyGroupId, RewardTableId, RequiredLevel, NextStageId 같은 항목을 넣을 수 있습니다. StageType은 일반 스테이지, 보스 스테이지, 튜토리얼 스테이지처럼 구분할 수 있고, ObjectiveType은 모든 적 처치, 목적지 도달, 보스 처치처럼 정의할 수 있습니다. EnemyGroupId와 RewardTableId를 사용하면 적 배치와 보상 정보를 별도 테이블로 분리해 연결할 수 있습니다.

프로그래머 관점에서는 스테이지 데이터가 여러 시스템에서 사용된다는 점이 중요합니다. 스테이지 선택 화면은 스테이지 이름과 난이도를 표시하고, 게임 시작 로직은 EnemyGroupId를 기준으로 적을 생성하며, 클리어 처리 로직은 RewardTableId를 기준으로 보상을 지급합니다. 또한 RequiredLevel이나 이전 스테이지 클리어 여부를 확인해 입장 가능 여부를 판단할 수 있습니다. 스테이지 데이터는 단순한 목록이 아니라 게임 진행 흐름의 중심 데이터입니다.

AI에게 스테이지 데이터 구조를 요청할 때는 필드와 연결 방식을 함께 전달해야 합니다. “스테이지 시스템을 만들어줘”보다 “스테이지 데이터 테이블은 StageId, DisplayName, ObjectiveType, EnemyGroupId, RewardTableId, RequiredLevel, NextStageId를 가진다. 플레이어가 스테이지를 선택하면 StageId로 데이터를 조회하고, 연결된 적 배치와 보상 테이블을 불러와 스테이지를 시작한다”라고 요청하는 편이 좋습니다. 좋은 스테이지 데이터 테이블은 스테이지 정보를 정리하는 표를 넘어, 게임 진행과 보상, 해금 구조를 연결하는 기준 데이터입니다.

Chapter 78. 퀘스트 데이터 테이블 작성하기

퀘스트 데이터 테이블은 게임 안에서 플레이어가 수행할 목표와 보상, 진행 조건을 정리하는 표입니다. 퀘스트는 단순히 “적을 처치하라”, “아이템을 가져오라” 같은 문장으로 끝나지 않습니다. 시작 조건, 완료 조건, 진행 상태, 보상, 선행 퀘스트, 반복 가능 여부까지 함께 관리해야 합니다. 따라서 퀘스트는 코드에 직접 작성하기보다 데이터 테이블로 정리하는 것이 좋습니다.

먼저 퀘스트를 구분할 수 있는 고유 ID가 필요합니다. 예를 들어 Quest_Main_001, Quest_Daily_001, Quest_Tutorial_001처럼 퀘스트의 종류와 순서를 알 수 있게 작성할 수 있습니다. 퀘스트 이름과 설명은 플레이어에게 표시되는 값이고, 퀘스트 ID는 코드와 저장 데이터에서 참조하는 기준입니다. 플레이어가 어떤 퀘스트를 수락했고, 어떤 퀘스트를 완료했는지 저장하려면 ID가 명확해야 합니다.

퀘스트 데이터 테이블에는 QuestId, DisplayName, Description, QuestType, StartCondition, CompleteCondition, TargetId, RequiredCount, RewardTableId, NextQuestId 같은 항목을 넣을 수 있습니다. QuestType은 메인 퀘스트, 서브 퀘스트, 일일 퀘스트, 튜토리얼 퀘스트처럼 구분할 수 있습니다. CompleteCondition은 적 처치, 아이템 수집, 특정 위치 도달, NPC 대화처럼 구체적인 조건으로 작성해야 합니다.

프로그래머 관점에서는 퀘스트 진행 상태를 별도로 관리해야 합니다. 퀘스트 테이블은 고정된 기준 데이터이고, 플레이어별 진행 상황은 저장 데이터입니다. 예를 들어 퀘스트 데이터에는 “슬라임 5마리 처치”라는 조건이 있고, 플레이어 진행 데이터에는 현재 처치 수 3마리가 저장될 수 있습니다. 기준 데이터와 진행 데이터를 분리해야 여러 플레이어 상태와 반복 퀘스트를 안정적으로 처리할 수 있습니다.

AI에게 퀘스트 시스템 구현을 요청할 때는 데이터 테이블과 진행 상태 구조를 함께 전달해야 합니다. “퀘스트 시스템을 만들어줘”보다 “퀘스트 데이터 테이블은 QuestId, QuestType, CompleteCondition, TargetId, RequiredCount, RewardTableId를 가진다. 플레이어별 퀘스트 진행 데이터는 QuestId, CurrentCount, IsCompleted를 저장한다”라고 요청하는 편이 좋습니다. 좋은 퀘스트 데이터 테이블은 퀘스트 문장을 모아둔 목록이 아니라, 목표 추적과 보상 지급, 진행도 저장을 연결하는 기준 데이터입니다.

Chapter 79. 밸런싱 수치를 표로 관리하기

밸런싱 수치는 게임의 체감 난이도와 성장 속도를 결정하는 중요한 데이터입니다. 캐릭터 체력, 공격력, 적 능력치, 경험치 보상, 아이템 가격, 스킬 쿨타임, 드랍 확률 같은 값은 작은 차이만으로도 플레이 경험을 크게 바꿀 수 있습니다. 이런 수치를 코드 안에 흩어놓으면 전체 균형을 확인하기 어렵습니다. 그래서 밸런싱 수치는 표로 정리해 관리하는 것이 좋습니다.

밸런싱 표의 장점은 여러 값을 한눈에 비교할 수 있다는 점입니다. 예를 들어 레벨별 필요 경험치와 공격력 증가량을 표로 정리하면 성장 곡선이 너무 빠른지, 특정 구간에서 갑자기 어려워지는지 확인하기 쉽습니다. 적 데이터도 체력, 공격력, 보상 경험치, 드랍 테이블을 함께 보면 어떤 적이 지나치게 강하거나 보상이 부족한지 판단할 수 있습니다.

밸런싱 수치를 표로 관리할 때는 기준 단위를 명확히 해야 합니다. 스테이지 기준인지, 레벨 기준인지, 아이템 등급 기준인지 먼저 정해야 합니다. 예를 들어 “Stage_01의 일반 적 체력은 30, Stage_02의 일반 적 체력은 45”처럼 스테이지 기준으로 관리할 수 있습니다. 또는 “레벨 1 캐릭터 공격력은 10, 레벨 2는 12”처럼 레벨 기준으로 관리할 수도 있습니다. 기준이 섞이면 수치 조정이 어려워집니다.

프로그래머 관점에서는 밸런싱 표가 실제 데이터 테이블과 연결될 수 있어야 합니다. 필드 이름은 코드에서 사용할 이름과 최대한 일치시키는 것이 좋습니다. 예를 들어 MaxHp, AttackPower, ExpReward, DropRate, Cooldown처럼 명확한 이름을 사용하면 구현과 검토가 쉬워집니다. 또한 수치가 정수인지, 실수인지, 퍼센트인지도 표에 표현해야 합니다. 0.1이 10%를 의미하는지, 0.1%를 의미하는지 모호하면 버그가 생길 수 있습니다.

AI에게 밸런싱 데이터 구조를 요청할 때도 표의 기준과 필드를 함께 전달해야 합니다. “밸런싱 데이터를 만들어줘”보다 “적 밸런싱 테이블은 EnemyId, MaxHp, AttackPower, ExpReward, GoldReward, DropTableId를 가진다. 스테이지가 올라갈수록 MaxHp와 AttackPower가 점진적으로 증가하도록 구성한다”라고 요청하는 편이 좋습니다. 좋은 밸런싱 기획은 감으로 수치를 정하는 것이 아니라, 비교 가능한 표를 통해 수치를 조정하고 게임의 난이도와 보상 흐름을 관리하는 문서입니다.

Chapter 80. 데이터 테이블을 ScriptableObject와 연결하기

Unity에서 데이터 테이블을 활용할 때 자주 사용하는 방식 중 하나가 ScriptableObject입니다. ScriptableObject는 캐릭터, 아이템, 스킬, 스테이지 같은 데이터를 에셋 형태로 관리할 수 있게 해 줍니다. 코드 안에 수치를 직접 넣는 대신, 데이터 에셋을 만들고 필요한 시스템이 그 데이터를 참조하도록 구성하면 유지보수와 밸런싱이 쉬워집니다.

데이터 테이블과 ScriptableObject를 연결하려면 먼저 테이블의 필드를 코드 구조로 옮겨야 합니다. 예를 들어 아이템 테이블에 ItemId, DisplayName, ItemType, Price, MaxStack이 있다면, 같은 필드를 가진 ItemData ScriptableObject 클래스를 만들 수 있습니다. 캐릭터 테이블이라면 CharacterId, MaxHp, AttackPower, MoveSpeed 같은 값을 ScriptableObject로 정의할 수 있습니다. 중요한 것은 테이블의 필드명과 코드의 변수명이 크게 어긋나지 않도록 유지하는 것입니다.

ScriptableObject의 장점은 Unity 에디터에서 데이터를 직접 확인하고 수정할 수 있다는 점입니다. 아이템 가격이나 스킬 쿨타임을 바꿀 때 코드를 수정하지 않고 에셋 값만 바꿀 수 있습니다. 또한 여러 프리팹이나 시스템이 같은 데이터 에셋을 참조할 수 있으므로 중복 데이터를 줄일 수 있습니다. 예를 들어 상점 UI와 인벤토리 UI가 같은 ItemData를 참조하면 아이템 이름과 아이콘이 서로 다르게 표시 되는 문제를 줄일 수 있습니다.

프로그래머 관점에서는 ScriptableObject를 기준 데이터로 사용하고, 플레이어별 상태 데이터와는 분리해야 합니다. ItemData는 아이템의 고정 정보이고, 플레이어가 그 아이템을 몇 개 가지고 있는지는 별도의 인벤토리 저장 데이터입니다. 마찬가지로 CharacterData는 캐릭터의 기본 능력치이고, 현재 체력이나 현재 경험치는 런타임 상태 데이터입니다. 기준 데이터와 상태 데이터를 섞으면 저장과 복구가 복잡해집니다.

AI에게 ScriptableObject 기반 데이터 구조를 요청할 때도 기준 데이터와 상태 데이터를 구분해 전달해야 합니다. “아이템 데이터를 ScriptableObject로 만들어줘”보다 “ItemData ScriptableObject는 ItemId, DisplayName, ItemType, Icon, Price, MaxStack을 가진다. 인벤토리는 ItemData를 참조하되, 보유 수량은 별도 InventoryItemState에 저장한다”라고 요청하는 편이 좋습니다. 좋은 데이터 기획은 표를 만드는 데서 끝나지 않고, 그 표를 Unity에서 실제로 참조할 수 있는 ScriptableObject 구조로 연결하는 것입니다.

Part 10. 시스템 흐름과 예외 조건 작성하기

Part 10에서는 게임 시스템이 정상적으로 동작하지 않는 상황까지 고려하는 방법을 다룹니다.

좋은 상세 기획서는 정상 흐름만 설명하지 않습니다. 승리와 패배 조건, 저장과 불러오기, 네트워크 끊김, 보상 중복 지급, 비정상 입력, 상태 전환 충돌 같은 예외 상황을 함께 정의해야 합니다.

이 파트에서는 시스템 흐름을 플로우차트와 상태 전환표로 정리하고, 구현 중 발생할 수 있는 모호한 판단을 줄이는 방법을 설명합니다.

Chapter 81. 좋은 상세 기획서는 예외 조건을 포함한다

좋은 상세 기획서는 정상적인 흐름만 설명하지 않습니다. 플레이어가 버튼을 눌렀을 때, 아이템을 구매했을 때, 스테이지를 클리어했을 때처럼 성공하는 상황만 작성하면 실제 구현에서 문제가 생길 수 있습니다. 게임은 항상 예외 상황이 발생합니다. 재화가 부족할 수 있고, 인벤토리가 가득 찰 수 있으며, 보상을 이미 받은 상태일 수도 있습니다. 이런 상황을 미리 정의해야 안정적인 시스템을 만들 수 있습니다.

예외 조건은 “원래 의도한 흐름이 불가능한 경우”를 의미합니다. 예를 들어 상점에서 아이템을 구매하려고 했지만 골드가 부족한 경우, 소비 아이템을 사용하려고 했지만 체력이 이미 가득 찬 경우, 스킬을 사용하려고 했지만 쿨타임이 남아 있는 경우가 있습니다. 이런 상황에서 게임이 어떤 메시지를 보여주고, 입력을 막고, 데이터를 변경하지 않을지 기획서에 작성해야 합니다.

예외 조건을 작성할 때는 기능별로 생각하는 것이 좋습니다. 상점 기능에는 재화 부족, 구매 제한 초과, 인벤토리 공간 부족 같은 예외가 있을 수 있습니다. 전투 기능에는 사망 상태에서 공격 입력, 무적 상태에서 피격 처리, 같은 공격의 중복 판정 같은 예외가 있을 수 있습니다. 저장 기능에는 저장 실패, 데이터 없음, 불러오기 오류 같은 예외가 필요할 수 있습니다. 기능마다 발생 가능한 문제를 미리 정리하면 구현 중 판단이 줄어듭니다.

프로그래머 관점에서 예외 조건은 방어 코드와 상태 검증으로 이어집니다. 어떤 기능을 실행하기 전에 조건을 검사하고, 조건을 만족하지 않으면 로직을 중단해야 합니다. 예를 들어 구매 처리 전에 보유 골드와 인벤토리 공간을 확인하고, 조건이 맞지 않으면 아이템 지급과 재화 차감을 실행하지 않아야 합니다. 예외 처리가 없으면 중복 지급, 잘못된 상태 변경, UI 오류 같은 버그가 발생할 수 있습니다.

AI에게 기능 구현을 요청할 때도 예외 조건을 함께 전달해야 합니다. “상점 구매 기능을 만들어줘”보다 “구매 전 골드가 충분한지 확인하고, 부족하면 구매를 중단하며 ‘골드가 부족합니다’ 메시지를 표시한다. 인벤토리가 가득 찬 경우에도 구매를 중단한다”라고 요청하는 편이 좋습니다. 좋은 상세 기획서는 성공 흐름뿐 아니라 실패와 예외 상황까지 포함해, 구현자가 안전하게 판단할 수 있는 기준을 제공하는 문서입니다.

Chapter 82. 게임 시작과 종료 흐름 작성하기

게임 시작과 종료 흐름은 플레이어가 게임에 진입하고 빠져나가는 과정을 정의하는 문서입니다. 타이틀 화면에서 시작 버튼을 눌렀을 때 어떤 데이터를 불러오고, 어떤 화면으로 이동하며, 어떤 상태로 게임이 시작되는지 명확히 작성해야 합니다. 종료 역시 단순히 게임을 끄는 동작이 아니라, 저장 처리와 확인 메시지, 진행 중 데이터 정리까지 포함하는 흐름입니다.

게임 시작 흐름은 먼저 진입 지점을 기준으로 정리해야 합니다. 새 게임을 시작하는지, 저장 데이터를 이어서 불러오는지, 특정 스테이지를 선택해 진입하는지에 따라 필요한 처리가 달라집니다. 예를 들어 새 게임은 초기 데이터를 생성하고 튜토리얼이나 첫 스테이지로 이동할 수 있습니다. 이어하기는 저장 데이터를 불러온 뒤 마지막 진행 위치나 로비 화면으로 이동할 수 있습니다.

시작 과정에서 필요한 데이터 초기화도 작성해야 합니다. 플레이어 체력, 보유 재화, 장착 장비, 진행 중 퀘스트, 선택한 스테이지, 난이도 설정 같은 값이 어디에서 결정되는지 명확해야 합니다. 스테이지 시작이라면 적 배치 데이터, 보상 테이블, 목표 조건을 불러와야 합니다. 이 흐름이 정리되지 않으면 게임 시작 시 이전 플레이 데이터가 남거나, 필요한 데이터가 초기화되지 않는 문제가 생길 수 있습니다.

게임 종료 흐름에는 플레이어가 종료를 요청했을 때의 처리와 시스템이 종료되는 상황을 함께 고려해야 합니다. 타이틀 화면에서 종료 버튼을 누를 수도 있고, 인게임에서 일시정지 메뉴를 통해 나갈 수도 있습니다. 이때 바로 종료할지, 확인창을 보여줄지, 자동 저장을 할지 정해야 합니다. 특히 진행 중인 스테이지를 나갈 때 보상을 유지할지, 소비한 아이템을 복구하지 않을지 같은 규칙도 필요합니다.

AI에게 게임 시작과 종료 흐름 구현을 요청할 때는 상태 전환과 저장 규칙을 함께 전달해야 합니다. “게임 시작과 종료를 만들어줘”보다 “타이틀에서 새 게임 버튼을 누르면 기본 플레이어 데이터를 생성하고 로비로 이동한다. 이어하기 버튼은 저장 데이터가 있을 때만 활성화한다. 인게임에서 종료를 선택하면 확인창을 표시하고, 확인 시 현재 진행 상태를 저장한 뒤 타이틀로 이동한다”라고 요청하는 편이 좋습니다. 좋은 시작과 종료 기획은 게임의 입구와 출구를 명확히 정의해 데이터 초기화, 저장, 화면 전환이 안정적으로 이어지도록 만드는 문서입니다.

Chapter 83. 승리와 패배 조건 작성하기

승리와 패배 조건은 게임이 언제 성공으로 끝나고, 언제 실패로 끝나는지 정의하는 기준입니다. 플레이어가 보스를 처치했는지, 목표 지점에 도달했는지, 제한 시간 안에 임무를 완료했는지에 따라 승리가 결정됩니다. 반대로 체력이 0이 되었는지, 제한 시간이 끝났는지, 보호 대상이 사망했는지에 따라 패배가 결정될 수 있습니다. 이 조건이 명확해야 게임 종료 처리와 보상 지급이 안정적으로 동작합니다.

승리 조건은 게임의 목표와 직접 연결되어야 합니다. “스테이지 클리어”라고만 작성하면 구현 기준이 부족합니다. “모든 적을 처치하면 승리”, “보스 체력이 0이 되면 승리”, “플레이어가 탈출 지점 트리거에 도달하면 승리”처럼 판단 가능한 조건으로 작성해야 합니다. 승리 조건은 UI 목표 문구, 이벤트 트리거, 보상 지급, 다음 스테이지 해금 조건의 기준이 됩니다.

패배 조건도 승리 조건만큼 중요합니다. 플레이어 체력이 0이 되면 즉시 패배인지, 부활 기회가 있다면 몇 번까지 가능한지 정해야 합니다. 제한 시간이 있는 스테이지라면 시간이 0이 되었을 때 실패 처리해야 합니다. 방어형 게임이라면 기지가 파괴되거나 보호 대상이 사망했을 때 패배로 처리할 수 있습니다. 패배 조건이 불명확하면 플레이어가 왜 실패했는지 이해하기 어렵습니다.

프로그래머 관점에서는 승리와 패배 조건의 우선순위도 정해야 합니다. 예를 들어 플레이어와 보스가 동시에 사망했을 때 승리로 처리할지, 패배로 처리할지 결정해야 합니다. 제한 시간이 0이 되는 순간 목표도 동시에 완료되었다면 어떤 결과를 우선할지도 필요합니다. 이런 예외 상황을 미리 정리하지 않으면 결과 화면, 보상 지급, 저장 데이터가 예상과 다르게 처리될 수 있습니다.

AI에게 승리와 패배 조건 구현을 요청할 때는 조건과 결과 처리를 함께 전달해야 합니다. “클리어와 실패를 만들어줘”보다 “보스 체력이 0이 되면 승리 처리하고, 플레이어 체력이 0이 되면 패배 처리한다. 승리 시 보상을 지급하고 다음 스테이지를 해금하며, 패배 시 보상을 지급하지 않고 재도전 버튼을 표시한다”라고 요청하는 편이 좋습니다. 좋은 승리와 패배 기획은 게임의 끝을 정하는 문서가 아니라, 성공과 실패를 판단하고 그 결과를 일관되게 처리하기 위한 기준 문서입니다.

Chapter 84. 저장과 불러오기 조건 작성하기

저장과 불러오기는 플레이어의 진행 상태를 유지하는 핵심 기능입니다. 게임을 종료했다가 다시 실행했을 때 레벨, 경험치, 보유 아이템, 클리어한 스테이지, 설정값이 유지되어야 플레이어는 자신의 진행이 안전하게 보관된다고 느낍니다. 따라서 상세 기획서에는 무엇을 저장하고, 언제 저장하며, 어떤 조건에서 불러올지 명확히 작성해야 합니다.

먼저 저장할 데이터를 구분해야 합니다. 플레이어 레벨, 현재 경험치, 보유 재화, 인벤토리, 장착 장비, 클리어한 스테이지, 진행 중인 퀘스트, 설정값 같은 데이터가 저장 대상이 될 수 있습니다. 반대로 일시적인 전투 상태나 잠깐 표시되는 UI 상태는 저장하지 않아도 됩니다. 모든 데이터를 저장하려고 하면 구조가 복잡해지므로, 다음 실행에서도 유지되어야 하는 값만 선별하는 것이 좋습니다.

저장 시점도 중요합니다. 스테이지 클리어 후, 아이템 구매 후, 장비 변경 후, 퀘스트 완료 후, 게임 종료 시점처럼 데이터가 의미 있게 바뀌는 순간에 저장할 수 있습니다. 너무 자주 저장하면 성능 부담이 생길 수 있고, 너무 적게 저장하면 종료나 오류 발생 시 진행 데이터가 사라질 수 있습니다. 자동 저장과 수동 저장을 함께 사용할지도 기획서에 작성해야 합니다.

불러오기 조건에는 저장 데이터가 있는 경우와 없는 경우를 모두 포함해야 합니다. 저장 데이터가 있으면 이어하기 버튼을 활성화하고, 저장 데이터가 없으면 새 게임만 가능하게 만들 수 있습니다. 저장 데이터가 손상되었거나 버전이 맞지 않는 경우도 고려해야 합니다. 이때 기본 데이터로 새로 시작할지, 오류 메시지를 보여줄지, 백업 데이터를 사용할지 정해야 합니다.

AI에게 저장과 불러오기 기능을 요청할 때는 저장 대상과 시점을 구체적으로 전달해야 합니다. “저장 시스템을 만들어줘”보다 “플레이어의 Level, CurrentExp, Gold, InventoryItems, ClearedStageIds를 저장한다. 스테이지 클리어와 아이템 구매 후 자동 저장하며, 게임 시작 시 저장 데이터가 있으면 불러오고 없으면 기본 데이터를 생성한다”라고 요청하는 편이 좋습니다. 좋은 저장 기획은 데이터를 보관하는 기능을 넘어서, 플레이어의 진행 상태를 안전하게 유지하고 복구하는 기준 문서입니다.

Chapter 85. 네트워크가 끊겼을 때의 처리 작성하기

온라인 기능이 있는 게임에서는 네트워크가 항상 안정적이라고 가정하면 안 됩니다. 로그인 중 연결이 끊길 수 있고, 상점 구매 도중 응답이 오지 않을 수 있으며, 멀티플레이 도중 서버와 연결이 끊길 수도 있습니다. 네트워크 끊김 처리가 기획서에 없으면 구현자는 어떤 데이터를 유지하고, 어떤 화면으로 보내고, 어떤 메시지를 보여줘야 하는지 판단하기 어렵습니다.

먼저 네트워크가 필요한 기능을 구분해야 합니다. 로그인, 서버 데이터 불러오기, 결제, 랭킹, 우편함, 멀티플레이 매칭, 실시간 전투처럼 서버와 통신하는 기능은 모두 끊김 상황을 고려해야 합니다. 반대로 완전한 싱글 플레이 기능이라면 네트워크가 끊겨도 계속 진행할 수 있습니다. 기능별로 “네트워크가 반드시 필요한지”, “오프라인에서도 가능한지”를 정리하는 것이 좋습니다.

네트워크가 끊겼을 때의 화면 처리도 명확히 작성해야 합니다. 플레이어에게 “네트워크 연결이 끊겼습니다”라는 메시지를 보여줄지, 재시도 버튼을 제공할지, 타이틀 화면으로 돌려보낼지 정해야 합니다. 일시적인 통신 실패라면 자동 재시도를 할 수 있고, 장시간 연결 실패라면 플레이어가 직접 재시도하거나 게임을 종료하도록 안내할 수 있습니다. 중요한 것은 플레이어가 현재 상황을 이해할 수 있게 만드는 것입니다.

프로그래머 관점에서는 네트워크 끊김이 데이터 무결성과 연결됩니다. 보상 지급, 상점 구매, 결제, 아이템 사용처럼 데이터가 변경되는 도중 연결이 끊기면 중복 지급이나 데이터 손실이 발생할 수 있습니다. 따라서 요청을 보냈지만 응답을 받지 못한 경우, 서버에 다시 상태를 확인할지, 로컬 처리를 취소할지, 임시 대기 상태로 돌지 기획서에 작성해야 합니다. 특히 재화와 아이템이 오가는 기능은 반드시 예외 조건을 정의해야 합니다.

AI에게 네트워크 끊김 처리를 요청할 때는 상황별 규칙을 구체적으로 전달해야 합니다. “네트워크 오류 처리를 해줘”보다 “서버 요청 중 네트워크가 끊기면 로딩 UI를 닫고 재시도 팝업을 표시한다. 상점 구매 요청 중 응답을 받지 못한 경우에는 로컬 재화를 차감하지 않고, 재접속 후 서버 구매 결과를 다시 확인한다”라고 요청하는 편이 좋습니다. 좋은 네트워크 예외 기획은 오류 메시지를 띄우는 문서가 아니라, 연결 실패 상황에서도 데이터와 플레이 흐름을 안전하게 보호하는 기준 문서입니다.

Chapter 86. 보상이 중복 지급되지 않게 조건 작성하기

보상 시스템에서 가장 조심해야 할 문제 중 하나는 중복 지급입니다. 플레이어가 스테이지 클리어 보상을 여러 번 받거나, 퀘스트 완료 보상을 반복해서 받거나, 네트워크 재시도 중 같은 아이템이 두 번 지급되면 게임 경제와 진행 상태가 쉽게 무너질 수 있습니다. 따라서 상세 기획서에는 보상이 언제 지급되고, 어떤 경우에는 다시 지급되지 않는지 명확히 작성해야 합니다.

먼저 보상 종류별로 중복 지급 가능 여부를 구분해야 합니다. 최초 클리어 보상은 한 번만 지급되어야 하고, 반복 클리어 보상은 매번 지급될 수 있습니다. 일일 보상은 하루에 한 번만 받을 수 있고, 이벤트 보상은 계정당 한 번만 받을 수 있습니다. 이처럼 보상마다 지급 횟수와 초기화 조건이 다르므로 기획서에 “1회 지급”, “반복 지급”, “일일 초기화”, “주간 초기화”처럼 기준을 적어두는 것이 좋습니다.

중복 지급을 막으려면 보상 지급 상태를 저장해야 합니다. 예를 들어 `ClearedStageIds`, `ClaimedRewardIds`, `CompletedQuestIds` 같은 데이터를 저장하면 이미 받은 보상인지 확인할 수 있습니다. 보상을 지급하기 전에는 반드시 해당 보상 ID가 이미 처리되었는지 검사해야 합니다. 이미 지급된 보상이라면 다시 지급하지 않고, 필요하다면 “이미 수령한 보상입니다”라는 메시지를 보여줄 수 있습니다.

프로그래머 관점에서는 보상 지급 순서도 중요합니다. 보상을 먼저 지급하고 나중에 지급 완료 상태를 저장하면, 저장 전에 오류가 발생했을 때 같은 보상이 다시 지급될 수 있습니다. 반대로 지급 완료 상태만 저장하고 실제 보상 지급에 실패하면 플레이어가 보상을 받지 못하는 문제가 생길 수 있습니다. 따라서 로컬 게임에서는 지급과 저장을 하나의 흐름으로 묶고, 서버 기반 게임에서는 서버가 최종 지급 여부를 판단하도록 설계하는 것이 안전합니다.

AI에게 보상 중복 방지 로직을 요청할 때는 지급 조건과 저장 기준을 함께 전달해야 합니다. “보상 지급 시스템을 만들어줘”보다 “보상 지급 전 `ClaimedRewardIds`에 `RewardId`가 있는지 확인한다. 없으면 보상을 지급하고 `RewardId`를 저장하며, 이미 있으면 보상을 지급하지 않는다. 최초 클리어 보상은 1회만 지급하고 반복 클리어 보상은 매번 지급한다”라고 요청하는 편이 좋습니다. 좋은 보상 기획은 무엇을 줄지 정하는 데서 끝나지 않고, 보상이 정확히 한 번만 지급되어야 하는 상황과 반복 지급되어야 하는 상황을 명확히 구분하는 문서입니다.

Chapter 87. 비정상 입력과 악용 가능성 정리하기

게임 시스템은 플레이어가 정상적인 순서로만 행동한다고 가정하면 쉽게 깨질 수 있습니다. 버튼을 매우 빠르게 여러 번 누르거나, 구매 요청을 반복하거나, 사망 상태에서 공격 입력을 하거나, 보상 수령 버튼을 중복 클릭하는 상황이 발생할 수 있습니다. 이런 비정상 입력과 악용 가능성을 미리 정리하지 않으면 중복 지급, 잘못된 상태 변경, UI 오류 같은 버그로 이어질 수 있습니다.

비정상 입력은 의도하지 않은 타이밍이나 상태에서 들어오는 입력을 의미합니다. 예를 들어 스킬 쿨타임 중에 스킬 버튼을 누르거나, 로딩 중에 다른 버튼을 누르거나, 팝업이 열린 상태에서 뒤쪽 화면의 버튼을 누르는 경우가 있습니다. 이런 상황에서는 입력을 무시할지, 안내 메시지를 보여줄지, 입력 자체를 차단할지 정해야 합니다. 기획서에는 기능별로 허용 상태와 차단 상태를 작성하는 것이 좋습니다.

악용 가능성은 플레이어가 시스템의 허점을 이용해 부당한 이득을 얻을 수 있는 경우입니다. 상점 구매 버튼을 연속으로 눌러 아이템을 중복 구매하거나, 보상 수령 요청을 반복해 같은 보상을 여러 번 받거나, 네트워크 오류를 이용해 재화 차감 없이 아이템을 얻는 상황이 있을 수 있습니다. 특히 재화, 보상, 아이템, 저장 데이터와 연결된 기능은 악용 가능성을 더 꼼꼼히 검토해야 합니다.

프로그래머 관점에서는 비정상 입력을 막기 위해 상태 검증과 중복 실행 방지 처리가 필요합니다. 버튼을 누른 직후에는 일정 시간 입력을 잠그거나, 요청 처리 중에는 같은 요청을 다시 보내지 않도록 막을 수 있습니다. 또한 기능을 실행하기 전에 현재 상태가 유효한지 확인해야 합니다. 예를 들어 플레이어가 사망 상태라면 이동, 공격, 스킬 사용 입력을 모두 무시해야 합니다.

AI에게 비정상 입력 처리를 요청할 때는 차단 조건과 처리 방식을 구체적으로 전달해야 합니다. “버그가 없게 만들어줘”보다 “구매 버튼을 누른 뒤 서버 응답이 올 때까지 버튼을 비활성화한다. 이미 보상을 수령한 RewardId는 다시 처리하지 않는다. 플레이어가 Dead 상태일 때는 공격과 이동 입력을 무시한다”라고 요청하는 편이 좋습니다. 좋은 상세 기획서는 정상 흐름뿐 아니라, 플레이어가 의도치 않게 또는 의도적으로 시스템을 흔들 수 있는 상황까지 고려하는 문서입니다.

Chapter 88. 상태 전환표 작성하기

상태 전환표는 게임 오브젝트나 시스템이 어떤 상태에서 어떤 조건을 만나면 다른 상태로 바뀌는지 정리한 표입니다. 플레이어 캐릭터, 적 AI, 스테이지 진행, UI 팝업, 퀘스트 시스템처럼 상태가 변하는 기능에는 전환 규칙이 필요합니다. 상태 전환표가 없으면 구현자는 “언제 이동 상태가 공격 상태로 바뀌는지”, “사망 상태에서 입력을 받아야 하는지” 같은 판단을 코드 작성 중에 임의로 하게 됩니다.

상태 전환표를 작성할 때는 먼저 상태 목록을 정의해야 합니다. 플레이어 캐릭터라면 Idle, Move, Attack, Hit, Dead 같은 상태가 있을 수 있습니다. 적 AI라면 Patrol, Chase, Attack, Return, Dead 같은 상태를 사용할 수 있습니다. 스테이지 진행이라면 Ready, Playing, Cleared, Failed 같은 상태로 정리할 수 있습니다. 중요한 것은 각 상태의 의미가 명확해야 한다는 점입니다.

다음으로 상태가 바뀌는 조건을 작성합니다. 예를 들어 플레이어가 Idle 상태에서 이동 입력을 받으면 Move 상태로 전환하고, 공격 입력을 받으면 Attack 상태로 전환할 수 있습니다. 체력이 0이 되면 어떤 상태에서도 Dead 상태로 전환되어야 합니다. 적 AI는 플레이어를 감지하면 Patrol에서 Chase로 바뀌고, 공격 범위 안에 들어오면 Attack으로 전환될 수 있습니다. 이런 조건을 표로 정리하면 누락된 전환을 찾기 쉽습니다.

프로그래머 관점에서는 상태 전환표가 상태 머신 구현의 기준이 됩니다. 표에는 현재 상태, 전환 조건, 다음 상태, 전환 시 실행할 동작을 포함할 수 있습니다. 예를 들어 “Attack 상태 진입 시 공격 애니메이션 재생”, “Dead 상태 진입 시 입력 차단과 사망 연출 실행”처럼 작성할 수 있습니다. 전환 조건과 진입 동작을 분리해두면 코드 구조도 더 명확해집니다.

AI에게 상태 머신 구현을 요청할 때는 상태 전환표를 문장으로 풀어 전달해야 합니다. “캐릭터 상태 머신을 만들어줘”보다 “플레이어 상태는 Idle, Move, Attack, Hit, Dead로 구성한다. 이동 입력이 있으면 Idle에서 Move로 전환하고, 공격 입력이 있으면 Idle 또는 Move에서 Attack으로 전환한다. 체력이 0이 되면 모든 상태에서 Dead로 전환한다”라고 요청하는 편이 좋습니다. 좋은 상태 전환표는 복잡한 조건을 한눈에 정리해, 게임 상태가 예측 가능한 방식으로 바뀌도록 만드는 기준 문서입니다.

Chapter 89. 플로우차트로 시스템 흐름 정리하기

플로우차트는 시스템의 흐름을 순서와 조건으로 시각화하는 문서입니다. 상세 기획서에서 문장으로만 설명하면 “어떤 조건을 먼저 확인하는지”, “실패했을 때 어디로 이동하는지”, “반복되는 처리가 있는지”를 한눈에 파악하기 어렵습니다. 플로우차트를 사용하면 게임 시작, 상점 구매, 보상 지급, 스테이지 클리어, 저장과 불러오기 같은 흐름을 더 명확하게 정리할 수 있습니다.

플로우차트를 작성할 때는 먼저 시작 지점과 종료 지점을 정해야 합니다. 예를 들어 상점 구매 흐름이라면 시작은 “구매 버튼 클릭”이고, 종료는 “구매 성공” 또는 “구매 실패”가 될 수 있습니다. 그 사이에는 보유 재화 확인, 인벤토리 공간 확인, 아이템 지급, 재화 차감, UI 갱신 같은 단계가 들어갑니다. 각 단계가 어떤 순서로 실행되는지 정리하면 구현자가 로직을 이해하기 쉬워집니다.

조건 분기는 플로우차트에서 특히 중요합니다. “골드가 충분한가?”, “인벤토리에 공간이 있는가?”, “이미 보상을 받았는가?” 같은 질문을 기준으로 흐름이 나뉩니다. 조건을 만족하면 다음 처리로 이동하고, 만족하지 않으면 메시지를 출력하거나 처리를 중단합니다. 이런 분기를 미리 작성하면 예외 조건을 빠뜨릴 가능성이 줄어듭니다.

프로그래머 관점에서 플로우차트는 코드의 조건문과 함수 호출 순서를 설계하는 데 도움이 됩니다. 구매 시스템이라면 CanPurchase, SpendCurrency, AddItem, RefreshUI 같은 함수 흐름으로 바꿀 수 있습니다. 스테이지 클리어 흐름이라면 목표 완료 확인, 보상 지급, 클리어 상태 저장, 결과 화면 표시 순서로 나눌 수 있습니다. 플로우차트는 기획 문서이면서 동시에 구현 흐름의 설계도 역할을 합니다.

AI에게 시스템 구현을 요청할 때도 플로우차트 내용을 문장으로 풀어 전달하면 좋습니다. “구매 시스템을 만들어줘”보다 “구매 버튼을 누르면 먼저 골드가 충분한지 확인하고, 부족하면 메시지를 출력한다. 충분하면 인벤토리 공간을 확인하고, 공간이 있으면 골드를 차감한 뒤 아이템을 추가하고 UI를 갱신한다”라고 요청하는 편이 좋습니다. 좋은 플로우차트는 복잡한 시스템을 단순한 실행 순서와 조건 분기로 나누어, 구현자가 실수 없이 따라갈 수 있게 만드는 문서입니다.

Part 11. 상세 기획서를 AI 프롬프트로 바꾸기

Part 11에서는 작성한 상세 기획서를 AI에게 전달할 수 있는 개발 요청 프롬프트로 바꾸는 방법을 다룹니다.

AI는 모호한 요청보다 목적, 조건, 입력, 출력, 제한 사항이 명확한 요청에서 더 좋은 결과를 냅니다.

이 파트에서는 기획서를 기능 구현 요청, 클래스 설계 요청, UI 구현 요청, 테스트 코드 요청으로 변환하는 방법을 설명합니다.

또한 AI가 만든 결과를 검토하고, 잘못된 결과를 다시 요청하며, 프롬프트 수정 이력을 관리하는 방법도 함께 다룹니다.

Chapter 90. AI는 모호한 기획서를 구현하지 못한다

AI는 빠르게 코드를 작성할 수 있지만, 모호한 기획 의도를 스스로 정확히 해석하지는 못합니다. “재미 있는 전투 시스템을 만들어줘”, “상점 기능을 적당히 구현해줘”, “좋은 UI를 만들어줘” 같은 요청은 결과가 매년 달라질 수 있습니다. AI는 사용자가 제공한 정보 안에서 판단하기 때문에, 기획서가 모호하면 구현 결과도 모호해집니다.

모호한 기획서의 문제는 구현 기준이 없다는 점입니다. 예를 들어 “적을 처치하면 보상을 준다”라고만 작성하면 어떤 보상을 줄지, 몇 개를 줄지, 확률은 어떻게 되는지, 이미 받은 보상은 다시 줄 수 있는지 알 수 없습니다. “레벨업하면 강해진다”라는 표현도 마찬가지입니다. 체력이 오르는지, 공격력이 오르는지, 어느 정도 증가하는지, 최대 레벨이 있는지 명확하지 않으면 AI는 임의로 구조를 만들어낼 수밖에 없습니다.

AI에게 좋은 결과를 얻으려면 기능의 목적, 입력, 처리 과정, 결과, 예외 조건을 함께 작성해야 합니다. 상점 구매 기능이라면 “구매 버튼을 누른다”가 입력이고, “보유 골드를 확인한다”가 처리 과정이며, “골드를 차감하고 아이템을 지급한다”가 결과입니다. 여기에 “골드가 부족하면 구매를 중단하고 메시지를 출력한다”라는 예외 조건까지 포함해야 구현 기준이 분명해집니다.

프로그래머 관점에서 기획서는 AI에게 전달할 명세서입니다. AI는 클래스 이름, 데이터 필드, 상태 전환, UI 요소, 조건 분기를 명확히 알려줄수록 더 안정적인 코드를 작성합니다. 반대로 기획서에 “적당히”, “자연스럽게”, “알아서”, “좋은 느낌으로” 같은 표현이 많으면 결과를 검토하고 수정하는 시간이 더 늘어납니다. 결국 모호함은 개발 시간을 줄이는 것이 아니라 뒤로 미루는 일에 가깝습니다.

AI에게 구현을 요청할 때는 모호한 표현을 구현 가능한 문장으로 바꾸어야 합니다. “인벤토리를 편하게 만들어줘”보다 “인벤토리는 장비, 소비, 재료 탭으로 나뉘며, 아이템을 선택하면 상세 정보와 사용 또는 장착 버튼을 표시한다”라고 작성하는 편이 좋습니다. 좋은 상세 기획서는 AI가 추측해야 할 부분을 줄이고, 구현자가 따라야 할 조건을 명확히 제시하는 문서입니다.

Chapter 91. 좋은 개발 요청 프롬프트의 구조

좋은 개발 요청 프롬프트는 AI에게 “무엇을 만들지”만 말하지 않습니다. 어떤 목적의 기능인지, 어떤 데이터를 사용해야 하는지, 어떤 순서로 처리해야 하는지, 어떤 예외 조건을 고려해야 하는지 함께 전달합니다. AI는 요청이 구체적일수록 더 안정적인 코드를 제안할 수 있습니다. 따라서 프롬프트는 짧은 명령문이 아니라, 구현자가 따라야 할 작업 명세에 가깝게 작성하는 것이 좋습니다.

개발 요청 프롬프트의 기본 구조는 “목적 → 환경 → 요구 기능 → 데이터 구조 → 처리 흐름 → 예외 조건 → 출력 형식”으로 정리할 수 있습니다. 예를 들어 Unity에서 상점 시스템을 만든다면 먼저 “Unity 6 C#으로 상점 구매 기능을 구현한다”라고 환경을 알려주고, 그다음 “보유 골드를 확인하고, 충분하면 아이템을 지급한다”처럼 요구 기능을 설명합니다. 마지막으로 골드 부족, 인벤토리 부족 같은 예외 조건을 덧붙입니다.

프롬프트에는 현재 프로젝트의 전제도 포함해야 합니다. 사용하는 엔진, 언어, UI 방식, 데이터 관리 방식, 네임스페이스, 기존 클래스 이름을 알려주면 AI가 더 일관된 코드를 만들 수 있습니다. 예를 들어 “아이템 데이터는 ScriptableObject인 ItemData를 사용하고, 인벤토리는 InventoryManager가 관리한다”라고 적으면 AI가 임의의 구조를 새로 만들 가능성이 줄어듭니다.

요구 기능은 가능한 한 동작 단위로 나누어 작성하는 것이 좋습니다. “인벤토리를 만들어줘”보다 “아이템 추가, 아이템 제거, 수량 증가, 수량 감소, 아이템 사용, UI 갱신 기능을 구현한다”라고 작성하는 편이 명확합니다. 기능을 작은 단위로 나누면 AI가 놓치는 부분을 줄일 수 있고, 결과를 검토하기도 쉬워집니다.

AI에게 좋은 결과를 얻으려면 프롬프트 안에 판단 기준을 넣어야 합니다. “좋게 만들어줘”보다 “구매 성공 시 골드를 차감하고 아이템을 추가한다. 실패 시 데이터는 변경하지 않는다. 모든 결과는 bool로 반환하고 실패 이유는 메시지로 제공한다”처럼 작성하는 것이 좋습니다. 좋은 개발 요청 프롬프트는 AI에게 아이디어를 던지는 문장이 아니라, 구현 조건과 완료 기준을 명확히 전달하는 개발 지시서입니다.

Chapter 92. 기획서를 기능 구현 요청으로 변환하기

상세 기획서는 사람이 읽기 위한 문서이지만, AI와 함께 개발할 때는 이를 기능 구현 요청으로 바꾸는 과정이 필요합니다. 기획서에는 목적, 규칙, 예외 조건이 문장으로 정리되어 있습니다. 그러나 AI에게 그대로 전달하면 범위가 넓거나 추상적으로 해석될 수 있습니다. 따라서 기획서의 내용을 구현 가능한 기능 단위로 나누고, 각 기능을 명확한 프롬프트로 변환해야 합니다.

먼저 기획서에서 구현해야 할 동사를 찾아야 합니다. “아이템을 구매한다”, “경험치를 획득한다”, “레벨업한다”, “보상을 지급한다”, “스테이지를 클리어한다” 같은 문장은 기능 구현 후보입니다. 이 동사를 기준으로 입력, 처리, 결과를 나누면 AI에게 전달하기 쉬운 요청이 됩니다. 예를 들어 “아이템을 구매한다”는 “구매 버튼 입력 → 재화 확인 → 아이템 지급 → UI 갱신”으로 분해할 수 있습니다.

기능 구현 요청에는 필요한 데이터도 함께 포함해야 합니다. 상점 구매 기능이라면 보유 재화, 상품 가격, 아이템 ID, 인벤토리 공간 정보가 필요합니다. 레벨업 기능이라면 현재 경험치, 필요 경험치, 레벨, 능력치 증가량이 필요합니다. AI는 어떤 데이터를 사용해야 하는지 알려주지 않으면 임의로 클래스를 만들거나 기존 구조와 맞지 않는 코드를 제안할 수 있습니다.

예외 조건도 반드시 구현 요청에 포함해야 합니다. 정상 흐름만 전달하면 골드 부족, 인벤토리 부족, 이미 받은 보상, 쿨타임 중 입력 같은 상황이 누락될 수 있습니다. 예를 들어 “골드가 부족하면 구매를 실패시키고 데이터는 변경하지 않는다”처럼 실패 조건을 명확히 작성해야 합니다. 예외 조건은 코드 품질과 버그 발생 가능성에 직접 영향을 줍니다.

AI에게 기능 구현을 요청할 때는 “기획 문장 → 기능 단위 → 데이터 → 처리 흐름 → 예외 조건” 순서로 변환하는 것이 좋습니다. “상점 시스템을 구현해줘”보다 “구매 버튼을 누르면 ShopItemData의 Price와 플레이어 Gold를 비교한다. Gold가 충분하면 Gold를 차감하고 ItemId를 인벤토리에 추가한다. Gold가 부족하면 토스트 메시지를 표시하고 아무 데이터도 변경하지 않는다”라고 작성하는 편이 좋습니다. 좋은 변환 과정은 기획서를 개발자가 바로 실행할 수 있는 작업 지시로 바꾸는 과정입니다.

Chapter 93. 기획서를 클래스 설계 요청으로 변환하기

상세 기획서는 기능의 목적과 규칙을 설명하지만, 실제 구현에서는 이 내용을 클래스 구조로 바꾸어야 합니다. AI에게 코드를 요청할 때도 단순히 “상점 시스템을 만들어줘”라고 하기보다, 어떤 클래스가 필요하고 각 클래스가 어떤 책임을 가져야 하는지 요청하는 편이 좋습니다. 클래스 설계 요청은 기획서를 코드 구조로 연결하는 중요한 단계입니다.

먼저 기획서에서 주요 명사를 찾아야 합니다. 아이템, 인벤토리, 상점, 플레이어, 스킬, 퀘스트, 보상 같은 단어는 클래스 후보가 될 수 있습니다. 예를 들어 아이템 기획서에서는 ItemData, InventoryItem, InventoryManager 같은 클래스가 필요할 수 있습니다. 상점 기획서에서는 ShopItemData, ShopManager, ShopUI처럼 데이터, 로직, UI를 나누어 생각할 수 있습니다.

다음으로 각 클래스의 책임을 정리해야 합니다. ItemData는 아이템의 고정 정보를 보관하고, InventoryManager는 아이템 추가와 제거를 처리하며, InventoryUI는 아이템 목록을 화면에 표시합니다. 하나의 클래스가 데이터 관리, 구매 처리, UI 갱신을 모두 맡으면 구조가 복잡해집니다. 따라서 AI에게 요청할 때도 “데이터 클래스, 로직 클래스, UI 클래스는 분리한다”처럼 책임 분리를 명확히 전달하는 것이 좋습니다.

클래스 설계 요청에는 필드와 메서드도 포함할 수 있습니다. 예를 들어 InventoryManager에는 AddItem, RemoveItem, HasItem, GetItemCount 같은 메서드가 필요할 수 있습니다. ShopManager에는 CanPurchase, PurchaseItem, GetShopItems 같은 메서드를 요청할 수 있습니다. 이처럼 기획서의 동작을 메서드로 바꾸면 AI가 더 구체적인 코드를 작성할 수 있습니다.

AI에게 클래스 설계를 요청할 때는 “기획 내용 → 클래스 후보 → 클래스 책임 → 주요 필드 → 주요 메서드” 순서로 전달하는 것이 좋습니다. “인벤토리 시스템 클래스를 설계해줘”보다 “ItemData는 아이템 고정 정보를 가진다. InventoryManager는 보유 아이템 수량을 관리하고 AddItem, RemoveItem을 제공한다. InventoryUI는 InventoryManager의 데이터를 받아 목록을 갱신한다”라고 요청하는 편이 좋습니다. 좋은 클래스 설계 요청은 기획서를 코드로 바로 옮기기 전에, 시스템의 책임과 구조를 먼저 정리하는 개발 설계 문서입니다.

Chapter 94. 기획서를 UI 구현 요청으로 변환하기

UI 기획서는 화면의 목적, 구성 요소, 표시 데이터, 버튼 동작, 화면 전환 흐름을 정리한 문서입니다. 하지만 AI에게 UI 구현을 요청할 때는 이 내용을 그대로 붙여 넣기보다 구현 가능한 요청으로 바꾸어야 합니다. “상점 화면을 만든다”라는 기획 문장은 너무 넓습니다. AI가 실제로 구현하려면 어떤 UI 요소가 필요하고, 어떤 데이터를 표시하며, 어떤 입력에 어떤 동작을 해야 하는지 알아야 합니다.

먼저 UI 화면의 목적을 한 문장으로 정리합니다. 예를 들어 “상점 화면은 플레이어가 보유 골드를 사용해 아이템을 구매하는 화면이다”처럼 작성합니다. 그다음 화면에 필요한 UI 요소를 나열합니다. 보유 골드 텍스트, 상품 목록, 상품 아이콘, 가격 텍스트, 구매 버튼, 닫기 버튼처럼 실제로 생성하거나 연결해야 할 요소를 구체적으로 적어야 합니다.

다음으로 각 UI 요소가 어떤 데이터와 연결되는지 작성합니다. 보유 골드 텍스트는 플레이어의 Gold 값을 표시하고, 상품 목록은 ShopItemData 리스트를 기반으로 생성하며, 구매 버튼은 선택한 상품의 가격과 보유 골드를 비교해야 합니다. UI는 단순한 화면 배치가 아니라 데이터와 상호작용하는 구조입니다. 따라서 데이터 연결 규칙이 빠지면 AI는 임의의 변수나 클래스를 만들어낼 가능성이 높습니다.

버튼 동작과 예외 조건도 구현 요청에 포함해야 합니다. 구매 버튼을 누르면 재화가 충분한지 확인하고, 충분하면 재화를 차감한 뒤 아이템을 인벤토리에 추가합니다. 재화가 부족하면 구매를 중단하고 토스트 메시지를 출력합니다. 닫기 버튼을 누르면 상점 화면을 닫거나 로비 화면으로 돌아갑니다. 이런 흐름을 구체적으로 작성해야 UI가 실제 게임 시스템과 연결됩니다.

AI에게 UI 구현을 요청할 때는 “화면 목적 → UI 요소 → 연결 데이터 → 버튼 동작 → 예외 조건 → 화면 전환” 순서로 작성하는 것이 좋습니다. “상점 UI를 만들어줘”보다 “상점 화면에는 보유 골드 텍스트, 상품 목록, 구매 버튼, 닫기 버튼이 있다. 상품 목록은 ShopItemData를 사용해 생성하고, 구매 버튼 클릭 시 Gold가 충분하면 아이템을 지급하며 부족하면 메시지를 표시한다”라고 요청하는 편이 좋습니다. 좋은 UI 구현 요청은 화면을 예쁘게 만드는 지시가 아니라, UI가 어떤 데이터와 기능을 연결해야 하는지 명확히 전달하는 개발 지시서입니다.

Chapter 95. 기획서를 테스트 코드 요청으로 변환하기

상세 기획서는 기능을 구현하는 기준이 될 뿐 아니라, 테스트 코드를 작성하는 기준도 됩니다. 기능이 의도대로 동작하는지 확인하려면 “무엇이 성공이고 무엇이 실패인지” 명확해야 합니다. 기획서에 입력 조건, 처리 결과, 예외 조건이 잘 정리되어 있다면 이를 그대로 테스트 케이스로 바꿀 수 있습니다. 좋은 기획서는 구현 요청뿐 아니라 검증 요청으로도 활용할 수 있습니다.

먼저 기획서에서 테스트해야 할 조건을 찾아야 합니다. 예를 들어 상점 구매 기획서라면 “골드가 충분하면 구매 성공”, “골드가 부족하면 구매 실패”, “인벤토리가 가득 차면 구매 실패” 같은 조건이 테스트 대상입니다. 레벨업 시스템이라면 “경험치가 필요 경험치 이상이면 레벨업”, “초과 경험치는 다음 레벨 경험치로 유지”, “최대 레벨에서는 더 이상 레벨업하지 않음” 같은 조건을 테스트할 수 있습니다.

테스트 코드 요청에는 입력값과 기대 결과를 함께 작성해야 합니다. “상점 구매 테스트를 만들어줘”보다 “플레이어 골드가 100이고 아이템 가격이 50일 때 구매하면 골드는 50이 되고 인벤토리에 아이템이 추가되어야 한다”라고 작성하는 편이 좋습니다. 테스트는 기능 설명이 아니라 검증 가능한 상황과 결과를 요구해야 합니다.

예외 조건은 테스트 코드에서 특히 중요합니다. 정상 흐름만 테스트하면 실제 플레이 중 발생하는 문제를 놓치기 쉽습니다. 골드 부족, 아이템 수량 0, 잘못된 itemId, 이미 수령한 보상, 쿨타임 중 스킬 사용 같은 상황을 테스트해야 합니다. 기획서에 예외 조건이 잘 정리되어 있으면 AI에게 테스트 코드 작성을 요청할 때도 빠뜨릴 가능성이 줄어듭니다.

AI에게 테스트 코드 작성을 요청할 때는 “기능명 → 테스트 조건 → 입력 데이터 → 기대 결과 → 사용 테스트 프레임워크” 순서로 전달하는 것이 좋습니다. 예를 들어 “Unity Test Framework를 사용해 ShopManager의 PurchaseItem 테스트를 작성한다. 골드가 충분하면 true를 반환하고 골드를 차감하며, 골드가 부족하면 false를 반환하고 데이터는 변경하지 않아야 한다”라고 요청할 수 있습니다. 좋은 테스트 코드 요청은 AI에게 코드를 만들게 하는 것이 아니라, 기획서의 규칙이 실제로 지켜지는지 검증하게 만드는 개발 지시서입니다.

Chapter 96. AI가 만든 결과를 검토하는 기준

AI가 만든 코드는 빠르게 결과를 보여주지만, 항상 기획 의도와 정확히 맞는 것은 아닙니다. AI는 사용자의 요청을 바탕으로 그럴듯한 구조를 만들 수 있지만, 프로젝트의 기존 코드 스타일, 데이터 구조, 예외 조건, 성능 기준까지 완벽히 이해한다고 볼 수 없습니다. 따라서 AI가 만든 결과는 그대로 사용하는 것이 아니라, 반드시 검토하고 수정하는 과정을 거쳐야 합니다.

첫 번째 검토 기준은 기획서의 요구사항을 충족하는지입니다. 요청한 기능이 모두 구현되었는지, 빠진 조건은 없는지, 예외 상황이 처리되었는지 확인해야 합니다. 예를 들어 상점 구매 기능이라면 골드가 충분할 때 구매되는지, 골드가 부족할 때 실패하는지, 인벤토리가 가득 찼을 때 데이터가 변경되지 않는지 검토해야 합니다. 정상 흐름만 동작한다고 해서 완성된 기능이라고 판단하면 안 됩니다.

두 번째 기준은 기존 프로젝트 구조와 잘 맞는지입니다. AI가 새로운 클래스를 임의로 만들거나, 이미 존재하는 Manager나 Data 클래스를 무시하고 다른 구조를 제안할 수 있습니다. 이 경우 코드는 동작 하더라도 프로젝트 전체 구조가 복잡해질 수 있습니다. 따라서 네임스페이스, 클래스 역할, 데이터 참조 방식, UI 갱신 방식이 기존 코드와 일관되는지 확인해야 합니다.

세 번째 기준은 데이터와 상태 변경이 안전한지입니다. 재화 차감, 아이템 지급, 경험치 증가, 저장 데이터 변경처럼 플레이어 진행에 영향을 주는 코드는 특히 주의해서 봐야 합니다. 같은 보상이 중복 지급되지 않는지, 실패 상황에서 데이터가 변경되지 않는지, 저장과 불러오기 흐름이 깨지지 않는지 검토해야 합니다. AI가 만든 코드에서 가장 위험한 부분은 “대충 동작하는 것처럼 보이지만 데이터가 틀어지는 코드”입니다.

AI에게 다시 수정을 요청할 때는 검토 결과를 구체적으로 전달해야 합니다. “이 코드 이상해”보다 “골드가 부족한 경우에도 인벤토리에 아이템이 추가된다. 구매 실패 시에는 Gold와 InventoryItems가 변경되지 않도록 수정해줘”라고 요청하는 편이 좋습니다. 좋은 검토 기준은 AI 결과를 믿을지 말지 판단하는 감각이 아니라, 기획서의 요구사항과 프로젝트 구조, 데이터 안정성을 기준으로 결과물을 점검하는 절차입니다.

Chapter 97. 잘못된 결과를 다시 요청하는 방법

AI가 만든 결과가 항상 한 번에 맞지는 않습니다. 요청한 기능이 일부 빠져 있거나, 기존 프로젝트 구조와 맞지 않거나, 예외 조건을 처리하지 못하는 경우가 있습니다. 이때 중요한 것은 “다시 해줘”라고 막연히 요청하는 것이 아니라, 무엇이 잘못되었고 어떤 방향으로 수정해야 하는지 구체적으로 알려주는 것입니다. 잘못된 결과를 다시 요청하는 능력은 AI와 함께 개발할 때 매우 중요합니다.

먼저 잘못된 부분을 기능 단위로 확인해야 합니다. 예를 들어 상점 구매 기능에서 골드가 부족한데도 아이템이 추가된다면, 문제는 “구매 실패 조건 처리”입니다. 인벤토리 UI가 갱신되지 않는다면 문제는 “데이터 변경 후 UI 반영”입니다. 이렇게 문제를 구체적인 기능이나 조건으로 나누면 AI에게 수정 요청을 명확히 전달할 수 있습니다.

수정 요청에는 현재 문제와 기대 결과를 함께 작성해야 합니다. “상점 코드가 이상해”보다 “현재 코드는 골드가 부족해도 아이템을 인벤토리에 추가한다. 골드가 부족하면 구매를 실패시키고, 골드와 인벤토리 데이터가 변경되지 않도록 수정해줘”라고 요청하는 편이 좋습니다. AI는 문제 상황과 원하는 결과를 함께 받아야 정확한 수정 방향을 잡을 수 있습니다.

기존 구조를 유지하라는 조건도 필요합니다. AI는 수정 과정에서 새로운 클래스를 만들거나, 기존 메서드 이름을 바꾸거나, 프로젝트 구조를 크게 변경할 수 있습니다. 따라서 “기존 ShopManager와 InventoryManager 구조는 유지하고, PurchaseItem 메서드 내부 로직만 수정해줘”처럼 변경 범위를 제한하는 것이 좋습니다. 수정 범위가 명확할수록 예상하지 못한 부작용을 줄일 수 있습니다.

AI에게 다시 요청할 때는 “문제 상황 → 원인으로 의심되는 부분 → 기대 동작 → 변경 제한” 순서로 작성하는 것이 좋습니다. 예를 들어 “구매 실패 시에도 UI가 성공처럼 갱신된다. 골드 부족이면 구매 실패 메시지만 표시하고, 인벤토리와 골드는 변경하지 않도록 수정해줘. 기존 클래스 이름과 public 메서드 시그니처는 바꾸지 말아줘”라고 요청할 수 있습니다. 좋은 재요청은 AI에게 다시 처음부터 만들게 하는 것이 아니라, 잘못된 부분을 좁혀 정확히 고치게 만드는 작업 지시입니다.

Chapter 98. 프롬프트 수정 이력 관리하기

AI와 함께 개발하다 보면 한 번의 프롬프트로 원하는 결과를 얻지 못하는 경우가 많습니다. 처음 요청한 내용이 모호했거나, AI가 기존 구조를 잘못 이해했거나, 예외 조건이 빠졌을 수 있습니다. 이때 프롬프트를 계속 수정하게 되는데, 어떤 요청을 했고 어떤 결과가 나왔으며 무엇을 고쳤는지 기록하지 않으면 같은 실수를 반복하기 쉽습니다. 그래서 프롬프트 수정 이력을 관리하는 습관이 필요합니다.

프롬프트 수정 이력에는 처음 요청한 프롬프트, AI가 만든 결과, 발견한 문제, 수정 요청 내용, 최종 반영 여부를 기록할 수 있습니다. 예를 들어 “상점 구매 기능 요청 → 골드 부족 예외 누락 → 골드 부족 시 구매 실패 조건 추가 요청 → 정상 동작 확인”처럼 정리할 수 있습니다. 이렇게 기록하면 나중에 비슷한 기능을 요청할 때 더 나은 프롬프트를 빠르게 만들 수 있습니다.

프로그래머 관점에서 프롬프트 수정 이력은 개발 로그와 비슷합니다. 어떤 기능을 어떤 기준으로 요청했는지 남겨두면 코드 변경 이유를 추적하기 쉬워집니다. 특히 AI가 생성한 코드가 여러 번 수정되었다면, 최종 코드만 봐서는 왜 이런 구조가 되었는지 알기 어렵습니다. 프롬프트와 수정 과정을 함께 남겨두면 기획 의도와 구현 과정이 연결됩니다.

프롬프트 이력은 너무 복잡하게 관리할 필요는 없습니다. 기능명, 날짜, 요청 목적, 사용한 프롬프트, 수정 사유, 최종 결과 정도만 기록해도 충분합니다. Git 커밋 메시지나 개발 일지, README, 별도 Markdown 문서에 정리할 수 있습니다. 중요한 것은 모든 대화를 완벽히 저장하는 것이 아니라, 다음 개발에 재사용할 수 있는 핵심 요청과 수정 이유를 남기는 것입니다.

AI에게 다시 비슷한 기능을 요청할 때는 이전 이력을 활용할 수 있습니다. “지난번 상점 구매 기능에서 골드 부족 예외가 누락되었으므로 이번에는 구매 전 조건 검사와 실패 시 데이터 미변경 조건을 반드시 포함해줘”처럼 작성하면 됩니다. 좋은 프롬프트 수정 이력은 AI 사용 기록을 보관하는 문서가 아니라, 개발자가 점점 더 정확한 요청을 만들 수 있게 돕는 학습 자료입니다.

Part 12. 미니 프로젝트로 상세 기획서 작성하기

Part 12에서는 지금까지 배운 내용을 바탕으로 하나의 미니 프로젝트 상세 기획서를 완성합니다.

게임 콘셉트 문서, 핵심 플레이 루프, 플레이어 조작 기획서, 전투 시스템 기획서, 아이템과 보상 기획서, UI 화면 기획서, 스테이지 기획서, 데이터 테이블을 순서대로 작성합니다.

마지막에는 이 기획서를 AI 구현 요청 프롬프트로 변환하고, 결과물을 검토하며 기획서를 수정하는 흐름까지 경험합니다.

이 파트의 목표는 단순히 문서 작성법을 아는 것이 아니라, 실제 개발에 사용할 수 있는 상세 기획서 한 벌을 완성하는 것입니다.

Chapter 99. 실습 프로젝트 개요 정하기

지금까지는 게임 상세 기획서를 구성하는 요소를 하나씩 살펴보았습니다. 이제부터는 작은 실습 프로젝트를 기준으로 실제 기획서를 작성해볼 차례입니다. 실습 프로젝트는 너무 크면 안 됩니다. 복잡한 오픈월드나 대규모 RPG를 대상으로 하면 기획서 작성 범위가 너무 넓어집니다. 이 장에서는 짧은 시간 안에 구조를 이해할 수 있는 작은 게임을 기준으로 개요를 정리합니다.

실습 프로젝트는 “작은 스테이지형 액션 RPG”로 가정할 수 있습니다. 플레이어는 하나의 캐릭터를 조작하고, 스테이지에 등장하는 적을 처치하며, 경험치와 골드를 얻고, 아이템을 획득합니다. 로비에서는 캐릭터 상태를 확인하고, 상점에서 포션을 구매할 수 있습니다. 스테이지를 클리어하면 보상을 받고 다음 스테이지로 이동할 수 있습니다.

이 프로젝트의 목적은 완성도 높은 상용 게임을 만드는 것이 아니라, 상세 기획서가 코드 구조와 AI 프롬프트로 어떻게 연결되는지 확인하는 것입니다. 따라서 기능 범위는 플레이어 조작, 전투, 적 AI, 경험치와 레벨업, 아이템과 보상, 인벤토리, 상점, 기본 UI 정도로 제한합니다. 기능을 작게 잡아야 각 시스템의 기획 문서와 구현 요청을 명확히 작성할 수 있습니다.

프로젝트 개요에는 게임의 장르, 목표, 핵심 플레이 루프, 주요 시스템, 화면 구성, 데이터 구조를 간단히 포함해야 합니다. 예를 들어 “플레이어는 스테이지에 입장해 적을 처치하고 보상을 얻은 뒤, 로비에서 성장과 준비를 하고 다음 스테이지에 도전한다”라고 정리할 수 있습니다. 이 한 문장이 이후 모든 기획서의 기준이 됩니다.

AI에게 실습 프로젝트 구현을 요청할 때도 프로젝트 개요가 먼저 필요합니다. “작은 게임을 만들어 줘”보다 “Unity 6 C#으로 작은 스테이지형 액션 RPG를 만든다. 플레이어는 적을 처치해 경험치와 골드를 얻고, 로비에서 상점과 인벤토리를 이용한 뒤 다음 스테이지에 도전한다”라고 작성하는 편이 좋습니다. 좋은 실습 프로젝트 개요는 기능을 많이 나열하는 문서가 아니라, 앞으로 작성할 상세 기획서의 범위와 방향을 정하는 출발점입니다.

Chapter 100. 게임 콘셉트 문서 작성하기

게임 콘셉트 문서는 실습 프로젝트가 어떤 게임인지 가장 짧고 명확하게 설명하는 문서입니다. 상세 기획서를 작성하기 전에 먼저 게임의 방향을 잡아야 합니다. 어떤 장르인지, 플레이어가 무엇을 하는지, 어떤 재미를 느끼게 할 것인지 정리하지 않으면 이후 시스템 기획, UI 기획, 데이터 설계가 흔들릴 수 있습니다. 콘셉트 문서는 모든 상세 기획의 기준점입니다.

실습 프로젝트의 콘셉트는 “짧은 스테이지를 반복해서 클리어하며 캐릭터를 성장시키는 액션 RPG”로 정리할 수 있습니다. 플레이어는 로비에서 스테이지를 선택하고, 전투 구간에 입장해 적을 처치합니다. 적을 처치하면 경험치와 골드를 얻고, 스테이지를 클리어하면 보상을 받습니다. 획득한 골드는 상점에서 포션을 구매하거나 성장 요소를 강화하는 데 사용할 수 있습니다.

게임 콘셉트 문서에는 핵심 재미도 포함해야 합니다. 이 실습 프로젝트의 핵심 재미는 “짧은 전투를 반복하며 캐릭터가 점점 강해지는 성장감”입니다. 복잡한 스토리나 대규모 콘텐츠보다, 작은 전투와 보상, 성장의 반복 구조에 집중합니다. 이렇게 핵심 재미를 한 문장으로 정리하면 어떤 기능을 넣고 어떤 기능을 제외할지 판단하기 쉬워집니다.

프로그래머 관점에서 콘셉트 문서는 구현 범위를 제한하는 역할도 합니다. “액션 RPG”라는 말만으로는 범위가 너무 넓습니다. 따라서 실습 프로젝트에서는 멀티플레이, 복잡한 장비 강화, 대규모 스킬 트리, 퀘스트 분기 같은 기능은 제외하고, 플레이어 이동, 공격, 적 처치, 경험치, 골드, 상점, 인벤토리, 스테이지 클리어만 포함한다고 정리할 수 있습니다. 범위를 줄여야 기획서와 구현 요청이 명확해집니다.

AI에게 프로젝트 구현을 요청할 때는 이 콘셉트 문서를 기반으로 전달해야 합니다. “액션 RPG를 만들어줘”보다 “Unity 6 C#으로 짧은 스테이지형 액션 RPG를 만든다. 플레이어는 적을 처치해 경험치와 골드를 얻고, 로비에서 상점과 인벤토리를 이용한 뒤 다음 스테이지에 도전한다. MVP 범위는 이동, 공격, 적 AI, 보상, 레벨업, 상점, 기본 UI로 제한한다”라고 요청하는 편이 좋습니다. 좋은 게임 콘셉트 문서는 게임의 매력을 설명하면서도, 구현할 범위와 제외할 범위를 명확히 정하는 출발 문서입니다.

Chapter 101. 핵심 플레이 루프 작성하기

핵심 플레이 루프는 플레이어가 게임 안에서 반복하게 되는 가장 중요한 행동 흐름입니다. 실습 프로젝트에서는 “로비에서 준비한다 → 스테이지에 입장한다 → 적을 처치한다 → 보상을 얻는다 → 캐릭터를 성장시킨다 → 다음 스테이지에 도전한다”가 기본 루프가 됩니다. 이 루프가 명확해야 어떤 시스템이 필요한지, 어떤 UI가 필요한지, 어떤 데이터를 저장해야 하는지 판단할 수 있습니다.

핵심 플레이 루프를 작성할 때는 플레이어의 행동을 순서대로 정리해야 합니다. 먼저 플레이어는 로비에서 현재 레벨과 보유 재화를 확인합니다. 이후 스테이지를 선택해 입장하고, 전투 중에는 이동과 공격을 사용해 적을 처치합니다. 스테이지를 클리어하면 경험치, 골드, 아이템 보상을 받고, 로비로 돌아와 상점이나 인벤토리를 이용합니다. 이 흐름이 다시 다음 스테이지 도전으로 이어집니다.

각 단계가 어떤 시스템과 연결되는지도 함께 작성해야 합니다. 로비 단계는 UI와 저장 데이터, 스테이지 선택 시스템과 연결됩니다. 전투 단계는 플레이어 조작, 적 AI, 데미지 계산, 체력 시스템과 연결됩니다. 보상 단계는 경험치, 재화, 아이템, 보상 테이블과 연결됩니다. 성장 단계는 레벨업, 능력치 상승, 인벤토리, 상점 시스템과 연결됩니다.

프로그래머 관점에서 핵심 플레이 루프는 기능 우선순위를 정하는 기준이 됩니다. 루프에 직접 포함되지 않는 기능은 MVP 단계에서 제외할 수 있습니다. 예를 들어 복잡한 퀘스트 분기나 장비 강화 시스템은 나중에 추가할 수 있지만, 적 처치, 보상 지급, 경험치 획득, 로비 복귀는 루프를 완성하기 위해 먼저 필요합니다. 핵심 루프가 완성되어야 게임은 작더라도 플레이 가능한 형태가 됩니다.

AI에게 구현을 요청할 때는 핵심 플레이 루프를 문장으로 명확히 전달해야 합니다. “액션 RPG를 만들어줘”보다 “플레이어는 로비에서 스테이지에 입장하고, 적을 처치해 경험치와 골드를 얻은 뒤, 스테이지 클리어 후 로비로 돌아와 성장과 정비를 하고 다음 스테이지에 도전한다”라고 작성하는 편이 좋습니다. 좋은 핵심 플레이 루프 문서는 게임의 반복 재미를 정의하고, 구현해야 할 주요 시스템의 우선순위를 정하는 기준 문서입니다.

Chapter 102. 플레이어 조작 기획서 작성하기

플레이어 조작 기획서는 플레이어가 캐릭터를 어떻게 움직이고 행동하게 할지 정리하는 문서입니다. 실습 프로젝트에서는 이동, 공격, 회피 또는 대시, 아이템 사용 정도의 조작만 정의해도 충분합니다. 조작 기획이 명확하지 않으면 입력 처리, 애니메이션 전환, 전투 판정, UI 안내가 서로 맞지 않을 수 있습니다. 따라서 조작 방식은 구현 전에 먼저 문서로 정리해야 합니다.

먼저 입력 방식을 정해야 합니다. PC 게임이라면 키보드와 마우스를 기준으로 WASD 이동, 마우스 좌클릭 공격, Space 대시, 숫자 키 아이템 사용처럼 작성할 수 있습니다. 모바일이라면 가상 조이스틱과 터치 버튼을 기준으로 정리할 수 있습니다. 입력 장치가 달라지면 UI와 코드 구조도 달라지므로, 실습 프로젝트에서는 우선 하나의 입력 방식을 기준으로 범위를 제한하는 것이 좋습니다.

다음으로 각 조작의 실행 조건을 정의해야 합니다. 이동은 살아 있는 상태에서만 가능하고, 공격은 쿨타임이 끝났을 때만 가능하며, 대시는 스테미나가 충분할 때만 가능하게 만들 수 있습니다. 사망 상태, 피격 상태, 로딩 중, 팝업이 열린 상태에서는 입력을 막아야 할 수도 있습니다. 조작 기획서에는 “언제 가능한가”뿐 아니라 “언제 불가능한가”도 함께 작성해야 합니다.

프로그래머 관점에서 조작 기획서는 입력, 상태, 애니메이션, 전투 시스템을 연결하는 기준입니다. 공격 입력이 들어오면 캐릭터 상태를 Attack으로 바꾸고, 공격 애니메이션을 재생하며, 특정 타이밍에 공격 판정을 발생시킬 수 있습니다. 대시 입력은 이동 방향, 대시 거리, 무적 여부, 쿨타임과 연결될 수 있습니다. 이런 연결 정보를 적어두면 AI에게 구현을 요청할 때도 훨씬 명확해집니다.

AI에게 플레이어 조작 구현을 요청할 때는 입력 키, 실행 조건, 결과 동작을 함께 전달해야 합니다. “플레이어 조작을 만들어줘”보다 “WASD로 이동하고, 마우스 좌클릭으로 공격한다. 플레이어가 Dead 상태일 때는 모든 입력을 무시한다. 공격 입력 시 Attack 상태로 전환하고 공격 애니메이션을 재생한 뒤 전방에 데미지 판정을 발생시킨다”라고 요청하는 편이 좋습니다. 좋은 플레이어 조작 기획서는 버튼 목록이 아니라, 입력이 게임 상태와 행동으로 어떻게 연결되는지 정의하는 문서입니다.

Chapter 103. 전투 시스템 기획서 작성하기

전투 시스템 기획서는 플레이어와 적이 어떻게 공격하고, 피격되고, 쓰러지는지 정의하는 문서입니다. 실습 프로젝트에서는 복잡한 콤보나 다양한 무기 시스템보다 기본 공격, 데미지 계산, 체력 감소, 피격 처리, 적 사망, 보상 지급 흐름을 먼저 정리하는 것이 좋습니다. 전투는 여러 시스템이 동시에 연결되는 영역이기 때문에 기획이 모호하면 구현 과정에서 오류가 많이 발생합니다.

먼저 전투의 기본 흐름을 작성해야 합니다. 플레이어가 공격 입력을 하면 공격 상태로 전환되고, 공격 애니메이션이 재생되며, 특정 타이밍에 공격 판정이 발생합니다. 공격 판정 범위 안에 적이 있으면 데미지를 계산하고 적의 체력을 감소시킵니다. 적의 체력이 0 이하가 되면 사망 상태로 전환하고, 경험치와 골드 보상을 지급합니다. 이 흐름을 문장으로 정리하면 전투 시스템의 기본 뼈대가 됩니다.

다음으로 수치 규칙을 정의해야 합니다. 플레이어 공격력, 적 방어력, 데미지 계산식, 치명타 여부, 공격 쿨타임, 피격 후 무적 시간 같은 값을 정리합니다. 예를 들어 “최종 데미지 = 공격력 - 방어력, 최소 데미지는 1”처럼 단순한 계산식부터 시작할 수 있습니다. 수치 규칙이 명확해야 AI가 임의의 계산식을 만들지 않고, 기획 의도에 맞는 코드를 생성할 수 있습니다.

전투 기획서에는 예외 조건도 포함해야 합니다. 플레이어가 Dead 상태일 때는 공격할 수 없고, 적이 이미 Dead 상태라면 추가 데미지를 받지 않아야 합니다. 공격 쿨타임 중에는 새로운 공격 입력을 무시해야 하며, 피격 무적 시간이 적용되는 동안에는 데미지를 중복으로 받지 않아야 합니다. 이런 조건을 작성하지 않으면 전투는 겉으로는 동작해도 실제 플레이에서 버그가 발생하기 쉽습니다.

AI에게 전투 시스템 구현을 요청할 때는 “공격 입력 → 공격 판정 → 데미지 계산 → 체력 감소 → 사망 처리 → 보상 지급 → 예외 조건” 순서로 전달하는 것이 좋습니다. “전투 시스템을 만들어줘”보다 “플레이어가 공격하면 전방 범위의 적을 검사하고, 살아 있는 적에게 데미지를 적용한다. 적 체력이 0 이하가 되면 Dead 상태로 전환하고 경험치와 골드를 지급한다. Dead 상태의 대상에게는 추가 데미지를 적용하지 않는다”라고 요청하는 편이 좋습니다. 좋은 전투 시스템 기획서는 액션의 느낌만 설명하는 문서가 아니라, 공격과 피격이 어떤 규칙으로 처리되는지 명확히 정의하는 개발 기준 문서입니다.

Chapter 104. 아이템과 보상 기획서 작성하기

아이템과 보상 기획서는 플레이어가 무엇을 얻고, 그것을 어디에 사용하며, 어떤 조건에서 지급되는지 정리하는 문서입니다. 실습 프로젝트에서는 복잡한 장비 세트나 강화 시스템보다 골드, 경험치, 포션, 간단한 재료 아이템 정도로 범위를 제한하는 것이 좋습니다. 보상 구조가 명확해야 플레이어가 전투를 반복할 이유가 생기고, 성장 루프도 자연스럽게 이어집니다. 아이템과 보상은 전투, 상점, 인벤토리, 데이터 테이블과 직접 연결됩니다.

먼저 아이템 종류를 구분해야 합니다. 소비 아이템은 사용하면 사라지고 효과를 즉시 적용하는 아이템입니다. 예를 들어 체력 포션은 사용 시 HP를 회복하고 수량이 1 감소합니다. 재료 아이템은 당장 사용하지 않더라도 나중에 제작이나 강화에 사용할 수 있는 아이템입니다. 실습 프로젝트에서는 아이템 유형을 너무 늘리기보다, 소비 아이템과 재화 중심으로 단순하게 시작하는 편이 좋습니다.

보상은 지급 조건과 지급 내용을 함께 작성해야 합니다. 적을 처치하면 경험치와 골드를 지급하고, 스테이지를 클리어하면 추가 골드나 아이템을 지급할 수 있습니다. 보스 몬스터를 처치했을 때는 일반 적보다 높은 보상을 주는 방식도 가능합니다. 이때 “보상을 준다”라고만 작성하면 부족합니다. 어떤 상황에서, 어떤 보상을, 몇 개 지급하는지 구체적으로 정리해야 합니다.

프로그래머 관점에서 아이템과 보상 기획서는 데이터 구조로 연결되어야 합니다. 아이템에는 ItemId, Name, ItemType, Description, Icon, MaxStack, EffectValue 같은 항목이 필요할 수 있습니다. 보상 테이블에는 RewardId, TargetType, GoldAmount, ExpAmount, ItemId, ItemCount, DropRate 같은 항목을 둘 수 있습니다. 이렇게 데이터 항목을 미리 정리하면 AI에게 ScriptableObject나 JSON, CSV 구조를 요청하기 쉬워집니다.

AI에게 아이템과 보상 시스템 구현을 요청할 때는 “아이템 종류 → 아이템 데이터 → 보상 조건 → 지급 방식 → 예외 조건” 순서로 전달하는 것이 좋습니다. “보상 시스템을 만들어줘”보다 “적 사망 시 EnemyRewardData를 참조해 경험치와 골드를 지급하고, DropRate에 따라 아이템을 인벤토리에 추가한다. 인벤토리에 공간이 부족하면 아이템 지급은 실패 처리하고 골드와 경험치는 정상 지급한다”라고 요청하는 편이 좋습니다. 좋은 아이템과 보상 기획서는 플레이어가 얻는 것의 목록이 아니라, 보상이 어떤 조건과 데이터 구조로 지급되는지 정의하는 개발 기준 문서입니다.

Chapter 105. UI 화면 기획서 작성하기

UI 화면 기획서는 플레이어가 게임의 기능을 어떻게 보고, 선택하고, 사용할지 정리하는 문서입니다. 실습 프로젝트에서는 타이틀 화면, 로비 화면, 인게임 HUD, 인벤토리 화면, 상점 화면, 결과 화면 정도를 먼저 정의하면 충분합니다. UI는 단순히 예쁘게 배치하는 요소가 아니라, 게임 시스템과 플레이어를 연결하는 인터페이스입니다. 따라서 각 화면의 목적과 기능을 명확히 작성해야 합니다.

먼저 화면 목록을 정리해야 합니다. 타이틀 화면은 게임 시작과 종료를 담당하고, 로비 화면은 캐릭터 정보, 스테이지 선택, 상점, 인벤토리 진입을 담당합니다. 인게임 HUD는 체력, 레벨, 경험치, 골드, 스킬 쿨타임, 포션 수량 등을 표시합니다. 결과 화면은 스테이지 클리어 후 획득한 경험치, 골드, 아이템을 보여줍니다. 화면 목록이 정리되면 필요한 UI 프리팹과 스크립트도 자연스럽게 보입니다.

다음으로 각 화면에 필요한 UI 요소를 작성해야 합니다. 예를 들어 로비 화면에는 캐릭터 이름, 레벨, 현재 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼이 필요할 수 있습니다. 인게임 HUD에는 HP 바, 경험치 바, 공격 버튼 또는 스킬 버튼, 포션 버튼, 일시정지 버튼이 필요합니다. 각 요소는 단순한 이름이 아니라 어떤 데이터를 표시하고 어떤 동작을 실행하는지 함께 적어야 합니다.

프로그래머 관점에서 UI 기획서는 데이터 바인딩과 이벤트 연결 기준이 됩니다. HP 바는 플레이어의 현재 HP와 최대 HP를 기준으로 갱신되고, 골드 텍스트는 보유 골드 값이 변경될 때 갱신되어야 합니다. 버튼은 단순히 클릭되는 오브젝트가 아니라 특정 시스템의 메서드와 연결됩니다. 예를 들어 상점 버튼은 `ShopUI.Open()`을 호출하고, 포션 버튼은 `InventoryManager.UseItem()`과 연결될 수 있습니다.

AI에게 UI 구현을 요청할 때는 “화면 목적 → UI 요소 → 표시 데이터 → 버튼 이벤트 → 갱신 조건 → 화면 전환” 순서로 전달하는 것이 좋습니다. “로비 UI를 만들어줘”보다 “로비 화면에는 레벨 텍스트, 골드 텍스트, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼이 있다. 골드 값이 변경되면 골드 텍스트를 갱신하고, 상점 버튼을 누르면 ShopUI를 열어준다”라고 요청하는 편이 좋습니다. 좋은 UI 화면 기획서는 화면 배치 설명이 아니라, UI가 어떤 데이터와 시스템 동작을 연결해야 하는지 정의하는 개발 기준 문서입니다.

Chapter 106. 스테이지 기획서 작성하기

스테이지 기획서는 플레이어가 어떤 공간에서, 어떤 목표를 가지고, 어떤 순서로 게임을 진행하는지 정리하는 문서입니다. 실습 프로젝트에서는 거대한 맵보다 짧은 전투 구간을 기준으로 스테이지를 설계하는 것이 좋습니다. 스테이지가 명확해야 적 배치, 보상, 난이도, 클리어 조건을 정리할 수 있습니다. 스테이지 기획서는 레벨 디자인과 시스템 기획을 연결하는 기준 문서입니다.

먼저 스테이지의 목표를 정의해야 합니다. 예를 들어 “모든 적을 처치하면 클리어”, “제한 시간 안에 보스를 처치하면 클리어”, “목표 지점에 도달하면 클리어”처럼 작성할 수 있습니다. 실습 프로젝트에서는 “스테이지에 등장하는 모든 적을 처치하면 클리어”라는 단순한 조건으로 시작하는 것이 좋습니다. 목표가 단순해야 전투, 보상, UI 결과 화면까지 쉽게 연결할 수 있습니다.

다음으로 스테이지 진행 흐름을 작성합니다. 플레이어는 로비에서 스테이지를 선택하고, 전투 맵에 입장합니다. 스테이지가 시작되면 적이 배치되어 있고, 플레이어는 적을 처치하며 진행합니다. 모든 적을 처치하면 클리어 판정이 발생하고, 보상 화면을 거쳐 로비로 돌아갑니다. 이 흐름은 스테이지 매니저, 적 스폰 시스템, 보상 시스템, 화면 전환 시스템과 연결됩니다.

스테이지 기획서에는 적 배치와 난이도도 포함해야 합니다. 1스테이지에는 약한 적 3마리, 2스테이지에는 약한 적 5마리와 강한 적 1마리, 3스테이지에는 보스 1마리처럼 단계적으로 구성할 수 있습니다. 난이도는 적 수, 적 체력, 공격력, 이동 속도, 보상량으로 조절할 수 있습니다. 이렇게 수치를 표로 정리하면 AI에게 데이터 테이블이나 ScriptableObject 생성을 요청하기 쉽습니다.

AI에게 스테이지 구현을 요청할 때는 “스테이지 목표 → 시작 조건 → 적 배치 → 클리어 조건 → 실패 조건 → 보상 → 화면 전환” 순서로 전달하는 것이 좋습니다. “스테이지 시스템을 만들어줘”보다 “로비에서 스테이지를 선택하면 해당 StageData를 불러오고, 지정된 적을 스폰한다. 모든 적이 Dead 상태가 되면 클리어 처리하고 보상 화면을 연다. 플레이어가 Dead 상태가 되면 실패 처리하고 로비로 돌아간다”라고 요청하는 편이 좋습니다. 좋은 스테이지 기획서는 맵 설명이 아니라, 플레이어가 어떤 조건에서 시작하고 어떤 조건으로 성공하거나 실패하는지 정의하는 개발 기준 문서입니다.

Chapter 107. 데이터 테이블 작성하기

데이터 테이블은 기획서에 작성한 수치와 규칙을 코드에서 사용할 수 있는 형태로 정리하는 문서입니다. 실습 프로젝트에서는 캐릭터, 적, 아이템, 보상, 스테이지 데이터를 테이블로 관리하면 좋습니다. 코드 안에 값을 직접 하드코딩하면 수정할 때마다 소스 코드를 열어야 하고, 밸런싱 과정도 불편해집니다. 데이터 테이블을 작성하면 기획 변경과 코드 구조를 분리할 수 있습니다.

먼저 어떤 데이터를 테이블로 관리할지 정해야 합니다. 플레이어 능력치는 레벨, 최대 HP, 공격력, 방어력, 필요 경험치로 정리할 수 있습니다. 적 데이터는 EnemyId, 이름, HP, 공격력, 이동 속도, 지급 경험치, 지급 골드로 정리할 수 있습니다. 아이템 데이터는 ItemId, 이름, 아이템 타입, 설명, 최대 수량, 효과값, 가격 같은 항목으로 구성할 수 있습니다.

다음으로 데이터 항목의 이름과 의미를 명확히 작성해야 합니다. 예를 들어 Attack이라는 이름만 쓰면 기본 공격력인지 최종 공격력인지 애매할 수 있습니다. BaseAttack, Defense, RewardGold, RequiredExp처럼 용도를 알 수 있는 이름을 사용하는 것이 좋습니다. 데이터 이름이 명확하면 AI에게 코드 생성을 요청할 때도 필드 이름과 역할이 흔들리지 않습니다.

데이터 테이블에는 타입과 단위도 함께 정리하는 것이 좋습니다. HP와 공격력은 정수인지 실수인지, 이동 속도는 초당 거리인지, 쿨타임은 초 단위인지 정해야 합니다. DropRate처럼 확률을 다루는 값은 0~1 범위인지 0~100 범위인지 명확히 해야 합니다. 단위가 빠진 데이터는 구현 과정에서 잘못 해석될 가능성이 높습니다.

AI에게 데이터 구조 구현을 요청할 때는 “테이블 목적 → 필드 목록 → 필드 타입 → 사용 위치 → 예외 조건” 순서로 전달하는 것이 좋습니다. “아이템 데이터를 만들어줘”보다 “ItemData에는 ItemId, Name, ItemType, Description, MaxStack, EffectValue, Price를 포함한다. ItemId는 중복될 수 없고, MaxStack은 1 이상이어야 한다. Unity에서는 ScriptableObject로 관리한다”라고 요청하는 편이 좋습니다. 좋은 데이터 테이블은 단순한 숫자 목록이 아니라, 기획 수치를 코드와 AI 구현 요청으로 연결하는 기준 문서입니다.

Chapter 108. AI 구현 요청 프롬프트 작성하기

실습 프로젝트의 기획서와 데이터 테이블을 작성했다면, 이제 이를 AI에게 전달할 구현 요청 프롬프트로 바꿀 수 있습니다. AI 구현 요청은 단순히 “이 기능을 만들어줘”라고 말하는 것이 아닙니다. 프로젝트 환경, 구현 목적, 필요한 클래스, 데이터 구조, 처리 흐름, 예외 조건, 결과 형식을 함께 전달해야 합니다. 프롬프트가 구체적일수록 AI가 만든 코드도 기획 의도에 가까워집니다.

먼저 프롬프트의 시작에는 개발 환경과 구현 범위를 작성합니다. 예를 들어 “Unity 6 C# 프로젝트에서 사용할 스테이지형 액션 RPG의 인벤토리 시스템을 구현한다”처럼 작성할 수 있습니다. 그다음 기존 구조가 있다면 클래스 이름, 네임스페이스, 데이터 관리 방식도 함께 알려줍니다. “아이템 데이터는 ScriptableObject인 ItemData를 사용하고, 보유 아이템은 InventoryItem 리스트로 관리한다”처럼 전제를 명확히 해야 AI가 임의의 구조를 만들 가능성이 줄어듭니다.

다음으로 구현할 기능을 작은 단위로 나누어 작성해야 합니다. 인벤토리 시스템이라면 아이템 추가, 아이템 제거, 수량 증가, 수량 감소, 아이템 사용, 보유 여부 확인 같은 기능으로 분리할 수 있습니다. 상점 시스템이라면 구매 가능 여부 확인, 골드 차감, 아이템 지급, 실패 메시지 반환으로 나눌 수 있습니다. 기능을 나누어 요청하면 AI가 빠뜨리는 부분을 줄이고, 나중에 테스트하기도 쉬워집니다.

예외 조건과 완료 기준도 반드시 포함해야 합니다. 아이템 수량이 0 이하가 되면 목록에서 제거할지, 인벤토리가 가득 찼을 때 어떻게 처리할지, 골드가 부족하면 데이터가 변경되지 않아야 하는지 작성해야 합니다. 또한 “컴파일 가능한 C# 코드로 작성”, “기존 public 메서드 이름 유지”, “간단한 사용 예시 포함”처럼 출력 형식도 지정할 수 있습니다. AI에게 기대하는 결과 형태를 알려주면 불필요한 설명이나 구조 변경을 줄일 수 있습니다.

AI에게 구현을 요청할 때는 “환경 → 목적 → 기존 구조 → 구현 기능 → 데이터 구조 → 처리 흐름 → 예외 조건 → 출력 형식” 순서로 작성하는 것이 좋습니다. 예를 들어 “Unity 6 C#에서 사용할 ShopManager를 구현한다. ShopItemData의 Price와 플레이어 Gold를 비교하고, 충분하면 Gold를 차감한 뒤 InventoryManager에 ItemId를 추가한다. Gold가 부족하거나 인벤토리가 가득 찬 경우에는 false를 반환하고 데이터는 변경하지 않는다”라고 요청할 수 있습니다. 좋은 AI 구현 요청 프롬프트는 AI에게 아이디어를 설명하는 문장이 아니라, 바로 코드로 옮길 수 있는 개발 작업 지시서입니다.

Chapter 109. 결과물을 검토하고 기획서 수정하기

AI에게 구현을 요청한 뒤에는 결과물을 반드시 검토해야 합니다. AI가 만든 코드는 요청한 기능을 빠르게 구현해줄 수 있지만, 기획 의도와 완전히 일치한다고 보장할 수는 없습니다. 따라서 결과물이 기획서의 요구사항을 충족하는지, 예외 조건을 제대로 처리하는지, 기존 프로젝트 구조와 어긋나지 않는지 확인해야 합니다. 검토 과정은 단순한 코드 확인이 아니라 기획서의 품질을 다시 점검하는 과정이기도 합니다.

먼저 기능이 기획서대로 동작하는지 확인합니다. 예를 들어 상점 구매 기능을 구현했다면 골드가 충분할 때 아이템이 지급되는지, 골드가 부족할 때 구매가 실패하는지, 실패 시 데이터가 변경되지 않는지 확인해야 합니다. 전투 시스템이라면 공격 판정, 데미지 계산, 적 사망 처리, 보상 지급 흐름이 기획서와 맞는지 봐야 합니다. 정상 동작뿐 아니라 실패 조건과 예외 상황까지 확인해야 합니다.

검토 과정에서 발견한 문제는 코드만 수정할 것이 아니라 기획서에도 반영해야 합니다. 구현하다 보면 처음 기획서에 없던 조건이 드러날 수 있습니다. 예를 들어 “인벤토리가 가득 찼을 때 보상을 어떻게 처리할 것인가”, “스테이지 클리어 직후 플레이어가 사망하면 어떤 결과를 우선할 것인가” 같은 문제가 생길 수 있습니다. 이런 내용은 코드에만 임시로 처리하지 말고, 기획서의 예외 조건 항목에 추가해야 합니다.

프로그래머 관점에서 기획서 수정은 실패가 아니라 정상적인 개발 과정입니다. 처음부터 완벽한 상세 기획서를 작성하기는 어렵습니다. 구현, 테스트, 플레이 과정을 거치면서 부족한 조건과 모호한 문장이 드러납니다. 중요한 것은 변경된 내용을 문서에 반영해 코드와 기획서가 서로 다른 방향으로 가지 않게 관리하는 것입니다.

AI에게 수정 구현을 요청할 때는 검토 결과와 수정된 기획 내용을 함께 전달하는 것이 좋습니다. “이 코드 수정해줘”보다 “기획서를 수정해 인벤토리가 가득 찬 경우 아이템 보상은 보류하지 않고 지급 실패로 처리하기로 했다. 이에 맞게 RewardManager에서 골드와 경험치는 지급하되, 아이템 추가 실패 시 경고 메시지를 반환하도록 수정해줘”라고 요청하는 편이 좋습니다. 좋은 결과물 검토는 AI가 만든 코드를 평가하는 데서 끝나지 않고, 기획서를 더 정확한 개발 기준 문서로 갱신하는 과정입니다.

Chapter 110. 최종 상세 기획서 완성하기

최종 상세 기획서는 지금까지 작성한 콘셉트, 플레이 루프, 조작, 전투, 아이템, 보상, UI, 스테이지, 데이터 테이블, AI 구현 요청 프롬프트를 하나로 정리한 문서입니다. 각각의 문서는 따로 존재할 수 있지만, 최종 단계에서는 서로 충돌하지 않는지 확인해야 합니다. 예를 들어 전투 보상 기획서의 골드 지급량과 데이터 테이블의 보상 수치가 다르면 구현 과정에서 혼란이 생깁니다. 최종 상세 기획서는 개발 기준을 하나로 통합하는 문서입니다.

먼저 문서의 전체 구조를 정리해야 합니다. 게임 개요, 핵심 플레이 루프, 주요 시스템 목록, 화면 목록, 데이터 테이블, 예외 조건, AI 구현 요청 프롬프트 순서로 구성할 수 있습니다. 이 순서로 정리하면 게임의 큰 방향에서 시작해 실제 구현 단위까지 자연스럽게 내려갈 수 있습니다. 프로그래머 관점에서는 이 문서가 곧 개발 작업 목록이자 검토 기준이 됩니다.

다음으로 각 문서 사이의 연결 관계를 확인해야 합니다. 플레이어 조작 기획서에서 공격 입력을 정의했다면, 전투 시스템 기획서에는 그 공격 입력이 어떤 판정을 발생시키는지 작성되어 있어야 합니다. 아이템 기획서에 포션이 있다면, UI 기획서에는 포션 수량 표시와 사용 버튼이 있어야 합니다. 스테이지 기획서에 보상 지급이 있다면, 보상 데이터 테이블에도 해당 보상이 존재해야 합니다. 최종 검토에서는 이런 연결이 끊기지 않았는지 확인해야 합니다.

예외 조건도 마지막으로 다시 점검해야 합니다. 플레이어가 사망했을 때 스테이지 결과는 어떻게 처리되는지, 보상이 중복 지급되지 않는지, 인벤토리가 가득 찼을 때 아이템 보상은 어떻게 되는지 확인해야 합니다. 구현 중에 새로 발견한 조건이 있다면 최종 기획서에 반영해야 합니다. 코드에만 처리하고 문서에 남기지 않으면 나중에 같은 기능을 수정할 때 기준을 잃기 쉽습니다.

AI에게 최종 구현을 요청할 때는 이 상세 기획서를 기준으로 기능을 순서대로 나누어 전달하는 것이 좋습니다. “전체 게임을 한 번에 만들어줘”보다 “먼저 데이터 구조를 만들고, 그다음 플레이어 조작, 전투, 보상, UI, 스테이지 흐름 순서로 구현한다”라고 요청하는 편이 안정적입니다. 좋은 최종 상세 기획서는 게임을 설명하는 문서에서 끝나지 않고, 개발자가 무엇을 어떤 순서로 만들고 검토해야 하는지 알려주는 실행 가능한 기준 문서입니다.

Part 13. 프로그래머의 기획 습관 만들기

Part 13에서는 상세 기획서를 일회성 문서가 아니라 개발 습관으로 만드는 방법을 다룹니다.

기획서는 처음 한 번 작성하고 끝나는 문서가 아니라, 구현 과정에서 계속 수정되고 업데이트되는 개발 기준입니다.

이 파트에서는 구현하면서 기획서를 갱신하는 방법, 코드 변경과 기획서 변경을 함께 관리하는 방법, Git으로 문서를 버전 관리하는 방법, README와 상세 기획서를 연결하는 방법을 설명합니다.

마지막으로 프로그래머가 기획 감각을 꾸준히 키우고, AI 시대에도 명확한 생각을 바탕으로 게임을 완성해가는 태도를 정리합니다.

Chapter 111. 기획서를 한 번에 완성하려 하지 않기

상세 기획서는 처음부터 완벽하게 작성되는 문서가 아닙니다. 게임 개발은 아이디어, 구현, 테스트, 수정이 반복되는 과정이기 때문에 기획서도 함께 변합니다. 처음 작성한 기획서는 방향을 잡기 위한 초안에 가깝고, 실제 개발을 진행하면서 부족한 조건과 모호한 부분이 드러납니다. 따라서 기획서를 한 번에 완성하려고 하면 오히려 시작이 늦어질 수 있습니다.

처음에는 게임의 큰 흐름만 정리해도 충분합니다. 게임 콘셉트, 핵심 플레이 루프, 주요 시스템 목록, 화면 목록 정도를 먼저 작성합니다. 이후 플레이어 조작, 전투, 보상, UI, 데이터 테이블처럼 구현과 직접 연결되는 문서를 단계적으로 추가하면 됩니다. 모든 세부 조건을 처음부터 정하려고 하면 아직 검증되지 않은 내용을 억지로 확정하게 됩니다.

프로그래머 관점에서 기획서는 구현을 막는 문서가 아니라 구현을 돕는 문서여야 합니다. 기능을 만들기 전에 필요한 만큼 작성하고, 구현하면서 부족한 부분을 보완하는 방식이 좋습니다. 예를 들어 전투 시스템을 만들기 전에는 공격 판정과 데미지 계산을 정리하고, 실제 구현 중 피격 무적 시간이나 넉백 처리가 필요하다는 것을 알게 되면 그때 기획서에 추가하면 됩니다.

기획서를 단계적으로 작성하면 AI와 협업하기도 쉬워집니다. 처음부터 전체 게임 기획서를 AI에게 전달하면 범위가 넓어져 결과가 흐려질 수 있습니다. 반대로 “이번에는 인벤토리 시스템 기획서만 기준으로 구현 요청을 작성한다”처럼 범위를 나누면 더 정확한 결과를 얻을 수 있습니다. 작은 단위의 기획서가 쌓이면 결국 전체 상세 기획서가 됩니다.

AI에게 도움을 요청할 때도 “전체 기획서를 완성해줘”보다 “현재 작성한 플레이 루프를 기준으로 필요한 시스템 목록을 정리해줘” 또는 “전투 시스템 초안에서 빠진 예외 조건을 찾아줘”처럼 단계적으로 요청하는 것이 좋습니다. 좋은 기획 습관은 처음부터 완벽한 문서를 만드는 것이 아니라, 개발 과정에 맞춰 기획서를 계속 갱신하는 것입니다. 상세 기획서는 완성품으로 시작하는 문서가 아니라, 프로젝트와 함께 점점 정확해지는 개발 기준 문서입니다.

Chapter 112. 구현하면서 기획서를 업데이트하는 습관

기획서는 구현 전에만 작성하는 문서가 아닙니다. 실제로 코드를 작성하다 보면 처음 기획서에서 놓친 조건, 잘못 예상한 흐름, 추가로 필요한 데이터가 계속 발견됩니다. 이때 코드는 수정하면서 기획서를 그대로 두면 문서와 실제 구현이 점점 달라집니다. 결국 시간이 지나면 기획서는 더 이상 개발 기준으로 사용할 수 없는 오래된 문서가 됩니다.

구현 중 발견한 변경 사항은 바로 기획서에 반영하는 습관이 필요합니다. 예를 들어 상점 시스템을 만들다가 “인벤토리가 가득 찼을 때 구매를 막아야 한다”는 조건이 추가되었다면, 코드에만 예외 처리를 넣지 말고 상점 기획서의 예외 조건 항목에도 기록해야 합니다. 전투 시스템에서 “적이 사망한 뒤에는 추가 데미지를 받지 않는다”는 처리를 추가했다면, 전투 기획서에도 같은 내용을 남겨야 합니다. 이렇게 해야 코드와 문서가 같은 기준을 공유합니다.

프로그래머 관점에서 기획서 업데이트는 불필요한 문서 작업이 아니라, 미래의 디버깅 비용을 줄이는 작업입니다. 나중에 기능을 수정할 때 “왜 이렇게 처리했지?”라는 의문이 생기면 코드만 보고 의도를 파악하기 어렵습니다. 하지만 기획서에 변경 이유와 조건이 남아 있으면 구현 의도를 빠르게 이해할 수 있습니다. 특히 AI가 만든 코드라면, 사람이 직접 작성한 코드보다 의도 추적이 더 중요합니다.

기획서를 업데이트할 때는 모든 내용을 길게 다시 쓸 필요는 없습니다. 변경된 규칙, 추가된 예외 조건, 수정된 데이터 항목, 구현 중 결정한 제한 사항만 짧게 기록해도 충분합니다. 예를 들어 “2026-06-21: 보상 지급 시 인벤토리가 가득 찬 경우 아이템 보상은 지급 실패 처리하고 골드와 경험치는 정상 지급한다”처럼 작성할 수 있습니다. 중요한 것은 완벽한 문장이 아니라 변경된 기준을 남기는 것입니다.

AI에게 추가 구현이나 수정 요청을 할 때도 업데이트된 기획서를 기준으로 전달해야 합니다. “이전 코드에 기능을 추가해줘”보다 “수정된 기획서 기준으로, 인벤토리가 가득 찬 경우 아이템 보상만 실패 처리하고 골드와 경험치는 지급되도록 RewardManager를 수정해줘”라고 요청하는 편이 좋습니다. 좋은 기획 습관은 문서를 한 번 작성하고 끝내는 것이 아니라, 구현 과정에서 드러난 판단과 변경 사항을 계속 반영해 기획서를 살아 있는 개발 기준으로 유지하는 것입니다.

Chapter 113. 코드 변경과 기획서 변경을 함께 관리하기

게임 개발에서는 코드만 바뀌는 것이 아니라 기획도 함께 바뀝니다. 처음에는 골드가 부족하면 구매만 실패하도록 정했지만, 구현 중 인벤토리 공간 부족 조건이 추가될 수 있습니다. 스테이지 클리어 조건도 “모든 적 처치”에서 “보스 처치”로 바뀔 수 있습니다. 이런 변경이 코드에만 반영되고 기획서에 남지 않으면, 나중에 어떤 동작이 의도된 것인지 판단하기 어려워집니다.

코드 변경과 기획서 변경은 가능한 한 같은 흐름에서 관리해야 합니다. 기능을 수정했다면 해당 기능의 기획서도 함께 확인해야 합니다. 예를 들어 RewardManager에서 보상 지급 조건을 바꿨다면 보상 기획서의 지급 조건, 예외 조건, 데이터 테이블도 같이 수정해야 합니다. StageManager의 클리어 조건을 바꿨다면 스테이지 기획서와 결과 화면 기획서도 함께 점검해야 합니다.

프로그래머 관점에서 코드와 기획서의 불일치는 버그의 원인이 됩니다. 코드에는 인벤토리가 가득 차면 아이템 보상을 지급하지 않도록 되어 있는데, 기획서에는 “스테이지 클리어 시 아이템을 지급한다”라고만 적혀 있다면 해석이 달라질 수 있습니다. 이후 AI에게 기능 수정을 요청할 때도 오래된 기획서를 기준으로 잘못된 코드를 만들 가능성이 있습니다. 따라서 문서와 코드의 기준을 맞추는 작업은 협업뿐 아니라 AI 활용에서도 중요합니다.

변경 이력을 관리할 때는 코드 커밋과 기획서 수정 내용을 연결하면 좋습니다. Git을 사용한다면 커밋 메시지에 “상점 구매 실패 조건 추가”처럼 코드 변경 이유를 적고, 기획서에도 같은 조건을 짧게 기록합니다. 예를 들어 “골드 부족 또는 인벤토리 공간 부족 시 구매 실패 처리”처럼 남길 수 있습니다. 이렇게 하면 나중에 코드를 보거나 문서를 볼 때 변경 이유를 추적하기 쉬워집니다.

AI에게 수정 요청을 할 때도 코드와 기획서의 변경 사항을 함께 전달하는 것이 좋습니다. “구매 로직을 수정해줘”보다 “기획서가 변경되어 골드 부족뿐 아니라 인벤토리 공간 부족도 구매 실패 조건에 포함된다. 기존 ShopManager 구조를 유지하면서 PurchaseItem이 두 조건을 모두 검사하도록 수정해줘”라고 요청하는 편이 좋습니다. 좋은 변경 관리 습관은 코드와 기획서를 따로 관리하는 것이 아니라, 두 문서가 같은 게임 규칙을 바라보도록 계속 동기화하는 것입니다.

Chapter 114. Git으로 기획서 버전 관리하기

Git은 소스 코드만 관리하는 도구가 아닙니다. Markdown으로 작성한 기획서, 데이터 테이블, README, AI 프롬프트 문서도 Git으로 함께 관리할 수 있습니다. 게임 개발에서는 코드와 기획이 함께 변하기 때문에 기획서도 버전 관리 대상에 포함하는 것이 좋습니다. 특히 AI와 함께 개발할 때는 어떤 기획을 기준으로 어떤 코드를 만들었는지 추적할 수 있어야 합니다.

기획서를 Git으로 관리하려면 먼저 문서 형식을 단순하게 유지하는 것이 좋습니다. .docx나 .pdf보다 .md 파일이 변경 이력을 확인하기 쉽습니다. 예를 들어 docs/game_concept.md, docs/combat_system.md, docs/item_reward.md, docs/ui_flow.md처럼 문서를 기능별로 나누어 관리할 수 있습니다. 이렇게 하면 특정 시스템의 기획 변경만 따로 확인하기 쉽습니다.

커밋 메시지에는 기획 변경의 이유를 짧게 남기는 것이 좋습니다. “문서 수정”보다 “상점 구매 실패 조건 추가”, “보상 지급 예외 조건 정리”, “스테이지 클리어 조건 변경”처럼 작성해야 나중에 변경 흐름을 이해하기 쉽습니다. 코드 커밋과 기획서 커밋을 함께 남길 수도 있고, 기획서 변경이 큰 경우에는 별도 커밋으로 분리할 수도 있습니다. 중요한 것은 변경된 게임 규칙이 Git 기록에 남아야 한다는 점입니다.

프로그래머 관점에서 Git은 기획서의 실험과 되돌리기를 가능하게 합니다. 특정 전투 규칙을 바꿨다가 플레이 감각이 나빠졌다면 이전 기획으로 되돌릴 수 있습니다. 보상 수치를 조정하다가 밸런스가 무너졌다면 어떤 시점에서 값이 바뀌었는지 확인할 수 있습니다. 기획서도 코드처럼 변경 이력이 남아야 판단 과정을 추적할 수 있습니다.

AI에게 구현을 요청할 때도 Git으로 관리된 기획서는 큰 도움이 됩니다. “현재 docs/combat_system.md 기준으로 전투 로직을 수정해줘”처럼 기준 문서를 명확히 지정할 수 있습니다. 또한 AI가 만든 코드가 마음에 들지 않으면 이전 커밋으로 되돌리고, 수정된 프롬프트로 다시 시도할 수 있습니다. 좋은 Git 활용 습관은 코드만 안전하게 보관하는 것이 아니라, 기획서와 코드가 같은 변경 이력 안에서 함께 성장하도록 관리하는 것입니다.

Chapter 115. README와 상세 기획서 연결하기

README는 프로젝트를 처음 보는 사람이 가장 먼저 읽는 문서입니다. 보통 설치 방법, 실행 방법, 프로젝트 소개를 적는 용도로 사용하지만, 게임 개발 프로젝트에서는 상세 기획서로 연결되는入口 역할도 할 수 있습니다. 특히 Git으로 기획서와 코드를 함께 관리한다면 README에서 주요 기획 문서의 위치를 안내하는 것이 좋습니다. README와 상세 기획서를 연결하면 프로젝트의 목적, 구조, 구현 기준을 빠르게 파악할 수 있습니다.

먼저 README에는 게임의 기본 개요를 짧게 작성합니다. 게임 제목, 장르, 핵심 플레이 루프, 개발 환경, 사용 엔진, 주요 시스템을 한눈에 볼 수 있게 정리합니다. 예를 들어 “Unity 6 C#으로 제작하는 스테이지형 액션 RPG이며, 플레이어는 스테이지에 입장해 적을 처치하고 보상을 얻어 성장한다”처럼 작성할 수 있습니다. README는 상세 기획서 전체를 반복하는 문서가 아니라, 프로젝트의 방향을 요약하는 문서입니다.

다음으로 README에서 상세 기획서 파일로 연결해야 합니다. docs/game_concept.md, docs/core_loop.md, docs/combat_system.md, docs/item_reward.md, docs/ui_flow.md, docs/stage_design.md, docs/data_table.md처럼 문서 목록을 정리하면 좋습니다. 이렇게 하면 프로젝트를 다시 열었을 때도 필요한 기획 문서를 빠르게 찾을 수 있습니다. 협업자가 있다면 어떤 문서를 기준으로 구현해야 하는지도 명확해집니다.

프로그래머 관점에서 README는 코드와 기획서를 연결하는 인덱스 역할을 합니다. Scripts/Combat 폴더의 전투 코드는 docs/combat_system.md를 기준으로 구현되었다고 안내할 수 있습니다. Scripts/UI 폴더의 화면 흐름은 docs/ui_flow.md와 연결할 수 있습니다. 이렇게 연결해두면 코드 구조를 볼 때 관련 기획서를 찾기 쉽고, 기획서를 수정할 때 어떤 코드에 영향이 있는지도 파악하기 쉬워집니다.

AI에게 프로젝트 분석이나 수정 구현을 요청할 때도 README는 유용합니다. “README를 기준으로 프로젝트 구조를 파악하고, docs/combat_system.md에 맞춰 전투 로직을 수정해줘”처럼 요청할 수 있습니다. 좋은 README는 단순한 실행 안내문이 아니라, 프로젝트 개요와 상세 기획서, 코드 구조를 이어주는 출발 문서입니다. 상세 기획서가 개발의 기준이라면 README는 그 기준 문서로 들어가는 지도라고 할 수 있습니다.

Chapter 116. 협업을 위한 기획 문서 작성법

협업을 위한 기획 문서는 혼자 보기 위한 메모와 다르게 작성해야 합니다. 혼자 개발할 때는 머릿속에 있는 전제를 생략해도 어느 정도 진행할 수 있지만, 팀으로 작업할 때는 생략된 전제가 곧 오해가 됩니다. 프로그래머, 기획자, 아티스트, UI 디자이너, QA가 같은 기능을 서로 다르게 이해하면 구현 결과도 달라집니다. 따라서 협업용 기획 문서는 누가 읽어도 같은 동작을 떠올릴 수 있도록 작성해야 합니다.

먼저 문서의 목적과 적용 범위를 분명히 해야 합니다. 예를 들어 “이 문서는 상점 구매 기능의 동작 규칙과 예외 조건을 정의한다”처럼 시작하면 좋습니다. 어떤 화면, 어떤 시스템, 어떤 데이터에 영향을 주는지 함께 적어두면 관련 작업자가 자신에게 필요한 내용을 빠르게 찾을 수 있습니다. 협업 문서에서 중요한 것은 많은 내용을 적는 것이 아니라, 필요한 사람이 필요한 기준을 쉽게 찾을 수 있게 만드는 것입니다.

기능 설명은 추상적인 표현보다 조건과 결과 중심으로 작성해야 합니다. “구매가 자연스럽게 처리된다”보다 “구매 버튼을 누르면 보유 골드와 상품 가격을 비교하고, 골드가 충분하면 골드를 차감한 뒤 아이템을 인벤토리에 추가한다”라고 쓰는 편이 좋습니다. 실패 조건도 반드시 포함해야 합니다. 골드 부족, 인벤토리 공간 부족, 이미 구매한 상품, 네트워크 오류처럼 협업자가 판단해야 할 상황을 문서에 남겨야 합니다.

문서에는 역할별로 필요한 정보도 포함해야 합니다. 프로그래머에게는 데이터 구조와 처리 흐름이 중요하고, UI 디자이너에게는 화면 요소와 상태 표시가 중요합니다. QA에게는 테스트 조건과 예외 상황이 필요합니다. 따라서 하나의 문서 안에서도 기능 개요, UI 요소, 데이터 항목, 처리 흐름, 예외 조건, 테스트 기준을 구분해 작성하면 협업 효율이 높아집니다.

AI와 함께 협업 문서를 다듬을 때는 “이 기획서를 프로그래머, UI 디자이너, QA가 각각 이해할 수 있도록 빠진 정보를 찾아줘”라고 요청할 수 있습니다. 또한 “모호한 표현을 조건과 결과 중심으로 바꿔줘”라고 요청하면 문서 품질을 높일 수 있습니다. 좋은 협업용 기획 문서는 의견을 많이 적은 문서가 아니라, 팀원이 같은 기준으로 구현하고 검토할 수 있게 만드는 공통 약속입니다.

Chapter 117. 혼자 개발할 때도 문서가 필요한 이유

혼자 개발한다고 해서 문서가 필요 없는 것은 아닙니다. 오히려 개인 프로젝트에서는 모든 결정을 혼자 기억해야 하기 때문에 문서의 역할이 더 중요해질 수 있습니다. 처음에는 머릿속에 있는 아이디어만으로 개발을 시작할 수 있지만, 시간이 지나면 왜 이런 구조로 만들었는지, 어떤 조건을 기준으로 기능을 구현했는지 잊어버리기 쉽습니다. 문서는 혼자 개발할 때도 프로젝트의 방향을 잃지 않게 해주는 기준입니다.

개인 개발에서 가장 흔한 문제는 범위가 계속 커지는 것입니다. 처음에는 작은 액션 게임을 만들려고 했지만, 개발 중에 장비 강화, 스킬 트리, 퀘스트, 상점, 업적, 저장 시스템까지 추가하고 싶어질 수 있습니다. 문서가 없으면 어떤 기능이 핵심이고 어떤 기능이 나중에 추가할 기능인지 판단하기 어렵습니다. 기획서를 작성해두면 현재 버전에서 반드시 만들어야 할 기능과 제외할 기능을 구분할 수 있습니다.

문서는 중단된 프로젝트를 다시 시작할 때도 도움이 됩니다. 개인 프로젝트는 회사 업무나 생활 일정 때문에 며칠, 몇 주, 몇 달씩 멈출 수 있습니다. 이때 상세 기획서, 데이터 테이블, README, 개발 일지가 남아 있으면 프로젝트의 구조를 빠르게 다시 파악할 수 있습니다. 문서가 없다면 코드를 다시 읽으며 기억을 되살려야 하고, 그 과정에서 개발 의욕이 떨어질 수 있습니다.

AI와 함께 개발할 때도 개인 문서는 중요한 기준이 됩니다. 혼자 개발하더라도 AI에게 기능 구현을 요청하려면 게임 규칙과 데이터 구조를 설명해야 합니다. “전에 만들던 전투 시스템 이어서 해줘”라고 요청하는 것보다 “전투 기획서는 `combat_system.md`를 기준으로 하고, 데미지 계산식과 사망 처리 조건은 여기에 정의되어 있다”라고 전달하는 편이 훨씬 정확합니다. 문서는 개인 개발자와 AI 사이의 공통 언어가 됩니다.

혼자 개발할 때의 문서는 거창할 필요가 없습니다. 게임 콘셉트, 핵심 플레이 루프, 주요 기능 목록, 데이터 테이블, 예외 조건, 구현 요청 프롬프트 정도만 정리해도 충분합니다. 좋은 개인 개발 문서는 남에게 보여주기 위한 완성된 보고서가 아니라, 미래의 자신이 프로젝트를 다시 이해하고 계속 개발할 수 있게 돕는 기록입니다. 혼자 개발할수록 문서는 협업 상대가 없는 빈자리를 채워주는 개발 기준이 됩니다.

Chapter 118. 작은 게임을 많이 기획해보는 연습

기획 감각은 큰 게임 하나를 오래 생각한다고 해서 빠르게 늘지 않습니다. 오히려 작은 게임을 여러 개 기획해보면서 게임의 구조를 반복해서 분해해보는 연습이 더 효과적입니다. 작은 게임은 범위가 작기 때문에 콘셉트, 핵심 플레이 루프, 조작, 규칙, 보상, UI 흐름을 짧은 시간 안에 정리할 수 있습니다. 이런 반복이 쌓이면 게임을 구조적으로 바라보는 힘이 생깁니다.

작은 게임 기획 연습은 1~2페이지 분량으로도 충분합니다. 예를 들어 벽돌깨기, 미니 슈팅, 간단한 디펜스, 클릭커, 퍼즐, 러너 게임처럼 규칙이 단순한 장르를 선택합니다. 그런 다음 “플레이어가 무엇을 하고, 무엇을 피하거나 달성하며, 어떤 보상을 얻는가”를 정리합니다. 처음부터 독창적인 게임을 만들려고 하기보다, 익숙한 장르를 기준으로 구조를 분석하는 것이 좋습니다.

작은 게임을 기획할 때도 핵심 플레이 루프는 반드시 작성해야 합니다. 예를 들어 러너 게임이라면 “자동으로 달린다 → 장애물을 피한다 → 코인을 획득한다 → 점수를 올린다 → 실패하면 기록을 확인한다”가 기본 루프가 됩니다. 디펜스 게임이라면 “웨이브가 시작된다 → 적이 이동한다 → 타워가 공격한다 → 적을 처치해 재화를 얻는다 → 타워를 강화한다”가 기본 루프가 됩니다. 루프를 정리하면 필요한 시스템이 자연스럽게 보입니다.

프로그래머 관점에서 작은 게임 기획은 구현 연습과도 잘 연결됩니다. 작은 기획서를 작성한 뒤 AI에게 구현 요청 프롬프트로 바꾸어보면, 어떤 문장이 부족한지 바로 확인할 수 있습니다. 예를 들어 “적이 나온다”라는 문장만으로는 스폰 위치, 등장 주기, 이동 경로, 체력, 보상 조건이 부족하다는 것을 알게 됩니다. 작은 게임을 많이 기획할수록 AI에게 전달해야 할 정보도 더 잘 보이게 됩니다.

AI와 함께 연습할 때는 “이 게임 아이디어를 1페이지 상세 기획서로 정리해줘”, “이 기획서에서 빠진 예외 조건을 찾아줘”, “이 내용을 구현 요청 프롬프트로 바꿔줘”처럼 단계적으로 요청할 수 있습니다. 중요한 것은 하나의 기획서를 완벽하게 만드는 것이 아니라, 다양한 게임의 구조를 반복해서 정리해보는 것입니다. 좋은 기획 연습은 거대한 세계관을 만드는 일이 아니라, 작은 게임 하나를 명확한 규칙과 구현 가능한 문서로 바꾸는 반복 훈련입니다.

Chapter 119. 프로그래머가 기획 감각을 키우는 방법

프로그래머에게 필요한 기획 감각은 멋진 아이디어를 떠올리는 능력만을 의미하지 않습니다. 게임을 기능, 규칙, 데이터, 상태, 흐름으로 나누어 바라보는 능력에 가깝습니다. 플레이어가 어떤 행동을 하고, 그 행동이 어떤 시스템을 거쳐 어떤 결과를 만드는지 설명할 수 있어야 합니다. 기획 감각이 있는 프로그래머는 막연한 재미를 구현 가능한 구조로 바꿀 수 있습니다.

가장 먼저 해야 할 연습은 게임을 플레이할 때 구조를 관찰하는 것입니다. 단순히 재미있다거나 재미없다고 느끼는 데서 끝나지 않고, 왜 그런 감정이 생겼는지 분석해야 합니다. 적 처치 후 보상이 바로 지급되는지, 성장 속도가 빠르지 느린지, UI가 어떤 정보를 먼저 보여주는지 살펴봅니다. 이런 관찰이 쌓이면 게임을 감각이 아니라 시스템의 조합으로 이해하게 됩니다.

두 번째 방법은 익숙한 게임을 작은 문서로 다시 작성해보는 것입니다. 좋아하는 게임 하나를 선택하고 콘셉트, 핵심 플레이 루프, 주요 시스템, 보상 구조, UI 흐름을 짧게 정리합니다. 이때 원작을 그대로 베끼는 것이 목적이 아니라, 게임이 어떤 규칙으로 작동하는지 분해하는 것이 목적입니다. 프로그래머는 이 과정을 통해 “재미”를 코드로 옮길 수 있는 단위로 나누는 연습을 할 수 있습니다.

세 번째 방법은 구현 가능한 범위로 생각하는 습관을 들이는 것입니다. 기획 감각이 부족하면 “오픈월드 RPG”, “실시간 멀티플레이 전투”, “무한 성장 시스템”처럼 너무 큰 단어로 게임을 설명하기 쉽습니다. 반대로 기획 감각이 있는 프로그래머는 “플레이어 이동, 적 추적, 공격 판정, 경험치 지급, 레벨업”처럼 작은 기능 단위로 쪼갭니다. 구현 범위를 볼 수 있어야 실제로 완성 가능한 게임을 기획할 수 있습니다.

AI를 활용하면 기획 감각을 훈련하는 속도를 높일 수 있습니다. “이 게임 아이디어를 핵심 플레이 루프로 정리해줘”, “이 기획에서 구현이 어려운 부분을 찾아줘”, “이 기능을 데이터 테이블로 바꿔줘”처럼 질문하면서 자신의 생각을 구조화할 수 있습니다. 좋은 기획 감각은 타고나는 능력이 아니라, 게임을 반복해서 분석하고 문서화하고 구현 단위로 나누는 과정에서 길러집니다. 프로그래머에게 기획 감각은 코드를 더 잘 쓰기 위한 출발점이자, AI에게 더 정확히 일을 맡기기 위한 언어입니다.

Chapter 120. AI 시대에도 결국 중요한 것은 명확한 생각이다

AI는 게임 개발의 속도를 크게 높여줍니다. 코드를 빠르게 생성하고, 오류를 분석하고, 문서를 정리하고, 기획 아이디어를 구현 가능한 형태로 바꾸는 데 도움을 줄 수 있습니다. 하지만 AI가 아무리 강력해져도 사용자가 무엇을 만들고 싶은지 명확히 설명하지 못하면 좋은 결과를 얻기 어렵습니다. 결국 AI 시대에도 개발의 출발점은 명확한 생각입니다.

명확한 생각은 단순히 아이디어가 있다는 뜻이 아닙니다. 플레이어가 무엇을 하고, 어떤 규칙에 따라 결과가 발생하며, 어떤 보상을 받고, 어떤 조건에서 실패하는지 설명할 수 있어야 합니다. “재미있는 전투 게임”이라는 표현은 좋은 출발점일 수 있지만, 구현 기준으로는 부족합니다. 공격 입력, 데미지 계산, 적 행동, 피격 처리, 보상 지급, 실패 조건까지 정리되어야 실제 개발로 이어질 수 있습니다.

프로그래머에게 명확한 생각은 구조화 능력으로 나타납니다. 게임을 콘셉트, 플레이 루프, 시스템, UI, 데이터, 예외 조건으로 나누어 바라볼 수 있어야 합니다. 이 구조가 있어야 코드도 안정적으로 설계되고, AI에게도 정확한 작업을 맡길 수 있습니다. AI는 생각을 대신해주는 도구가 아니라, 정리된 생각을 빠르게 실행으로 옮기도록 돕는 도구에 가깝습니다.

AI를 잘 활용하는 사람은 프롬프트를 길게 쓰는 사람이 아니라, 무엇을 요청해야 하는지 아는 사람입니다. 어떤 클래스가 필요한지, 어떤 데이터가 필요한지, 어떤 예외 조건을 처리해야 하는지 알고 있어야 AI의 결과를 판단할 수 있습니다. AI가 만든 코드가 맞는지 틀린지 검토하려면 결국 사용자의 기준이 필요합니다. 그 기준이 바로 상세 기획서입니다.

이 책에서 다룬 상세 기획서는 문서 작성을 위한 문서가 아닙니다. 프로그래머가 자신의 게임을 더 분명하게 이해하고, 구현 가능한 단위로 나누고, AI와 협업할 수 있도록 만드는 생각의 구조입니다. AI 시대에는 더 많은 코드를 더 빠르게 만들 수 있지만, 무엇을 만들지 명확히 정하지 못하면 빠른 속도는 오히려 혼란을 키울 수 있습니다. 좋은 게임 개발은 좋은 코드에서만 시작하지 않습니다. 명확한 생각, 구조화된 기획, 그리고 그것을 실행으로 옮기는 꾸준한 검토에서 시작합니다.

부록 A. 게임 상세 기획서 기본 템플릿

게임 상세 기획서는 게임의 아이디어를 구현 가능한 개발 기준으로 정리한 문서입니다. 이 템플릿은 프로그래머가 직접 기획서를 작성하거나, AI에게 구현 요청을 전달하기 전에 사용할 수 있는 기본 구조입니다. 모든 항목을 처음부터 완벽하게 채울 필요는 없습니다. 프로젝트 규모에 맞게 필요한 항목부터 작성하고, 구현 과정에서 계속 보완하면 됩니다.

1. 게임 기본 정보

- 게임 제목:
- 장르:
- 플랫폼:
- 개발 엔진:
- 개발 언어:
- 대상 플레이어:
- 예상 플레이 시간:
- 프로젝트 목표:

예시 문장:

이 게임은 Unity 6와 C#으로 개발하는 스테이지형 액션 RPG입니다. 플레이어는 짧은 스테이지에 입장해 적을 처치하고, 경험치와 골드를 획득해 캐릭터를 성장시킵니다. MVP 단계에서는 플레이어 이동, 공격, 적 AI, 보상, 인벤토리, 상점, 기본 UI만 구현합니다.

2. 게임 콘셉트

- 게임을 한 문장으로 설명:
- 플레이어가 하는 핵심 행동:
- 게임의 핵심 재미:
- 포함할 기능:
- 제외할 기능:

예시 문장:

플레이어는 로비에서 스테이지를 선택하고, 전투 구간에 입장해 적을 처치한 뒤 보상을 획득합니다. 이 게임의 핵심 재미는 짧은 전투를 반복하며 캐릭터가 점점 강해지는 성장감입니다. 복잡한 퀘스트, 멀티플레이, 대규모 장비 강화 시스템은 MVP 범위에서 제외합니다.

3. 핵심 플레이 루프

- 시작 지점:

- 반복 행동:
- 보상:
- 성장:
- 다음 목표:

기본 흐름:

로비에서 준비한다 → 스테이지에 입장한다 → 적을 처치한다 → 보상을 얻는다 → 캐릭터를 성장시킨다 → 다음 스테이지에 도전한다

상세 설명:

플레이어는 로비에서 현재 레벨, 보유 골드, 인벤토리를 확인합니다. 이후 스테이지를 선택해 입장하고, 적을 처치해 경험치와 골드를 얻습니다. 스테이지를 클리어하면 보상 화면을 확인한 뒤 로비로 돌아와 상점이나 인벤토리를 이용합니다.

4. 주요 시스템 목록

- 플레이어 조작 시스템:
- 전투 시스템:
- 적 AI 시스템:
- 체력 및 데미지 시스템:
- 경험치 및 레벨업 시스템:
- 아이템 시스템:
- 인벤토리 시스템:
- 보상 시스템:
- 상점 시스템:
- 스테이지 시스템:
- UI 시스템:
- 저장 시스템:

각 시스템은 “무엇을 담당하는지” 한 문장으로 작성합니다.

예시:

전투 시스템은 플레이어와 적의 공격, 피격, 데미지 계산, 사망 처리를 담당합니다. 보상 시스템은 적 처치 또는 스테이지 클리어 시 경험치, 골드, 아이템을 지급합니다.

5. 플레이어 조작 기획

- 이동 입력:
- 공격 입력:

- 대시 입력:
- 아이템 사용 입력:
- 조작 가능 조건:
- 조작 불가능 조건:

예시:

플레이어는 WASD로 이동하고, 마우스 좌클릭으로 기본 공격을 합니다. 플레이어가 Dead 상태일 때는 모든 입력을 무시합니다. 공격 쿨타임이 끝나지 않은 상태에서는 새로운 공격 입력을 처리하지 않습니다.

6. 전투 시스템 기획

- 공격 방식:
- 공격 판정:
- 데미지 계산식:
- 피격 처리:
- 사망 처리:
- 보상 지급 조건:
- 예외 조건:

예시:

플레이어가 공격 입력을 하면 Attack 상태로 전환하고 공격 애니메이션을 재생합니다. 공격 판정 범위 안에 살아 있는 적이 있으면 데미지를 적용합니다. 적의 HP가 0 이하가 되면 Dead 상태로 전환하고 경험치와 골드를 지급합니다. 이미 Dead 상태인 적에게는 추가 데미지를 적용하지 않습니다.

7. 아이템과 보상 기획

- 아이템 종류:
- 소비 아이템:
- 재료 아이템:
- 재화:
- 보상 지급 조건:
- 드랍 확률:
- 보상 예외 조건:

예시:

적을 처치하면 경험치와 골드를 지급합니다. 일부 적은 확률에 따라 아이템을 드랍합니다. 인벤토리에 공간이 부족하면 아이템 지급은 실패 처리하지만, 골드와 경험치는 정상 지급합니다.

8. UI 화면 기획

- 타이틀 화면:
- 로비 화면:
- 인게임 HUD:
- 인벤토리 화면:
- 상점 화면:
- 결과 화면:
- 설정 화면:
- 팝업과 토스트 메시지:

예시:

로비 화면에는 플레이어 레벨, 보유 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼을 표시합니다. 인게임 HUD에는 HP 바, 경험치 바, 골드, 포션 수량을 표시합니다. 스테이지 클리어 후에는 결과 화면에서 획득 경험치, 골드, 아이템을 보여줍니다.

9. 스테이지 기획

- 스테이지 ID:
- 스테이지 이름:
- 클리어 조건:
- 실패 조건:
- 등장 적:
- 보상:
- 난이도:
- 다음 스테이지 조건:

예시:

스테이지에 등장하는 모든 적을 처치하면 클리어됩니다. 플레이어 HP가 0 이하가 되면 실패 처리하고 로비로 돌아갑니다. 클리어 시 경험치, 골드, 아이템 보상을 지급합니다.

10. 데이터 테이블 목록

- PlayerLevelData:
- EnemyData:
- ItemData:
- RewardData:

- StageData:
- ShopItemData:
- SkillData:

예시 필드:

EnemyData에는 EnemyId, Name, MaxHp, AttackPower, Defense, MoveSpeed, RewardExp, RewardGold를 포함합니다. ItemData에는 ItemId, Name, ItemType, Description, MaxStack, EffectValue, Price를 포함합니다.

11. 예외 조건 정리

- 플레이어가 사망한 상태에서 입력이 들어온 경우:
- 적이 이미 사망한 상태에서 공격을 받은 경우:
- 골드가 부족한 상태에서 구매를 시도한 경우:
- 인벤토리가 가득 찬 상태에서 아이템을 획득한 경우:
- 보상이 중복 지급될 가능성이 있는 경우:
- 스테이지 클리어와 실패 조건이 동시에 발생한 경우:
- 저장 데이터가 없거나 손상된 경우:

예시:

스테이지 클리어 보상은 한 번만 지급되어야 합니다. 보상 지급 중 오류가 발생하면 중복 지급을 방지하기 위해 보상 지급 여부를 먼저 기록하거나, 지급 완료 상태를 명확히 관리해야 합니다.

12. AI 구현 요청 프롬프트

기본 구조:

환경 → 목적 → 기존 구조 → 구현 기능 → 데이터 구조 → 처리 흐름 → 예외 조건 → 출력 형식

예시:

Unity 6 C# 프로젝트에서 사용할 상점 구매 시스템을 구현해줘. 아이템 데이터는 ShopItemData를 사용하고, 플레이어 골드는 PlayerCurrencyManager가 관리한다. 구매 버튼을 누르면 상품 가격과 보유 골드를 비교한다. 골드가 충분하고 인벤토리에 공간이 있으면 골드를 차감하고 아이템을 InventoryManager에 추가한다. 골드가 부족하거나 인벤토리가 가득 찬 경우에는 false를 반환하고 어떤 데이터도 변경하지 않는다. 기존 클래스 이름과 public 메서드 시그니처는 변경하지 말아줘.

13. 검토 체크리스트

- 게임 콘셉트가 한 문장으로 설명되는가?
- 핵심 플레이 루프가 명확한가?
- MVP 범위와 제외할 기능이 구분되어 있는가?

- 각 시스템의 책임이 분리되어 있는가?
- 입력 조건과 불가능 조건이 작성되어 있는가?
- 정상 흐름과 예외 조건이 모두 작성되어 있는가?
- 데이터 테이블의 필드 이름과 타입이 명확한가?
- UI가 어떤 데이터를 표시하는지 정리되어 있는가?
- 보상 지급 조건과 중복 지급 방지 조건이 있는가?
- AI에게 전달할 구현 요청 프롬프트로 변환할 수 있는가?

14. 변경 이력

- 날짜:
- 변경한 항목:
- 변경 이유:
- 관련 코드:
- 관련 데이터:
- AI 프롬프트 수정 여부:

예시:

2026-06-21: 인벤토리가 가득 찬 경우 아이템 보상은 지급 실패 처리하고, 골드와 경험치는 정상 지급하도록 보상 예외 조건을 수정했습니다. RewardManager와 RewardData 문서를 함께 수정해야 합니다.

이 템플릿은 프로젝트의 모든 내용을 한 번에 완성하기 위한 양식이 아닙니다. 처음에는 게임 개요, 핵심 플레이 루프, 주요 시스템 목록만 작성해도 충분합니다. 이후 구현과 테스트를 반복하면서 조작, 전투, 보상, UI, 데이터, 예외 조건을 계속 보완하면 됩니다. 좋은 게임 상세 기획서는 고정된 문서가 아니라, 프로젝트와 함께 점점 정확해지는 개발 기준 문서입니다.

부록 B. 게임 콘셉트 문서 템플릿

게임 콘셉트 문서는 게임의 방향을 가장 짧고 명확하게 정리하는 문서입니다. 상세 기획서를 작성하기 전에 콘셉트가 먼저 정리되어 있어야 이후 플레이 루프, 시스템, UI, 데이터 구조를 일관되게 설계할 수 있습니다. 이 문서는 거창한 세계관 설정서가 아니라, “이 게임은 무엇이며, 플레이어가 무엇을 반복하고, 어떤 재미를 느끼는가”를 설명하는 기준 문서입니다.

1. 게임 기본 정보

- 게임 제목:
- 장르:
- 플랫폼:
- 개발 엔진:
- 개발 언어:
- 플레이 방식:
- 예상 플레이 시간:
- 개발 목표:

예시:

- 게임 제목: 스테이지 액션 RPG 실습 프로젝트
- 장르: 스테이지형 액션 RPG
- 플랫폼: PC
- 개발 엔진: Unity 6
- 개발 언어: C#
- 플레이 방식: 싱글 플레이
- 예상 플레이 시간: 5~10분 단위 반복 플레이
- 개발 목표: 전투, 보상, 성장, 상점, 인벤토리 흐름을 가진 작은 게임 구현

2. 한 문장 콘셉트

이 게임을 한 문장으로 설명합니다.

작성 예시:

이 게임은 플레이어가 짧은 스테이지에 입장해 적을 처치하고, 보상을 얻어 캐릭터를 성장시키는 스테이지형 액션 RPG입니다.

작성 기준:

- 장르가 드러나는가?

- 플레이어의 핵심 행동이 드러나는가?
- 반복 플레이 구조가 보이는가?
- 구현 범위가 너무 넓게 느껴지지 않는가?

3. 핵심 플레이 경험

플레이어가 이 게임에서 어떤 경험을 하게 될지 정리합니다.

- 플레이어가 가장 자주 하는 행동:
- 플레이어가 달성해야 하는 목표:
- 플레이어가 얻는 보상:
- 플레이어가 느껴야 할 감정:
- 게임을 반복하게 만드는 이유:

예시:

플레이어는 스테이지에 입장해 적을 처치하고, 경험치와 골드를 획득합니다. 전투가 끝나면 로비로 돌아와 캐릭터 상태를 확인하고, 상점에서 포션을 구매하거나 인벤토리를 정리합니다. 이 게임의 핵심 경험은 짧은 전투를 반복하면서 캐릭터가 조금씩 강해지는 성장감입니다.

4. 핵심 재미

이 게임에서 가장 중요하게 전달해야 할 재미를 작성합니다.

- 핵심 재미 한 문장:
- 이 재미를 만드는 시스템:
- 이 재미를 방해하는 요소:
- MVP에서 반드시 필요한 기능:

예시:

이 게임의 핵심 재미는 짧은 전투를 반복하며 캐릭터가 점점 강해지는 성장감입니다. 이를 위해 전투, 보상 지급, 경험치 획득, 레벨업, 상점, 인벤토리 시스템이 필요합니다. 반대로 복잡한 스토리 분기, 대규모 장비 강화, 멀티플레이는 MVP 단계에서 제외합니다.

5. 대상 플레이어

이 게임을 누가 플레이할지 정리합니다.

- 대상 플레이어:
- 플레이어의 예상 경험 수준:
- 선호하는 플레이 방식:
- 짧은 플레이를 원하는가, 긴 플레이를 원하는가:

- 난이도 방향:

예시:

이 게임은 짧은 시간 동안 전투와 성장을 반복하고 싶은 플레이어를 대상으로 합니다. 복잡한 조작보다 간단한 이동과 공격을 중심으로 구성하며, 스테이지를 반복할수록 성장하는 느낌을 주는 것을 목표로 합니다.

6. 게임의 기본 흐름

게임 시작부터 종료까지의 큰 흐름을 작성합니다.

기본 흐름:

타이틀 화면 → 로비 화면 → 스테이지 선택 → 전투 시작 → 적 처치 → 스테이지 클리어 → 보상 획득 → 로비 복귀 → 성장 및 정비 → 다음 스테이지 도전

상세 설명:

플레이어는 타이틀 화면에서 게임을 시작하고 로비 화면으로 이동합니다. 로비에서는 캐릭터 상태와 보유 골드를 확인하고, 스테이지에 입장합니다. 스테이지에서 모든 적을 처치하면 클리어 처리되고, 보상을 받은 뒤 다시 로비로 돌아옵니다.

7. 포함할 기능

MVP 단계에서 반드시 구현할 기능을 작성합니다.

- 플레이어 이동:
- 기본 공격:
- 적 AI:
- 체력과 데미지:
- 적 사망 처리:
- 경험치와 골드 보상:
- 레벨업:
- 인벤토리:
- 상점:
- 스테이지 클리어:
- 기본 UI:

예시:

MVP에서는 플레이어 이동, 기본 공격, 적 추적, 데미지 계산, 적 사망, 보상 지급, 레벨업, 인벤토리, 상점, 스테이지 클리어 화면을 구현합니다. 이 기능들이 연결되어야 게임의 핵심 플레이 루프가 완성됩니다.

8. 제외할 기능

현재 버전에서 구현하지 않을 기능을 작성합니다.

- 멀티플레이:
- 복잡한 장비 강화:
- 스킬 트리:
- 퀘스트 분기:
- 대규모 스토리:
- 세이브 슬롯 여러 개:
- 랭킹 시스템:
- 유료 결제:
- 온라인 서버 연동:

예시:

MVP 단계에서는 멀티플레이, 대규모 장비 강화, 복잡한 스킬 트리, 퀘스트 분기, 온라인 서버 연동을 제외합니다. 이 기능들은 핵심 플레이 루프가 완성된 뒤 확장 기능으로 검토합니다.

9. 주요 시스템 요약

각 시스템을 한 문장으로 요약합니다.

- 플레이어 조작 시스템:
- 전투 시스템:
- 적 AI 시스템:
- 보상 시스템:
- 성장 시스템:
- 아이템 시스템:
- 인벤토리 시스템:
- 상점 시스템:
- 스테이지 시스템:
- UI 시스템:

예시:

전투 시스템은 플레이어와 적의 공격, 피격, 데미지 계산, 사망 처리를 담당합니다. 보상 시스템은 적 처치와 스테이지 클리어 시 경험치, 골드, 아이템을 지급합니다. 상점 시스템은 플레이어가 골드를 사용해 소비 아이템을 구매할 수 있도록 합니다.

10. 화면 구성 요약

게임에 필요한 화면을 정리합니다.

- 타이틀 화면:
- 로비 화면:
- 인게임 HUD:
- 인벤토리 화면:
- 상점 화면:
- 스테이지 결과 화면:
- 설정 화면:

예시:

로비 화면에서는 캐릭터 레벨, 보유 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼을 표시합니다. 인게임 HUD에서는 HP, 경험치, 골드, 포션 수량을 표시합니다. 결과 화면에서는 획득한 경험치, 골드, 아이템을 보여줍니다.

11. 아트와 분위기 방향

아트 스타일과 분위기를 간단히 정리합니다.

- 전체 분위기:
- 캐릭터 스타일:
- 적 스타일:
- UI 스타일:
- 색감:
- 연출 방향:

예시:

전체 분위기는 가볍고 직관적인 캐주얼 액션 RPG를 지향합니다. UI는 복잡한 장식보다 정보 전달을 우선하며, 전투 연출은 타격감과 보상 획득 피드백을 명확하게 보여주는 방향으로 구성합니다.

12. 개발 범위 요약

현재 버전에서 완성해야 할 범위를 정리합니다.

- 목표 버전:
- 구현 기간:
- 핵심 완성 기준:
- 플레이 가능한 최소 조건:

- 테스트 기준:

예시:

MVP 버전의 완성 기준은 플레이어가 로비에서 스테이지에 입장하고, 적을 처치해 보상을 얻은 뒤 다시 로비로 돌아오는 흐름이 동작하는 것입니다. 이 흐름 안에서 전투, 보상, 성장, UI가 최소한으로 연결되어야 합니다.

13. AI 구현 요청용 콘셉트 요약

AI에게 전달할 수 있도록 짧은 프롬프트 형태로 정리합니다.

예시:

Unity 6 C#으로 스테이지형 액션 RPG 실습 프로젝트를 만든다. 플레이어는 로비에서 스테이지를 선택하고, 전투 구간에 입장해 적을 처치한다. 적을 처치하면 경험치와 골드를 얻고, 스테이지를 클리어하면 보상 화면을 확인한 뒤 로비로 돌아온다. MVP 범위는 플레이어 이동, 기본 공격, 적 AI, 데미지 계산, 보상 지급, 레벨업, 인벤토리, 상점, 기본 UI로 제한한다. 멀티플레이, 복잡한 장비 강화, 스킬 트리, 퀘스트 분기는 제외한다.

14. 검토 체크리스트

- 게임을 한 문장으로 설명할 수 있는가?
- 플레이어의 핵심 행동이 명확한가?
- 핵심 재미가 구체적으로 작성되어 있는가?
- 포함할 기능과 제외할 기능이 구분되어 있는가?
- 핵심 플레이 루프가 보이는가?
- 주요 시스템이 구현 가능한 단위로 정리되어 있는가?
- 화면 구성이 게임 흐름과 연결되어 있는가?
- MVP 범위가 너무 크지 않은가?
- AI에게 전달할 수 있는 콘셉트 요약이 있는가?

게임 콘셉트 문서는 이후 모든 상세 기획서의 기준이 됩니다. 콘셉트가 모호하면 전투, 보상, UI, 데이터 구조도 함께 흔들립니다. 반대로 콘셉트가 명확하면 어떤 기능을 먼저 만들고, 어떤 기능을 제외해야 하는지 판단하기 쉬워집니다. 좋은 게임 콘셉트 문서는 게임의 매력을 설명하면서도, 개발자가 실제로 구현할 수 있는 범위를 정리하는 출발 문서입니다.

부록 C. 핵심 플레이 루프 작성 템플릿

핵심 플레이 루프는 플레이어가 게임에서 가장 자주 반복하는 행동 흐름을 정리한 문서입니다. 게임의 재미는 대부분 이 반복 구조 안에서 만들어집니다. 플레이어가 무엇을 하고, 어떤 결과를 얻고, 다시 무엇을 하게 되는지 명확해야 필요한 시스템과 UI, 데이터 구조를 정리할 수 있습니다. 이 템플릿은 게임의 반복 흐름을 구현 가능한 형태로 정리하기 위한 기본 양식입니다.

1. 게임 기본 정보

- 게임 제목:
- 장르:
- 플랫폼:
- 개발 엔진:
- 개발 언어:
- 플레이 방식:
- 핵심 플레이 시간:
- 반복 플레이 여부:

예시:

- 게임 제목: 스테이지 액션 RPG 실습 프로젝트
- 장르: 스테이지형 액션 RPG
- 플랫폼: PC
- 개발 엔진: Unity 6
- 개발 언어: C#
- 플레이 방식: 싱글 플레이
- 핵심 플레이 시간: 스테이지당 3~5분
- 반복 플레이 여부: 스테이지 클리어 후 로비로 돌아와 성장하고 다음 스테이지에 도전

2. 핵심 플레이 루프 한 문장 요약

플레이어가 반복하는 흐름을 한 문장으로 작성합니다.

작성 예시:

플레이어는 로비에서 스테이지에 입장하고, 적을 처치해 보상을 얻은 뒤, 캐릭터를 성장시켜 다음 스테이지에 도전합니다.

작성 기준:

- 플레이어의 시작 지점이 보이는가?

- 반복 행동이 명확한가?
- 보상과 성장 요소가 포함되어 있는가?
- 다음 행동으로 이어지는 구조가 있는가?

3. 기본 루프 구조

게임의 반복 흐름을 단계별로 작성합니다.

기본 형식:

준비 → 도전 → 행동 → 결과 → 보상 → 성장 → 재도전

실습 프로젝트 예시:

로비에서 준비한다 → 스테이지에 입장한다 → 적을 처치한다 → 스테이지를 클리어한다 → 경험치와 골드를 얻는다 → 캐릭터를 성장시킨다 → 다음 스테이지에 도전한다

상세 설명:

플레이어는 로비에서 캐릭터 상태와 보유 골드를 확인합니다. 이후 스테이지를 선택해 전투에 입장하고, 적을 처치해 경험치와 골드를 획득합니다. 스테이지를 클리어하면 보상 화면을 확인하고 로비로 돌아옵니다. 로비에서는 상점이나 인벤토리를 이용한 뒤 다음 스테이지에 다시 도전합니다.

4. 루프 단계별 정리

각 단계가 어떤 역할을 하는지 작성합니다.

단계	플레이어 행동	시스템 반응	결과
준비	로비에서 상태 확인	캐릭터 정보와 골드 표시	스테이지 도전 준비
도전	스테이지 선택	StageData 로드	전투 시작
행동	이동과 공격	적 AI, 데미지 계산	적 체력 감소
결과	모든 적 처치	클리어 판정	결과 화면 표시
보상	보상 확인	경험치, 골드, 아이템 지급	데이터 갱신
성장	상점/인벤토리 사용	아이템 구매, 포션 사용	다음 도전 준비
재도전	다음 스테이지 선택	난이도 상승	루프 반복

이 표는 기획 내용을 시스템 단위로 분해하는 데 도움이 됩니다. 각 단계가 어떤 코드와 연결되는지 함께 생각하면 이후 구현 요청 프롬프트로 변환하기 쉽습니다.

5. 루프에 필요한 주요 시스템

핵심 플레이 루프를 완성하기 위해 필요한 시스템을 작성합니다.

- 로비 시스템:

- 스테이지 선택 시스템:
- 플레이어 조작 시스템:
- 전투 시스템:
- 적 AI 시스템:
- 체력 및 데미지 시스템:
- 보상 시스템:
- 경험치 및 레벨업 시스템:
- 인벤토리 시스템:
- 상점 시스템:
- UI 시스템:
- 저장 시스템:

예시:

로비 시스템은 플레이어 상태, 골드, 스테이지 선택 버튼을 표시합니다. 전투 시스템은 플레이어와 적의 공격, 피격, 사망 처리를 담당합니다. 보상 시스템은 적 처치와 스테이지 클리어 시 경험치, 골드, 아이템을 지급합니다.

6. 루프에 필요한 UI 화면

핵심 플레이 루프에 필요한 화면을 작성합니다.

- 타이틀 화면:
- 로비 화면:
- 스테이지 선택 화면:
- 인게임 HUD:
- 결과 화면:
- 인벤토리 화면:
- 상점 화면:
- 설정 화면:

예시:

로비 화면에서는 캐릭터 레벨, 보유 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼을 표시합니다. 인게임 HUD에서는 HP, 경험치, 포션 수량, 골드를 표시합니다. 결과 화면에서는 스테이지 클리어 여부와 획득한 보상을 보여줍니다.

7. 루프에 필요한 데이터

핵심 플레이 루프에서 사용하는 데이터를 작성합니다.

- PlayerData:
- PlayerLevelData:
- EnemyData:
- StageData:
- RewardData:
- ItemData:
- InventoryData:
- ShopItemData:

예시:

StageData에는 StageId, StageName, EnemyList, ClearCondition, RewardId를 포함합니다. RewardData에는 RewardId, ExpAmount, GoldAmount, ItemId, ItemCount, DropRate를 포함합니다. PlayerLevelData에는 Level, RequiredExp, MaxHp, AttackPower를 포함합니다.

8. 보상과 성장 연결

플레이어가 루프를 반복해야 하는 이유를 작성합니다.

- 적 처치 보상:
- 스테이지 클리어 보상:
- 보상 사용처:
- 성장 방식:
- 성장 후 변화:
- 다음 도전 동기:

예시:

플레이어는 적을 처치해 경험치와 골드를 얻습니다. 경험치가 일정 수치에 도달하면 레벨업하고, 최대 HP와 공격력이 증가합니다. 골드는 상점에서 포션을 구매하는 데 사용합니다. 성장한 플레이어는 더 높은 난이도의 스테이지에 도전할 수 있습니다.

9. 실패 조건과 재도전 흐름

플레이어가 실패했을 때의 흐름을 작성합니다.

- 실패 조건:
- 실패 시 데이터 처리:
- 실패 화면:
- 보상 지급 여부:
- 로비 복귀 여부:

- 재도전 방식:

예시:

플레이어 HP가 0 이하가 되면 스테이지 실패로 처리합니다. 실패 시 스테이지 클리어 보상은 지급하지 않지만, 이미 처치한 적의 경험치와 골드는 유지할 수 있습니다. 실패 화면을 표시한 뒤 로비로 돌아가며, 플레이어는 포션을 구매하거나 다시 같은 스테이지에 도전할 수 있습니다.

10. 예외 조건

핵심 플레이 루프에서 발생할 수 있는 예외 상황을 작성합니다.

- 스테이지 시작 중 로딩 실패:
- 스테이지 클리어와 플레이어 사망이 동시에 발생:
- 보상 지급 중 인벤토리 공간 부족:
- 결과 화면이 중복으로 열림:
- 보상이 중복 지급됨:
- 로비 복귀 중 저장 실패:
- 잘못된 StageId 선택:

예시:

스테이지 클리어와 플레이어 사망 조건이 동시에 발생한 경우, 먼저 발생한 이벤트를 기준으로 결과를 결정합니다. 보상 지급은 한 번만 처리되어야 하며, 결과 화면이 중복으로 열리지 않도록 StageResult 상태를 관리합니다.

11. MVP 기준 루프

가장 작은 플레이 가능 버전의 루프를 작성합니다.

MVP 루프 예시:

로비 화면 표시 → 스테이지 시작 버튼 클릭 → 적 3마리 스폰 → 플레이어가 적 처치 → 모든 적 사망 시 클리어 → 경험치와 골드 지급 → 결과 화면 표시 → 로비 복귀

MVP에서 제외할 기능:

- 복잡한 스킬 시스템
- 장비 강화
- 퀘스트 분기
- 멀티플레이
- 보스 패턴
- 온라인 랭킹
- 유료 결제

MVP 루프는 게임의 핵심이 실제로 동작하는지 확인하기 위한 최소 단위입니다. 이 흐름이 완성되면 이후 기능을 확장할 수 있습니다.

12. AI 구현 요청용 루프 요약

AI에게 전달할 수 있는 형태로 핵심 플레이 루프를 정리합니다.

예시:

Unity 6 C#으로 스테이지형 액션 RPG의 핵심 플레이 루프를 구현해줘. 플레이어는 로비 화면에서 스테이지 시작 버튼을 누르고, StageData를 기준으로 전투 스테이지에 입장한다. 스테이지가 시작되면 EnemyData에 따라 적을 스폰하고, 플레이어가 모든 적을 처치하면 클리어 처리한다. 클리어 시 RewardData를 기준으로 경험치와 골드를 지급하고 결과 화면을 표시한다. 결과 화면에서 확인 버튼을 누르면 로비로 돌아간다. 플레이어가 Dead 상태가 되면 실패 처리하고 클리어 보상은 지급하지 않는다.

13. 검토 체크리스트

- 플레이어가 반복하는 행동이 명확한가?
- 루프의 시작과 끝이 분명한가?
- 보상과 성장 요소가 연결되어 있는가?
- 다음 도전으로 이어지는 이유가 있는가?
- 각 단계가 필요한 시스템과 연결되어 있는가?
- 필요한 UI 화면이 정리되어 있는가?
- 필요한 데이터 테이블이 정리되어 있는가?
- 실패 조건과 재도전 흐름이 있는가?
- 예외 조건이 작성되어 있는가?
- MVP 기준 루프가 너무 크지 않은가?
- AI 구현 요청 프롬프트로 변환할 수 있는가?

핵심 플레이 루프는 게임의 중심 구조입니다. 루프가 명확하면 어떤 시스템을 먼저 만들어야 하는지 판단하기 쉽고, 루프가 모호하면 기능을 많이 만들어도 게임이 완성되지 않은 느낌을 줄 수 있습니다. 좋은 핵심 플레이 루프 문서는 플레이어의 반복 행동과 보상, 성장, 재도전 흐름을 하나의 개발 기준으로 연결하는 문서입니다.

부록 D. 플레이어 조작 기획서 템플릿

플레이어 조작 기획서는 플레이어가 캐릭터를 어떻게 움직이고, 공격하고, 상호작용하는지 정리하는 문서입니다. 조작은 입력, 캐릭터 상태, 애니메이션, 전투 판정, UI 안내와 직접 연결됩니다. 따라서 단순히 키 배치만 적는 것이 아니라, 입력이 어떤 조건에서 가능하고 어떤 결과를 만드는지까지 작성해야 합니다. 이 템플릿은 플레이어 조작을 구현 가능한 형태로 정리하기 위한 기본 양식입니다.

1. 게임 기본 정보

- 게임 제목:
- 장르:
- 플랫폼:
- 개발 엔진:
- 개발 언어:
- 입력 방식:
- 카메라 방식:
- 조작 기준:

예시:

- 게임 제목: 스테이지 액션 RPG 실습 프로젝트
- 장르: 스테이지형 액션 RPG
- 플랫폼: PC
- 개발 엔진: Unity 6
- 개발 언어: C#
- 입력 방식: 키보드 + 마우스
- 카메라 방식: 탑다운 또는 3인칭 고정 카메라
- 조작 기준: 플레이어가 캐릭터를 직접 이동하고, 마우스 또는 키 입력으로 공격한다

2. 조작 콘셉트

플레이어 조작의 방향을 한 문장으로 작성합니다.

작성 예시:

플레이어는 WASD로 캐릭터를 이동하고, 마우스 좌클릭으로 기본 공격을 하며, Space 키로 짧은 대시를 사용해 적의 공격을 피합니다.

작성 기준:

- 이동 방식이 명확한가?

- 공격 방식이 명확한가?
- 회피나 대시가 있는가?
- 조작이 게임 장르와 어울리는가?
- 입력 장치가 분명한가?

3. 입력 목록

플레이어가 사용할 입력을 정리합니다.

기능	입력	설명	비고
이동	WASD	캐릭터를 상하좌우로 이동	카메라 기준 또는 월드 기준 선택
기본 공격	마우스 좌클릭	전방 또는 마우스 방향으로 공격	공격 쿨타임 적용
대시	Space	현재 이동 방향으로 짧게 이동	쿨타임 또는 스테미나 필요
포션 사용	숫자 키 1	체력 포션 사용	보유 수량 필요
상호작용	E	NPC, 상자, 포털과 상호작용	상호작용 범위 필요
일시정지	Esc	일시정지 메뉴 열기	전투 중 가능 여부 결정

이 표는 입력과 기능을 연결하는 기본 문서입니다. 이후 구현 단계에서는 각 입력이 어떤 상태와 조건에서 처리되는지 추가로 정의해야 합니다.

4. 이동 기획

- 이동 입력:
- 이동 방향:
- 이동 속도:
- 이동 가능 상태:
- 이동 불가능 상태:
- 카메라 기준 이동 여부:
- 이동 애니메이션:
- 정지 애니메이션:

예시:

플레이어는 WASD 입력으로 이동합니다. 이동 방향은 월드 좌표 기준이며, 이동 속도는 MoveSpeed 값을 사용합니다. 플레이어가 Idle 또는 Move 상태일 때만 이동할 수 있습니다. Attack, Hit, Dead, StageClear 상태에서는 이동 입력을 무시합니다.

5. 공격 기획

- 공격 입력:
- 공격 방식:
- 공격 방향:
- 공격 판정 범위:
- 공격 데미지:
- 공격 쿨타임:
- 공격 가능 상태:
- 공격 불가능 상태:
- 공격 애니메이션:
- 공격 판정 발생 타이밍:

예시:

플레이어는 마우스 좌클릭으로 기본 공격을 합니다. 공격 입력이 들어오면 Attack 상태로 전환하고 공격 애니메이션을 재생합니다. 애니메이션 재생 중 지정된 타이밍에 전방 공격 판정을 생성합니다. 공격 쿨타임이 끝나지 않았거나 플레이어가 Dead 상태이면 공격 입력을 무시합니다.

6. 대시 또는 회피 기획

- 대시 입력:
- 대시 방향:
- 대시 거리:
- 대시 속도:
- 대시 쿨타임:
- 무적 시간 여부:
- 대시 가능 상태:
- 대시 불가능 상태:
- 대시 중 충돌 처리:
- 대시 애니메이션:

예시:

플레이어는 Space 키를 눌러 현재 이동 방향으로 대시합니다. 이동 입력이 없을 때는 캐릭터가 바라보는 방향으로 대시합니다. 대시 중에는 0.2초 동안 피격을 무시합니다. 대시 쿨타임이 끝나지 않았거나 플레이어가 Hit, Attack, Dead 상태이면 대시할 수 없습니다.

7. 아이템 사용 기획

- 아이템 사용 입력:
- 사용 가능한 아이템:
- 사용 조건:
- 사용 결과:
- 수량 감소 여부:
- 사용 실패 조건:
- UI 갱신 조건:

예시:

플레이어는 숫자 키 1을 눌러 체력 포션을 사용합니다. 체력 포션을 사용하면 현재 HP가 EffectValue만큼 회복되고, 포션 수량이 1 감소합니다. 현재 HP가 최대 HP인 경우에는 포션을 사용하지 않고 안내 메시지를 표시합니다. 포션 수량이 0이면 입력을 무시하고 토스트 메시지를 출력합니다.

8. 상호작용 기획

- 상호작용 입력:
- 상호작용 대상:
- 상호작용 가능 거리:
- 상호작용 가능 상태:
- 상호작용 결과:
- 우선순위 규칙:
- 상호작용 UI 표시 여부:

예시:

플레이어는 E 키를 눌러 가까운 상호작용 대상과 상호작용합니다. 상호작용 대상은 NPC, 상자, 포털이 될 수 있습니다. 여러 대상이 범위 안에 있을 경우 가장 가까운 대상을 우선합니다. 상호작용 가능한 대상이 있을 때는 화면에 안내 UI를 표시합니다.

9. 캐릭터 상태와 조작 가능 조건

플레이어 상태별 입력 가능 여부를 정리합니다.

상태	이동	공격	대시	아이템 사용	상호작용
Idle	가능	가능	가능	가능	가능
Move	가능	가능	가능	가능	가능
Attack	불가능 또는 제한	불가능	불가능	불가능	불가능

Dash	불가능	불가능	불가능	불가능	불가능
Hit	불가능	불가능	불가능	가능 여부 결정	불가능
Dead	불가능	불가능	불가능	불가능	불가능
StageClear	불가능	불가능	불가능	불가능	불가능

이 표는 입력 차단 규칙을 구현할 때 기준이 됩니다. 상태별 조작 가능 조건이 없으면 공격 중 이동, 사망 후 아이템 사용, 클리어 후 추가 입력 같은 버그가 발생할 수 있습니다.

10. 애니메이션 연결

- Idle 애니메이션:
- Move 애니메이션:
- Attack 애니메이션:
- Dash 애니메이션:
- Hit 애니메이션:
- Dead 애니메이션:
- 상태 전환 조건:
- 애니메이션 이벤트 사용 여부:

예시:

이동 입력이 없으면 Idle 애니메이션을 재생하고, 이동 입력이 있으면 Move 애니메이션을 재생합니다. 공격 입력이 들어오면 Attack 애니메이션을 재생하며, 공격 판정은 애니메이션 이벤트를 통해 발생시킵니다. HP가 0 이하가 되면 Dead 애니메이션을 재생하고 모든 입력을 차단합니다.

11. UI 안내

- 조작 안내 표시 여부:
- 상호작용 안내:
- 쿨타임 표시:
- 아이템 수량 표시:
- 입력 불가 안내:
- 튜토리얼 메시지:

예시:

인게임 HUD에는 포션 수량과 대시 쿨타임을 표시합니다. 상호작용 가능한 대상이 가까이 있으면 “E: 상호작용” 안내를 표시합니다. 포션 수량이 부족하거나 스킬 쿨타임 중 입력이 들어오면 짧은 토스트 메시지를 출력합니다.

12. 예외 조건

- 플레이어가 Dead 상태일 때 입력이 들어온 경우:
- 공격 쿨타임 중 공격 입력이 들어온 경우:
- 대시 쿨타임 중 대시 입력이 들어온 경우:
- 포션이 없는데 아이템 사용 입력이 들어온 경우:
- 상호작용 대상이 없는데 E 키를 누른 경우:
- UI 팝업이 열린 상태에서 조작 입력이 들어온 경우:
- 스테이지 클리어 후 추가 입력이 들어온 경우:
- 로딩 중 입력이 들어온 경우:

예시:

UI 팝업이 열린 상태에서는 이동, 공격, 대시 입력을 무시합니다. 단, 팝업 닫기 입력은 처리할 수 있습니다. 플레이어가 Dead 상태가 되면 모든 조작 입력을 차단하고 결과 화면으로 전환합니다.

13. 필요한 데이터

- MoveSpeed:
- AttackPower:
- AttackRange:
- AttackCooldown:
- DashDistance:
- DashCooldown:
- DashInvincibleTime:
- PotionItemId:
- InteractionRange:

예시:

MoveSpeed는 플레이어 이동 속도입니다. AttackCooldown은 기본 공격을 다시 사용할 수 있을 때까지의 시간입니다. DashInvincibleTime은 대시 중 피격을 무시하는 시간입니다. 모든 시간 단위는 초로 통일합니다.

14. AI 구현 요청용 프롬프트

예시:

Unity 6 C#에서 사용할 플레이어 조작 시스템을 구현해줘. 플레이어는 WASD로 이동하고, 마우스 좌클릭으로 기본 공격을 하며, Space 키로 대시한다. 플레이어 상태는 Idle, Move, Attack, Dash, Hit, Dead로 관리한다. Dead 상태에서는 모든 입력을 무시한다. 공격 입력 시 Attack 상태로 전환하

고 공격 애니메이션을 재생하며, 공격 쿨타임이 끝난 경우에만 전방 범위의 적에게 데미지를 적용한다. 대시는 현재 이동 방향으로 실행하고, 대시 중 0.2초 동안 피격을 무시한다. 기존 public 메서드 이름은 변경하지 말고, 상태별 입력 가능 조건을 코드에 반영해줘.

15. 검토 체크리스트

- 입력 장치가 명확한가?
- 이동, 공격, 대시, 아이템 사용 입력이 정리되어 있는가?
- 각 조작의 실행 조건이 작성되어 있는가?
- 각 조작의 불가능 조건이 작성되어 있는가?
- 캐릭터 상태별 입력 가능 여부가 정리되어 있는가?
- 조작과 애니메이션 연결이 명확한가?
- 공격 판정 발생 타이밍이 정의되어 있는가?
- UI 팝업, 사망, 로딩 중 입력 차단 조건이 있는가?
- 필요한 데이터와 단위가 정리되어 있는가?
- AI 구현 요청 프롬프트로 변환할 수 있는가?

플레이어 조작 기획서는 입력 목록을 적는 문서에서 끝나면 안 됩니다. 입력이 어떤 조건에서 처리되고, 어떤 상태와 애니메이션으로 이어지며, 어떤 시스템을 호출하는지까지 정리해야 합니다. 좋은 조작 기획서는 플레이어의 버튼 입력을 게임 안의 행동, 상태 변화, 전투 결과로 연결하는 개발 기준 문서입니다.

부록 E. 전투 시스템 기획서 템플릿

전투 시스템 기획서는 플레이어와 적이 어떻게 공격하고, 피격되고, 사망하며, 보상을 발생시키는지 정의하는 문서입니다. 전투는 입력, 상태, 애니메이션, 판정, 수치 계산, 보상, UI 피드백이 함께 연결되는 핵심 시스템입니다. 따라서 “공격한다”, “데미지를 받는다”처럼 단순히 작성하는 것보다, 어떤 조건에서 어떤 판정과 계산이 발생하는지 구체적으로 정리해야 합니다. 이 템플릿은 전투 시스템을 구현 가능한 기준으로 정리하기 위한 기본 양식입니다.

1. 전투 시스템 기본 정보

- 게임 제목:
- 장르:
- 개발 엔진:
- 개발 언어:
- 전투 방식:
- 카메라 시점:
- 공격 방식:
- 주요 전투 대상:
- 전투 목표:

예시:

- 게임 제목: 스테이지 액션 RPG 실습 프로젝트
- 장르: 스테이지형 액션 RPG
- 개발 엔진: Unity 6
- 개발 언어: C#
- 전투 방식: 실시간 액션 전투
- 카메라 시점: 탑다운 또는 3인칭
- 공격 방식: 기본 공격 중심
- 주요 전투 대상: 일반 적, 보스 적
- 전투 목표: 스테이지 안의 모든 적을 처치한다

2. 전투 콘셉트

전투 시스템의 방향을 한 문장으로 작성합니다.

작성 예시:

플레이어는 이동과 기본 공격을 사용해 적의 공격을 피하면서 적을 처치하고, 적을 처치하면 경험치와 골드를 획득합니다.

작성 기준:

- 플레이어가 어떤 행동을 하는가?
- 적은 어떤 방식으로 대응하는가?
- 전투의 목표가 명확한가?
- 전투 결과가 보상과 연결되는가?
- MVP 범위에서 구현 가능한가?

3. 전투 기본 흐름

전투가 시작되고 종료될 때까지의 흐름을 작성합니다.

기본 흐름:

스테이지 시작 → 적 스폰 → 플레이어 이동 및 공격 → 적 피격 → 데미지 계산 → 적 체력 감소 → 적 사망 → 보상 지급 → 모든 적 처치 시 스테이지 클리어

상세 설명:

스테이지가 시작되면 StageData를 기준으로 적이 배치됩니다. 플레이어는 이동과 공격을 사용해 적을 처치합니다. 공격 판정에 적이 포함되면 데미지를 계산하고 적의 HP를 감소시킵니다. 적의 HP가 0 이하가 되면 Dead 상태로 전환하고 보상을 지급합니다.

4. 플레이어 공격 기획

- 공격 입력:
- 공격 종류:
- 공격 방향:
- 공격 판정 방식:
- 공격 범위:
- 공격 데미지:
- 공격 쿨타임:
- 공격 가능 상태:
- 공격 불가능 상태:
- 공격 애니메이션:
- 공격 판정 발생 타이밍:

예시:

플레이어는 마우스 좌클릭으로 기본 공격을 실행합니다. 공격 입력이 들어오면 플레이어는 Attack 상태로 전환하고 공격 애니메이션을 재생합니다. 애니메이션 이벤트 시점에 전방 원형 또는 박스 형태의 공격 판정을 생성합니다. 판정 범위 안에 살아 있는 적이 있으면 데미지를 적용합니다.

5. 적 공격 기획

- 적 공격 조건:
- 적 공격 범위:
- 적 공격 데미지:
- 적 공격 쿨타임:
- 적 공격 대상:
- 적 공격 전 준비 시간:
- 적 공격 후 딜레이:
- 적 공격 애니메이션:
- 적 공격 판정 타이밍:

예시:

적은 플레이어가 공격 범위 안에 들어오면 공격 상태로 전환합니다. 공격 애니메이션이 재생된 뒤 지정된 타이밍에 플레이어에게 데미지를 적용합니다. 공격 후에는 일정 시간 동안 다음 공격을 하지 않습니다.

6. 데미지 계산 규칙

- 기본 공격력:
- 방어력:
- 최종 데미지 계산식:
- 최소 데미지:
- 치명타 여부:
- 치명타 확률:
- 치명타 배율:
- 속성 보정:
- 버프/디버프 적용 여부:

예시:

최종 데미지는 공격력 - 방어력으로 계산합니다. 계산 결과가 1보다 작으면 최소 데미지 1을 적용합니다. MVP 단계에서는 치명타, 속성 보정, 버프와 디버프는 제외하고 기본 공격력과 방어력만 사용합니다.

7. 체력과 사망 처리

- 최대 HP:
- 현재 HP:
- HP 감소 조건:
- HP 회복 조건:
- HP가 0 이하가 되었을 때:
- 사망 상태:
- 사망 애니메이션:
- 사망 후 충돌 처리:
- 사망 후 입력 처리:
- 사망 후 보상 지급 여부:

예시:

대상의 현재 HP가 0 이하가 되면 Dead 상태로 전환합니다. Dead 상태가 된 적은 추가 데미지를 받지 않으며, 공격 대상에서도 제외됩니다. 적 사망 시 경험치와 골드를 지급합니다. 플레이어가 Dead 상태가 되면 모든 입력을 차단하고 스테이지 실패 처리로 전환합니다.

8. 피격 처리

- 피격 조건:
- 피격 애니메이션:
- 피격 경직 여부:
- 넉백 여부:
- 넉백 방향:
- 넉백 거리:
- 피격 무적 시간:
- 중복 피격 방지:
- 피격 이펙트:
- 피격 사운드:

예시:

플레이어가 적의 공격 판정에 맞으면 HP가 감소하고 Hit 상태로 전환합니다. Hit 상태에서는 짧은 시간 동안 이동과 공격을 제한할 수 있습니다. 피격 후 0.5초 동안 무적 시간을 적용해 동일한 공격으로 중복 피해를 받지 않도록 합니다.

9. 전투 상태 정의

전투에서 사용하는 상태를 정리합니다.

상태	설명	가능한 행동	전환 조건
Idle	대기 상태	이동, 공격 가능	입력 없음
Move	이동 상태	이동, 공격 가능	이동 입력
Attack	공격 상태	이동 제한, 추가 공격 제한	공격 입력
Hit	피격 상태	입력 제한	피격 발생
Dash	회피 상태	피격 무시 가능	대시 입력
Dead	사망 상태	모든 행동 불가	HP 0 이하

상태 정의는 전투 시스템의 안정성을 높입니다. 상태별 가능한 행동이 정리되어 있으면 공격 중 중복 공격, 사망 후 이동, 피격 중 대시 같은 문제를 줄일 수 있습니다.

10. 적 AI 전투 행동

- 인식 범위:
- 추적 조건:
- 공격 조건:
- 복귀 조건:
- 사망 조건:
- 이동 속도:
- 공격 우선순위:
- 타깃 재탐색 조건:

예시:

적은 플레이어가 인식 범위 안에 들어오면 추적을 시작합니다. 플레이어가 공격 범위 안에 들어오면 이동을 멈추고 공격합니다. 플레이어가 인식 범위를 벗어나면 원래 위치로 돌아가거나 추적을 중단합니다. Dead 상태가 되면 모든 AI 행동을 중지합니다.

11. 보스 전투 기획

- 보스 등장 조건:
- 보스 HP:
- 보스 공격 패턴:
- 패턴 전환 조건:
- 페이즈 구분:

- 보스 사망 조건:
- 보스 보상:
- 보스 전투 예외 조건:

예시:

보스는 스테이지 마지막에 등장합니다. HP가 50% 이하가 되면 2페이지로 전환하고 공격 속도가 증가합니다. MVP 단계에서는 보스 패턴을 기본 공격과 범위 공격 두 가지로 제한합니다.

12. 보상 연결

- 적 처치 보상:
- 보스 처치 보상:
- 스테이지 클리어 보상:
- 경험치 지급 조건:
- 골드 지급 조건:
- 아이템 드랍 조건:
- 중복 지급 방지 조건:

예시:

일반 적이 사망하면 EnemyData에 정의된 경험치와 골드를 지급합니다. 보스가 사망하면 추가 보상을 지급하고 스테이지 클리어 조건을 검사합니다. 보상은 적이 Dead 상태로 처음 전환될 때 한 번만 지급해야 합니다.

13. 전투 UI 피드백

- 플레이어 HP 표시:
- 적 HP 표시:
- 데미지 숫자 표시:
- 피격 이펙트:
- 사망 이펙트:
- 보상 획득 표시:
- 스킬 쿨타임 표시:
- 상태 이상 표시:

예시:

플레이어 HP는 인게임 HUD의 HP 바로 표시합니다. 적이 피해를 받으면 데미지 숫자를 표시하고, 적 HP 바를 갱신합니다. 적이 사망하면 사망 이펙트를 재생하고 보상 획득 메시지를 표시합니다.

14. 전투 데이터 목록

- PlayerCombatData:
- EnemyData:
- BossData:
- SkillData:
- DamageData:
- RewardData:
- StageData:

예시 필드:

EnemyData에는 EnemyId, Name, MaxHp, AttackPower, Defense, MoveSpeed, DetectRange, AttackRange, AttackCooldown, RewardExp, RewardGold를 포함합니다. PlayerCombatData에는 MaxHp, AttackPower, Defense, AttackRange, AttackCooldown, InvincibleTime을 포함합니다.

15. 예외 조건

- 플레이어가 Dead 상태일 때 공격 입력이 들어온 경우:
- 적이 Dead 상태인데 추가 공격을 받은 경우:
- 공격 쿨타임 중 공격 입력이 들어온 경우:
- 공격 판정이 같은 적에게 여러 번 적용되는 경우:
- 피격 무적 시간 중 데미지를 받은 경우:
- 적 사망 보상이 중복 지급되는 경우:
- 보스 사망과 플레이어 사망이 동시에 발생한 경우:
- 스테이지 종료 후 전투 입력이 들어온 경우:
- 전투 중 UI 팝업이 열린 경우:

예시:

Dead 상태의 대상은 공격 판정 대상에서 제외합니다. 보상은 사망 처리 시 한 번만 지급되며, 이미 보상 지급이 완료된 적은 다시 보상을 지급하지 않습니다. 스테이지 클리어 또는 실패 이후에는 모든 전투 입력과 AI 행동을 중지합니다.

16. AI 구현 요청용 프롬프트

예시:

Unity 6 C#에서 사용할 전투 시스템을 구현해줘. 플레이어는 마우스 좌클릭으로 기본 공격을 하고, 공격 시 Attack 상태로 전환한다. 공격 애니메이션 이벤트 시점에 전방 공격 판정을 발생시키고, 판정

범위 안에 있는 살아 있는 적에게 데미지를 적용한다. 데미지는 공격력에서 방어력을 뺀 값이며 최소 데미지는 1이다. 적의 HP가 0 이하가 되면 Dead 상태로 전환하고 EnemyData에 정의된 경험치와 골드를 한 번만 지급한다. Dead 상태의 적은 추가 데미지를 받지 않고, 보상도 중복 지급되지 않도록 처리해줘.

17. 테스트 기준

- 플레이어 공격이 적에게 데미지를 적용하는가?
- 공격 범위 밖의 적은 데미지를 받지 않는가?
- 공격 쿨타임 중 중복 공격이 막히는가?
- 방어력을 반영해 데미지가 계산되는가?
- 최소 데미지가 적용되는가?
- 적 HP가 0 이하가 되면 Dead 상태가 되는가?
- Dead 상태 적에게 추가 데미지가 적용되지 않는가?
- 적 사망 보상이 한 번만 지급되는가?
- 플레이어가 Dead 상태일 때 입력이 차단되는가?
- 스테이지 종료 후 전투가 중지되는가?

18. 검토 체크리스트

- 전투 기본 흐름이 명확한가?
- 플레이어 공격 조건이 작성되어 있는가?
- 적 공격 조건이 작성되어 있는가?
- 데미지 계산식이 정의되어 있는가?
- 체력과 사망 처리 규칙이 명확한가?
- 피격, 낙백, 무적 시간 조건이 있는가?
- 전투 상태별 가능한 행동이 정리되어 있는가?
- 보상 지급 조건과 중복 지급 방지 조건이 있는가?
- 전투 UI 피드백이 정의되어 있는가?
- 전투 데이터 필드와 단위가 정리되어 있는가?
- 예외 조건이 충분히 작성되어 있는가?
- AI 구현 요청 프롬프트로 변환할 수 있는가?

전투 시스템 기획서는 액션의 분위기만 설명하는 문서가 아닙니다. 공격 입력, 판정, 데미지 계산, 피격, 사망, 보상 지급까지 하나의 흐름으로 연결해야 합니다. 좋은 전투 시스템 기획서는 플레이어의 공격 한 번이 어떤 상태 변화와 데이터 변경으로 이어지는지 명확히 설명하는 개발 기준 문서입니다.

부록 F. 아이템 데이터 테이블 템플릿

아이템 데이터 테이블은 게임에 등장하는 아이템의 고정 정보를 정리하는 문서입니다. 아이템 이름, 종류, 설명, 가격, 최대 보유 수량, 효과값 같은 정보를 코드와 분리해 관리하면 밸런싱과 수정이 쉬워집니다. 특히 AI에게 아이템 시스템 구현을 요청할 때는 데이터 항목이 명확해야 합니다. 이 템플릿은 아이템 정보를 구현 가능한 데이터 구조로 정리하기 위한 기본 양식입니다.

1. 아이템 데이터 기본 정보

- 데이터 테이블 이름:
- 사용 목적:
- 관리 방식:
- 사용 시스템:
- 데이터 작성자:
- 마지막 수정일:

예시:

- 데이터 테이블 이름: ItemData
- 사용 목적: 게임 내 아이템의 고정 정보 관리
- 관리 방식: ScriptableObject, CSV, JSON, Google Sheets 중 선택
- 사용 시스템: 인벤토리, 상점, 보상, 아이템 사용 시스템
- 마지막 수정일: 2026-06-21

2. 아이템 분류

아이템은 용도에 따라 먼저 분류해야 합니다.

- 소비 아이템:
- 장비 아이템:
- 재료 아이템:
- 퀘스트 아이템:
- 재화 아이템:
- 기타 아이템:

예시:

소비 아이템은 사용하면 즉시 효과가 발생하고 수량이 감소합니다. 장비 아이템은 캐릭터 능력치에 영향을 줍니다. 재료 아이템은 제작이나 강화에 사용할 수 있습니다. MVP 단계에서는 소비 아이템과 재료 아이템만 사용하고, 장비 아이템은 이후 확장 기능으로 분리할 수 있습니다.

3. 아이템 데이터 필드 정의

필드명	타입	설명	예시
ItemId	int 또는 string	아이템 고유 ID	1001
ItemName	string	아이템 이름	체력 포션
ItemType	enum/string	아이템 종류	Consumable
Description	string	아이템 설명	HP를 50 회복합니다
IconKey	string	아이콘 리소스 키	icon_potion_hp
MaxStack	int	최대 중첩 수량	99
EffectType	enum/string	아이템 효과 종류	Heal
EffectValue	int/float	효과 수치	50
Price	int	상점 구매 가격	100
SellPrice	int	판매 가격	25
IsUsable	bool	사용 가능 여부	true
IsSellable	bool	판매 가능 여부	true

필드명은 코드에서 그대로 사용할 가능성이 높기 때문에 의미가 명확해야 합니다. Value처럼 애매한 이름보다 EffectValue, Price, MaxStack처럼 역할이 드러나는 이름을 사용하는 것이 좋습니다.

4. 아이템 데이터 예시

ItemId	ItemName	ItemType	Description	MaxStack	EffectType	EffectValue	Price	IsUsable
1001	체력 포션	Consumable	HP를 50 회복합니다	99	HealHp	50	100	true
1002	대형 체력 포션	Consumable	HP를 150 회복합니다	99	HealHp	150	250	true
2001	작은 수정 조각	Material	강화에 사용하는 기본 재료입니다	999	None	0	0	false
3001	스테이지 열쇠	Key	특정 스테이지 입장에 필요합니다	1	None	0	0	false

테이블에는 최소한 게임에서 실제로 사용하는 아이템만 넣는 것이 좋습니다. 나중에 사용할지도 모르는 아이템을 너무 많이 작성하면 관리 비용만 늘어납니다.

5. ItemId 규칙

- 소비 아이템 ID 범위:
- 장비 아이템 ID 범위:

- 재료 아이템 ID 범위:
- 퀘스트 아이템 ID 범위:
- 중복 허용 여부:
- 삭제된 ID 재사용 여부:

예시:

- 1000번대: 소비 아이템
- 2000번대: 재료 아이템
- 3000번대: 열쇠 및 특수 아이템
- 4000번대: 장비 아이템

ItemId는 중복될 수 없습니다. 한 번 사용한 ItemId는 아이템을 삭제하더라도 가능한 한 재사용하지 않는 것이 좋습니다. 저장 데이터나 보상 테이블에서 기존 ID를 참조할 수 있기 때문입니다.

6. ItemType 정의

ItemType	설명	사용 가능 여부	대표 예시
Consumable	사용 시 효과가 발생하고 수량이 감소하는 아이템	가능	체력 포션
Material	제작, 강화 등에 사용하는 재료	불가능	수정 조각
Equipment	장착해 능력치를 변경하는 아이템	가능	검, 방어구
Key	특정 콘텐츠 해금에 사용하는 아이템	조건부	스테이지 열쇠
Currency	재화로 사용하는 아이템	불가능	골드, 보석

아이템 타입은 인벤토리 UI, 사용 가능 여부, 정렬 기준, 상점 판매 여부에 영향을 줍니다. 따라서 문자열을 임의로 사용하기보다 enum으로 관리하는 것이 좋습니다.

7. EffectType 정의

EffectType	설명	EffectValue 의미
None	효과 없음	0
HealHp	HP 회복	회복량
IncreaseAttack	공격력 증가	증가량
IncreaseDefense	방어력 증가	증가량
RestoreStamina	스태미나 회복	회복량
UnlockStage	스테이지 해금	StageId

예시:

체력 포션의 EffectType은 HealHp이고, EffectValue는 회복량입니다. EffectValue가 50이면 사용 시 HP를 50 회복합니다. 효과가 없는 재료 아이템은 EffectType을 None으로 설정합니다.

8. 중첩 수량 규칙

- MaxStack이 1인 아이템:
- MaxStack이 99인 아이템:
- MaxStack이 999인 아이템:
- 수량이 0이 되었을 때 처리:
- 최대 수량을 초과해 획득했을 때 처리:

예시:

소비 아이템은 최대 99개까지 중첩할 수 있습니다. 재료 아이템은 최대 999개까지 중첩할 수 있습니다. 장비 아이템은 기본적으로 중첩하지 않으며 MaxStack을 1로 설정합니다. 아이템 사용 후 수량이 0이 되면 인벤토리 목록에서 제거합니다.

9. 상점 관련 필드

- 구매 가능 여부:
- 구매 가격:
- 판매 가능 여부:
- 판매 가격:
- 구매 제한 수량:
- 해금 조건:
- 표시 순서:

추가 필드 예시:

필드명	타입	설명
CanBuy	bool	상점에서 구매 가능한지
CanSell	bool	상점에 판매 가능한지
UnlockStageId	int	특정 스테이지 클리어 후 해금
DisplayOrder	int	상점 표시 순서

상점에서 사용하는 아이템은 아이템 데이터와 별도 상점 데이터로 분리할 수도 있습니다. 아이템의 기본 정보는 ItemData가 담당하고, 상점 가격이나 해금 조건은 ShopItemData에서 관리하는 방식도 좋습니다.

10. 인벤토리 표시 규칙

- 아이템 이름 표시:
- 아이콘 표시:
- 수량 표시:
- 설명 표시:
- 사용 버튼 표시 여부:
- 판매 버튼 표시 여부:
- 정렬 기준:
- 필터 기준:

예시:

인벤토리 화면에서는 아이템 아이콘, 이름, 보유 수량을 표시합니다. 소비 아이템은 사용 버튼을 표시하고, 재료 아이템은 사용 버튼을 표시하지 않습니다. 아이템은 ItemType 순서와 DisplayOrder를 기준으로 정렬합니다.

11. 보상 시스템과 연결

- 적 처치 보상에서 사용하는 아이템:
- 스테이지 클리어 보상에서 사용하는 아이템:
- 드랍 확률 적용 여부:
- 보상 지급 실패 조건:
- 인벤토리 공간 부족 처리:

예시:

보상 시스템은 RewardData에서 ItemId를 참조해 아이템을 지급합니다. 인벤토리에 공간이 부족하면 아이템 지급은 실패 처리하고, 골드와 경험치는 정상 지급합니다. 존재하지 않는 ItemId가 보상 데이터에 들어 있으면 오류 로그를 출력하고 아이템 지급을 중단합니다.

12. 데이터 검증 규칙

- ItemId는 중복될 수 없다.
- ItemName은 비어 있으면 안 된다.
- MaxStack은 1 이상이어야 한다.
- Price는 0 이상이어야 한다.
- EffectType이 None이면 EffectValue는 0이어야 한다.
- IsUsable이 false이면 사용 버튼을 표시하지 않는다.
- ItemType과 EffectType의 조합이 자연스러운지 확인한다.

- IconKey가 비어 있으면 기본 아이콘을 사용한다.

데이터 검증 규칙은 개발 중 오류를 줄이는 데 도움이 됩니다. AI에게 데이터 로더나 검증 도구를 요청할 때도 이 규칙을 함께 전달하면 좋습니다.

13. AI 구현 요청용 프롬프트

예시:

Unity 6 C#에서 사용할 ItemData 구조를 설계해줘. ItemData는 ScriptableObject로 관리하고, ItemId, ItemName, ItemType, Description, IconKey, MaxStack, EffectType, EffectValue, Price, SellPrice, IsUsable, IsSellable 필드를 가진다. ItemType은 Consumable, Material, Equipment, Key, Currency로 구분하고, EffectType은 None, HealHp, IncreaseAttack, IncreaseDefense, RestoreStamina로 구분한다. ItemId는 중복될 수 없고, MaxStack은 1 이상이어야 한다. 인벤토리 UI와 보상 시스템에서 ItemId를 기준으로 아이템을 조회할 수 있도록 설계해줘.

14. 테스트 기준

- ItemId가 중복되면 오류를 감지하는가?
- MaxStack이 0 이하인 아이템을 막는가?
- 소비 아이템 사용 시 수량이 감소하는가?
- 수량이 0이 되면 인벤토리에서 제거되는가?
- IsUsable이 false인 아이템은 사용할 수 없는가?
- 존재하지 않는 ItemId를 참조하면 오류를 처리하는가?
- 상점 가격이 음수인 데이터를 막는가?
- 보상 시스템이 ItemId를 기준으로 아이템을 지급하는가?
- 인벤토리 공간 부족 시 아이템 지급 실패가 처리되는가?

15. 검토 체크리스트

- 아이템 타입이 명확히 구분되어 있는가?
- ItemId 규칙이 정리되어 있는가?
- 필드명과 타입이 구현에 적합한가?
- 효과 타입과 효과값의 의미가 명확한가?
- 중첩 수량 규칙이 작성되어 있는가?
- 상점, 인벤토리, 보상 시스템과 연결되어 있는가?
- 데이터 검증 규칙이 포함되어 있는가?
- AI 구현 요청 프롬프트로 변환할 수 있는가?

- MVP에 필요한 아이템만 포함되어 있는가?
- 나중에 확장할 아이템과 현재 구현할 아이템이 구분되어 있는가?

아이템 데이터 테이블은 아이템 목록을 단순히 정리하는 표가 아닙니다. 인벤토리, 상점, 보상, 성장 시스템이 같은 데이터를 기준으로 동작하도록 만드는 공통 기준입니다. 좋은 아이템 데이터 테이블은 아이템의 이름과 설명뿐 아니라, 코드에서 사용할 ID, 타입, 효과, 수량, 가격, 예외 조건까지 명확히 정의하는 개발 문서입니다.

부록 G. UI 화면 기획서 템플릿

UI 화면 기획서는 플레이어가 게임 정보를 어떻게 확인하고, 어떤 버튼을 눌러 어떤 기능을 실행하는지 정리하는 문서입니다. UI는 단순한 화면 장식이 아니라 게임 시스템과 플레이어를 연결하는 인터페이스입니다. 따라서 화면 이름, 표시 데이터, 버튼 동작, 화면 전환, 예외 조건을 함께 작성해야 합니다. 이 템플릿은 UI 화면을 구현 가능한 기준으로 정리하기 위한 기본 양식입니다.

1. UI 기획 기본 정보

- 게임 제목:
- 장르:
- 플랫폼:
- 개발 엔진:
- UI 방식:
- 입력 방식:
- 해상도 기준:
- UI 기획 목적:

예시:

- 게임 제목: 스테이지 액션 RPG 실습 프로젝트
- 장르: 스테이지형 액션 RPG
- 플랫폼: PC
- 개발 엔진: Unity 6
- UI 방식: UGUI
- 입력 방식: 키보드 + 마우스
- 해상도 기준: 1920x1080
- UI 기획 목적: 로비, 인게임 HUD, 인벤토리, 상점, 결과 화면의 표시 정보와 버튼 동작 정의

2. UI 전체 화면 목록

게임에 필요한 화면을 정리합니다.

화면 이름	목적	진입 조건	종료 조건
TitleUI	게임 시작	앱 실행	시작 버튼 클릭
LobbyUI	캐릭터 상태 확인과 스테이지 선택	타이틀 시작, 스테이지 종료	스테이지 시작
InGameHUD	전투 중 상태 표시	스테이지 시작	클리어 또는 실패

InventoryUI	보유 아이템 확인과 사용	로비 또는 인게임에서 열기	닫기 버튼
ShopUI	아이템 구매	로비에서 상점 버튼 클릭	닫기 버튼
ResultUI	스테이지 결과 확인	클리어 또는 실패	확인 버튼

화면 목록은 UI 구현 범위를 정하는 기준입니다. 처음부터 모든 화면을 만들기보다 핵심 플레이 루프에 필요한 화면부터 작성하는 것이 좋습니다.

3. 화면 전환 흐름

UI 화면이 어떤 순서로 열리고 닫히는지 작성합니다.

기본 흐름:

타이틀 화면 → 로비 화면 → 스테이지 선택 → 인게임 HUD → 결과 화면 → 로비 화면

상세 설명:

플레이어가 타이틀 화면에서 시작 버튼을 누르면 로비 화면으로 이동합니다. 로비에서 스테이지 시작 버튼을 누르면 인게임 HUD가 표시되고 전투가 시작됩니다. 스테이지가 클리어되거나 실패하면 결과 화면을 표시합니다. 결과 화면에서 확인 버튼을 누르면 다시 로비 화면으로 돌아갑니다.

4. 타이틀 화면 기획

- 화면 목적:
- 표시 요소:
- 버튼 목록:
- 버튼 동작:
- 표시 데이터:
- 진입 조건:
- 종료 조건:
- 예외 조건:

예시:

타이틀 화면은 게임 시작 전 가장 먼저 표시되는 화면입니다. 게임 제목, 시작 버튼, 설정 버튼, 종료 버튼을 표시합니다. 시작 버튼을 누르면 저장 데이터를 불러온 뒤 로비 화면으로 이동합니다. 저장 데이터가 없으면 기본 데이터를 생성합니다.

5. 로비 화면 기획

- 화면 목적:
- 표시 요소:
- 캐릭터 정보:

- 재화 정보:
- 버튼 목록:
- 버튼 동작:
- 갱신 조건:
- 연결 시스템:

예시:

로비 화면은 플레이어가 스테이지에 입장하기 전 캐릭터 상태를 확인하고 정비하는 화면입니다. 플레이어 레벨, 현재 경험치, 보유 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼을 표시합니다. 골드가 변경되면 골드 텍스트를 갱신하고, 레벨업이 발생하면 레벨과 경험치 바를 갱신합니다.

6. 인게임 HUD 기획

- 화면 목적:
- 표시 요소:
- HP 표시:
- 경험치 표시:
- 골드 표시:
- 아이템 수량 표시:
- 쿨타임 표시:
- 버튼 목록:
- 갱신 조건:

예시:

인게임 HUD는 전투 중 필요한 정보를 표시합니다. HP 바, 경험치 바, 골드, 포션 수량, 대시 쿨타임, 일시정지 버튼을 표시합니다. 플레이어가 데미지를 받으면 HP 바를 갱신하고, 포션을 사용하면 포션 수량을 갱신합니다. 스테이지가 종료되면 HUD를 숨기고 결과 화면을 표시합니다.

7. 인벤토리 화면 기획

- 화면 목적:
- 아이템 목록 표시 방식:
- 표시 데이터:
- 필터 기능:
- 정렬 기준:
- 사용 버튼:
- 판매 버튼:

- 닫기 버튼:
- 예외 조건:

예시:

인벤토리 화면은 플레이어가 보유한 아이템을 확인하고 사용할 수 있는 화면입니다. 아이템 아이콘, 이름, 보유 수량, 설명을 표시합니다. 소비 아이템을 선택하면 사용 버튼을 표시하고, 사용 시 아이템 효과를 적용한 뒤 수량을 감소시킵니다. 수량이 0이 되면 목록에서 제거합니다.

8. 상점 화면 기획

- 화면 목적:
- 상품 목록 표시 방식:
- 표시 데이터:
- 구매 버튼:
- 보유 골드 표시:
- 구매 성공 처리:
- 구매 실패 처리:
- 닫기 버튼:
- 예외 조건:

예시:

상점 화면은 플레이어가 골드를 사용해 아이템을 구매하는 화면입니다. 상품 목록에는 아이템 이름, 아이콘, 가격, 설명을 표시합니다. 구매 버튼을 누르면 보유 골드와 가격을 비교합니다. 골드가 충분하고 인벤토리에 공간이 있으면 골드를 차감하고 아이템을 추가합니다. 골드가 부족하면 구매 실패 메시지를 표시합니다.

9. 결과 화면 기획

- 화면 목적:
- 클리어 여부 표시:
- 획득 경험치:
- 획득 골드:
- 획득 아이템:
- 레벨업 여부:
- 확인 버튼:
- 로비 복귀 조건:
- 예외 조건:

예시:

결과 화면은 스테이지 종료 후 플레이어가 얻은 결과를 확인하는 화면입니다. 클리어 또는 실패 여부, 획득 경험치, 골드, 아이템 목록을 표시합니다. 확인 버튼을 누르면 로비 화면으로 돌아갑니다. 보상은 결과 화면이 열릴 때 한 번만 지급되어야 하며, 화면을 다시 열어도 중복 지급되지 않아야 합니다.

10. 팝업과 토스트 메시지

- 확인 팝업:
- 경고 팝업:
- 보상 팝업:
- 토스트 메시지:
- 표시 위치:
- 표시 시간:
- 중복 표시 처리:

예시:

골드가 부족한 상태에서 구매 버튼을 누르면 “골드가 부족합니다”라는 토스트 메시지를 표시합니다. 스테이지 포기 버튼을 누르면 확인 팝업을 표시하고, 플레이어가 확인을 누른 경우에만 로비로 돌아갑니다. 같은 토스트 메시지가 짧은 시간 안에 반복 호출되면 기존 메시지를 갱신하거나 무시합니다.

11. UI 데이터 연결

UI 요소와 실제 데이터를 연결합니다.

UI 요소	연결 데이터	갱신 조건
HP Bar	Player.CurrentHp / Player.MaxHp	데미지, 회복 발생
Exp Bar	Player.CurrentExp / RequiredExp	경험치 획득
Gold Text	Player.Gold	골드 획득, 구매
Potion Count	InventoryItem.Count	포션 사용, 획득
Stage Name Text	StageData.StageName	스테이지 선택
Reward List	RewardResultData	스테이지 종료

UI 데이터 연결이 명확해야 AI에게 구현을 요청할 때 임의의 변수를 만들지 않게 할 수 있습니다.

12. 버튼 이벤트 정의

버튼 이름	위치 화면	동작	호출 시스템
StartButton	TitleUI	로비 화면으로 이동	GameFlowManager

StageStartButton	LobbyUI	선택한 스테이지 시작	StageManager
InventoryButton	LobbyUI	인벤토리 화면 열기	InventoryUI
ShopButton	LobbyUI	상점 화면 열기	ShopUI
BuyButton	ShopUI	선택 상품 구매	ShopManager
UseItemButton	InventoryUI	선택 아이템 사용	InventoryManager
ResultConfirmButton	ResultUI	로비로 복귀	GameFlowManager

버튼 이벤트는 UI와 시스템을 연결하는 중요한 기준입니다. 버튼 이름, 위치, 동작, 호출 시스템을 명확히 작성해야 구현 실수를 줄일 수 있습니다.

13. 화면 상태 규칙

- 화면이 열릴 때:
- 화면이 닫힐 때:
- 입력 차단 여부:
- 게임 일시정지 여부:
- 다른 화면과 동시 표시 가능 여부:
- 화면 우선순위:

예시:

상점 화면이나 인벤토리 화면이 열려 있는 동안에는 플레이어 이동과 공격 입력을 차단합니다. 결과 화면이 열리면 인게임 HUD를 숨깁니다. 팝업이 열린 상태에서는 배경 UI 버튼 입력을 막습니다.

14. UI 예외 조건

- 보유 골드 데이터가 없을 때:
- 아이템 데이터가 존재하지 않을 때:
- 버튼을 빠르게 여러 번 누른 경우:
- 결과 화면이 중복으로 열린 경우:
- 보상이 중복 지급될 가능성이 있는 경우:
- UI가 열린 상태에서 스테이지가 종료된 경우:
- 인벤토리 아이템 수량이 0이 된 경우:
- 화면 전환 중 다른 버튼을 누른 경우:

예시:

구매 버튼을 빠르게 여러 번 눌러도 구매 처리는 한 번씩 순차적으로 실행되어야 합니다. 결과 화면은 스테이지 종료 시 한 번만 열려야 하며, 보상 지급도 한 번만 처리되어야 합니다. 화면 전환 중에는 주요 버튼 입력을 잠시 차단합니다.

15. UI 프리팹 목록

- TitleUI.prefab:
- LobbyUI.prefab:
- InGameHUD.prefab:
- InventoryUI.prefab:
- InventorySlot.prefab:
- ShopUI.prefab:
- ShopItemSlot.prefab:
- ResultUI.prefab:
- ToastMessage.prefab:
- ConfirmPopup.prefab:

예시:

인벤토리 화면은 InventoryUI.prefab으로 구성하고, 아이템 한 칸은 InventorySlot.prefab을 사용합니다. 상점 상품 한 칸은 ShopItemSlot.prefab으로 생성합니다. 반복되는 UI 요소는 프리팹으로 분리해 재사용합니다.

16. UI 스크립트 목록

- TitleUI:
- LobbyUI:
- InGameHUD:
- InventoryUI:
- InventorySlotUI:
- ShopUI:
- ShopItemSlotUI:
- ResultUI:
- ToastUI:
- ConfirmPopupUI:
- UIManager:

예시:

InGameHUD는 HP, 경험치, 골드, 포션 수량 표시를 담당합니다. InventoryUI는 인벤토리 목록을 갱신하고, InventorySlotUI는 개별 아이템 표시를 담당합니다. UIManager는 화면 열기와 닫기, 팝업 표시를 관리합니다.

17. AI 구현 요청용 프롬프트

예시:

Unity 6 UGUI로 스테이지형 액션 RPG의 기본 UI를 구현해줘. 필요한 화면은 LobbyUI, InGameHUD, InventoryUI, ShopUI, ResultUI이다. LobbyUI에는 레벨 텍스트, 골드 텍스트, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼이 있다. InGameHUD에는 HP 바, 경험치 바, 포션 수량, 골드 텍스트를 표시한다. ShopUI의 구매 버튼은 ShopManager.PurchaseItem을 호출하고, InventoryUI의 사용 버튼은 InventoryManager.UseItem을 호출한다. 결과 화면은 스테이지 종료 시 한 번만 열리고, 보상 중복 지급이 발생하지 않도록 처리해줘.

18. 테스트 기준

- 타이틀에서 로비 화면으로 정상 전환되는가?
- 로비에서 스테이지 시작 버튼이 동작하는가?
- HP 변경 시 HP 바가 갱신되는가?
- 골드 변경 시 골드 텍스트가 갱신되는가?
- 인벤토리 아이템 수량이 UI에 반영되는가?
- 상점 구매 성공 시 골드와 아이템 UI가 갱신되는가?
- 구매 실패 시 토스트 메시지가 표시되는가?
- 결과 화면이 중복으로 열리지 않는가?
- 팝업이 열린 상태에서 배경 버튼 입력이 막히는가?
- 화면 전환 중 잘못된 입력이 처리되지 않는가?

19. 검토 체크리스트

- 화면 목록이 핵심 플레이 루프와 연결되어 있는가?
- 각 화면의 목적이 명확한가?
- UI 요소와 표시 데이터가 정리되어 있는가?
- 버튼 이벤트와 호출 시스템이 작성되어 있는가?
- 화면 전환 흐름이 명확한가?
- 팝업과 토스트 메시지 규칙이 있는가?
- 화면 상태와 입력 차단 규칙이 있는가?
- UI 프리팹과 스크립트 목록이 정리되어 있는가?

- 예외 조건이 충분히 작성되어 있는가?
- AI 구현 요청 프롬프트로 변환할 수 있는가?

UI 화면 기획서는 화면을 예쁘게 꾸미기 위한 문서가 아닙니다. 어떤 정보를 보여주고, 어떤 버튼이 어떤 시스템을 호출하며, 화면이 어떤 조건에서 열리고 닫히는지 정의하는 문서입니다. 좋은 UI 기획서는 플레이어의 입력과 게임 데이터를 화면 흐름으로 연결하는 개발 기준 문서입니다.

부록 H. 레벨 디자인 문서 템플릿

레벨 디자인 문서는 플레이어가 어떤 공간에서, 어떤 목표를 가지고, 어떤 순서로 플레이를 진행하는지 정리하는 문서입니다. 레벨은 단순한 맵이 아니라 게임 규칙, 적 배치, 난이도, 보상, 실패 조건이 함께 작동하는 플레이 단위입니다. 특히 스테이지형 게임에서는 각 레벨의 목표와 진행 흐름이 명확해야 전체 플레이 루프가 안정적으로 완성됩니다. 이 템플릿은 레벨과 스테이지를 구현 가능한 기준으로 정리하기 위한 기본 양식입니다.

1. 레벨 기본 정보

- 레벨 ID:
- 레벨 이름:
- 레벨 유형:
- 등장 위치:
- 예상 플레이 시간:
- 권장 플레이어 레벨:
- 난이도:
- 클리어 조건:
- 실패 조건:
- 보상 ID:

예시:

- 레벨 ID: Stage_001
- 레벨 이름: 숲의 입구
- 레벨 유형: 일반 전투 스테이지
- 예상 플레이 시간: 3분
- 권장 플레이어 레벨: 1
- 난이도: 쉬움
- 클리어 조건: 모든 적 처치
- 실패 조건: 플레이어 HP 0 이하
- 보상 ID: Reward_Stage_001

2. 레벨 콘셉트

레벨의 목적과 분위기를 한 문장으로 작성합니다.

작성 예시:

이 레벨은 플레이어가 기본 이동과 공격을 익히고, 약한 적을 처치하며 전투와 보상 흐름을 경험하는 첫 번째 전투 스테이지입니다.

작성 기준:

- 이 레벨에서 플레이어가 무엇을 배우는가?
- 어떤 행동을 반복하게 되는가?
- 이전 레벨과 어떤 차이가 있는가?
- 다음 레벨로 이어지는 역할이 있는가?
- 난이도가 현재 플레이어 성장 상태와 맞는가?

3. 레벨 진행 흐름

레벨 시작부터 종료까지의 흐름을 작성합니다.

기본 흐름:

스테이지 입장 → 시작 연출 또는 안내 → 적 배치 확인 → 전투 진행 → 모든 적 처치 → 클리어 판정 → 보상 지급 → 결과 화면 → 로비 복귀

상세 설명:

플레이어가 로비에서 스테이지 시작 버튼을 누르면 해당 레벨로 이동합니다. 레벨이 시작되면 플레이어는 시작 위치에 배치되고, 적은 지정된 위치에 스폰됩니다. 플레이어가 모든 적을 처치하면 클리어 조건이 충족되고, 보상 화면을 표시한 뒤 로비로 돌아갑니다.

4. 레벨 목표

- 주요 목표:
- 보조 목표:
- 튜토리얼 목표:
- 숨겨진 목표:
- 실패 조건:

예시:

주요 목표는 스테이지 안에 등장하는 모든 적을 처치하는 것입니다. 보조 목표는 포션을 사용하지 않고 클리어하는 것입니다. MVP 단계에서는 보조 목표와 숨겨진 목표는 제외하고, 모든 적 처치만 클리어 조건으로 사용합니다.

5. 플레이어 시작 조건

- 시작 위치:
- 시작 방향:
- 시작 HP:

- 시작 아이템:
- 시작 카메라 위치:
- 시작 UI 상태:
- 시작 연출 여부:

예시:

플레이어는 맵의 하단 중앙에서 시작합니다. 시작 시 HP는 현재 저장된 플레이어 HP를 기준으로 하며, 인게임 HUD를 표시합니다. 시작 연출은 MVP 단계에서는 생략하고, 1초 후 적 AI가 활성화되도록 설정합니다.

6. 맵 구조

- 맵 크기:
- 주요 이동 경로:
- 막힌 공간:
- 안전 구역:
- 전투 구역:
- 보상 구역:
- 출구 또는 포털 위치:
- 카메라 제한 영역:

예시:

맵은 작은 직사각형 전투 공간으로 구성합니다. 플레이어는 맵 전체를 자유롭게 이동할 수 있지만, 외곽에는 이동할 수 없는 벽을 배치합니다. 적은 맵 중앙과 상단에 배치하고, 클리어 후에는 결과 화면으로 바로 전환합니다. MVP 단계에서는 별도의 출구 오브젝트를 사용하지 않습니다.

7. 적 배치

적 ID	적 이름	수량	스폰 위치	등장 조건	비고
Enemy_001	슬라임	3	맵 중앙	스테이지 시작 시	기본 적
Enemy_002	고블린	1	맵 상단	슬라임 2마리 처치 후	약간 강한 적

적 배치는 난이도와 플레이 흐름에 직접 영향을 줍니다. 한 번에 너무 많은 적을 배치하면 초반 난이도가 급격히 올라갈 수 있습니다. 반대로 적 수가 너무 적으면 전투 루프가 짧게 끝나 보상과 성장의 의미가 약해질 수 있습니다.

8. 적 스폰 규칙

- 스폰 방식:

- 스폰 타이밍:
- 웨이브 수:
- 웨이브 간격:
- 최대 동시 등장 수:
- 재스폰 여부:
- 스폰 위치 고정 여부:
- 스폰 연출:

예시:

Stage_001에서는 스테이지 시작 시 슬라임 3마리를 즉시 스폰합니다. 모든 적은 고정 위치에 등장하며, 재스폰은 없습니다. Stage_002부터는 웨이브 방식으로 적을 등장시킬 수 있습니다.

9. 난이도 설계

- 권장 플레이어 레벨:
- 적 평균 HP:
- 적 평균 공격력:
- 적 수:
- 보스 여부:
- 예상 클리어 시간:
- 포션 사용 예상 횟수:
- 난이도 조절 요소:

예시:

Stage_001은 플레이어가 기본 공격을 2~3회 사용하면 적을 처치할 수 있도록 설계합니다. 적의 공격력은 낮게 설정하고, 플레이어가 실수해도 바로 사망하지 않도록 합니다. 이 레벨의 목적은 도전보다 기본 전투 흐름을 학습하는 것입니다.

10. 오브젝트 배치

오브젝트 ID	오브젝트 이름	위치	역할	상호작용 여부
Obj_001	나무 장애물	좌측 상단	이동 경로 제한	불가능
Obj_002	회복 상자	우측 하단	포션 제공	가능
Obj_003	출구 포털	맵 상단	클리어 후 이동	조건부

오브젝트는 플레이어의 이동 경로와 전투 판단에 영향을 줍니다. MVP 단계에서는 장애물과 회복 상자를 최소한으로 사용하고, 레벨 구조가 복잡해지지 않도록 합니다.

11. 보상 설계

- 적 처치 보상:
- 스테이지 클리어 보상:
- 보너스 보상:
- 아이템 드랍:
- 경험치:
- 골드:
- 보상 지급 조건:
- 보상 중복 방지 조건:

예시:

일반 적 처치 시 EnemyData에 정의된 경험치와 골드를 지급합니다. 스테이지 클리어 시 RewardData에 정의된 추가 골드를 지급합니다. 스테이지 클리어 보상은 한 번만 지급되어야 하며, 결과 화면을 다시 열어도 중복 지급되지 않아야 합니다.

12. 실패 처리

- 실패 조건:
- 실패 시 화면:
- 실패 시 보상:
- 실패 시 저장 여부:
- 로비 복귀 여부:
- 재도전 방식:

예시:

플레이어 HP가 0 이하가 되면 스테이지 실패로 처리합니다. 실패 시 스테이지 클리어 보상은 지급하지 않습니다. 이미 처치한 적의 보상을 유지할지 여부는 프로젝트 정책에 따라 결정합니다. 실패 화면에서 확인 버튼을 누르면 로비로 돌아갑니다.

13. 튜토리얼 요소

- 이동 안내:
- 공격 안내:
- 회피 안내:
- 아이템 사용 안내:
- 목표 안내:
- 안내 표시 조건:

- 안내 종료 조건:

예시:

첫 번째 스테이지에서는 시작 직후 “WASD로 이동하세요” 메시지를 표시합니다. 플레이어가 일정 거리 이상 이동하면 안내를 숨깁니다. 첫 번째 적과 마주치면 “마우스 좌클릭으로 공격하세요” 메시지를 표시합니다. MVP 단계에서는 튜토리얼 메시지를 토스트 형태로 간단히 표시할 수 있습니다.

14. UI 연결

- 스테이지 이름 표시:
- 목표 표시:
- 남은 적 수 표시:
- HP 표시:
- 보상 결과 표시:
- 실패 메시지:
- 클리어 메시지:

예시:

인게임 HUD에는 현재 HP와 남은 적 수를 표시합니다. 모든 적을 처치하면 “스테이지 클리어” 메시지를 표시하고 결과 화면으로 전환합니다. 플레이어가 사망하면 “스테이지 실패” 메시지를 표시합니다.

15. 레벨 데이터 테이블

필드명	타입	설명	예시
StageId	string/int	스테이지 고유 ID	Stage_001
StageName	string	스테이지 이름	숲의 입구
RequiredLevel	int	권장 또는 입장 가능 레벨	1
Difficulty	int/string	난이도	Easy
EnemyGroupId	string/int	등장 적 그룹 ID	EnemyGroup_001
RewardId	string/int	클리어 보상 ID	Reward_Stage_001
ClearCondition	enum/string	클리어 조건	KillAllEnemies
FailCondition	enum/string	실패 조건	PlayerDead
TimeLimit	float	제한 시간	0

레벨 데이터는 스테이지 시스템, 적 스폰 시스템, 보상 시스템, UI 결과 화면과 연결됩니다. 따라서 필드 이름과 의미를 명확히 작성해야 합니다.

16. 예외 조건

- 잘못된 Stageld로 입장한 경우:
- EnemyGroupId가 존재하지 않는 경우:
- RewardId가 존재하지 않는 경우:
- 모든 적 처치와 플레이어 사망이 동시에 발생한 경우:
- 결과 화면이 중복으로 열린 경우:
- 보상이 중복 지급되는 경우:
- 스테이지 로딩 중 입력이 들어온 경우:
- 스테이지 종료 후 적 AI가 계속 동작하는 경우:
- 플레이어가 맵 밖으로 나간 경우:

예시:

모든 적 처치와 플레이어 사망이 동시에 발생한 경우에는 이벤트 발생 순서를 기준으로 결과를 결정합니다. 결과 화면은 한 번만 열려야 하며, 스테이지 종료 후에는 적 AI와 전투 입력을 모두 중지합니다.

17. AI 구현 요청용 프롬프트

예시:

Unity 6 C#에서 사용할 스테이지 시스템을 구현해줘. StageData에는 Stageld, StageName, RequiredLevel, EnemyGroupId, RewardId, ClearCondition, FailCondition을 포함한다. 로비에서 스테이지를 선택하면 해당 StageData를 불러오고, EnemyGroupId에 정의된 적들을 지정된 위치에 스폰한다. ClearCondition이 KillAllEnemies인 경우 모든 적이 Dead 상태가 되면 스테이지 클리어로 처리한다. 플레이어가 Dead 상태가 되면 실패 처리한다. 클리어 시 RewardId를 기준으로 보상을 한 번만 지급하고 결과 화면을 표시한다. 결과 화면이 중복으로 열리지 않도록 상태를 관리해줘.

18. 테스트 기준

- 올바른 Stageld로 스테이지에 입장되는가?
- 잘못된 Stageld 입력 시 오류가 처리되는가?
- StageData 기준으로 적이 스폰되는가?
- 모든 적 처치 시 클리어 처리되는가?
- 플레이어 사망 시 실패 처리되는가?
- 클리어 보상이 한 번만 지급되는가?
- 결과 화면이 중복으로 열리지 않는가?
- 실패 시 클리어 보상이 지급되지 않는가?

- 스테이지 종료 후 적 AI가 중지되는가?
- 로비로 정상 복귀되는가?

19. 검토 체크리스트

- 레벨 ID와 이름이 명확한가?
- 레벨의 목적이 한 문장으로 정리되어 있는가?
- 클리어 조건과 실패 조건이 작성되어 있는가?
- 플레이어 시작 조건이 정리되어 있는가?
- 적 배치와 스폰 규칙이 명확한가?
- 난이도 기준이 작성되어 있는가?
- 보상 지급 조건과 중복 방지 조건이 있는가?
- UI 표시 정보가 정리되어 있는가?
- 레벨 데이터 테이블 필드가 구현에 적합한가?
- 예외 조건이 충분히 작성되어 있는가?
- AI 구현 요청 프롬프트로 변환할 수 있는가?

레벨 디자인 문서는 맵의 모양만 설명하는 문서가 아닙니다. 플레이어가 어떤 목표로 시작하고, 어떤 적과 마주치며, 어떤 조건으로 성공하거나 실패하고, 어떤 보상을 얻는지 정의하는 문서입니다. 좋은 레벨 디자인 문서는 공간, 규칙, 난이도, 보상, UI 흐름을 하나의 플레이 경험으로 연결하는 개발 기준 문서입니다.

부록 I. AI 개발 요청 프롬프트 템플릿

AI 개발 요청 프롬프트는 기획서의 내용을 실제 구현 작업으로 바꾸기 위한 문서입니다. 좋은 프롬프트는 “기능을 만들어줘”라는 짧은 명령이 아니라, 개발 환경, 구현 목적, 기존 구조, 데이터, 처리 흐름, 예외 조건, 출력 형식을 함께 전달하는 작업 지시서입니다. AI는 사용자가 명확하게 설명한 만큼 더 정확한 코드를 제안할 수 있습니다. 이 템플릿은 게임 상세 기획서를 AI에게 전달할 수 있는 구현 요청 형태로 바꾸기 위한 기본 양식입니다.

1. 프롬프트 기본 정보

- 요청 기능 이름:
- 관련 기획서:
- 관련 데이터 테이블:
- 개발 엔진:
- 개발 언어:
- 사용 프레임워크:
- 요청 목적:
- 요청 날짜:
- 요청 상태:

예시:

- 요청 기능 이름: 상점 구매 시스템 구현
- 관련 기획서: docs/shop_system.md
- 관련 데이터 테이블: ShopItemData, ItemData
- 개발 엔진: Unity 6
- 개발 언어: C#
- 요청 목적: 골드를 사용해 아이템을 구매하는 기능 구현
- 요청 상태: 초안 / 수정 요청 / 완료 / 보류

2. 개발 환경 설명

AI가 코드를 작성할 때 필요한 환경 정보를 작성합니다.

- 엔진:
- 언어:
- 프로젝트 유형:
- UI 방식:

- 데이터 관리 방식:
- 입력 시스템:
- 네임스페이스:
- 코딩 스타일:

예시:

Unity 6 C# 프로젝트에서 사용할 기능입니다. UI는 UGUI를 사용하고, 아이템 데이터는 ScriptableObject로 관리합니다. 네임스페이스는 Game.Systems.Shop을 사용합니다. 가능한 한 기존 클래스 구조를 유지하고, public 메서드 이름은 변경하지 않습니다.

3. 구현 목적

이 기능이 왜 필요한지 한 문장으로 작성합니다.

작성 예시:

플레이어가 로비의 상점 화면에서 골드를 사용해 포션 아이템을 구매할 수 있도록 상점 구매 기능을 구현합니다.

작성 기준:

- 어떤 기능을 구현하는가?
- 어떤 화면이나 시스템에서 사용하는가?
- 플레이어에게 어떤 결과를 제공하는가?
- 핵심 플레이 루프와 어떻게 연결되는가?

4. 기존 프로젝트 구조

이미 존재하는 클래스, 데이터, 메서드가 있다면 작성합니다.

- 기존 클래스:
- 기존 메서드:
- 기존 데이터 구조:
- 기존 UI:
- 변경하면 안 되는 요소:
- 재사용해야 하는 요소:

예시:

현재 프로젝트에는 InventoryManager, PlayerCurrencyManager, ItemData, ShopItemData, ShopUI가 존재합니다. InventoryManager.AddItem(int itemId, int count) 메서드를 사용해 아이템을 추가합니다. PlayerCurrencyManager는 플레이어의 골드를 관리합니다. 기존 public 메서드 시그니처는 변경하지 않습니다.

5. 구현할 기능 목록

기능을 작은 단위로 나누어 작성합니다.

- 구매 가능 여부 확인:
- 골드 확인:
- 인벤토리 공간 확인:
- 골드 차감:
- 아이템 지급:
- UI 갱신:
- 실패 메시지 반환:
- 로그 출력:
- 테스트 코드 작성:

예시:

상점 구매 시스템은 상품 가격과 플레이어 골드를 비교해야 합니다. 골드가 충분하고 인벤토리에 공간이 있으면 골드를 차감하고 아이템을 지급합니다. 구매에 실패하면 실패 이유를 반환하고, 골드와 인벤토리 데이터는 변경하지 않습니다.

6. 입력 데이터

기능 실행에 필요한 입력값을 작성합니다.

입력값	타입	설명	예시
shopItemId	int/string	구매할 상점 상품 ID	1001
itemId	int/string	지급할 아이템 ID	1001
price	int	상품 가격	100
playerGold	int	플레이어 보유 골드	300
itemCount	int	구매 수량	1

입력 데이터가 명확해야 AI가 임의의 변수를 만들지 않습니다. 필요한 값이 기존 클래스에서 오는지, 매개변수로 받는지, 데이터 테이블에서 조회하는지 함께 작성하면 좋습니다.

7. 출력 결과

기능 실행 후 반환해야 할 결과를 작성합니다.

- 성공 여부:
- 실패 이유:
- 변경된 골드:

- 지급된 아이템:
- UI 갱신 여부:
- 로그 메시지:
- 이벤트 호출 여부:

예시:

구매 성공 시 true를 반환하고, 플레이어 골드를 차감한 뒤 인벤토리에 아이템을 추가합니다. 구매 실패 시 false를 반환하고 실패 이유 메시지를 제공합니다. 실패 시 골드와 인벤토리 데이터는 변경하지 않습니다.

8. 처리 흐름

기능이 실행되는 순서를 단계별로 작성합니다.

기본 흐름 예시:

구매 버튼을 누른다.

선택한 ShopItemData를 조회한다.

상품 가격과 플레이어 보유 골드를 비교한다.

인벤토리에 아이템을 추가할 수 있는지 확인한다.

조건이 모두 충족되면 골드를 차감한다.

인벤토리에 아이템을 추가한다.

상점 UI와 골드 UI를 갱신한다.

구매 성공 메시지를 표시한다.

처리 흐름은 코드의 메서드 순서와 거의 연결됩니다. AI에게 이 흐름을 전달하면 누락된 조건이나 순서 오류를 줄일 수 있습니다.

9. 예외 조건

정상 흐름 외에 반드시 처리해야 할 조건을 작성합니다.

- 상품 데이터가 존재하지 않는 경우:
- 아이템 데이터가 존재하지 않는 경우:
- 골드가 부족한 경우:
- 인벤토리 공간이 부족한 경우:
- 구매 수량이 0 이하인 경우:
- 구매 버튼을 빠르게 여러 번 누른 경우:
- 상점 화면이 닫힌 상태에서 구매 요청이 들어온 경우:
- 저장 데이터가 갱신되지 않은 경우:

예시:

골드가 부족하면 구매를 실패 처리하고, 골드와 인벤토리 데이터는 변경하지 않습니다. 인벤토리 공간이 부족한 경우에도 구매를 실패 처리합니다. 상품 데이터가 존재하지 않으면 오류 로그를 출력하고 false를 반환합니다.

10. 변경 제한 조건

AI가 임의로 변경하면 안 되는 조건을 작성합니다.

- 기존 클래스 이름 변경 금지:
- 기존 public 메서드 시그니처 변경 금지:
- 새로운 Manager 클래스 생성 금지:
- 기존 데이터 구조 유지:
- 특정 네임스페이스 사용:
- 특정 UI 방식 유지:
- 외부 패키지 추가 금지:

예시:

기존 InventoryManager, PlayerCurrencyManager, ShopUI 클래스 이름은 변경하지 않습니다. InventoryManager.AddItem 메서드를 재사용합니다. 새로운 전역 Manager 클래스를 만들지 말고, 현재 구조 안에서 필요한 메서드만 추가합니다.

11. 출력 형식 지정

AI가 어떤 형태로 답변해야 하는지 작성합니다.

- C# 코드만 출력:
- 설명과 코드 함께 출력:
- 변경된 파일 단위로 출력:
- 클래스별로 분리:
- 테스트 코드 포함:
- 사용 예시 포함:
- 주석 포함 여부:
- 기존 코드 수정 위치 표시:

예시:

변경이 필요한 C# 클래스 코드를 파일 단위로 작성해줘. 각 클래스 상단에 파일 경로를 주석으로 표시해줘. 코드 설명은 짧게만 작성하고, 컴파일 가능한 형태로 제공해줘. Unity Test Framework를 사용한 간단한 테스트 코드도 함께 작성해줘.

12. 테스트 요청

AI에게 함께 요청할 테스트 기준을 작성합니다.

- 정상 구매 테스트:
- 골드 부족 테스트:
- 인벤토리 부족 테스트:
- 잘못된 상품 ID 테스트:
- 구매 실패 시 데이터 미변경 테스트:
- 구매 성공 시 UI 갱신 테스트:

예시:

Unity Test Framework를 사용해 ShopManager.PurchaseItem 테스트를 작성해줘. 골드가 충분한 경우 true를 반환하고 골드가 차감되며 아이템이 추가되어야 합니다. 골드가 부족한 경우 false를 반환하고 골드와 인벤토리 데이터는 변경되지 않아야 합니다.

13. AI 개발 요청 프롬프트 완성 예시

예시 프롬프트:

Unity 6 C# 프로젝트에서 사용할 상점 구매 시스템을 구현해줘. UI는 UGUI를 사용하고, 아이템 데이터는 ScriptableObject인 ItemData, 상점 상품 데이터는 ShopItemData로 관리한다. 기존 클래스는 InventoryManager, PlayerCurrencyManager, ShopUI가 있으며, 기존 클래스 이름과 public 메서드 시그니처는 변경하지 않아줘.

구매 기능은 ShopManager.PurchaseItem(int shopItemId)로 구현한다. 이 메서드는 ShopItemData를 조회하고, 상품 가격과 플레이어 보유 골드를 비교한다. 골드가 충분하고 인벤토리에 공간이 있으면 골드를 차감하고 InventoryManager.AddItem(itemId, count)를 호출해 아이템을 추가한다. 구매가 성공하면 true를 반환하고, 상점 UI와 골드 UI를 갱신한다.

골드가 부족하거나 인벤토리 공간이 부족하면 false를 반환하고, 골드와 인벤토리 데이터는 변경하지 않는다. 상품 데이터나 아이템 데이터가 존재하지 않는 경우에는 오류 로그를 출력하고 false를 반환한다. 구매 버튼을 빠르게 여러 번 눌러도 데이터가 중복 변경되지 않도록 처리해줘.

변경이 필요한 C# 코드를 파일 단위로 작성해줘. 가능하면 ShopManager, ShopItemData, 필요한 결과 타입을 분리해서 작성해줘. Unity Test Framework를 사용해 정상 구매, 골드 부족, 인벤토리 부족, 잘못된 상품 ID 테스트도 함께 작성해줘.

14. 재요청 프롬프트 템플릿

AI 결과가 잘못되었을 때 사용할 수 있는 재요청 양식입니다.

기본 구조:

문제 상황 → 잘못된 동작 → 기대 동작 → 변경 제한 → 다시 요청할 내용

예시:

현재 작성된 코드는 골드가 부족한 경우에도 인벤토리에 아이템을 추가합니다. 기획서 기준으로 골드가 부족하면 구매를 실패 처리해야 하며, 골드와 인벤토리 데이터는 변경되지 않아야 합니다. 기존 ShopManager 구조와 public 메서드 시그니처는 유지하고, PurchaseItem 내부 로직만 수정해줘. 실패 이유를 반환할 수 있도록 결과 타입도 함께 정리해줘.

15. 프롬프트 수정 이력

- 날짜:
- 기능명:
- 최초 요청:
- 발견한 문제:
- 수정 요청:
- 최종 결과:
- 관련 커밋:
- 관련 기획서:

예시:

2026-06-21 / 상점 구매 시스템 / 최초 요청에서는 골드 부족 조건만 포함했으나, 구현 검토 중 인벤토리 공간 부족 조건이 누락된 것을 확인했습니다. 수정 요청에서 골드 부족과 인벤토리 공간 부족 모두 구매 실패 조건으로 추가했습니다. 관련 기획서는 docs/shop_system.md입니다.

16. 검토 체크리스트

- 개발 환경이 명확히 작성되어 있는가?
- 구현 목적이 한 문장으로 설명되어 있는가?
- 기존 프로젝트 구조가 포함되어 있는가?
- 구현할 기능이 작은 단위로 나누어져 있는가?
- 입력 데이터와 출력 결과가 작성되어 있는가?
- 처리 흐름이 순서대로 정리되어 있는가?
- 예외 조건이 충분히 포함되어 있는가?
- 변경하면 안 되는 조건이 작성되어 있는가?
- 출력 형식이 지정되어 있는가?
- 테스트 요청이 포함되어 있는가?
- 잘못된 결과를 다시 요청할 수 있는 재요청 문장이 있는가?
- 프롬프트 수정 이력을 남길 수 있는가?

AI 개발 요청 프롬프트는 AI에게 코드를 부탁하는 짧은 문장이 아닙니다. 기획서의 규칙을 개발자가 실행 가능한 조건으로 바꾸고, AI가 그 조건 안에서 코드를 만들도록 안내하는 작업 지시서입니다. 좋은 프롬프트는 AI의 능력을 끌어내는 문장이 아니라, 사용자의 생각과 기획 기준을 명확히 전달하는 개발 문서입니다.

부록 J. 기획서 검토 체크리스트

기획서 검토 체크리스트는 작성한 문서가 실제 개발 기준으로 사용할 수 있는지 확인하기 위한 도구입니다. 게임 콘셉트, 플레이 루프, 조작, 전투, UI, 데이터, 예외 조건이 서로 연결되어 있는지 점검해야 합니다. 특히 AI와 함께 개발할 때는 기획서의 모호한 문장이 그대로 잘못된 코드로 이어질 수 있습니다. 따라서 구현 전에 기획서를 한 번 검토하고, 구현 후에도 다시 수정하는 습관이 필요합니다.

1. 게임 콘셉트 검토

- 게임을 한 문장으로 설명할 수 있는가?
- 장르가 명확한가?
- 플레이어가 가장 자주 하는 행동이 드러나는가?
- 핵심 재미가 구체적으로 작성되어 있는가?
- 대상 플레이어가 명확한가?
- MVP 범위가 정리되어 있는가?
- 제외할 기능이 작성되어 있는가?
- 게임의 목표가 너무 크거나 모호하지 않은가?
- 콘셉트가 실제 구현 가능한 범위인가?
- AI에게 전달할 수 있는 짧은 콘셉트 요약이 있는가?

검토 예시:

“액션 RPG를 만든다”는 너무 넓은 표현입니다. “플레이어가 짧은 스테이지에 입장해 적을 처치하고, 보상을 얻어 성장하는 스테이지형 액션 RPG”처럼 플레이어 행동과 반복 구조가 드러나야 합니다.

2. 핵심 플레이 루프 검토

- 루프의 시작 지점이 명확한가?
- 플레이어의 반복 행동이 순서대로 작성되어 있는가?
- 행동의 결과가 보상으로 연결되는가?
- 보상이 성장으로 이어지는가?
- 성장이 다음 도전으로 이어지는가?
- 실패했을 때의 흐름이 있는가?
- 로비, 전투, 보상, 성장 흐름이 끊기지 않는가?
- MVP에서 반드시 필요한 루프만 남아 있는가?
- 루프에 포함되지 않는 기능을 나중에 미뤘는가?
- 루프를 AI 구현 요청으로 바꿀 수 있는가?

검토 예시:

“스테이지에 입장한다 → 적을 처치한다 → 보상을 얻는다 → 로비로 돌아온다 → 성장한다 → 다음 스테이지에 도전한다”처럼 반복 흐름이 닫힌 구조여야 합니다.

3. 주요 시스템 목록 검토

- 필요한 시스템이 모두 나열되어 있는가?
- 각 시스템의 책임이 한 문장으로 설명되어 있는가?
- 시스템 간 역할이 중복되지 않는가?
- 데이터 관리 시스템과 UI 시스템이 구분되어 있는가?
- 전투, 보상, 성장, 스테이지 시스템이 연결되어 있는가?
- 현재 버전에 필요한 시스템과 나중에 추가할 시스템이 구분되어 있는가?
- 각 시스템을 클래스 설계 요청으로 변환할 수 있는가?
- 시스템 이름이 코드 클래스 이름으로 사용하기에 적절한가?
- 불필요하게 큰 Manager 클래스가 생기지 않도록 책임이 나뉘어 있는가?
- AI에게 시스템 단위로 구현 요청을 보낼 수 있는가?

검토 예시:

“게임 시스템”처럼 큰 이름 하나로 묶기보다 CombatSystem, RewardManager, InventoryManager, StageManager, UIManager처럼 책임을 나누는 것이 좋습니다.

4. 플레이어 조작 기획 검토

- 입력 장치가 명확한가?
- 이동 입력이 정의되어 있는가?
- 공격 입력이 정의되어 있는가?
- 대시, 회피, 아이템 사용 등 추가 입력이 정리되어 있는가?
- 각 입력의 실행 조건이 작성되어 있는가?
- 각 입력의 불가능 조건이 작성되어 있는가?
- 캐릭터 상태별 입력 가능 여부가 정리되어 있는가?
- 입력과 애니메이션 전환 조건이 연결되어 있는가?
- UI 팝업, 사망, 로딩 중 입력 차단 조건이 있는가?
- AI에게 입력 처리 구현을 요청할 수 있을 만큼 구체적인가?

검토 예시:

“공격 버튼을 누르면 공격한다”보다 “마우스 좌클릭 입력 시 Attack 상태로 전환하고, 공격 쿨타임이 끝난 경우에만 공격 애니메이션을 재생한다”처럼 조건이 있어야 합니다.

5. 전투 시스템 검토

- 공격 입력과 공격 판정이 구분되어 있는가?
- 공격 범위와 판정 타이밍이 정의되어 있는가?
- 데미지 계산식이 작성되어 있는가?
- 최소 데미지, 방어력, 치명타 적용 여부가 정리되어 있는가?
- 피격 처리와 무적 시간이 정의되어 있는가?
- HP가 0 이하일 때의 사망 처리가 명확한가?
- Dead 상태 대상에게 추가 데미지가 적용되지 않는가?
- 보상 지급 조건과 사망 처리가 연결되어 있는가?
- 보상 중복 지급 방지 조건이 있는가?
- 전투 종료 후 입력과 AI가 중지되는가?

검토 예시:

적이 사망했을 때 경험치와 골드를 지급한다면, “Dead 상태로 처음 전환되는 순간 한 번만 보상 지급”이라는 조건을 반드시 작성해야 합니다.

6. 아이템과 보상 기획 검토

- 아이템 종류가 구분되어 있는가?
- 소비 아이템과 재료 아이템의 차이가 명확한가?
- 아이템 사용 조건과 사용 결과가 작성되어 있는가?
- 수량이 0이 되었을 때의 처리가 있는가?
- 보상 지급 조건이 명확한가?
- 적 처치 보상과 스테이지 클리어 보상이 구분되어 있는가?
- 보상 데이터가 아이템 데이터와 연결되어 있는가?
- 드랍 확률의 범위가 명확한가?
- 인벤토리 공간 부족 시 처리 방식이 있는가?
- 보상 중복 지급 방지 조건이 있는가?

검토 예시:

“아이템을 지급한다”라고만 쓰면 부족합니다. 어떤 ItemId를 몇 개 지급하는지, 인벤토리가 가득 찼을 때 어떻게 처리하는지까지 작성해야 합니다.

7. UI 화면 기획 검토

- 화면 목록이 정리되어 있는가?

- 각 화면의 목적이 한 문장으로 작성되어 있는가?
- 각 화면에 표시할 데이터가 정리되어 있는가?
- 버튼 목록과 버튼 동작이 작성되어 있는가?
- 화면 전환 흐름이 명확한가?
- UI가 어떤 시스템 메서드를 호출하는지 정리되어 있는가?
- 데이터 변경 시 UI 갱신 조건이 있는가?
- 팝업과 토스트 메시지 규칙이 있는가?
- 화면 중복 열림 방지 조건이 있는가?
- AI에게 UI 구현 요청을 보낼 수 있을 만큼 구체적인가?

검토 예시:

상점 화면에는 상품 목록, 보유 골드, 구매 버튼이 있어야 합니다. 구매 버튼은 ShopManager.PurchaseItem을 호출하고, 성공 또는 실패 결과에 따라 UI를 갱신해야 합니다.

8. 스테이지와 레벨 디자인 검토

- 스테이지 ID와 이름이 명확한가?
- 스테이지 목표가 작성되어 있는가?
- 클리어 조건과 실패 조건이 구분되어 있는가?
- 플레이어 시작 위치가 정의되어 있는가?
- 적 배치와 스폰 규칙이 작성되어 있는가?
- 난이도 기준이 정리되어 있는가?
- 스테이지 보상이 작성되어 있는가?
- 결과 화면과 로비 복귀 흐름이 연결되어 있는가?
- 잘못된 StageId나 RewardId에 대한 예외 처리가 있는가?
- 스테이지 종료 후 전투 시스템이 중지되는가?

검토 예시:

“모든 적을 처치하면 클리어”라는 조건을 사용한다면, 적이 스폰되지 않았거나 EnemyGroupData가 비어 있는 경우를 어떻게 처리할지도 정해야 합니다.

9. 데이터 테이블 검토

- 데이터 테이블 목록이 정리되어 있는가?
- 각 테이블의 목적이 명확한가?
- 필드 이름이 구체적인가?

- 필드 타입이 작성되어 있는가?
- 수치 단위가 명확한가?
- ID 중복 방지 규칙이 있는가?
- 참조하는 ID가 존재하는지 검증할 수 있는가?
- 기본값과 허용 범위가 정리되어 있는가?
- 코드에서 사용할 구조와 연결되어 있는가?
- AI에게 ScriptableObject, JSON, CSV 구조로 요청할 수 있는가?

검토 예시:

DropRate가 0~1 범위인지 0~100 범위인지 정하지 않으면 구현자가 다르게 해석할 수 있습니다. 확률, 시간, 거리, 속도 같은 값은 반드시 단위를 작성해야 합니다.

10. 예외 조건 검토

- 플레이어 사망 상태에서 입력이 들어온 경우가 작성되어 있는가?
- 적이 사망한 후 추가 공격을 받은 경우가 작성되어 있는가?
- 보상이 중복 지급될 가능성이 정리되어 있는가?
- 인벤토리가 가득 찬 경우의 처리가 있는가?
- 골드가 부족한 경우의 처리가 있는가?
- 잘못된 데이터 ID를 참조한 경우의 처리가 있는가?
- 화면이 중복으로 열리는 경우의 처리가 있는가?
- 스테이지 클리어와 실패가 동시에 발생한 경우의 기준이 있는가?
- 저장 실패나 데이터 손상 상황이 고려되어 있는가?
- 예외 조건이 테스트 케이스로 변환 가능한가?

검토 예시:

예외 조건은 나중에 생각할 문제가 아닙니다. 예외 조건이 빠진 기획서는 AI에게 구현을 요청했을 때 정상 흐름만 동작하는 불안정한 코드로 이어지기 쉽습니다.

11. AI 프롬프트 변환 가능성 검토

- 기획서를 기능 구현 요청으로 바꿀 수 있는가?
- 클래스 설계 요청으로 바꿀 수 있는가?
- UI 구현 요청으로 바꿀 수 있는가?
- 테스트 코드 요청으로 바꿀 수 있는가?
- 입력 데이터와 출력 결과가 작성되어 있는가?

- 처리 흐름이 순서대로 정리되어 있는가?
- 예외 조건이 명확한가?
- 기존 클래스와 변경 제한 조건이 작성되어 있는가?
- AI가 임의로 구조를 만들 가능성이 줄어들었는가?
- 결과 검토 기준이 있는가?

검토 예시:

“전투 시스템을 구현해줘”보다 “공격 입력, 판정 범위, 데미지 계산식, 사망 처리, 보상 지급, 예외 조건”이 모두 포함된 프롬프트로 바꿀 수 있어야 합니다.

12. 구현 후 검토

- 구현 결과가 기획서 요구사항을 충족하는가?
- 코드가 기획서와 다른 규칙으로 동작하지 않는가?
- 구현 중 새로 생긴 조건을 기획서에 반영했는가?
- 데이터 테이블과 코드의 필드명이 일치하는가?
- UI 표시 정보와 실제 데이터가 일치하는가?
- 테스트 결과가 기획서의 기대 결과와 일치하는가?
- AI가 만든 코드에서 임의로 추가된 구조가 없는가?
- 수정된 코드와 기획서가 함께 버전 관리되는가?
- README에서 관련 기획서로 연결되는가?
- 다음 구현 요청에 반영할 프롬프트 수정 이력이 남아 있는가?

검토 예시:

코드가 정상 동작하더라도 기획서와 다르게 구현되었다면 둘 중 하나를 수정해야 합니다. 중요한 것은 코드와 문서가 같은 게임 규칙을 바라보도록 맞추는 것입니다.

13. 최종 검토 체크리스트

- 게임 콘셉트가 명확하다.
- 핵심 플레이 루프가 닫힌 구조로 작성되어 있다.
- 주요 시스템의 책임이 분리되어 있다.
- 조작, 전투, 보상, UI, 스테이지가 서로 연결되어 있다.
- 데이터 테이블의 필드와 타입이 명확하다.
- 정상 흐름과 예외 조건이 모두 작성되어 있다.
- MVP 범위와 제외할 기능이 구분되어 있다.

- AI 구현 요청 프롬프트로 변환할 수 있다.
- 테스트 코드 요청으로 변환할 수 있다.
- 구현 후 수정된 내용을 기획서에 반영할 수 있다.
- Git 또는 문서 이력으로 변경 사항을 관리할 수 있다.
- README에서 주요 기획서로 접근할 수 있다.

기획서 검토는 문장의 맞춤법을 확인하는 일이 아닙니다. 게임 규칙이 명확한지, 시스템이 서로 연결되어 있는지, 예외 조건이 빠지지 않았는지, AI와 개발자가 같은 기준으로 구현할 수 있는지를 확인하는 과정입니다. 좋은 기획서 검토 체크리스트는 문서를 예쁘게 다듬는 도구가 아니라, 개발 중 발생할 수 있는 혼란과 버그를 줄이는 안전장치입니다.

부록 K. 상세 기획서 예시 문장 모음

상세 기획서를 작성하다 보면 어떤 문장으로 기능을 설명해야 할지 막힐 때가 있습니다. 이 부록은 게임 콘셉트, 플레이 루프, 조작, 전투, 보상, UI, 데이터, 예외 조건, AI 구현 요청에 사용할 수 있는 예시 문장을 모아둔 자료입니다. 그대로 복사해 사용해도 되고, 프로젝트 상황에 맞게 수정해도 됩니다. 중요한 것은 감상적인 표현보다 조건, 동작, 결과가 드러나는 문장으로 작성하는 것입니다.

1. 게임 콘셉트 예시 문장

이 게임은 플레이어가 짧은 스테이지에 입장해 적을 처치하고, 보상을 얻어 캐릭터를 성장시키는 스테이지형 액션 RPG입니다.

플레이어는 로비에서 캐릭터 상태를 확인하고, 스테이지에 입장해 전투를 진행한 뒤, 보상을 사용해 다음 도전을 준비합니다.

이 게임의 핵심 재미는 짧은 전투를 반복하며 캐릭터가 조금씩 강해지는 성장감입니다.

MVP 단계에서는 플레이어 이동, 기본 공격, 적 AI, 보상 지급, 레벨업, 인벤토리, 상점, 기본 UI만 구현합니다.

멀티플레이, 대규모 장비 강화, 복잡한 스킬 트리, 퀘스트 분기는 현재 버전의 구현 범위에서 제외합니다.

2. 핵심 플레이 루프 예시 문장

플레이어는 로비에서 스테이지를 선택하고, 전투 구간에 입장해 적을 처치합니다.

적을 처치하면 경험치와 골드를 획득하고, 모든 적을 처치하면 스테이지 클리어로 처리합니다.

스테이지 클리어 후에는 결과 화면에서 획득한 보상을 확인하고 로비로 돌아갑니다.

로비에서는 상점과 인벤토리를 이용해 다음 스테이지 도전을 준비합니다.

핵심 플레이 루프는 “준비 → 도전 → 전투 → 보상 → 성장 → 재도전” 구조로 구성됩니다.

3. 플레이어 조작 예시 문장

플레이어는 WASD 입력으로 캐릭터를 이동합니다.

플레이어는 마우스 좌클릭으로 기본 공격을 실행합니다.

공격 입력이 들어오면 플레이어는 Attack 상태로 전환하고 공격 애니메이션을 재생합니다.

공격 쿨타임이 끝나지 않은 상태에서는 새로운 공격 입력을 무시합니다.

플레이어가 Dead 상태일 때는 이동, 공격, 대시, 아이템 사용 입력을 모두 무시합니다.

UI 팝업이 열린 상태에서는 전투 입력을 차단하고, 팝업 닫기 입력만 처리합니다.

4. 캐릭터 상태 예시 문장

플레이어 상태는 Idle, Move, Attack, Hit, Dash, Dead로 구분합니다.

Idle 상태에서는 이동, 공격, 대시 입력을 처리할 수 있습니다.

Attack 상태에서는 추가 공격 입력과 대시 입력을 처리하지 않습니다.

Hit 상태에서는 짧은 시간 동안 이동과 공격을 제한합니다.

Dead 상태가 되면 모든 입력을 차단하고 사망 애니메이션을 재생합니다.

상태 전환은 입력, 피격, HP 변화, 애니메이션 종료 이벤트를 기준으로 처리합니다.

5. 전투 시스템 예시 문장

플레이어가 공격하면 전방 범위 안에 있는 살아 있는 적을 검사합니다.

공격 판정에 포함된 적에게 데미지를 적용합니다.

최종 데미지는 공격력에서 방어력을 뺀 값으로 계산하며, 최소 데미지는 1로 처리합니다.

적의 현재 HP가 0 이하가 되면 Dead 상태로 전환합니다.

Dead 상태의 적은 추가 데미지를 받지 않으며, 공격 대상에서도 제외됩니다.

적이 Dead 상태로 처음 전환될 때 경험치와 골드를 한 번만 지급합니다.

6. 적 AI 예시 문장

적은 플레이어가 인식 범위 안에 들어오면 추적을 시작합니다.

적은 플레이어가 공격 범위 안에 들어오면 이동을 멈추고 공격 상태로 전환합니다.

공격 후에는 지정된 쿨타임 동안 다시 공격할 수 없습니다.

플레이어가 인식 범위를 벗어나면 적은 추적을 중단하고 대기 상태로 돌아갑니다.

적이 Dead 상태가 되면 모든 AI 행동을 중지합니다.

스테이지가 종료되면 살아 있는 적의 AI도 함께 비활성화합니다.

7. 아이템 예시 문장

아이템은 소비 아이템, 재료 아이템, 장비 아이템, 열쇠 아이템으로 구분합니다.

소비 아이템은 사용 시 즉시 효과가 적용되고 수량이 1 감소합니다.

체력 포션은 사용 시 플레이어의 HP를 EffectValue만큼 회복합니다.

현재 HP가 최대 HP인 경우 체력 포션을 사용하지 않고 안내 메시지를 표시합니다.

아이템 수량이 0이 되면 인벤토리 목록에서 제거합니다.

사용할 수 없는 아이템은 인벤토리 화면에서 사용 버튼을 표시하지 않습니다.

8. 보상 예시 문장

적을 처치하면 EnemyData에 정의된 경험치와 골드를 지급합니다.

스테이지를 클리어하면 RewardData에 정의된 클리어 보상을 지급합니다.

보상은 스테이지 결과 화면이 열릴 때 한 번만 지급합니다.

이미 지급된 보상은 결과 화면을 다시 열어도 중복 지급되지 않습니다.

인벤토리에 공간이 부족하면 아이템 보상은 지급 실패 처리합니다.

아이템 보상 지급에 실패해도 골드와 경험치는 정상 지급합니다.

9. 상점 예시 문장

상점 화면에는 상품 목록, 보유 골드, 구매 버튼, 닫기 버튼을 표시합니다.

상품 목록은 ShopItemData를 기준으로 생성합니다.

구매 버튼을 누르면 상품 가격과 플레이어 보유 골드를 비교합니다.

골드가 충분하고 인벤토리에 공간이 있으면 골드를 차감하고 아이템을 지급합니다.

골드가 부족하면 구매를 실패 처리하고 데이터는 변경하지 않습니다.

구매 실패 시 실패 이유를 토스트 메시지로 표시합니다.

10. 인벤토리 예시 문장

인벤토리 화면에는 보유 아이템의 아이콘, 이름, 수량, 설명을 표시합니다.

아이템은 ItemType과 DisplayOrder를 기준으로 정렬합니다.

소비 아이템을 선택하면 사용 버튼을 표시합니다.

아이템 사용에 성공하면 수량을 감소시키고 UI를 갱신합니다.

아이템 수량이 0이 되면 해당 아이템 슬롯을 제거합니다.

존재하지 않는 ItemId를 참조한 경우 오류 로그를 출력하고 해당 아이템은 표시하지 않습니다.

11. UI 화면 예시 문장

로비 화면은 플레이어가 스테이지에 입장하기 전 캐릭터 상태를 확인하고 정비하는 화면입니다.

로비 화면에는 플레이어 레벨, 경험치, 보유 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼을 표시합니다.

인게임 HUD에는 HP 바, 경험치 바, 골드, 포션 수량을 표시합니다.

HP가 변경되면 HP 바를 즉시 갱신합니다.

상점 버튼을 누르면 ShopUI를 열고, 인벤토리 버튼을 누르면 InventoryUI를 엽니다.

결과 화면에서 확인 버튼을 누르면 로비 화면으로 돌아갑니다.

12. 스테이지 예시 문장

로비에서 스테이지 시작 버튼을 누르면 선택한 StageData를 불러옵니다.

StageData에 정의된 EnemyGroupId를 기준으로 적을 스폰합니다.

스테이지에 등장한 모든 적이 Dead 상태가 되면 클리어 처리합니다.

플레이어가 Dead 상태가 되면 스테이지 실패로 처리합니다.

스테이지 클리어 시 RewardId를 기준으로 보상을 지급합니다.

스테이지 종료 후에는 적 AI와 플레이어 전투 입력을 모두 중지합니다.

13. 데이터 테이블 예시 문장

EnemyData에는 EnemyId, Name, MaxHp, AttackPower, Defense, MoveSpeed, RewardExp, RewardGold를 포함합니다.

ItemData에는 ItemId, ItemName, ItemType, Description, MaxStack, EffectType, EffectValue, Price를 포함합니다.

StageData에는 StageId, StageName, EnemyGroupId, RewardId, ClearCondition, FailCondition을 포함합니다.

RewardData에는 RewardId, ExpAmount, GoldAmount, ItemId, ItemCount, DropRate를 포함합니다.

ItemId는 중복될 수 없으며, 삭제된 ItemId는 가능한 한 재사용하지 않습니다.

DropRate는 0부터 1 사이의 값으로 관리합니다.

14. 예외 조건 예시 문장

플레이어가 Dead 상태일 때 입력이 들어오면 해당 입력을 무시합니다.

적이 Dead 상태인 경우 추가 데미지를 적용하지 않습니다.

공격 쿨타임 중 공격 입력이 들어오면 입력을 무시합니다.

골드가 부족한 상태에서 구매를 시도하면 구매 실패로 처리합니다.

인벤토리가 가득 찬 상태에서 아이템을 획득하면 아이템 지급을 실패 처리합니다.

결과 화면은 스테이지 종료 시 한 번만 열려야 합니다.

보상 지급은 중복으로 실행되지 않도록 지급 완료 상태를 기록합니다.

잘못된 StageId가 전달되면 스테이지 시작을 중단하고 오류 로그를 출력합니다.

15. AI 구현 요청 예시 문장

Unity 6 C#에서 사용할 전투 시스템을 구현해줘.

기존 PlayerController, CombatController, EnemyController 구조는 유지해줘.

플레이어가 공격하면 전방 범위의 살아 있는 적에게 데미지를 적용하도록 만들어줘.

데미지는 공격력에서 방어력을 뺀 값으로 계산하고, 최소 데미지는 1로 처리해줘.

적의 HP가 0 이하가 되면 Dead 상태로 전환하고 보상을 한 번만 지급해줘.

Dead 상태의 적은 추가 데미지를 받지 않도록 처리해줘.

코드는 컴파일 가능한 C# 형태로 작성하고, 필요한 클래스는 파일 단위로 분리해줘.

Unity Test Framework를 사용한 기본 테스트 코드도 함께 작성해줘.

16. AI 재요청 예시 문장

현재 코드는 골드가 부족한 경우에도 아이템을 인벤토리에 추가합니다.

기획서 기준으로 골드가 부족하면 구매를 실패 처리해야 하며, 골드와 인벤토리 데이터는 변경되지 않아야 합니다.

기존 ShopManager 구조와 public 메서드 시그니처는 유지하고, PurchaseItem 내부 로직만 수정해줘.

구매 실패 이유를 반환할 수 있도록 결과 타입을 정리해줘.

현재 결과 화면이 두 번 열리면서 보상이 중복 지급됩니다.

스테이지 종료 상태를 관리해 결과 화면이 한 번만 열리고, 보상도 한 번만 지급되도록 수정해줘.

17. 테스트 코드 요청 예시 문장

ShopManager의 PurchaseItem 기능에 대한 테스트 코드를 작성해줘.

골드가 충분한 경우 구매가 성공하고, 골드가 차감되며, 인벤토리에 아이템이 추가되어야 합니다.

골드가 부족한 경우 구매가 실패하고, 골드와 인벤토리 데이터는 변경되지 않아야 합니다.

잘못된 ShopItemId가 전달되면 false를 반환하고 오류 로그를 출력해야 합니다.

적 HP가 0 이하가 되면 Dead 상태로 전환되고 보상이 한 번만 지급되는지 테스트해줘.

Dead 상태의 적에게 다시 데미지를 적용해도 보상이 중복 지급되지 않아야 합니다.

18. 변경 이력 예시 문장

2026-06-21: 상점 구매 실패 조건에 인벤토리 공간 부족을 추가했습니다.

2026-06-21: 스테이지 클리어 보상이 중복 지급되지 않도록 RewardClaimed 상태를 추가했습니다.

2026-06-21: Dead 상태의 적은 공격 판정 대상에서 제외하도록 전투 예외 조건을 수정했습니다.

2026-06-21: 포션 사용 조건에 “현재 HP가 최대 HP인 경우 사용 실패” 조건을 추가했습니다.

2026-06-21: DropRate 단위를 0~100에서 0~1 범위로 변경했습니다.

19. 모호한 문장 개선 예시

모호한 문장: 적이 적당히 강하게 나온다. 개선 문장: Stage_001에는 MaxHp 30, AttackPower 5인 슬라임 3마리를 배치합니다.

모호한 문장: 보상을 잘 준다. 개선 문장: 일반 적 처치 시 경험치 10과 골드 5를 지급하고, 스테이지 클리어 시 골드 50을 추가 지급합니다.

모호한 문장: 공격하면 적이 맞는다. 개선 문장: 공격 애니메이션 이벤트 시점에 전방 2m 범위의 살아 있는 적을 검사하고, 첫 번째 대상에게 데미지를 적용합니다.

모호한 문장: 상점에서 아이템을 살 수 있다. 개선 문장: 구매 버튼을 누르면 보유 골드와 상품 가격을 비교하고, 골드가 충분하면 골드를 차감한 뒤 인벤토리에 아이템을 추가합니다.

모호한 문장: UI를 갱신한다. 개선 문장: 골드 값이 변경되면 LobbyUI와 ShopUI의 GoldText를 즉시 갱신합니다.

20. 검토용 마무리 문장 예시

이 기능은 MVP 범위에 포함되며, 핵심 플레이 루프를 완성하기 위해 우선 구현합니다.

이 기능은 현재 버전에서는 제외하고, 전투와 보상 루프가 완성된 뒤 확장 기능으로 검토합니다.

이 조건은 구현 중 발견된 예외 상황이며, 이후 동일한 기능을 수정할 때 반드시 참고해야 합니다.

이 문서는 AI 구현 요청의 기준이 되며, 코드 변경 시 함께 업데이트해야 합니다.

이 기획서는 현재 구현 기준이며, 테스트 결과에 따라 수치와 예외 조건이 수정될 수 있습니다.

상세 기획서의 문장은 멋있을 필요보다 정확할 필요가 있습니다. 좋은 문장은 플레이어 행동, 시스템 반응, 데이터 변경, 예외 조건이 분명하게 드러나는 문장입니다. 이 예시 문장 모음은 기획서를 빠르게 채우기 위한 자료이기도 하지만, 동시에 모호한 아이디어를 구현 가능한 개발 언어로 바꾸는 연습 도구입니다.

부록 L. 미니 프로젝트 최종 기획서 예시

이 부록은 지금까지 작성한 게임 콘셉트, 핵심 플레이 루프, 조작, 전투, 아이템, 보상, UI, 스테이지, 데이터 테이블, AI 구현 요청 프롬프트를 하나로 묶은 최종 상세 기획서 예시입니다. 실제 프로젝트에서는 이 문서를 그대로 복사해 사용하는 것이 아니라, 자신의 게임 구조에 맞게 항목을 수정하면 됩니다. 중요한 것은 게임의 아이디어를 구현 가능한 규칙과 데이터, 화면 흐름, 예외 조건으로 정리하는 것입니다.

1. 프로젝트 기본 정보

- 게임 제목: 스테이지 액션 RPG 실습 프로젝트
- 장르: 스테이지형 액션 RPG
- 플랫폼: PC
- 개발 엔진: Unity 6
- 개발 언어: C#
- 플레이 방식: 싱글 플레이
- 목표 버전: MVP
- 예상 플레이 시간: 스테이지당 3~5분
- 프로젝트 목표: 전투, 보상, 성장, 인벤토리, 상점, UI 흐름을 갖춘 작은 게임 구현

이 프로젝트는 대규모 액션 RPG를 만드는 것이 아니라, 상세 기획서를 기준으로 게임 시스템을 구조화하고 AI 구현 요청으로 변환하는 과정을 연습하기 위한 미니 프로젝트입니다.

2. 게임 콘셉트

이 게임은 플레이어가 짧은 스테이지에 입장해 적을 처치하고, 보상을 얻어 캐릭터를 성장시키는 스테이지형 액션 RPG입니다.

플레이어는 로비에서 스테이지를 선택하고, 전투 구간에 입장해 적을 처치합니다. 적을 처치하면 경험치와 골드를 얻고, 스테이지를 클리어하면 추가 보상을 획득합니다. 로비로 돌아온 플레이어는 상점에서 포션을 구매하거나 인벤토리를 확인한 뒤 다음 스테이지에 도전합니다.

이 게임의 핵심 재미는 짧은 전투를 반복하며 캐릭터가 점점 강해지는 성장감입니다. MVP 단계에서는 플레이어 이동, 기본 공격, 적 AI, 체력과 데미지, 보상, 레벨업, 인벤토리, 상점, 기본 UI만 구현합니다. 멀티플레이, 복잡한 장비 강화, 스킬 트리, 퀘스트 분기, 온라인 서버 연동은 제외합니다.

3. 핵심 플레이 루프

핵심 플레이 루프는 다음과 같습니다.

로비에서 준비한다 → 스테이지에 입장한다 → 적을 처치한다 → 보상을 얻는다 → 캐릭터를 성장시킨다 → 다음 스테이지에 도전한다

플레이어는 로비 화면에서 현재 레벨, 경험치, 보유 골드, 인벤토리 상태를 확인합니다. 스테이지 시작 버튼을 누르면 전투 스테이지에 입장합니다. 전투 중에는 이동과 기본 공격을 사용해 적을 처치합니다. 모든 적을 처치하면 스테이지 클리어로 처리하고 결과 화면을 표시합니다. 결과 화면에서 보상을 확인한 뒤 로비로 돌아와 다음 도전을 준비합니다.

4. 주요 시스템 목록

플레이어 조작 시스템

플레이어 이동, 기본 공격, 대시, 포션 사용 입력을 처리합니다. 플레이어가 Dead 상태일 때는 모든 입력을 무시합니다.

전투 시스템

플레이어와 적의 공격, 피격, 데미지 계산, HP 감소, 사망 처리를 담당합니다. 적이 사망하면 보상 시스템에 보상 지급을 요청합니다.

적 AI 시스템

적은 플레이어가 인식 범위 안에 들어오면 추적하고, 공격 범위 안에 들어오면 공격합니다. 적이 Dead 상태가 되면 모든 AI 행동을 중지합니다.

보상 시스템

적 처치와 스테이지 클리어 시 경험치, 골드, 아이템을 지급합니다. 보상은 중복 지급되지 않도록 지급 완료 상태를 관리합니다.

성장 시스템

플레이어가 경험치를 획득하고, 필요 경험치 이상이 되면 레벨업합니다. 레벨업 시 최대 HP와 공격력이 증가합니다.

인벤토리 시스템

플레이어가 보유한 아이템과 수량을 관리합니다. 소비 아이템 사용, 아이템 추가, 아이템 제거, 수량 갱신을 담당합니다.

상점 시스템

플레이어가 골드를 사용해 포션 아이템을 구매할 수 있도록 합니다. 골드 부족 또는 인벤토리 공간 부족 시 구매를 실패 처리합니다.

UI 시스템

로비 화면, 인게임 HUD, 인벤토리 화면, 상점 화면, 결과 화면을 관리합니다. 게임 데이터가 변경되면 관련 UI를 갱신합니다.

스테이지 시스템

StageData를 기준으로 스테이지를 시작하고 적을 스폰합니다. 모든 적이 Dead 상태가 되면 클리어 처리하고, 플레이어가 Dead 상태가 되면 실패 처리합니다.

5. 플레이어 조작 기획

플레이어는 WASD로 이동합니다. 이동 방향은 월드 좌표 기준으로 처리합니다. 플레이어가 Idle 또는 Move 상태일 때만 이동할 수 있습니다.

플레이어는 마우스 좌클릭으로 기본 공격을 실행합니다. 공격 입력이 들어오면 Attack 상태로 전환하고 공격 애니메이션을 재생합니다. 공격 판정은 애니메이션 이벤트 시점에 발생합니다. 공격 쿨타임이 끝나지 않은 상태에서는 새로운 공격 입력을 무시합니다.

플레이어는 Space 키로 대시를 사용할 수 있습니다. 대시는 현재 이동 방향으로 짧게 이동하며, 대시 중 0.2초 동안 피격을 무시합니다. 플레이어가 Attack, Hit, Dead 상태일 때는 대시할 수 없습니다.

플레이어는 숫자 키 1로 체력 포션을 사용할 수 있습니다. 체력 포션을 사용하면 HP가 회복되고 포션 수량이 1 감소합니다. 포션 수량이 0이거나 HP가 최대 HP인 경우에는 사용 실패 메시지를 표시합니다.

6. 전투 시스템 기획

플레이어가 공격하면 전방 범위 안에 있는 살아 있는 적을 검사합니다. 공격 판정에 포함된 적에게 데미지를 적용합니다. 최종 데미지는 공격력 - 방어력으로 계산하며, 최소 데미지는 1로 처리합니다.

적의 현재 HP가 0 이하가 되면 Dead 상태로 전환합니다. Dead 상태의 적은 추가 데미지를 받지 않으며, 공격 대상에서도 제외됩니다. 적이 Dead 상태로 처음 전환될 때 경험치와 골드를 한 번만 지급합니다.

적은 플레이어가 인식 범위 안에 들어오면 추적을 시작합니다. 플레이어가 공격 범위 안에 들어오면 이동을 멈추고 공격합니다. 적의 공격 판정에 플레이어가 맞으면 플레이어 HP가 감소합니다. 플레이어 HP가 0 이하가 되면 Dead 상태로 전환하고 스테이지 실패로 처리합니다.

7. 아이템과 보상 기획

MVP 단계에서 사용하는 아이템은 체력 포션과 재료 아이템으로 제한합니다. 체력 포션은 소비 아이템이며, 사용 시 HP를 회복하고 수량이 1 감소합니다. 재료 아이템은 현재 버전에서는 수집만 가능하고 사용 기능은 구현하지 않습니다.

적 처치 보상은 EnemyData를 기준으로 지급합니다. 일반 적은 경험치와 골드를 지급합니다. 스테이지 클리어 보상은 RewardData를 기준으로 지급합니다. 스테이지 클리어 보상은 결과 화면이 열릴 때 한 번만 지급됩니다.

인벤토리에 공간이 부족한 경우 아이템 보상은 지급 실패 처리합니다. 단, 골드와 경험치는 정상 지급합니다. 존재하지 않는 ItemId가 보상 데이터에 포함되어 있으면 오류 로그를 출력하고 해당 아이템 지급만 중단합니다.

8. UI 화면 기획

로비 화면

로비 화면은 플레이어가 스테이지에 입장하기 전 캐릭터 상태를 확인하고 정비하는 화면입니다. 플레이어 레벨, 경험치, 보유 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼을 표시합니다.

인게임 HUD

인게임 HUD는 전투 중 필요한 정보를 표시합니다. HP 바, 경험치 바, 골드, 포션 수량을 표시합니다. HP가 변경되면 HP 바를 갱신하고, 포션을 사용하면 포션 수량을 갱신합니다.

인벤토리 화면

인벤토리 화면은 보유 아이템의 아이콘, 이름, 수량, 설명을 표시합니다. 소비 아이템을 선택하면 사용 버튼을 표시합니다. 수량이 0이 된 아이템은 목록에서 제거합니다.

상점 화면

상점 화면은 상품 목록, 보유 골드, 구매 버튼, 닫기 버튼을 표시합니다. 구매 버튼을 누르면 보유 골드와 상품 가격을 비교합니다. 구매 성공 시 골드를 차감하고 아이템을 인벤토리에 추가합니다. 실패 시 토스트 메시지를 표시합니다.

결과 화면

결과 화면은 스테이지 종료 후 클리어 여부, 획득 경험치, 골드, 아이템 목록을 표시합니다. 확인 버튼을 누르면 로비 화면으로 돌아갑니다. 결과 화면은 스테이지 종료 시 한 번만 열려야 합니다.

9. 스테이지 기획

Stage_001: 숲의 입구

- 권장 레벨: 1
- 클리어 조건: 모든 적 처치
- 실패 조건: 플레이어 HP 0 이하
- 등장 적: 슬라임 3마리
- 예상 플레이 시간: 3분
- 클리어 보상: 경험치 30, 골드 50

Stage_001은 플레이어가 이동과 기본 공격을 익히는 첫 번째 전투 스테이지입니다. 적의 공격력은 낮게 설정하고, 플레이어가 기본 전투 흐름을 이해할 수 있도록 구성합니다.

Stage_002: 고블린 초소

- 권장 레벨: 2
- 클리어 조건: 모든 적 처치

- 실패 조건: 플레이어 HP 0 이하
- 등장 적: 슬라임 3마리, 고블린 2마리
- 예상 플레이 시간: 4분
- 클리어 보상: 경험치 50, 골드 80

Stage_002는 Stage_001보다 적 수와 공격 빈도를 늘려 난이도를 조금 높입니다. 플레이어가 포션 사용과 대시를 함께 활용하도록 유도합니다.

10. 데이터 테이블 예시

PlayerLevelData

Level	RequiredExp	MaxHp	AttackPower	Defense
1	100	100	10	2
2	150	120	14	3
3	220	140	18	4

EnemyData

EnemyId	Name	MaxHp	AttackPower	Defense	MoveSpeed	RewardExp	RewardGold
Enemy_001	슬라임	30	5	0	2.0	10	5
Enemy_002	고블린	50	8	1	2.5	18	10

ItemData

ItemId	ItemName	ItemType	Description	MaxStack	EffectType	EffectValue	Price
1001	체력 포션	Consumable	HP를 50 회복합니다	99	HealHp	50	100
2001	작은 수정 조각	Material	강화 재료입니다	999	None	0	0

StageData

StageId	StageName	RequiredLevel	EnemyGroupId	RewardId	ClearCondition	FailCondition
Stage_001	숲의 입구	1	EnemyGroup_001	Reward_Stage_001	KillAllEnemies	PlayerDead
Stage_002	고블린 초소	2	EnemyGroup_002	Reward_Stage_002	KillAllEnemies	PlayerDead

11. 예외 조건

플레이어가 Dead 상태일 때는 이동, 공격, 대시, 아이템 사용 입력을 모두 무시합니다. 적이 Dead 상태인 경우 추가 데미지를 적용하지 않습니다. 적 사망 보상은 한 번만 지급됩니다.

골드가 부족한 상태에서 상점 구매를 시도하면 구매 실패로 처리하고, 골드와 인벤토리 데이터는 변경하지 않습니다. 인벤토리 공간이 부족한 경우에도 구매를 실패 처리합니다.

스테이지 클리어와 플레이어 사망이 동시에 발생한 경우에는 먼저 처리된 상태를 기준으로 결과를 결정합니다. 결과 화면은 한 번만 열려야 하며, 보상도 한 번만 지급되어야 합니다. 잘못된 StageId, RewardId, ItemId가 전달되면 오류 로그를 출력하고 해당 처리를 중단합니다.

12. AI 구현 요청 프롬프트 예시

Unity 6 C#으로 스테이지형 액션 RPG MVP를 구현해줘. 플레이어는 WASD로 이동하고, 마우스 좌클릭으로 기본 공격을 하며, Space 키로 대시한다. 플레이어 상태는 Idle, Move, Attack, Dash, Hit, Dead로 관리한다. Dead 상태에서는 모든 입력을 무시한다.

전투 시스템은 플레이어 공격 시 전방 범위 안의 살아 있는 적에게 데미지를 적용한다. 데미지는 공격력에서 방어력을 뺀 값으로 계산하고 최소 데미지는 1로 처리한다. 적 HP가 0 이하가 되면 Dead 상태로 전환하고 EnemyData에 정의된 경험치와 골드를 한 번만 지급한다.

StageData를 기준으로 스테이지를 시작하고, EnemyGroupId에 따라 적을 스폰한다. 모든 적이 Dead 상태가 되면 스테이지 클리어로 처리하고 RewardData 기준으로 보상을 지급한다. 플레이어가 Dead 상태가 되면 실패 처리한다. 결과 화면은 한 번만 열리고 보상이 중복 지급되지 않도록 상태를 관리해줘.

UI는 로비 화면, 인게임 HUD, 인벤토리 화면, 상점 화면, 결과 화면으로 구성한다. 로비 화면에는 레벨, 경험치, 골드, 스테이지 시작 버튼, 상점 버튼, 인벤토리 버튼을 표시한다. 인게임 HUD에는 HP, 경험치, 골드, 포션 수량을 표시한다. 상점 구매 시 골드가 부족하거나 인벤토리 공간이 부족하면 구매 실패 메시지를 표시하고 데이터는 변경하지 않는다.

13. 테스트 기준

- 플레이어가 이동, 공격, 대시를 정상적으로 수행하는가?
- Dead 상태에서 모든 입력이 차단되는가?
- 공격 범위 안의 적에게만 데미지가 적용되는가?
- 적 HP가 0 이하가 되면 Dead 상태로 전환되는가?
- 적 사망 보상이 한 번만 지급되는가?
- 모든 적 처치 시 스테이지 클리어가 처리되는가?
- 플레이어 사망 시 스테이지 실패가 처리되는가?
- 스테이지 클리어 보상이 중복 지급되지 않는가?
- 상점 구매 성공 시 골드가 차감되고 아이템이 지급되는가?

- 구매 실패 시 골드와 인벤토리 데이터가 변경되지 않는가?
- UI가 데이터 변경에 맞춰 갱신되는가?
- 결과 화면이 중복으로 열리지 않는가?

14. 변경 이력

- 2026-06-21: MVP 범위를 플레이어 이동, 전투, 보상, 인벤토리, 상점, 기본 UI로 제한했습니다.
- 2026-06-21: 적 사망 보상은 Dead 상태로 처음 전환될 때 한 번만 지급하도록 정의했습니다.
- 2026-06-21: 인벤토리 공간 부족 시 아이템 보상은 실패 처리하고, 골드와 경험치는 정상 지급하도록 예외 조건을 추가했습니다.
- 2026-06-21: 결과 화면 중복 표시와 보상 중복 지급 방지 조건을 추가했습니다.

15. 최종 검토

이 미니 프로젝트의 최종 목표는 거대한 게임을 완성하는 것이 아닙니다. 작은 게임이라도 콘셉트, 플레이 루프, 시스템, UI, 데이터, 예외 조건, AI 구현 요청이 서로 연결되는 구조를 경험하는 것입니다. 이 기획서를 기준으로 구현하면, 무엇을 먼저 만들고 어떤 조건을 검토해야 하는지 명확해집니다.

좋은 최종 상세 기획서는 모든 아이디어를 담는 문서가 아닙니다. 현재 버전에서 실제로 구현할 기능을 정리하고, 제외할 기능을 명확히 하며, AI와 개발자가 같은 기준으로 작업할 수 있도록 만드는 실행 가능한 개발 문서입니다.

에필로그

게임 개발에서 프로그래머의 역할은 오랫동안 “기획서를 코드로 구현하는 사람”에 가까웠습니다. 기획자가 문서를 작성하고, 프로그래머는 그 문서를 해석해 시스템을 만들었습니다. 하지만 개인 개발, 소규모 팀 개발, AI와 함께하는 개발이 늘어나면서 이 경계는 점점 흐려지고 있습니다. 이제 프로그래머도 자신이 만들 게임을 설명할 수 있어야 합니다. 무엇을 만들지, 어떤 규칙으로 동작할지, 어떤 데이터가 필요할지 스스로 정리할 수 있어야 합니다.

AI 시대가 되면서 코드를 작성하는 속도는 빨라졌습니다. 예전에는 며칠이 걸리던 기능도 AI의 도움을 받으면 훨씬 빠르게 초안을 만들 수 있습니다. 그러나 개발 속도가 빨라졌다고 해서 생각의 과정이 사라지는 것은 아닙니다. 오히려 무엇을 요청해야 하는지, 어떤 결과가 맞는지, 어떤 예외 조건이 빠졌는지 판단하는 능력이 더 중요해졌습니다. AI는 좋은 도구이지만, 기준 없이 사용하는 도구는 프로젝트를 더 빠르게 혼란스럽게 만들 수도 있습니다.

이 책에서 다룬 상세 기획서는 거창한 문서가 아닙니다. 게임 콘셉트, 핵심 플레이 루프, 조작, 전투, 보상, UI, 데이터, 예외 조건을 구현 가능한 형태로 정리하는 개발 기준 문서입니다. 프로그래머가 기획서를 작성한다는 것은 기획자의 역할을 빼앗는다는 뜻이 아닙니다. 자신이 구현할 시스템을 더 정확하게 이해하고, 팀원이나 AI에게 더 명확히 설명할 수 있는 능력을 갖춘다는 뜻입니다. 결국 좋은 코드는 명확한 생각에서 시작됩니다.

작은 게임 하나를 만들 때도 문서는 필요합니다. 어떤 기능을 먼저 만들지, 어떤 기능을 제외할지, 어떤 데이터로 관리할지, 어떤 조건에서 실패할지 정리해야 합니다. 문서가 없으면 개발 중에 판단이 흔들리고, AI에게 요청할 때도 매번 설명이 달라집니다. 반대로 기획서가 있으면 코드 변경, 테스트, 디버깅, AI 재요청이 모두 같은 기준을 바라볼 수 있습니다. 문서는 개발을 느리게 만드는 짐이 아니라, 개발 방향을 잃지 않게 해주는 지도입니다.

앞으로 AI는 더 많은 코드를 만들고, 더 많은 도구와 연결될 것입니다. 하지만 어떤 게임을 만들지 결정하는 일, 그 게임의 재미를 구조화하는 일, 구현 결과가 의도와 맞는지 검토하는 일은 여전히 개발자의 몫입니다. 프로그래머가 게임을 설명할 수 있을 때, AI는 더 강력한 협업 도구가 됩니다. 이 책이 프로그래머가 자신의 아이디어를 더 명확하게 정리하고, 작은 게임부터 꾸준히 완성해가는 데 도움이 되기를 바랍니다. 좋은 게임 개발은 코드 한 줄에서만 시작하지 않습니다. 명확한 생각, 구조화된 기획, 그리고 끝까지 검토하는 습관에서 시작합니다.